150 Algorithms

in Pseudocode

Ordered by importance for coding interviews

Contents

How to Read This Book 1
Notation . . . 1

Topic Index 3

Tier 1 · Must Know Cold 5
1. Two Sum . . . 6
2. Prefix Sum + Hash Map . . . 7
3. Product of Array Except Self . . . 9
4. Longest Consecutive Sequence . . . 11
5. Group Anagrams . . . 13
6. Reverse a Linked List . . . 15
7. 3Sum . . . 16
8. Container With Most Water . . . 18
9. Trapping Rain Water . . . 20
10. Floyd's Tortoise and Hare . . . 22
11. Longest Substring Without Repeating Characters . . . 24
12. Find All Anagrams in a String . . . 25
13. Minimum Window Substring . . . 27
14. Binary Search . . . 29
15. Binary Search on the Answer . . . 31
16. Search in a Rotated Sorted Array . . . 33
17. Merge Sort . . . 35
18. Quick Sort . . . 37
19. Top-K Elements . . . 39
20. Merge K Sorted Arrays . . . 41
21. Find Median from Data Stream . . . 43
22. Valid Parentheses . . . 45
23. Monotonic Stack . . . 47
24. Min Stack . . . 49
25. Decode String . . . 51
26. Merge Intervals . . . 53
27. Meeting Rooms II . . . 55
28. Kadane's Algorithm . . . 57
29. Best Time to Buy and Sell Stock . . . 59
30. Subsets . . . 61
31. Permutations . . . 63
32. Binary Tree Level-Order Traversal . . . 65
33. Diameter and Max Path Sum . . . 67
34. Lowest Common Ancestor . . . 69
35. Validate a Binary Search Tree . . . 71

36. Serialize and Deserialize a Binary Tree . . . 73
37. Depth-First Search on a Graph . . . 75
38. Number of Islands . . . 77
39. BFS Shortest Path . . . 79
40. Kahn's Algorithm . . . 81
41. Union-Find . . . 83
42. Dijkstra's Algorithm . . . 85
43. Trie . . . 87
44. LRU Cache . . . 89
45. Design Hit Counter . . . 91
46. Design Twitter . . . 93
47. Snapshot Array . . . 95
48. Design Tic-Tac-Toe . . . 97
49. Flatten Nested List Iterator . . . 99
50. Key-Value Store with Transactions . . . 101

Tier 2 · Should Know Well **104**

51. Hash Map with Separate Chaining . . . 105
52. Dutch National Flag . . . 107
53. Longest Repeating Character Replacement . . . 109
54. Minimum Size Subarray Sum . . . 111
55. Median of Two Sorted Arrays . . . 113
56. Search a 2D Matrix . . . 115
57. Find Peak Element . . . 117
58. Longest Increasing Subsequence . . . 119
59. Heapify . . . 121
60. Quickselect . . . 123
61. Bucket Sort . . . 125
62. Largest Rectangle in Histogram . . . 127
63. Basic Calculator . . . 129
64. Sliding Window Maximum . . . 131
65. Insert Interval . . . 133
66. Interval List Intersections . . . 135
67. Jump Game . . . 137
68. Jump Game II . . . 138
69. Gas Station . . . 140
70. Interval Scheduling . . . 142
71. Task Scheduler . . . 144
72. Partition Labels . . . 146
73. Combination Sum . . . 148
74. Word Search . . . 150
75. Construct Binary Tree from Preorder and Inorder . . . 152
76. BST Search, Insert, Delete . . . 154
77. BST Iterator . . . 156
78. Multi-Source BFS . . . 158
79. Word Ladder . . . 160
80. Clone Graph . . . 162
81. Cycle Detection in a Directed Graph . . . 164
82. Topological Sort with DFS . . . 166

83. Alien Dictionary . . . 168
84. Connected Components . . . 170
85. Word Search II . . . 172
86. KMP String Matching . . . 174
87. Longest Palindromic Substring . . . 176
88. Encode and Decode Strings . . . 178
89. Game of Life . . . 180
90. Fibonacci . . . 182
91. Insert Delete GetRandom O(1) . . . 184
92. Time-Based Key-Value Store . . . 186
93. LFU Cache . . . 188
94. Cache with Expiring Keys . . . 191
95. Design Browser History . . . 193
96. Design Underground System . . . 195
97. Design Search Autocomplete System . . . 197
98. My Calendar I and II . . . 200
99. Design Snake Game . . . 202
100. URL Shortener . . . 204

Tier 3 · Good to Know **206**

101. Hash Map with Open Addressing . . . 207
102. Boyer-Moore Majority Vote . . . 209
103. Next Permutation . . . 211
104. Reorder List . . . 213
105. Median of K Sorted Arrays . . . 215
106. Random Pick with Weight . . . 217
107. Segment Tree . . . 219
108. Counting Sort . . . 221
109. Heap Sort . . . 223
110. Circular Queue . . . 225
111. Candy . . . 227
112. Hand of Straights . . . 229
113. N-Queens . . . 231
114. Bipartite Check . . . 233
115. Cycle Detection in an Undirected Graph . . . 235
116. Critical Connections . . . 237
117. Bellman-Ford . . . 239
118. Kruskal's Minimum Spanning Tree . . . 241
119. Prim's Minimum Spanning Tree . . . 243
120. Rabin-Karp . . . 245
121. Longest Common Prefix . . . 247
122. Text Justification . . . 249
123. String Compression . . . 251
124. Reverse Words in a String . . . 253
125. Multiply Strings . . . 255
126. Spiral Matrix . . . 257
127. Rotate Image . . . 259
128. GCD and LCM . . . 261
129. Sieve of Eratosthenes . . . 263

130. Fast Exponentiation 265
131. Reservoir Sampling 267
132. Orientation and Segment Intersection 269
133. Convex Hull 271
134. Max Points on a Line 273
135. Single Number 275
136. Count Set Bits 277
137. Bitmask Subset Enumeration 279
138. Rate Limiter 281
139. Consistent Hashing 283
140. Stock Price Tracker 285
141. All O(1) Data Structure 287
142. Range Module 290
143. In-Memory File System 292
144. Design a Text Editor 295
145. Unique ID Generator 297
146. Bloom Filter 299
147. Count-Min Sketch 301
148. Quadtree 304
149. Job Scheduler 307
150. Inverted Index 309

How to Read This Book

This book holds one hundred and fifty algorithms in the order to learn them. The order is importance for a coding interview: how often the idea appears, how many other problems it unlocks, and how much it costs to be missing it. Tier 1, the first fifty, are the algorithms to know cold. Tier 2, the next fifty, come up often enough that you should know them well. Tier 3, the last fifty, are good to know and separate a prepared candidate from a strong one. Inside a tier, related algorithms sit next to each other, so each tier is one pass over every topic and the next tier deepens it. Thirty-two of the entries are designs, from the LRU cache to the inverted index, because design coding and system design are where interviews increasingly end.

Every entry fits on a page or two and always reads in the same order. It opens with the problem in plain words and a tiny example. Then comes the idea: when a brute force exists, one sentence names it and its cost, and the rest explains the insight that beats it as a picture of the data and the motion of the algorithm. A framed picture traces the algorithm on the example, and every number in it was produced by running the code. The pseudocode follows, then a closing paragraph on the cost, the signals in a problem statement that call for the algorithm, and the classic mistake, and finally a few LeetCode problems to practise on. A reference such as #41 points to entry 41 and is a link.

The companion book, *150 Algorithms in Python*, has the same entries in the same order, each one a short runnable file whose variable names match this pseudocode line for line.

Notation

The pseudocode reads like careful English. Indentation alone marks a block, so there are no END lines. Keywords are in capitals, ← assigns and = compares, so no line ever has to be read twice. Ranges are inclusive at both ends, arrays start at index 0, and containers are written with the operation you would say out loud.

```
FUNCTION Name(a, b)               // RETURN hands back a value
    x ← 0                         // assignment
    IF x = 0 AND b ≠ null         // comparison: = ≠ < ≤ > ≥
        ...
    ELSE IF x > 0 OR NOT done
        ...
    ELSE
        ...
    FOR i FROM 0 TO n − 1         // inclusive at both ends
    FOR i FROM n − 1 DOWNTO 0     // counting down
    FOR EACH x IN items
    WHILE lo < hi
    floor(a / b), a mod b, ∞      // integer division, remainder, infinity
    seen ← empty map              // also: empty set, list, stack, queue, min-heap
    seen[key] ← value             // write; "key IN seen" tests membership
    stack.push(x), stack.pop(), stack.top()
    queue.enqueue(x), queue.dequeue()
```

```
    heap.push(x), heap.pop(), heap.top()
    a[i..j]                       // the slice from i to j, both included
```

Topic Index

The book is ordered by importance. This index groups the same entries by topic; every number is a link.

Topic	Entries
Arrays & Hashing	#1 Two Sum · #2 Prefix Sum + Hash Map · #3 Product of Array Except Self · #4 Longest Consecutive Sequence · #102 Boyer-Moore Majority Vote · #103 Next Permutation
Hash Map Internals	#51 Hash Map with Separate Chaining · #101 Hash Map with Open Addressing
Two Pointers	#6 Reverse a Linked List · #7 3Sum · #8 Container With Most Water · #9 Trapping Rain Water · #52 Dutch National Flag · #104 Reorder List
Sliding Window	#11 Longest Substring Without Repeating Characters · #12 Find All Anagrams in a String · #13 Minimum Window Substring · #53 Longest Repeating Character Replacement · #54 Minimum Size Subarray Sum
Binary Search	#14 Binary Search · #15 Binary Search on the Answer · #16 Search in a Rotated Sorted Array · #55 Median of Two Sorted Arrays · #56 Search a 2D Matrix · #57 Find Peak Element · #58 Longest Increasing Subsequence · #105 Median of K Sorted Arrays · #106 Random Pick with Weight
Sorting	#17 Merge Sort · #18 Quick Sort · #61 Bucket Sort · #108 Counting Sort · #109 Heap Sort
Heaps	#19 Top-K Elements · #20 Merge K Sorted Arrays · #59 Heapify · #60 Quickselect
Stacks	#22 Valid Parentheses · #23 Monotonic Stack · #25 Decode String · #62 Largest Rectangle in Histogram · #63 Basic Calculator
Queues	#64 Sliding Window Maximum · #110 Circular Queue
Intervals	#26 Merge Intervals · #27 Meeting Rooms II · #65 Insert Interval · #66 Interval List Intersections
Greedy	#28 Kadane's Algorithm · #29 Best Time to Buy and Sell Stock · #67 Jump Game · #68 Jump Game II · #69 Gas Station · #70 Interval Scheduling · #71 Task Scheduler · #72 Partition Labels · #111 Candy · #112 Hand of Straights
Backtracking	#30 Subsets · #31 Permutations · #73 Combination Sum · #74 Word Search · #113 N-Queens
Trees	#75 Construct Binary Tree from Preorder and Inorder
DFS	#33 Diameter and Max Path Sum · #34 Lowest Common Ancestor · #36 Serialize and Deserialize a Binary Tree · #37 Depth-First Search on a Graph · #38 Number of Islands
BFS	#32 Binary Tree Level-Order Traversal · #39 BFS Shortest Path · #78 Multi-Source BFS · #79 Word Ladder · #114 Bipartite Check
Graphs	#80 Clone Graph · #116 Critical Connections
Cycle Detection	#10 Floyd's Tortoise and Hare · #81 Cycle Detection in a Directed Graph · #115 Cycle Detection in an Undirected Graph

Topic	Entries
Topological Sort	#40 Kahn's Algorithm · #82 Topological Sort with DFS · #83 Alien Dictionary
Connected Components	#41 Union-Find · #84 Connected Components
Shortest Path	#42 Dijkstra's Algorithm · #117 Bellman-Ford
MST	#118 Kruskal's Minimum Spanning Tree · #119 Prim's Minimum Spanning Tree
BST	#35 Validate a Binary Search Tree · #76 BST Search, Insert, Delete · #77 BST Iterator
Tries	#43 Trie · #85 Word Search II
Range Queries	#107 Segment Tree
Strings	#5 Group Anagrams · #86 KMP String Matching · #87 Longest Palindromic Substring · #88 Encode and Decode Strings · #120 Rabin-Karp · #121 Longest Common Prefix · #122 Text Justification · #123 String Compression · #124 Reverse Words in a String · #125 Multiply Strings
Matrix	#89 Game of Life · #126 Spiral Matrix · #127 Rotate Image
Math	#90 Fibonacci · #128 GCD and LCM · #129 Sieve of Eratosthenes · #130 Fast Exponentiation · #131 Reservoir Sampling
Geometry	#132 Orientation and Segment Intersection · #133 Convex Hull · #134 Max Points on a Line
Bit Manipulation	#135 Single Number · #136 Count Set Bits · #137 Bitmask Subset Enumeration
Design	#21 Find Median from Data Stream · #24 Min Stack · #44 LRU Cache · #45 Design Hit Counter · #46 Design Twitter · #47 Snapshot Array · #48 Design Tic-Tac-Toe · #49 Flatten Nested List Iterator · #50 Key-Value Store with Transactions · #91 Insert Delete GetRandom O(1) · #92 Time-Based Key-Value Store · #93 LFU Cache · #94 Cache with Expiring Keys · #95 Design Browser History · #96 Design Underground System · #97 Design Search Autocomplete System · #98 My Calendar I and II · #99 Design Snake Game · #100 URL Shortener · #138 Rate Limiter · #139 Consistent Hashing · #140 Stock Price Tracker · #141 All O(1) Data Structure · #142 Range Module · #143 In-Memory File System · #144 Design a Text Editor · #145 Unique ID Generator · #146 Bloom Filter · #147 Count-Min Sketch · #148 Quadtree · #149 Job Scheduler · #150 Inverted Index

Tier 1 · Must Know Cold

These fifty algorithms are the core of the interview. Each one is the standard answer to a whole family of problems: hashing to remember the past, two pointers and sliding windows for sequences, binary search for anything monotone, sorting and heaps for order, stacks for nesting, BFS and DFS for every tree and graph, Kahn's algorithm, union-find and Dijkstra for dependencies, components and distances, the trie for prefixes, and the design classics that open every design-coding round: the LRU cache, the hit counter, the news feed, the snapshot array, the iterator and the transactional key-value store. Learn them until you can write each one from a blank page without hesitation.

1. Two Sum

Hash Map of Complements · Arrays & Hashing · Tier 1

Given an array of integers and a target, return the indices of the two numbers that add up to the target; exactly one such pair exists. In `[2, 7, 11, 15]` with target `9` the answer is `[0, 1]`, because `2 + 7 = 9`.

The brute force tries every pair, `n(n − 1) / 2` of them, for `O(n²)` time. Its waste is that every number keeps asking the same question, “did my partner appear earlier?”, and answers it by rescanning the past. A hash map answers that question in `O(1)`. Walk the array once; at index `i` the partner the current number needs is `target − nums[i]`. If that value is already a key in the map, the pair is complete. If not, store the current number with its index and move on, so that a later number can find it.

Picture the map as the memory of everything to the left of the pointer. Each number looks backward exactly once, through the map, instead of across the whole prefix. Checking before storing is what stops a number from pairing with itself when the target is twice its value.

```
target = 9
  i   nums[i]   need = 9 − nums[i]   need in seen?   seen after the step
  0      2              7                 no          {2: 0}
  1      7              2              yes → 0        stop: answer [0, 1]
```

```
FUNCTION TwoSum(nums, target)
    seen ← empty map                        // value → index
    FOR i FROM 0 TO len(nums) − 1
        need ← target − nums[i]
        IF need IN seen                     // partner appeared earlier
            RETURN [seen[need], i]
        seen[nums[i]] ← i                   // store only after checking
    RETURN empty list
```

One pass with constant-time lookups gives `O(n)` time, and the map costs `O(n)` space. Reach for this pattern whenever a problem asks for a pair with a fixed sum, difference or other exact relationship and the input is unsorted: the map turns “search the past” into a single lookup. When the array is sorted, two pointers closing in from both ends do the same job in `O(1)` space; that loop is the heart of 3Sum (#7). The classic mistake is storing `nums[i]` before the lookup, which pairs `3` with itself for target `6` in `[3, 2, 4]` and returns `[0, 0]`.

Practice: LC 1 Two Sum · LC 167 Two Sum II - Input Array Is Sorted · LC 454 4Sum II · LC 15 3Sum

2. Prefix Sum + Hash Map

Subarray Sum Equals K · Arrays & Hashing · Tier 1

Given an array of integers, negatives allowed, and a target `k`, count the contiguous subarrays whose sum is exactly `k`. In `[2, −1, 1, 2]` with `k = 2` the answer is 4: `[2]`, `[2, −1, 1]`, `[−1, 1, 2]` and the final `[2]`.

The brute force fixes every start, extends every end with a running sum and pays `O(n²)`. The insight is that every subarray sum is a difference of two prefix sums. If `prefix` is the sum of everything up to the current index and some earlier prefix equals `prefix − k`, the stretch between them sums to exactly `k`. The question at each index becomes Two Sum (#1) looking backward: how many earlier prefixes equal `prefix − k`? A map from prefix sum to the number of times it has occurred answers it in one lookup. The map is seeded with `0 → 1`, the empty prefix before index 0, so that a subarray starting at index 0 has a partner to find.

Picture the prefix sums as a walk of heights, `0, 2, 1, 2, 4` here. Every subarray is a step from an earlier height to a later one, and the algorithm counts the steps that climb exactly `k`. The map stores counts, not indices, because a height can come back: `2` occurs twice, and both visits pair with the final `4`. That return is why a sliding window fails. With only positive numbers the walk climbs strictly, so each right end has at most one matching left end, and a window that shrinks whenever its sum passes `k` finds it. A negative number lets the walk fall back, the window sum stops being monotone, and the window cannot tell which end to move; on this array it finds only 3 of the 4.

```
k = 2; prefix_count starts as {0: 1} for the empty prefix
  x   prefix   prefix − k   found   total   prefix_count after
  2      2          0         1       1     {0: 1, 2: 1}
 −1      1         −1         0       1     {0: 1, 2: 1, 1: 1}
  1      2          0         1       2     {0: 1, 2: 2, 1: 1}
  2      4          2         2       4     {0: 1, 2: 2, 1: 1, 4: 1}
the four subarrays:  [2]   [2, −1, 1]   [−1, 1, 2]   [2]
```

```
FUNCTION SubarraySum(nums, k)
    prefix_count ← empty map              // prefix sum → how many times seen
    prefix_count[0] ← 1                   // the empty prefix, before index 0
    prefix ← 0
    total ← 0
    FOR EACH x IN nums
        prefix ← prefix + x               // sum of nums up to and including x
        // each earlier prefix equal to prefix − k starts one subarray of sum k
        total ← total + prefix_count.get(prefix − k, 0)
        prefix_count[prefix] ← prefix_count.get(prefix, 0) + 1   // record last
    RETURN total
```

One pass with constant-time map operations gives `O(n)` time, and the map costs `O(n)` space. Reach for this pattern whenever a problem asks about subarrays with an exact sum, especially when negative numbers rule out a window. Key the map by `prefix mod k` to count sums divisible by `k`, or store each prefix's first index instead of a count to find the longest such subarray, as in Contiguous Array with every `0` read as `−1`. The classic mistakes are forgetting the seed, which misses every subarray that starts at index 0 (2 of the 4 here), and recording the current prefix before counting, which pairs a prefix with itself when `k = 0`: on `[0, 0, 0]` that reports 9 subarrays instead of 6.

Practice: LC 560 Subarray Sum Equals K · LC 974 Subarray Sums Divisible by K · LC 523 Continuous Subarray Sum · LC 525 Contiguous Array

3. Product of Array Except Self

Prefix and Suffix Products · Arrays & Hashing · Tier 1

Given an integer array, return an array whose entry `i` is the product of every element except `nums[i]`, without using division. For `[1, 2, 3, 4]` the answer is `[24, 12, 8, 6]`; entry 2, for one, is `1 × 2 × 4 = 8`.

The brute force multiplies the other `n − 1` numbers for every index, `O(n²)` work. Dividing the total product by `nums[i]` looks like a shortcut, but a single zero wrecks it: the total becomes 0, and at the zero's own index the division is by zero. The insight is that "everything except `i`" is two unbroken blocks, the elements left of `i` and the elements right of `i`. So `answer[i]` is a prefix product times a suffix product, and a running product delivers every prefix product in one left-to-right pass and every suffix product in one right-to-left pass.

The output array doubles as the prefix table, which is where the `O(1)` extra space comes from. The first pass writes the running `prefix` into `answer[i]` before multiplying `nums[i]` into `prefix`, so each entry holds exactly the product of the elements strictly to its left. The second pass starts a running `suffix` at the right end, multiplies it into `answer[i]`, and only then absorbs `nums[i]`. Picture two waves crossing the array from opposite ends, each leaving one factor behind at every index. Zeros and negatives need no special case, because nothing is ever divided: a zero simply zeroes every block that contains it, so with one zero every entry except the zero's own is 0, and with two zeros every entry is 0.

```
nums                    1             2            3            4
left of i               (empty) 1     1            1×2 = 2      1×2×3 = 6
right of i              2×3×4 = 24    3×4 = 12     4            (empty) 1
answer = left × right   24            12           8            6

pass 1, left to right                 pass 2, right to left
 i   answer[i] ← prefix   new prefix    i   answer[i] × suffix   new suffix
 0            1                1       3      6 × 1  = 6            4
 1            1                2       2      2 × 4  = 8           12
 2            2                6       1      1 × 12 = 12          24
 3            6               24       0      1 × 24 = 24          24
answer [1, 1, 2, 6]                   answer [24, 12, 8, 6]
```

```
FUNCTION ProductExceptSelf(nums)
    n ← len(nums)
    answer ← array of n copies of 1
    prefix ← 1                          // product of nums[0..i−1]
    FOR i FROM 0 TO n − 1
        answer[i] ← prefix              // everything left of i
        prefix ← prefix * nums[i]       // only now let nums[i] in
    suffix ← 1                          // product of nums[i+1..n−1]
    FOR i FROM n − 1 DOWNTO 0           // right to left
        answer[i] ← answer[i] * suffix  // times everything right of i
        suffix ← suffix * nums[i]
    RETURN answer
```

Two linear passes give `O(n)` time, and besides the output only `prefix` and `suffix` are kept, `O(1)` extra space. Reach for prefix and suffix accumulations whenever the answer at a position combines everything on its left with everything on its right: the same two sweeps build the wall arrays of Trapping Rain

Water (#9) and hand out the candies of #111, and with sums in place of products they find the pivot index whose left sum equals its right sum. The classic mistake is multiplying `nums[i]` into `prefix` before writing `answer[i]`, which leaves every element inside its own product: on the example the first pass writes `[1, 2, 6, 24]`, and the answer comes out `[24, 24, 24, 24]`.

Practice: LC 238 Product of Array Except Self · LC 724 Find Pivot Index · LC 42 Trapping Rain Water · LC 1352 Product of the Last K Numbers

4. Longest Consecutive Sequence

Count Only from the Start of a Run · Arrays & Hashing · Tier 1

Given an unsorted array of integers, return the length of the longest run of consecutive values, values that climb by exactly 1 from one to the next, wherever they sit in the array. In `[100, 4, 200, 1, 3, 2]` the answer is `4`, the run `1, 2, 3, 4`.

Sorting lines the runs up, and one scan that counts steps of exactly 1 and skips repeats finds the longest, `O(n log n)` in all. The linear solution throws the order away and keeps only membership. Put every value in a hash set; then any value can ask in `O(1)` whether its neighbor is present, and a run can be traced by stepping up one value at a time. Stepping up from every value would retrace the same run over and over, `n + (n − 1) + ... + 1` steps on `1, 2, ..., n`, which is `O(n²)`. The fix is to walk only from the start of a run, a value `num` whose left neighbor `num − 1` is missing. Every other value sits inside a run whose start will count it.

Picture the values as dots on a number line. Each dot looks once at the spot to its left. A dot with an empty spot on its left is the bottom of a run, and only that dot walks right, step by step, until the next spot is empty. Each value is therefore visited at most twice, once when the loop tests it as a possible start and once when the walk from its run's start steps onto it, so the work stays linear however the runs are laid out. Duplicates collapse in the set, and negative values need nothing special, since `num − 1` and `num + 1` behave the same below zero.

```
number line   ●─●─●─●   …   ●   …   ●
              1  2  3  4      100     200
start?        ✓  ✗  ✗  ✗       ✓       ✓      ✓ when num − 1 is not in the set

the loop visits the set in its own order
  num   num − 1 in set?   action                  length   best
    1         no          walk 1 → 2 → 3 → 4        4        4
    2         yes         skip                                4
    3         yes         skip                                4
  100         no          walk 100                  1        4
    4         yes         skip                                4
  200         no          walk 200                  1        4
12 lookups in all: two per distinct value
```

```
FUNCTION LongestConsecutive(nums)
    values ← set(nums)                    // O(1) lookups; duplicates collapse
    best ← 0
    FOR EACH num IN values                // the set, so each start walks once
        IF num − 1 IN values
            CONTINUE                      // inside a run: its start counts it
        length ← 1
        WHILE num + length IN values
            length ← length + 1           // one more step up the run
        best ← max(best, length)
    RETURN best
```

Building the set and visiting each value at most twice give `O(n)` time, and the set takes `O(n)` space. Reach for this pattern when a problem asks for consecutive values rather than consecutive positions and wants to beat a sort: the set turns "is my neighbor here?" into one lookup, and the start test makes

each run be counted exactly once. Union-Find (#41) solves the same problem by joining each `x` with `x + 1`, with more machinery and no gain. The classic mistake is looping over `nums` instead of the set. Every copy of a start then walks its whole run again, so 5,000 copies of `1` in front of the run `1, 2, ..., 5000` cost about 25 million steps instead of 5,000.

Practice: LC 128 Longest Consecutive Sequence · LC 298 Binary Tree Longest Consecutive Sequence · LC 2501 Longest Square Streak in an Array

5. Group Anagrams

Canonical-Key Hashing · Strings · Tier 1

Given a list of lowercase words, group together the words that are anagrams of each other, words that use the same letters the same number of times. `["eat", "tea", "tan", "ate", "nat", "bat"]` becomes `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]`, with the groups in the order they first appear.

The brute force tests every pair of words for being anagrams, $O(n^2)$ tests at $O(L)$ each for words of length L. The insight is to stop comparing words with one another and give each word a canonical key: a value that comes out identical for two words exactly when they are anagrams. Grouping then becomes a single pass that files each word in a hash map under its key. Two keys work. The sorted letters, `"aet"` for eat, tea and ate, cost $O(L \log L)$ per word. The 26 letter counts cost $O(L)$, and that is the key the code uses, frozen into a tuple because a map key must be immutable to be hashed.

Picture each key as a barcode of 26 slots, one per letter from `a` to `z`. Shuffling a word's letters never changes its barcode, so anagrams print the same stripes and the map only has to drop equal barcodes into the same bucket. The order of the letters is forgotten; how many of each letter there are survives.

```
          abcdefghijklmnopqrstuvwxyz    (letter counts, "." for zero)
"eat"  →  1...1.............1......    new key: group 0
"tea"  →  1...1.............1......    same key: group 0
"tan"  →  1............1.....1......    new key: group 1
"ate"  →  1...1.............1......    same key: group 0
"nat"  →  1............1.....1......    same key: group 1
"bat"  →  11................1......    new key: group 2
result: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

```
FUNCTION LetterCounts(word)
    counts ← array of 26 zeros
    FOR EACH ch IN word
        slot ← ord(ch) − ord('a')          // 'a' → 0, 'b' → 1, ..., 'z' → 25
        counts[slot] ← counts[slot] + 1
    RETURN tuple(counts)                   // immutable, so it can be a map key

FUNCTION GroupAnagrams(words)
    groups ← empty map                     // count key → words with that key
    FOR EACH word IN words
        key ← LetterCounts(word)
        IF key NOT IN groups
            groups[key] ← empty list       // first word of a new group
        groups[key].append(word)
    RETURN groups.values()                 // one list per key, first-seen order
```

Each word costs $O(L)$ to count plus a constant 26 to hash its key, so grouping n words takes $O(n \cdot L)$ time, and the groups take $O(n \cdot L)$ space. Reach for a canonical key whenever a problem says group, or treats two inputs as equal up to some rearrangement. The same count array decides Valid Anagram with one comparison and, slid along a string as a fixed-width window, finds every anagram of a pattern. The classic mistake is a key that throws information away: `set(word)` forgets multiplicity and merges `"aab"` with `"abb"`, and the sum of the character codes collides on `"ad"` and `"bc"`. A plain list as the key fails outright, because a list cannot be hashed.

Practice: LC 49 Group Anagrams · LC 242 Valid Anagram · LC 438 Find All Anagrams in a String

6. Reverse a Linked List

Three Pointers: prev, curr, next · Two Pointers · Tier 1

Given the head of a singly linked list, reverse the list in place and return the new head. `1 → 2 → 3 → 4 → 5` becomes `5 → 4 → 3 → 2 → 1`, built from the same five nodes.

There is no index to swap by, and copying the values into a new list wastes `O(n)` space. The insight is that reversing a list means turning every `next` arrow around, and an arrow can be turned the moment we stand on its node, provided we first save where it pointed. So walk the list once with three pointers. `curr` is the node being processed, `prev` is the node it must now point back to, and `next_node` holds the rest of the list for the instant its link is cut. Every step saves, turns and advances: `next_node ← curr.next`, then `curr.next ← prev`, then `prev` and `curr` each move one node forward.

Picture two chains. `prev` heads the reversed part, `curr` heads the part still in its original order, and each step lifts the front node off the second chain and lays it on the front of the first, like moving plates from one stack to another. The invariant is that the two chains together hold every node, none lost and none repeated. When `curr` runs off the end, the first chain is the whole list reversed, and `prev` is its head.

```
step   prev (reversed part)         curr (still to reverse)
start  null                         1 → 2 → 3 → 4 → 5 → null
1      1 → null                     2 → 3 → 4 → 5 → null
2      2 → 1 → null                 3 → 4 → 5 → null
3      3 → 2 → 1 → null             4 → 5 → null
4      4 → 3 → 2 → 1 → null         5 → null
5      5 → 4 → 3 → 2 → 1 → null     null            return prev
```

```
FUNCTION ReverseList(head)
    prev ← null                         // head of the reversed part
    curr ← head                         // head of the part still to reverse
    WHILE curr ≠ null
        next_node ← curr.next           // save the rest before the cut
        curr.next ← prev                // turn this arrow around
        prev ← curr                     // the reversed part gains curr
        curr ← next_node                // step into the saved rest
    RETURN prev                         // curr ran off the end
```

Each node is visited once and only three pointers are kept, so the reversal costs `O(n)` time and `O(1)` space. The recursive version reverses everything after the head first, then hooks the head behind its old neighbor with `head.next.next ← head` and `head.next ← null`; it is shorter but holds `O(n)` frames on the call stack, which overflows on a long enough list. Reach for this loop whenever a problem reverses a list or a stretch of one: between two positions, in groups of `k`, or the second half of a list to compare it with the first. The classic mistake is writing `curr.next ← prev` before saving `curr.next`, which cuts the rest of the list loose; the runner-up is returning `curr`, which is `null` at the end, instead of `prev`.

Practice: LC 206 Reverse Linked List · LC 92 Reverse Linked List II · LC 25 Reverse Nodes in k-Group · LC 234 Palindrome Linked List

7. 3Sum

Sort, Fix One, Converge Two Pointers · Two Pointers · Tier 1

Given an array of integers, return every unique triplet of values that sums to zero; the same triplet must not appear twice, whichever positions it comes from. In `[-1, 0, 1, 2, -1, -4]` the answer is `[[-1, -1, 2], [-1, 0, 1]]`.

The brute force tries all `n(n - 1)(n - 2) / 6` triples of positions, `O(n^3)`, and still needs a set to strip repeated triplets. Sorting first changes the shape of the problem. Fix the first value `nums[i]`; what remains is a pair in the sorted suffix that sums to `-nums[i]`, which is Two Sum II, solved by two pointers converging from the ends of the suffix. If the sum is too small, only moving `left` rightward can raise it, because every value to the left of `right` pairs with `nums[left]` for a sum no larger. If it is too big, only moving `right` leftward can lower it. Each comparison retires one value for good, so one sweep is `O(n)` and the whole search is $O(n^2)$.

Sorting also tames the duplicates, because equal values now sit side by side. An `i` whose value equals the one before it would rediscover the same triplets, so it is skipped; after a hit, `left` steps past every copy of the value just used. The early exit comes free: once `nums[i] > 0`, all three values are positive and nothing later can reach zero. Picture `i` pinned at the left end while `left` and `right` squeeze the rest of the line from both sides.

```
index      0    1    2    3    4    5
sorted    -4   -1   -1    0    1    2
 i  nums[i]  left  right   sum   action
 0    -4       1     5     -3    too small: left + 1
 0    -4       2     5     -3    too small: left + 1
 0    -4       3     5     -2    too small: left + 1
 0    -4       4     5     -1    too small: left + 1, pointers meet
 1    -1       2     5      0    record [-1, -1, 2], move both
 1    -1       3     4      0    record [-1, 0, 1], move both, pointers cross
 2    -1                         same value as nums[1]: skip
 3     0       4     5      3    too big: right - 1, pointers meet
```

```
FUNCTION ThreeSum(nums)
    nums ← sorted(nums)                    // equal values now sit together
    triplets ← empty list
    n ← len(nums)
    FOR i FROM 0 TO n - 3
        IF nums[i] > 0
            BREAK                          // smallest of the three is positive
        IF i > 0 AND nums[i] = nums[i - 1]
            CONTINUE                       // same first value, same triplets
        left ← i + 1
        right ← n - 1
        WHILE left < right                 // Two Sum II on nums[i+1..n-1]
            total ← nums[i] + nums[left] + nums[right]
            IF total < 0
                left ← left + 1            // too small: a bigger second value
            ELSE IF total > 0
                right ← right - 1          // too big: a smaller third value
            ELSE
                triplets.append([nums[i], nums[left], nums[right]])
```

```
                left ← left + 1
                right ← right − 1
                WHILE left < right AND nums[left] = nums[left − 1]
                    left ← left + 1        // skip repeats of the second value
    RETURN triplets
```

The sort costs `O(n log n)` and is dominated by the `n` linear sweeps, so the total is $O(n^2)$ time, and the only extra memory besides the output is the sorted copy. Reach for sort, fix one, converge two whenever a problem asks for pairs, triplets or quadruplets with a target sum and wants values rather than indices: 3Sum Closest keeps the nearest total instead of exact hits, and 4Sum wraps one more fixed loop around the same sweep. The classic mistake is skipping duplicates by looking ahead, `nums[i] = nums[i + 1]`, instead of back. On the example it skips the first `−1` and loses `[−1, −1, 2]`, whose second `−1` was needed at `left`.

Practice: LC 15 3Sum · LC 167 Two Sum II - Input Array Is Sorted · LC 16 3Sum Closest · LC 18 4Sum

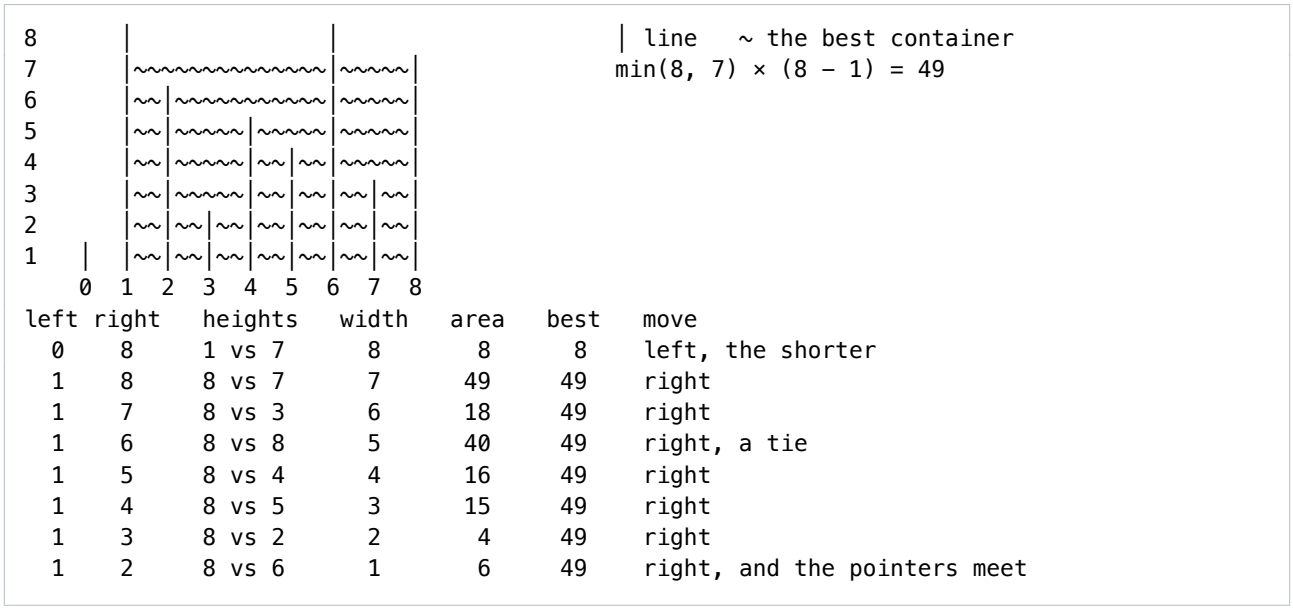
8. Container With Most Water

Always Move the Shorter Wall · Two Pointers · Tier 1

Vertical lines with the given heights stand at positions `0, 1, ..., n − 1`. Choose two of them so that, together with the x-axis, they hold the most water, and return that amount: lines `i` and `j` hold `min(height[i], height[j]) × (j − i)`, the shorter line times the distance between them. For `[1, 8, 6, 2, 5, 4, 8, 3, 7]` the answer is 49, from the lines at positions 1 and 8, `min(8, 7) × 7`.

The brute force measures all `n(n − 1) / 2` pairs, $O(n^2)$. Two pointers start from the widest container, `left = 0` and `right = n − 1`, and narrow it one wall at a time, and the whole argument is about which wall moves. Suppose `height[left]` is the shorter. Every other container still in play that keeps the line at `left` pairs it with a line strictly between the two, so it is narrower, and its water level is still capped by `height[left]`, so it is no taller. None of them can beat the area just measured, and `left` can be retired for good. Moving the taller wall instead keeps the same cap and only loses width. On a tie both walls are spent, and the code simply moves `right`.

Picture all the pairs as a triangle of candidates, one row per left line and one column per right line. Each comparison measures one pair and crosses off every other pair that uses the retired wall, a whole row or column of the triangle at once. After `n − 1` steps the pointers meet, and every pair has been measured or ruled out. In the drawing the best container spans positions 1 to 8, and the lines in between stand in the water without holding any of it back, because a line has no width.



left	right	heights	width	area	best	move
0	8	1 vs 7	8	8	8	left, the shorter
1	8	8 vs 7	7	49	49	right
1	7	8 vs 3	6	18	49	right
1	6	8 vs 8	5	40	49	right, a tie
1	5	8 vs 4	4	16	49	right
1	4	8 vs 5	3	15	49	right
1	3	8 vs 2	2	4	49	right
1	2	8 vs 6	1	6	49	right, and the pointers meet

```
FUNCTION MaxArea(height)
    left ← 0
    right ← len(height) − 1
    best ← 0
    WHILE left < right
        width ← right − left
        area ← min(height[left], height[right]) * width
        best ← max(best, area)
        IF height[left] < height[right]
            left ← left + 1               // left wall is shorter: retire it
        ELSE
```

```
            right ← right − 1              // right wall is shorter or tied
    RETURN best
```

Each step moves one pointer inward, so the scan is `O(n)` time with `O(1)` space. Reach for converging pointers whenever a pair is scored by its distance and by the smaller of its two ends, or by their sum, so that one comparison proves a whole side useless; Two Sum II on a sorted array is the same squeeze with sums. Trapping Rain Water (#9) looks similar but turns the geometry around: there every bar is a floor holding its own column of water, the answer is a sum over all columns, and the pointer behind the lower running maximum moves; here only the two chosen lines matter, and the answer is a single rectangle. The classic mistake is moving the taller wall: on the example `left` never leaves the line of height 1, and the scan returns `8` instead of `49`.

Practice: LC 11 Container With Most Water · LC 42 Trapping Rain Water · LC 167 Two Sum II - Input Array Is Sorted · LC 881 Boats to Save People

9. Trapping Rain Water

Two Pointers with Running Walls · Two Pointers · Tier 1

Given the heights of bars of width 1 standing side by side, return how many units of rain water they trap between them. `[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]` traps `6`.

Look at one column at a time. The water above bar `i` rises to the lower of the two walls around it, the tallest bar on its left and the tallest bar on its right, bar `i` itself included, so the column holds `min(max left, max right) − height[i]` units. The brute force rescans both sides for every column, $O(n^2)$. Two arrays, prefix maxima from the left and suffix maxima from the right, precompute both walls in $O(n)$ time at the price of $O(n)$ extra space.

The two-pointer version needs neither array, because a column only needs its lower wall exactly. Walk `left` and `right` inward, with `left_max` the tallest bar seen from the left end and `right_max` the tallest seen from the right end. If `left_max < right_max`, the true right wall of column `left` is at least `right_max` and so already taller; the lower wall is `left_max`, and the column is settled. Add its water and move `left`. Otherwise the mirror argument settles column `right`. The pointer behind the lower wall always moves, so the two meet on a tallest bar, which holds no water.

```
3                     #                 # bar   ~ water
2               # ~ ~ ~ # # ~ #
1           # ~ # # ~ # # # # # #
height    0 1 0 2 1 0 1 3 2 1 2 1
water     0 0 1 0 1 2 1 0 0 1 0 0   total 6
by        L L L L L L · R R R R R   (· where the pointers meet)
left right  left_max right_max   settles    water  total
  0   11        0         1      left  0      +0      0
  1   11        1         1      right 11     +0      0
  1   10        1         2      left  1      +0      0
  2   10        1         2      left  2      +1      1
  3   10        2         2      right 10     +0      1
  3    9        2         2      right 9      +1      2
  3    8        2         2      right 8      +0      2
  3    7        2         3      left  3      +0      2
  4    7        2         3      left  4      +1      3
  5    7        2         3      left  5      +2      5
  6    7        2         3      left  6      +1      6
```

```
FUNCTION Trap(height)
    left ← 0
    right ← len(height) − 1
    left_max ← 0                          // tallest bar in height[0..left]
    right_max ← 0                         // tallest bar in height[right..n−1]
    water ← 0
    WHILE left < right
        left_max ← max(left_max, height[left])
        right_max ← max(right_max, height[right])
        IF left_max < right_max           // right wall taller: settle left
            water ← water + left_max − height[left]
            left ← left + 1
        ELSE                              // left wall as tall: settle right
            water ← water + right_max − height[right]
            right ← right − 1
```

```
    RETURN water
```

Each step settles one column, so the scan is `O(n)` time with `O(1)` extra space. Reach for this whenever the answer at a position depends on the extremes on both sides of it: the lower side decides, so the lower side moves. Container With Most Water (#8) is the cousin, where the area between two lines is capped by the shorter line and the pointer at the shorter line moves. Trapping Rain Water II lifts the problem onto a grid, where a min-heap of boundary cells plays the lower wall. The classic mistake is adding `left_max − height[left]` before raising `left_max` to include the current bar, which goes negative whenever that bar is a new tallest.

Practice: LC 42 Trapping Rain Water · LC 11 Container With Most Water · LC 407 Trapping Rain Water II

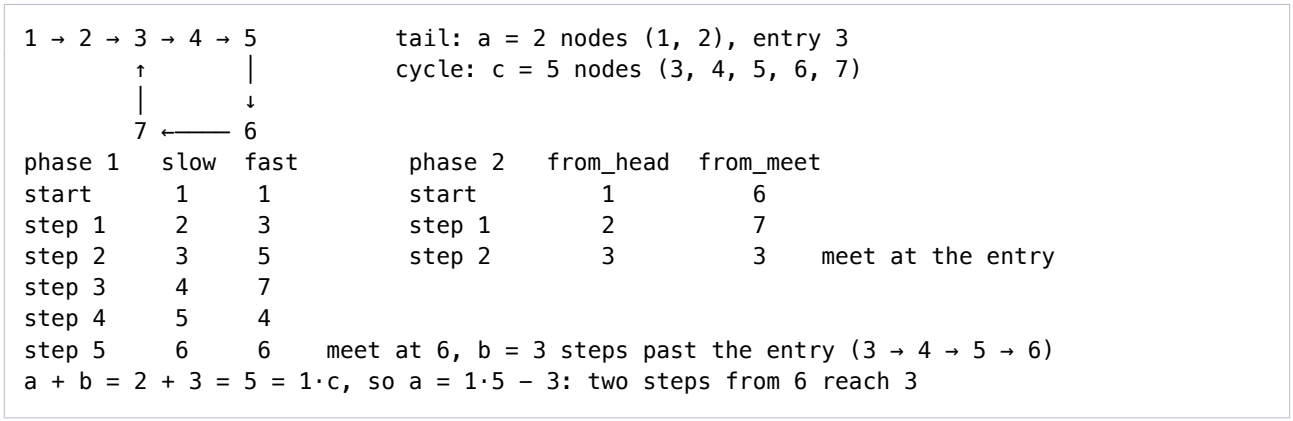
10. Floyd's Tortoise and Hare

Detect a Cycle and Find Its Entry · Cycle Detection · Tier 1

Given the head of a linked list whose last node may link back to an earlier node, decide whether there is a cycle and return the node where it begins, using `O(1)` extra space. In the list `1 → 2 → … → 7` whose tail links back to node `3`, the answer is node `3`.

The brute force keeps a hash set of visited nodes and returns the first node seen twice: `O(n)` time, but `O(n)` space. Floyd's idea replaces the memory with a difference in speed. A slow pointer moves one step at a time and a fast pointer two. Without a cycle, fast runs off the end. With one, both pointers end up circling the loop, and because fast gains exactly one node per step, the gap between them shrinks by one each step and closes on a meeting instead of jumping over it.

Finding the entry takes one more observation. Say the tail holds `a` nodes before the entry, the cycle holds `c`, and the pointers meet `b` steps past the entry. Fast has walked twice as far as slow and stands on the same node, so the extra distance, which equals slow's whole walk, is a whole number of laps. Slow's walk is `a` steps of tail plus `b` steps into the cycle, plus any full laps, so `a + b = k·c` for some whole `k`, that is `a = k·c − b`. Walking `a` steps from the meeting point therefore lands exactly on the entry, and so does walking `a` steps from the head. Restart one pointer at the head, step both by one, and they meet at the entry.




```
FUNCTION MeetingPoint(head)
    slow ← head
    fast ← head
    WHILE fast ≠ null AND fast.next ≠ null
        slow ← slow.next
        fast ← fast.next.next
        IF slow = fast                      // the same node, not an equal value
            RETURN slow
    RETURN null                             // fast fell off the end: no cycle

FUNCTION CycleEntry(head)
    meet ← MeetingPoint(head)
    IF meet = null
        RETURN null
    from_head ← head
    from_meet ← meet
    WHILE from_head ≠ from_meet             // both are a steps from the entry
        from_head ← from_head.next
```

```
        from_meet ← from_meet.next
    RETURN from_head
```

Slow takes at most `n` steps before the meeting and the second phase takes `a` more, so the search is `O(n)` time with `O(1)` space. The same two pointers work on any sequence where each value determines the next, an implicit linked list. In Find the Duplicate Number, index `i` points to `nums[i]`; the duplicate is the only node with more than one arrow coming in, so it is the entry of the cycle. In Happy Number, each number points to the sum of the squares of its digits, and the number is happy exactly when the cycle its walk falls into is `1 → 1`. The classic mistake is testing `slow = fast` before the first move, where both sit on the head, or comparing values instead of nodes, which finds a cycle in `5 → 5 → 5`.

Practice: LC 141 Linked List Cycle · LC 142 Linked List Cycle II · LC 287 Find the Duplicate Number · LC 202 Happy Number

11. Longest Substring Without Repeating Characters

Variable Window with Last-Seen Index · Sliding Window · Tier 1

Given a string, return the length of its longest substring in which no character appears twice. In `"abcbad"` the answer is 4, the window `"cbad"`.

The brute force starts a scan at every index and extends it until a character repeats, $O(n^2)$ work. The insight is that neighboring answers overlap: when the window `s[left..right]` meets a repeat, the next valid window only has to drop the stretch up to the earlier copy, never start over. So keep one window and slide both of its edges forward. `right` advances one character per step; `left` moves only when the new character already sits inside the window, and then it jumps straight past the earlier copy, because no window that still holds both copies can be valid. A map from each character to the index of its latest occurrence makes that jump a single lookup.

Picture the window as an elastic band on the string: the right edge moves steadily, the left edge waits and then leaps. The one subtlety is a stale entry. The map still remembers characters that have already left the window, so the jump happens only when `last_seen[ch] ≥ left`; otherwise `left` would move backward and readmit a repeat. Below, the `a` at index 4 was last seen at index 0, already outside the window, so `left` stays at 2.

```
          a b c b a d     last_seen[ch]           left  best
right 0   a               –                        0    1
right 1   a b             –                        0    2
right 2   a b c           –                        0    3
right 3       c b         1, inside: left = 2      2    3
right 4       c b a       0, left of window        2    3
right 5       c b a d     –                        2    4
```

```
FUNCTION LengthOfLongestSubstring(s)
    last_seen ← empty map                  // char → index of latest occurrence
    left ← 0
    best ← 0
    FOR right FROM 0 TO len(s) − 1
        ch ← s[right]
        IF ch IN last_seen AND last_seen[ch] ≥ left
            left ← last_seen[ch] + 1       // jump past the copy in the window
        last_seen[ch] ← right
        best ← max(best, right − left + 1)
    RETURN best
```

Each index enters the window once and `left` only moves forward, so the scan is $O(n)$ time, and the map holds one entry per distinct character. Reach for a variable window whenever a problem asks for the longest contiguous stretch whose condition survives shrinking: no repeats, at most `k` distinct characters, at most `k` replacements. When the condition depends on counts rather than positions, keep a count map and advance `left` one step at a time instead of jumping, as Minimum Window Substring (#13) does. The classic mistake is dropping the `last_seen[ch] ≥ left` test: on `"abba"` the final `a` drags `left` back to index 1 and the answer comes out 3 instead of 2.

Practice: LC 3 Longest Substring Without Repeating Characters · LC 159 Longest Substring with At Most Two Distinct Characters · LC 340 Longest Substring with At Most K Distinct Characters · LC 424 Longest Repeating Character Replacement

12. Find All Anagrams in a String

Fixed-Size Window with Letter Counts · Sliding Window · Tier 1

Given lowercase strings `s` and `p`, return the start index of every substring of `s` that is an anagram of `p`: the same letters, each the same number of times, in any order. For `s = "cbaebabacd"` and `p = "abc"` the answer is `[0, 6]`, the windows `"cba"` and `"bac"`.

An anagram of `p` has exactly `len(p)` letters, so only windows of that one width can qualify. The brute force rebuilds the letter counts of each such window from scratch, `O(n · m)` for `n = len(s)` and `m = len(p)`, and more if it sorts each one instead. But neighboring windows differ only at their ends. Slide the window one step and at most two counts change: the letter entering on the right goes up by one, and the letter leaving on the left goes down by one. The 26 counts are the canonical key of Group Anagrams (#5), kept current instead of recomputed, and a window is an anagram exactly when its counts equal those of `p`.

Picture a frame of fixed width riding along the string with a tally of 26 slots attached. Both edges move in lockstep, which is what sets this window apart from the variable windows of #11 and #13, where the right edge runs ahead and the left edge follows its own rule: it jumps past a repeat in #11, and in #13 it shrinks the window for as long as the window still covers every needed letter. Each step compares two arrays of 26 counts, a constant amount of work. A counter of how many of the 26 slots currently agree removes even that: a slide touches two slots, so the counter moves by at most two per step, and the window is an anagram when all 26 agree.

```
p = "abc", need = a1 b1 c1
start  c b a e b a b a c d    in   out   window counts   anagram?
0      └─┘                   –    –     a1 b1 c1       ✓
1        └─┘                 e    c     a1 b1 e1       ✗
2          └─┘               b    b     a1 b1 e1       ✗
3            └─┘             a    a     a1 b1 e1       ✗
4              └─┘           b    e     a1 b2          ✗
5                └─┘         a    b     a2 b1          ✗
6                  └─┘       c    a     a1 b1 c1       ✓
7                    └─┘     d    b     a1 c1 d1       ✗
answer [0, 6]
```

```
FUNCTION LetterCounts(text)
    counts ← array of 26 zeros
    FOR EACH ch IN text
        slot ← ord(ch) − ord('a')          // 'a' → 0, 'b' → 1, ..., 'z' → 25
        counts[slot] ← counts[slot] + 1
    RETURN counts

FUNCTION FindAnagrams(s, p)
    m ← len(p)
    starts ← empty list
    IF m > len(s)
        RETURN starts                      // no window fits
    need ← LetterCounts(p)
    window ← LetterCounts(s[0..m−1])       // the first window
    IF window = need
        starts.append(0)                   // the first window counts too
    FOR right FROM m TO len(s) − 1
```

```
        entering ← ord(s[right]) − ord('a')
        leaving ← ord(s[right − m]) − ord('a')
        window[entering] ← window[entering] + 1      // s[right] slides in
        window[leaving] ← window[leaving] − 1        // s[right − m] slides out
        IF window = need                    // 26 slot comparisons
            starts.append(right − m + 1)
    RETURN starts
```

Building the first window costs `O(m)`, and each of the `n − m` slides costs two updates and a 26-slot comparison, so the scan is `O(26 · n)` time, linear for a fixed alphabet, with two arrays of 26 counts for space. Reach for a fixed-size window whenever a problem asks about every substring or subarray of one given length: anagrams and permutations of a pattern, or the best sum or average of `k` consecutive values. Permutation in String is the yes-or-no version of this problem and returns `true` at the first match. The classic mistake is comparing only inside the sliding loop and forgetting the very first window: on the example that loses index 0 and returns `[6]`.

Practice: LC 438 Find All Anagrams in a String · LC 567 Permutation in String · LC 643 Maximum Average Subarray I · LC 30 Substring with Concatenation of All Words

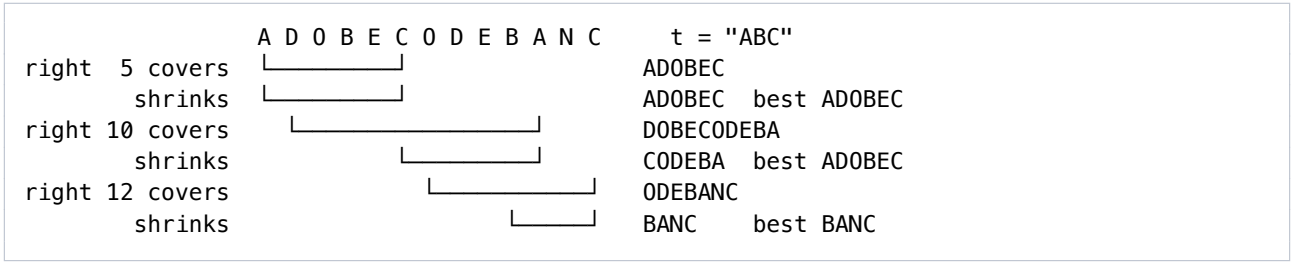
13. Minimum Window Substring

Expand Right, Shrink Left · Sliding Window · Tier 1

Given strings `s` and `t`, return the shortest substring of `s` that contains every character of `t`, repeats included, or `""` if there is none. For `s = "ADOBECODEBANC"` and `t = "ABC"` the answer is `"BANC"`.

The brute force tests every substring against the counts of `t`, $O(n^2)$ windows before any checking. The insight is that the windows worth testing form a chain. For each right end only the shortest covering window that ends there matters, and as the right end moves forward, the left end of that window never moves back. So one window slides with two edges: `right` expands until the window covers `t`, then `left` shrinks it for as long as it still covers `t`, recording each covering window on the way. The first character whose loss breaks the cover hands control back to `right`. Unlike the jumping left edge of #11, `left` here advances one step at a time, because coverage depends on counts rather than positions.

Re-checking the cover at every step would cost a loop over the characters of `t`, so the window keeps two pieces of bookkeeping. `need[ch]` is how many more copies of `ch` the window lacks, and it goes negative when the window holds spares. `missing` counts the characters of `t` still uncovered, repeats included, and drops only when an arriving character fills a real gap, that is, when its `need` was positive. The window covers `t` exactly when `missing = 0`, a constant-time test.




```
FUNCTION CharCounts(text)
    counts ← empty map
    FOR EACH ch IN text
        counts[ch] ← counts.get(ch, 0) + 1
    RETURN counts

FUNCTION MinWindow(s, t)
    IF len(t) = 0
        RETURN ""                          // nothing to cover
    need ← CharCounts(t)                   // char → copies the window lacks
    missing ← len(t)                       // chars of t not yet covered
    left ← 0
    best_start ← 0
    best_len ← len(s) + 1                  // longer than any real window
    FOR right FROM 0 TO len(s) − 1
        ch ← s[right]
        IF need.get(ch, 0) > 0
            missing ← missing − 1          // this copy fills a real gap
        need[ch] ← need.get(ch, 0) − 1     // below zero: a spare copy
        WHILE missing = 0                  // covers t: record, then shrink
            IF right − left + 1 < best_len
                best_start ← left
                best_len ← right − left + 1
            drop ← s[left]
```

```
            need[drop] ← need[drop] + 1
            IF need[drop] > 0
                missing ← missing + 1      // dropped a copy the window needed
            left ← left + 1
    IF best_len > len(s)
        RETURN ""                          // no window ever covered t
    RETURN s[best_start..best_start + best_len − 1]
```

Each index enters the window once and leaves it at most once, so the work is `O(len(s) + len(t))` time, and the map holds one count per distinct character. Reach for expand-right, shrink-left whenever a problem asks for the shortest window that contains something; with the width pinned to `len(t)`, the same counts find every permutation or anagram of a pattern inside a string. The classic mistake is decrementing `missing` for every arriving character of `t`, spares included: with `s = "AAB"` and `t = "AB"`, the second `A` claims the slot meant for `B`, `missing` reaches 0 on `"AA"`, and the answer comes back without a `B`.

Practice: LC 76 Minimum Window Substring · LC 567 Permutation in String · LC 438 Find All Anagrams in a String · LC 30 Substring with Concatenation of All Words

14. Binary Search

The Lower-Bound Template · Binary Search · Tier 1

Given an array sorted in non-decreasing order and a target, return the first index whose value is at least the target, or `len(nums)` if every value is smaller. In `[1, 3, 3, 5, 8, 9]` the answer for target `3` is `1`, the first of the two 3s; for target `4` it is `3`, the slot where 4 would be inserted; for target `10` it is `6`, one past the end.

A linear scan finds the answer in `O(n)` and ignores the order. Sorted order turns the question "is `nums[i] ≥ target`?" into a row of answers that reads ✗ ✗ ✗ then ✓ ✓ ✓, because once a value is big enough, every value to its right is too. The lower bound is the boundary between the two runs, and binary search finds a boundary by halving the unknown part. Keep a half-open window `[lo, hi)` with one invariant: every index left of `lo` is a ✗ and every index from `hi` on is a ✓, with the position `len(nums)` counting as a ✓. Probe the middle. A ✗ at `mid` condemns `mid` and everything left of it, so `lo ← mid + 1`. A ✓ at `mid` might be the first ✓, so it becomes the new right edge: `hi ← mid`. When `lo = hi` the window is empty and `lo` sits exactly on the boundary.

Every other search falls out of this template. Exact search checks whether the lower bound actually holds the target. The insert position is the lower bound itself. The upper bound, the first index whose value is greater than the target, is the same loop with < replaced by ≤, so that equal values count as too small. The last occurrence sits just before the upper bound, so first and last position take two searches, and upper bound minus lower bound is the number of copies. First Bad Version is the template with the comparison replaced by a call to `isBadVersion(mid)`.

```
index            0    1    2    3    4    5    6
nums             1    3    3    5    8    9
nums[i] ≥ 3?     ✗    ✓    ✓    ✓    ✓    ✓    (end counts as ✓)
                      ▲ lower bound = 1

   lo   hi   mid   nums[mid]   < 3?   move
    0    6     3       5        no    hi ← 3     mid is a ✓: keep it
    0    3     1       3        no    hi ← 1     mid is a ✓: keep it
    0    1     0       1        yes   lo ← 1     mid is a ✗: drop it
    1    1   window empty: return lo = 1
```

```
FUNCTION LowerBound(nums, target)        // first i with nums[i] ≥ target
    lo ← 0
    hi ← len(nums)                       // half-open [lo, hi)
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)
        IF nums[mid] < target
            lo ← mid + 1                 // mid and its left are too small
        ELSE
            hi ← mid                     // mid qualifies: keep it in play
    RETURN lo

FUNCTION UpperBound(nums, target)        // first i with nums[i] > target
    lo ← 0
    hi ← len(nums)
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)
        IF nums[mid] ≤ target            // equal values count as too small
```

```
            lo ← mid + 1
        ELSE
            hi ← mid
    RETURN lo

FUNCTION Search(nums, target)
    i ← LowerBound(nums, target)
    IF i < len(nums) AND nums[i] = target
        RETURN i
    RETURN -1

FUNCTION SearchRange(nums, target)
    first ← LowerBound(nums, target)
    IF first = len(nums) OR nums[first] ≠ target
        RETURN [-1, -1]
    RETURN [first, UpperBound(nums, target) - 1]
```

Each probe halves the window, so a search costs `O(log n)` time and `O(1)` space. Reach for it when the input is sorted and the question is "first", "last", "how many" or "where would it go", and more generally whenever a yes/no question about an index flips exactly once from no to yes, which is binary search on the answer (#15). Python's `bisect_left` and `bisect_right` compute exactly `lower_bound` and `upper_bound`; mention them, but write the loop by hand in an interview. The classic mistake is mixing conventions: `hi ← mid − 1` in this half-open template can jump past the answer, and `WHILE lo ≤ hi` with `hi ← mid` either spins forever once `lo = hi` or reads `nums[len(nums)]`.

Practice: LC 704 Binary Search · LC 35 Search Insert Position · LC 34 Find First and Last Position of Element in Sorted Array · LC 278 First Bad Version

15. Binary Search on the Answer

Search a Monotone Yes/No Predicate · Binary Search · Tier 1

Koko has piles of bananas and `h` hours. She picks one eating speed `k`; each hour she eats `k` bananas from a single pile, and when fewer than `k` are left in that pile she still spends the whole hour on it. Return the smallest `k` that finishes every pile within `h` hours; `h` is at least the number of piles, so some speed always works. For piles `[3, 6, 7, 11]` and `h = 8` the answer is `4`: at speed 4 the piles take `1 + 2 + 2 + 3 = 8` hours, while at speed 3 they take `1 + 2 + 3 + 4 = 10`.

Nothing here is sorted, yet binary search applies, because the search runs over the answer instead of over the array. Trying every speed from 1 upward costs `O(n · m)` for the largest pile `m`. The insight is that feasibility is monotone: if Koko finishes at speed `k`, she finishes at every faster speed, so the question "can she finish at speed `k`?" reads ✗ ✗ ✗ then ✓ ✓ ✓ along the number line of speeds. The answer is the first ✓, exactly the boundary that the lower-bound template of #14 finds. Only the probe changes: instead of comparing `nums[mid]` with a target, it simulates a whole meal at speed `mid` in one pass, summing `ceil(pile / mid)` hours.

Every problem of this kind has the same three parts. The range `[lo, hi]` holds every candidate answer, with `lo` the smallest conceivable value and `hi` a value that surely works; here they are `1` and `max(piles)`, since at that speed every pile takes one hour. The check `feasible(x)` is a simple simulation, usually `O(n)`, whose answer flips once from no to yes. The loop keeps a ✓ at `mid` with `hi ← mid`, drops a ✗ with `lo ← mid + 1`, and stops when `lo = hi`. "Largest value that still works" is the mirror image: search for the first value that fails and step back one.

```
speed          1   2   3   4   5   6   7   8   9  10  11
hours         27  15  10   8   8   6   5   5   5   5   4
hours ≤ 8?     ✗   ✗   ✗   ✓   ✓   ✓   ✓   ✓   ✓   ✓   ✓
                           ▲ the answer: the first ✓

   lo   hi   mid   hours at mid   ≤ 8?   move
    1   11     6           6      yes   hi ← 6
    1    6     3          10      no    lo ← 4
    4    6     5           8      yes   hi ← 5
    4    5     4           8      yes   hi ← 4
    4    4   lo = hi: the smallest speed is 4
```

```
FUNCTION CanFinish(piles, h, speed)      // the yes/no check
    hours ← 0
    FOR EACH pile IN piles
        hours ← hours + ceil(pile / speed)   // a partial hour costs a full one
    RETURN hours ≤ h

FUNCTION MinEatingSpeed(piles, h)
    lo ← 1                               // the slowest possible speed
    hi ← max(piles)                      // one pile per hour: always enough
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)
        IF CanFinish(piles, h, mid)
            hi ← mid                     // mid works; a slower speed might too
        ELSE
            lo ← mid + 1                 // mid fails, so does every slower one
    RETURN lo
```

The check runs once per halving of the speed range, so the cost is `O(n log m)` time and `O(1)` space. The signals are phrases such as “minimum capacity”, “smallest speed”, “minimize the largest” or “maximize the minimum”, together with a check that is easy for any fixed answer and flips only once as the answer grows. The classic mistakes sit in the check and in the range: `pile // mid` rounds down and forgets the partial last hour, which makes slow speeds look feasible, and starting `lo` at `0` can divide by zero.

Practice: LC 875 Koko Eating Bananas · LC 1011 Capacity To Ship Packages Within D Days · LC 410 Split Array Largest Sum · LC 1482 Minimum Number of Days to Make m Bouquets

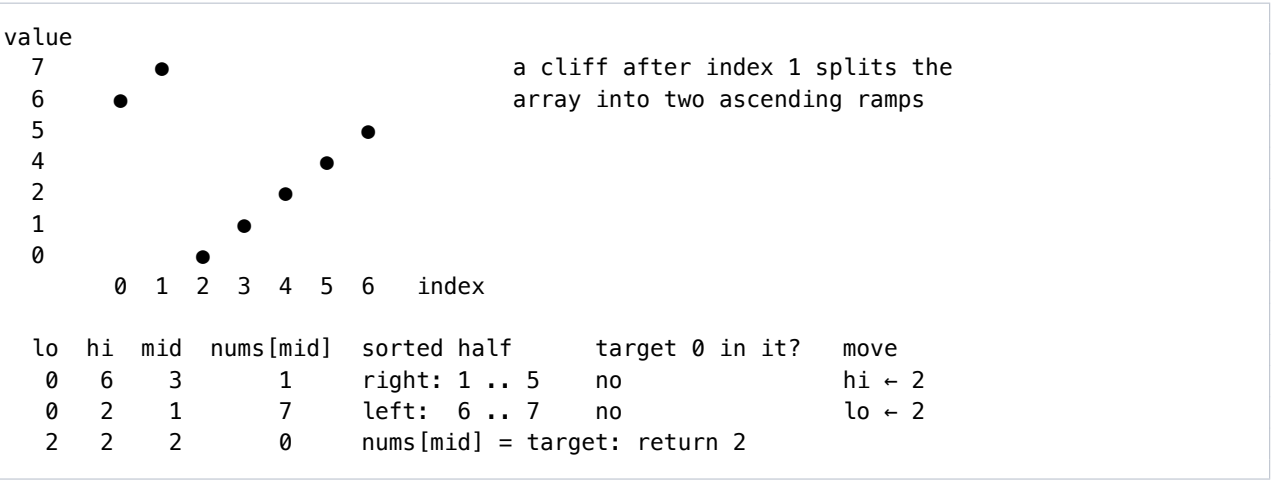
16. Search in a Rotated Sorted Array

One Half Is Always Sorted · Binary Search · Tier 1

A sorted array of distinct values was rotated at an unknown pivot, so `[0, 1, 2, 4, 5, 6, 7]` may now read `[6, 7, 0, 1, 2, 4, 5]`. Return the index of a target, or `−1` if it is absent, in `O(log n)` time. In `[6, 7, 0, 1, 2, 4, 5]` the target `0` sits at index `2`.

A linear scan costs `O(n)`. Finding the rotation point first and then binary searching the correct side reaches `O(log n)` in two passes, but one pass is enough. Picture the array as two ascending ramps with a cliff between them. Cut the window at any `mid` and the cliff falls into at most one of the two halves, so the other half is a single ramp, fully sorted. One comparison tells which: if `nums[lo] ≤ nums[mid]`, the left half climbs without a break, because a cliff inside it would leave `nums[mid]` below `nums[lo]`; otherwise the cliff is on the left and the right half is sorted. A sorted half answers "could the target be in here?" with two comparisons against its ends. If the target lies inside its range, search there; if not, it can only be in the other half. Either way half the window disappears, exactly as in plain binary search. The window is the closed `[lo, hi]` of an exact search, since the loop reads `nums[hi]` as a real element.

Two relatives use the same picture. To find the minimum (LC 153), compare `nums[mid]` with `nums[hi]`: if `nums[mid] > nums[hi]`, the cliff and the minimum just after it lie to the right of `mid`, so `lo ← mid + 1`; otherwise `hi ← mid`. With duplicates (LC 81), `nums[lo] = nums[mid] = nums[hi]` hides which half is sorted: `[1, 1, 1, 0, 1]` and `[1, 0, 1, 1, 1]` look identical at those three probes. The safe move is to drop one copy from each end, which makes the worst case `O(n)`.



lo	hi	mid	nums[mid]	sorted half	target 0 in it?	move
0	6	3	1	right: 1 .. 5	no	hi ← 2
0	2	1	7	left: 6 .. 7	no	lo ← 2
2	2	2	0	nums[mid] = target: return 2		

```
FUNCTION SearchRotated(nums, target)
    lo ← 0
    hi ← len(nums) − 1                    // closed window [lo, hi]
    WHILE lo ≤ hi
        mid ← floor((lo + hi) / 2)
        IF nums[mid] = target
            RETURN mid
        IF nums[lo] ≤ nums[mid]           // left half is sorted
            IF nums[lo] ≤ target AND target < nums[mid]
                hi ← mid − 1              // target inside the left half
            ELSE
                lo ← mid + 1              // so it must be on the right
        ELSE                              // right half is sorted
```

```
        IF nums[mid] < target AND target ≤ nums[hi]
            lo ← mid + 1                // target inside the right half
        ELSE
            hi ← mid − 1                // so it must be on the left
    RETURN −1
```

Each step discards half of the window, so the search takes `O(log n)` time and `O(1)` space. The signals are words like “rotated” or “shifted” next to “sorted”, together with a demand for `O(log n)`. The classic mistake is writing `nums[lo] < nums[mid]` instead of ≤: when the window shrinks to two elements, `lo = mid` and the left half is the single element `nums[mid]`, which is sorted, but the strict test calls the right half sorted instead; on `[3, 1]` with target `1` it discards the `1` and returns `−1`.

Practice: LC 33 Search in Rotated Sorted Array · LC 153 Find Minimum in Rotated Sorted Array · LC 81 Search in Rotated Sorted Array II · LC 154 Find Minimum in Rotated Sorted Array II

17. Merge Sort

Divide, Sort Halves, Merge · Sorting · Tier 1

Sort an array of integers into ascending order and return the result as a new list. `[5, 2, 4, 7, 1, 3, 2, 6]` becomes `[1, 2, 2, 3, 4, 5, 6, 7]`.

Insertion sort, the brute force, slides each value left past every larger one and costs $O(n^2)$ on reversed input. Merge sort starts from a cheaper fact: two lists that are already sorted combine into one sorted list in a single pass, because the smallest value not yet taken always sits at the front of one of them. So cut the array in half, sort each half by the same method, and merge the two sorted halves. The recursion bottoms out at lists of zero or one value, which are sorted by definition, and all the real work happens on the way back up, in the merges.

The merge is two fingers, one on the front of each half: compare the two fronts, take the smaller, advance that finger, and when one half runs out, copy the rest of the other. Taking the left front on a tie, ≤ rather than <, keeps equal values in their original order, which is what makes merge sort stable. Picture the recursion as a tree of cuts. Every row of the tree holds all n values, cut into more and more pieces, and after about `log2 n` rounds of cutting the pieces are single values. On the way back up each row is merged with `O(n)` work in total, because every value is copied once per row.

```
split  [5 2 4 7 1 3 2 6]                          one list of 8
       [5 2 4 7]           [1 3 2 6]               2 lists of 4
       [5 2]     [4 7]     [1 3]     [2 6]         4 lists of 2
       [5] [2]   [4] [7]   [1] [3]   [2] [6]       8 lists of 1: sorted
merge  [2 5]     [4 7]     [1 3]     [2 6]         8 values copied
       [2 4 5 7]           [1 2 3 6]               8 values copied
       [1 2 2 3 4 5 6 7]                          8 values copied
```

```
FUNCTION Merge(left, right)
    merged ← empty list
    i ← 0                                  // front of left
    j ← 0                                  // front of right
    WHILE i < len(left) AND j < len(right)
        IF left[i] ≤ right[j]              // a tie takes the left one: stable
            merged.append(left[i])
            i ← i + 1
        ELSE
            merged.append(right[j])
            j ← j + 1
    WHILE i < len(left)                    // one side is used up: copy the rest
        merged.append(left[i])
        i ← i + 1
    WHILE j < len(right)
        merged.append(right[j])
        j ← j + 1
    RETURN merged

FUNCTION MergeSort(nums)
    IF len(nums) ≤ 1
        RETURN copy of nums                // 0 or 1 values: sorted already
    mid ← floor(len(nums) / 2)
    left ← MergeSort(nums[0..mid − 1])
```

```
    right ← MergeSort(nums[mid..len(nums) − 1])
    RETURN Merge(left, right)
```

There are `log2 n` levels of merging, rounded up, each costing `O(n)`, so the time is `O(n log n)` in every case, plus `O(n)` extra space for the merged lists. Reach for merge sort when the sort must be stable, when the data is a linked list (LC 148 splits at the middle with slow and fast pointers and needs no random access), and when a problem counts pairs `i < j` with `nums[i] > nums[j]`. That inversion count is the classic extension: when the merge takes `right[j]` while `len(left) − i` left values still wait, all of them are larger and came earlier in the array, so adding `len(left) − i` at that step counts every inversion; LC 315 and LC 493 are this count with a twist. The classic mistake is stopping when the main loop ends: `Merge([1, 4], [2, 3])` runs out of right values at `[1, 2, 3]` and loses the `4`.

Practice: LC 912 Sort an Array · LC 148 Sort List · LC 315 Count of Smaller Numbers After Self · LC 493 Reverse Pairs

18. Quick Sort

Partition Around a Random Pivot · Sorting · Tier 1

Sort an array of integers in place, rearranging it without a second array. `[6, 3, 7, 4, 1, 5, 2]` becomes `[1, 2, 3, 4, 5, 6, 7]`.

Merge sort (#17) does its work after the recursive calls, merging sorted halves into a new list. Quick sort does its work before them, in place. Partition the range around one value, the pivot: every smaller value moves to its left and every other value to its right. The pivot now sits in its final position, and the two sides are independent problems that never need to be combined. Sort each side the same way and the whole range is sorted.

The Lomuto partition is a single pass. Pick a random index, swap that value to the end and call it `pivot`. Then `i` scans the range from left to right while `store` marks where the next small value goes, under the invariant that `nums[lo..store-1]` holds only values less than the pivot. When `nums[i]` is smaller, swap it into `nums[store]` and advance `store`. At the end one swap moves the pivot to `store`, right between the two regions. Picture a wall that slides to the right: everything left of the wall is smaller than the pivot, everything between the wall and `i` is not.

```
random pick: index 3, pivot 4, parked at the end: 6 3 7 2 1 5 4
  i   nums[i]   < 4?   nums after the step (| = store)   store
  0      6       no    |6 3 7 2 1 5 4                      0
  1      3       yes    3|6 7 2 1 5 4                     1
  2      7       no     3|6 7 2 1 5 4                     1
  3      2       yes    3 2|7 6 1 5 4                     2
  4      1       yes    3 2 1|6 7 5 4                     3
  5      5       no     3 2 1|6 7 5 4                     3
pivot swapped to store: 3 2 1 4 7 5 6, so p = 3
then sort 3 2 1 and 7 5 6; the 4 never moves again
```

```
FUNCTION Partition(nums, lo, hi)
    pick ← RandomInt(lo, hi)            // lo ≤ pick ≤ hi
    swap(nums[pick], nums[hi])          // park the pivot at the end
    pivot ← nums[hi]
    store ← lo                          // invariant: nums[lo..store-1] < pivot
    FOR i FROM lo TO hi - 1
        IF nums[i] < pivot              // a small value joins the left region
            swap(nums[store], nums[i])
            store ← store + 1
    swap(nums[store], nums[hi])         // pivot between the regions
    RETURN store

FUNCTION SortRange(nums, lo, hi)
    IF lo ≥ hi                          // 0 or 1 values: sorted already
        RETURN
    p ← Partition(nums, lo, hi)
    SortRange(nums, lo, p - 1)          // the pivot at p is final: skip it
    SortRange(nums, p + 1, hi)

FUNCTION QuickSort(nums)
    SortRange(nums, 0, len(nums) - 1)
    RETURN nums
```

A balanced split gives `log2 n` levels of `O(n)` partitioning, so `O(n log n)` time. A pivot that is always the smallest or largest value peels off one element per level and costs `O(n²)`; with a fixed last-element pivot, an already sorted array does exactly that. A random pivot makes no input order bad: on distinct values the expected time is `O(n log n)` and the expected stack depth `O(log n)`. Quick sort is not stable, because a long swap can carry a value past an equal one. Hoare's original partition moves two indices inward from both ends and swaps each misplaced pair; it makes fewer swaps and splits a run of equal values down the middle. Reach for quick sort when the sort must be in place and fast on average, and for its partition alone in kth largest (#60). The classic mistake is ignoring duplicates, and this two-way partition makes it: with the `<` test, an array of equal values leaves the left side empty at every level, so even a random pivot goes quadratic and the recursion runs n levels deep; the three-way partition of #52 fixes it.

Practice: LC 912 Sort an Array · LC 215 Kth Largest Element in an Array · LC 75 Sort Colors

19. Top-K Elements

Keep a Size-K Min-Heap · Heaps · Tier 1

Given an unsorted array and an integer `k`, return the kth largest value, where duplicates count as separate values; the same method returns the k largest values themselves. In `[3, 1, 5, 12, 2, 11]` with `k = 3` the three largest are 12, 11 and 5, so the answer is 5.

Sorting answers it in `O(n log n)` but orders all n values when only k of them matter. Keep instead a min-heap of the k largest values seen so far. Push each value; when the heap grows to `k + 1` values, pop its smallest. The popped value can never be among the k largest, because the k values still in the heap are all at least as large. After the last value the heap holds exactly the k largest, and its top, the smallest of them, is the kth largest.

A min-heap for a question about the largest feels backwards, and that is the point. The top of the heap is the weakest member of the current top k, the gatekeeper at the door: a newcomer that cannot beat it is popped right back out, and one that can beat it pushes the gatekeeper out instead. The heap never holds more than `k + 1` values, so each push and pop costs `O(log k)`, not `O(log n)`.

```
k = 3; heap contents listed smallest first, so the top comes first
  value   after the push   size > 3?   popped   heap kept
    3     3                no                    3
    1     1 3              no                    1 3
    5     1 3 5            no                    1 3 5
   12     1 3 5 12         yes           1       3 5 12
    2     2 3 5 12         yes           2       3 5 12
   11     3 5 11 12        yes           3       5 11 12
heap top 5 is the 3rd largest; the heap holds the top three
```

```
FUNCTION TopK(nums, k)
    heap ← empty min-heap
    FOR EACH value IN nums
        heap.push(value)
        IF len(heap) > k                  // one too many: drop the smallest
            heap.pop()
    RETURN heap

FUNCTION KthLargest(nums, k)
    heap ← TopK(nums, k)
    RETURN heap.top()                     // the smallest of the k largest
```

Each of the n values costs one push and at most one pop on a heap of at most `k + 1` values, so the time is `O(n log k)` and the space `O(k)`. That beats sorting when k is small, and it works on an endless stream: LC 703 keeps this heap alive and answers after every new value. Signals: k largest, k most frequent, k closest. K Closest Points to Origin is the same loop with a distance key in the opposite direction: keeping the k smallest distances needs a max-heap, which `heapq` gives when you push `(-(x² + y²), x, y)`, so the popped point is the farthest so far. Top K Frequent Elements counts first and runs the loop on `(count, value)` pairs, or uses the buckets of #61. Quickselect (#60) is expected `O(n)` but reorders the array and needs all of it at once. The classic mistake is reaching for a max-heap because the problem says largest: a size-k max-heap evicts the biggest values, exactly the ones to keep.

Practice: LC 215 Kth Largest Element in an Array · LC 703 Kth Largest Element in a Stream · LC 347 Top K Frequent Elements · LC 973 K Closest Points to Origin

20. Merge K Sorted Arrays

A Heap of the Front Elements · Heaps · Tier 1

Given k arrays, each sorted in ascending order, merge them into one sorted array. `[[1, 4, 7], [2, 5], [0, 3, 6]]` becomes `[0, 1, 2, 3, 4, 5, 6, 7]`.

Concatenating and sorting costs `O(N log N)` for N values in total and throws away the order the arrays already have. Merging the arrays one at a time into a growing result recopies that result k times, `O(Nk)`. The insight is that the next value of the output is always the smallest of the k fronts, one front per array, and a min-heap is the structure that hands back the smallest of a changing set. Put each array's first value in the heap; then pop the smallest front, append it to the output, and push the next value of the same array in its place.

Picture the arrays as k rows with a finger on the front of each. The heap holds what the fingers point at, never more than k values. Each pop moves exactly one finger one step to the right, and a row whose finger falls off the end simply stops taking part. A heap entry is the tuple `(value, row, col)`: the value orders the heap, and `row` and `col` say where the successor lives, at `arrays[row][col + 1]`.

```
rows: 0 → [1 4 7]   1 → [2 5]   2 → [0 3 6]; the heap starts with 0 1 2
  pop (value, row, col)   then push      heap after   merged
  (0, 2, 0)               (3, 2, 1)      1 2 3        0
  (1, 0, 0)               (4, 0, 1)      2 3 4        0 1
  (2, 1, 0)               (5, 1, 1)      3 4 5        0 1 2
  (3, 2, 1)               (6, 2, 2)      4 5 6        0 1 2 3
  (4, 0, 1)               (7, 0, 2)      5 6 7        0 1 2 3 4
  (5, 1, 1)               row 1 done     6 7          0 1 2 3 4 5
  (6, 2, 2)               row 2 done     7            0 1 2 3 4 5 6
  (7, 0, 2)               row 0 done     empty        0 1 2 3 4 5 6 7
```

```
FUNCTION MergeKSorted(arrays)
    heap ← empty min-heap                  // of (value, row, col)
    FOR row FROM 0 TO len(arrays) − 1
        IF len(arrays[row]) > 0            // an empty array has no front
            heap.push((arrays[row][0], row, 0))
    merged ← empty list
    WHILE len(heap) > 0
        (value, row, col) ← heap.pop()     // the smallest front of all
        merged.append(value)
        IF col + 1 < len(arrays[row])      // that row has a next value
            heap.push((arrays[row][col + 1], row, col + 1))
    RETURN merged
```

Every value enters and leaves the heap once, and the heap never holds more than k entries, so the time is `O(N log k)` and the extra space `O(k)` beyond the output. Merging the arrays in pairs, level by level as in merge sort (#17), reaches the same `O(N log k)` without a heap. Signals: several sorted sources, k sorted lists, the kth smallest across sorted rows, or a range that must touch every list. LC 23 is this loop on linked lists: push `(node.val, i, node)` for the head of list `i`, and after a pop push `node.next` if it exists. A sorted matrix (LC 378) is a stack of sorted rows: seed the heap with every row's first value, pop k times, and the kth pop is the kth smallest. The classic mistake in Python is pushing `(node.val, node)`: when two values tie, the heap compares the nodes themselves and raises a `TypeError`, so put the list index in the middle as a tie-breaker.

Practice: LC 23 Merge k Sorted Lists · LC 378 Kth Smallest Element in a Sorted Matrix · LC 632 Smallest Range Covering Elements from K Lists · LC 373 Find K Pairs with Smallest Sums

21. Find Median from Data Stream

Two Heaps Meet in the Middle · Design · Tier 1

Numbers arrive one at a time, and after any arrival we must report the median of everything seen so far: the middle value when the count is odd, the mean of the two middle values when it is even. The stream 5, 15, 1, 3 has medians 5, 10, 5 and 4 after each arrival.

The brute force keeps a sorted list and inserts each number in its place, which costs `O(n)` per arrival because the tail has to shift. But the median never needs the full order. It needs only the cut through the middle of the sorted stream and the one or two values touching that cut. So keep the smaller half in a max-heap `low`, whose top is its largest value, and the larger half in a min-heap `high`, whose top is its smallest. The two heaps are the two halves of the sorted stream, and their tops face each other across the cut. Two invariants hold after every call: every value in `low` is at most every value in `high`, and `low` holds as many values as `high` or exactly one more. With an odd count the median is the top of `low`; with an even count it is the mean of the two tops.

Adding a number keeps both invariants without ever asking which half it belongs to. Push it into `low`, pop the largest value of `low` and push that into `high`. The value that crosses is the largest of the old smaller half and the newcomer, so the order between the halves survives. The crossing makes `high` one bigger; if it is now larger than `low`, its smallest value walks back, which is safe for the same reason in mirror image.

```
stream 5, 15, 1, 3              low = max-heap ]│[ high = min-heap

add   low → high   high → low      low ]│[ high          median
  5        5            5           5 ]│[                   5
 15       15            -           5 ]│[ 15               10.0
  1        5            5         1 5 ]│[ 15                  5
  3        5            -         1 3 ]│[ 5 15               4.0

the values touching the wall are the two heap tops
```

```
CLASS MedianFinder
    FUNCTION Init()
        self.low ← empty max-heap          // the smaller half
        self.high ← empty min-heap         // the larger half

    FUNCTION AddNum(num)
        self.low.push(num)
        top_low ← self.low.pop()           // the largest of low and num
        self.high.push(top_low)
        IF len(self.high) > len(self.low)  // low may lead by one, not trail
            top_high ← self.high.pop()
            self.low.push(top_high)

    FUNCTION FindMedian()
        IF len(self.low) > len(self.high)
            RETURN self.low.top()          // odd count: low holds the middle
        RETURN (self.low.top() + self.high.top()) / 2
```

Each `AddNum` makes at most three pushes and two pops, so it costs `O(log n)`; `FindMedian` reads two tops in `O(1)`, and the heaps hold all `n` values. Reach for two heaps when a problem asks for the median

or any middle rank of a collection that keeps changing, or when items move from a waiting pool into a chosen pool ranked the opposite way, as in IPO with a min-heap of capital and a max-heap of profit. The sliding-window median adds removals, handled lazily: count the values that have left the window and discard them only when they surface at a top. The classic mistake lives in Python, where `heapq` builds only min-heaps: `low` stores negated values, so a number goes in as `-num` and comes out with a second minus, and dropping either sign silently mixes the halves. A close second is `//` in the even case, which turns the median 1.5 of `[1, 2]` into 1.

Practice: LC 295 Find Median from Data Stream · LC 480 Sliding Window Median · LC 502 IPO

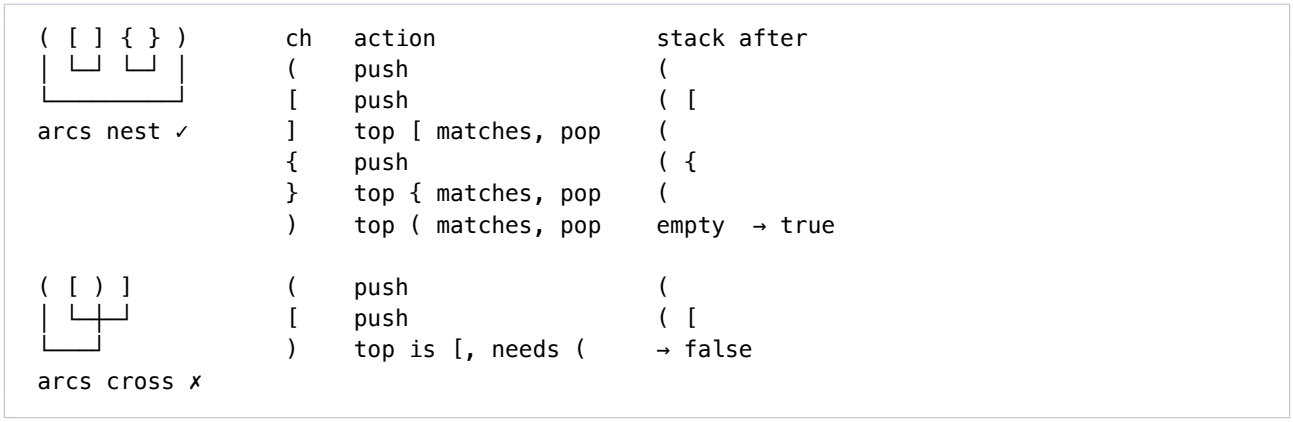
22. Valid Parentheses

The Matching Stack · Stacks · Tier 1

Given a string made of the brackets `()[]{}`, decide whether it is valid: every opener is closed by a bracket of the same kind, and the bracket opened most recently closes first. `"([]{})"` is valid; `"([)]"` is not, even though each kind is opened once and closed once.

The brute force deletes adjacent matched pairs such as `()` until nothing changes and checks that the string vanished, which costs $O(n^2)$ when the nesting is deep. The better view is geometric. Draw an arc from every opener to the closer that ends it. A valid string is a set of arcs that nest like boxes inside boxes, never crossing, each joining two brackets of the same kind. Nesting means that a closer always ends the innermost arc still open, the one opened most recently, and “most recent first” is exactly a stack.

So walk the string once. Push every opener. When a closer arrives, the top of the stack must be its partner; popping it closes that arc. A closer that meets an empty stack has no arc to close, and a closer that meets the wrong kind on top would cross an arc that is still open, so either way the answer is `false` on the spot. At the end the stack must be empty, because any opener left on it is an arc that never closed.




```
FUNCTION IsValid(s)
    partner ← map {')' → '(', ']' → '[', '}' → '{'}  // closer → its opener
    stack ← empty stack                              // openers still waiting
    FOR EACH ch IN s
        IF ch IN partner
            IF len(stack) = 0 OR stack.top() ≠ partner[ch]
                RETURN false                         // nothing open, or wrong kind
            stack.pop()                              // the innermost opener closes
        ELSE
            stack.push(ch)
    RETURN len(stack) = 0                            // leftovers never closed
```

Each character is pushed or popped at most once, so the scan takes $O(n)$ time, and the stack can hold all n openers, $O(n)$ space. Reach for the matching stack whenever things must close in the reverse order of opening: brackets, HTML tags, nested function calls, an encoded string such as `3[a2[c]]`, the groups of an arithmetic expression (#63). With a single bracket kind the stack shrinks to a counter of open brackets, and Minimum Add to Make Parentheses Valid just adds the closers that find the counter at zero to the openers left at the end. Minimum Remove to Make Valid Parentheses pushes indices instead of

characters, so it knows exactly which unmatched brackets to delete, and Longest Valid Parentheses also keeps indices, measuring each valid run from the last unmatched index left on the stack. The classic mistakes are returning `true` without checking that the stack is empty, which accepts `"(("`, and reading the top of an empty stack on a leading closer as in `")("`, which crashes instead of returning `false`. Keeping one counter per kind is the third: the counts of `"([)]"` balance perfectly and the crossing goes unseen.

Practice: LC 20 Valid Parentheses · LC 921 Minimum Add to Make Parentheses Valid · LC 1249 Minimum Remove to Make Valid Parentheses · LC 32 Longest Valid Parentheses

23. Monotonic Stack

Next Greater Element · Stacks · Tier 1

Given daily temperatures, return for each day how many days you must wait for a strictly warmer one, or 0 if none comes. For `[73, 74, 75, 71, 69, 72, 76, 73]` the answer is `[1, 1, 4, 2, 1, 1, 0, 0]`: the 75 on day 2 waits four days for the 76.

The brute force scans right from every day until something warmer appears, $O(n^2)$ on a falling sequence. Its waste is that a cold day gets rescanned by every earlier day still waiting behind it. Turn the question around: instead of each day searching forward, let each new day answer everyone it beats. The days still waiting form a staircase that never rises from left to right, because a warmer later day would already have answered the earlier one. Keep that staircase on a stack of indices. A new day knocks down every step colder than itself, starting at the top, and each fallen day `j` learns its answer, `i - j`. The first step that is not colder stops the pops, and the new day becomes the top step, waiting for its own answer.

The stack holds indices, not temperatures, because the answer is a distance and a temperature is one lookup away. Only a strictly warmer day pops, so equal temperatures sit side by side and keep waiting together.

```
i  temp   pops: answer[j] = i - j          stack after (index:temp)
0   73    -                                0:73
1   74    answer[0] = 1                    1:74
2   75    answer[1] = 1                    2:75
3   71    -                                2:75  3:71
4   69    -                                2:75  3:71  4:69
5   72    answer[4] = 1, answer[3] = 2     2:75  5:72
6   76    answer[5] = 1, answer[2] = 4     6:76
7   73    -                                6:76  7:73
end: days 6 and 7 never pop and keep 0 → [1, 1, 4, 2, 1, 1, 0, 0]
```

```
FUNCTION DailyTemperatures(temperatures)
    n ← len(temperatures)
    answer ← array of n zeros           // 0 stays for days nothing beats
    stack ← empty stack                 // waiting indices, temps never rising
    FOR i FROM 0 TO n - 1
        WHILE len(stack) > 0 AND temperatures[stack.top()] < temperatures[i]
            j ← stack.pop()             // day i is the first warmer day after j
            answer[j] ← i - j
        stack.push(i)                   // day i waits for its own answer
    RETURN answer
```

Every index is pushed once and popped at most once, so the nested loops do $O(n)$ work in total, plus $O(n)$ space for the stack. Reach for a monotonic stack whenever each element needs its nearest greater or smaller neighbor on one side: the next warmer day, the next greater element, a stock span, the walls of the largest rectangle in a histogram (#62). The variants change only the comparison and the direction. For the previous smaller element, keep values rising on the stack, pop everything greater than or equal to the new value, and the top that remains is the answer. For a circular array, walk the indices twice with `i mod n`, so elements near the end find their answer near the front. Online Stock Span is the previous-greater variant run as a stream, where each popped day's span is added to the new one. The classic mistakes are pushing temperatures instead of indices, which loses the distance, and

popping on ≤ instead of <, which answers a day with an equal one that is not warmer and turns `[70, 70, 71]` into `[1, 1, 0]` instead of `[2, 1, 0]`.

Practice: LC 739 Daily Temperatures · LC 496 Next Greater Element I · LC 503 Next Greater Element II · LC 901 Online Stock Span

24. Min Stack

Every Entry Remembers the Minimum Below It · Design · Tier 1

Design a stack that supports `push(val)`, `pop()`, `top()` and `get_min()`, which returns the smallest value on the stack, all in `O(1)` time; `pop`, `top` and `get_min` are called only when the stack is not empty. After `push(-2)`, `push(0)` and `push(-3)`, `get_min()` returns -3. After one `pop()`, `top()` returns 0 and `get_min()` returns -2 again.

Scanning the stack inside `get_min` costs `O(n)`, and a single variable holding the minimum breaks on the first pop that removes it: the -3 leaves, the variable still says -3, and nothing remembers that the minimum used to be -2. The way out is a fact about stacks. While an entry sits on the stack, the entries below it never change: all of them were there when it arrived, and none can leave before it does. So the minimum of an entry and everything beneath it is fixed for the entry's whole life, and the entry computes it once, at its push, as the smaller of its own value and the minimum recorded by the entry just below. Each entry is a pair, (value, minimum so far), and `get_min` reads the second half of the top pair.

Read the second column from the bottom up and it is a running minimum, a staircase that only steps down: flat when a larger value arrives, one step lower when a new minimum does. A pop removes the top step and repairs nothing, because the pair it uncovers recorded its minimum when the stack looked exactly as it does now. The minimum that the popped value had hidden comes back on its own, and a second copy of the minimum gets a pair of its own, so it comes and goes without disturbing the first.

```
after push(-2), push(0), push(-3)              after pop()
           value  min                               value  min
  top → |   -3 |   -3 | ← get_min = -3
        |    0 |   -2 |                    top → |    0 |   -2 | ← get_min = -2
        |   -2 |   -2 |                          |   -2 |   -2 |
        └──────┴──────┘                          └──────┴──────┘

operation   how the pair is made          entries, bottom → top       returns
push(-2)    nothing below: min = -2       (-2, -2)
push(0)     min(0, -2) = -2               (-2, -2) (0, -2)
push(-3)    min(-3, -2) = -3              (-2, -2) (0, -2) (-3, -3)
get_min()   read the top pair             (-2, -2) (0, -2) (-3, -3)    -3
pop()       -3 leaves with its record     (-2, -2) (0, -2)
top()       read the top pair             (-2, -2) (0, -2)              0
get_min()   the record below is intact    (-2, -2) (0, -2)             -2
push(-2)    min(-2, -2) = -2              (-2, -2) (0, -2) (-2, -2)
pop()       the copy leaves               (-2, -2) (0, -2)
get_min()   the first -2 still counts     (-2, -2) (0, -2)             -2
```

```
CLASS MinStack
    FUNCTION Init()
        self.entries ← empty stack          // pairs, bottom to top

    FUNCTION Push(val)
        current_min ← val
        IF len(self.entries) > 0
            below_min ← self.entries.top()[1]     // min of everything below
            current_min ← min(val, below_min)
        self.entries.push((val, current_min))
```

```
    FUNCTION Pop()
        self.entries.pop()                  // the pair below keeps its record

    FUNCTION Top()
        RETURN self.entries.top()[0]        // the value half

    FUNCTION GetMin()
        RETURN self.entries.top()[1]        // the minimum half
```

Each operation touches only the top of the stack, so all four run in `O(1)` time, at the price of two numbers per entry, `O(n)` space. Reach for this whenever a stack must also answer a question about everything it holds, such as its minimum, maximum or sum; two min stacks back to back make a queue that reports its minimum too, every operation `O(1)` amortized. The two-stack variant saves space when new minimums are rare: a second stack records a value when it is at most the current minimum, and popping a value equal to the latest record pops the record too. Max Stack mirrors the pairs for `peekMax`, but `popMax` deletes from the middle, which they cannot absorb, so the usual design adds a max-heap beside the stack and deletes lazily, `O(log n)` amortized per call. The classic mistake, in the two-stack variant, is recording only values strictly smaller than the minimum: the demo's second -2 then goes unrecorded, its pop removes the only recorded -2, and `get_min` finds the record stack empty while `[-2, 0]` is still stored.

Practice: LC 155 Min Stack · LC 716 Max Stack · LC 232 Implement Queue using Stacks

25. Decode String

One Stack Frame per Bracket · Stacks · Tier 1

Decode a string in which `k[text]` stands for `text` repeated `k` times. Groups may nest, `k` is a positive integer that may have several digits, and digits appear only as repeat counts. `"3[a2[c]]"` decodes to `"accaccacc"`: the inner `2[c]` becomes `cc`, the group then reads `acc`, and the outer 3 writes it three times.

The brute force finds an innermost group, the first `]` and the last `[` before it, expands it in place and starts over, rebuilding the whole string once per group. One left-to-right pass is enough if it can pause a level when a bracket opens and resume it when the bracket closes, and pausing and resuming in last-opened, first-closed order is a stack. Keep `current`, the decoded text of the innermost open level, and `count`, the number being read. On `[` push a frame with the text before the bracket and its count, then start the inner level empty. On `]` the group is finished: pop the frame, and the outer level resumes as `before + current × repeat`. Letters are appended to `current`, and digits build the count as `count * 10 + digit`, so 12 reads as twelve.

Picture each bracket pair as a box. Every open box owns one frame on the stack, holding what its outer level had written before the box opened, and only the innermost open box is being written. Closing a box flattens it into plain text, `repeat` copies of its contents, glued onto the end of the outer level's text, which is exactly the `before` in the frame on top.

```
3 [ a 2 [ c ] ]
  |     └───┘ |     frame ("a", 2) closes: "a" + "c" × 2 = "acc"
  └────────────┘     frame ("", 3) closes:  "" + "acc" × 3 = "accaccacc"

ch  count  current       frames (before, count)   step
3   3      ""            empty                    count ← 3
[   0      ""            ("", 3)                  park ("", 3)
a   0      "a"           ("", 3)                  append a
2   2      "a"           ("", 3)                  count ← 2
[   0      ""            ("", 3) ("a", 2)         park ("a", 2)
c   0      "c"           ("", 3) ("a", 2)         append c
]   0      "acc"         ("", 3)                  "a" + "c" × 2
]   0      "accaccacc"   empty                    "" + "acc" × 3
```

```
FUNCTION DecodeString(s)
    stack ← empty stack                 // frames: (text before the '[', count)
    current ← ""                        // decoded text of the innermost level
    count ← 0                           // the repeat count being read
    FOR EACH ch IN s
        IF ch is a digit
            count ← count * 10 + int(ch)        // counts can have many digits
        ELSE IF ch = '['
            stack.push((current, count))        // park the outer level
            current ← ""                        // the inner level starts empty
            count ← 0
        ELSE IF ch = ']'
            before, repeat ← stack.pop()
            current ← before + current × repeat // resume the outer level
        ELSE
            current ← current + ch
```

```
    RETURN current
```

The loop reads each input character once, and each `]` builds one string no longer than the output, so the cost is measured by the output length, not the input length: the 10 characters of `9[9[9[a]]]` decode to 729, built as strings of 9, 81 and 729. The trap is a long text followed by many groups, as in `abcdef1[x]1[y]1[z]`, where every `]` recopies the text before it; collecting each level's pieces in a list and joining them once avoids that. The recursive version lets the call stack hold the frames, calling itself after each `[` and getting back the decoded group and the index past its `]`, the same shape as Basic Calculator (#63). Reach for a stack of frames whenever a nested input must suspend an outer level while an inner one is built, as in Number of Atoms, where each frame is a map of element counts and the multiplier follows the `)`. The classic mistake is forgetting to reset `count` after `[`, which reads the inner `2` of `3[a2[c]]` as 32; treating each digit as the whole count turns `12[x]` into `xx`.

Practice: LC 394 Decode String · LC 726 Number of Atoms · LC 1190 Reverse Substrings Between Each Pair of Parentheses

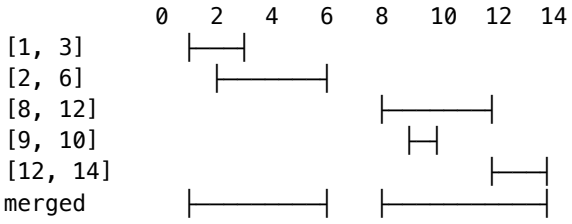
26. Merge Intervals

Sort by Start, Sweep and Extend · Intervals · Tier 1

Given closed intervals `[start, end]` in any order, merge every group that overlaps or touches and return the result sorted by start. `[[1, 3], [8, 12], [2, 6], [9, 10], [12, 14]]` becomes `[[1, 6], [8, 14]]`: `[2, 6]` overlaps `[1, 3]`, `[9, 10]` lies inside `[8, 12]`, and `[12, 14]` touches it at 12.

The brute force compares every pair and merges any two that overlap, repeating until nothing changes, which is at least `O(n²)`. Sorting by start removes all of that searching. Lay the sorted intervals on a number line and sweep from left to right. Each interval starts at or after every interval before it, so the only one it can reach is the merged interval being built at the right end of the output, `merged[−1]`. One comparison settles it. If the new start is at most the last merged end, the two share a point and the last interval stretches to cover the new one. Otherwise there is a gap on the line, and no later interval can bridge it, because every later interval starts no earlier than the new one. The last merged interval is final, and a new one begins.

The stretch takes the larger of the two ends, not the new end, because a short interval can sit entirely inside the current one, as `[9, 10]` sits inside `[8, 12]`. The test is ≤ rather than < because closed intervals that touch share a point, so `[8, 12]` and `[12, 14]` become one.



sorted	start ≤ last merged end?	merged after
[1, 3]	merged is empty → append	[1, 3]
[2, 6]	2 ≤ 3 → end ← max(3, 6) = 6	[1, 6]
[8, 12]	8 > 6 → append	[1, 6] [8, 12]
[9, 10]	9 ≤ 12 → end ← max(12, 10) = 12	[1, 6] [8, 12]
[12, 14]	12 ≤ 12 → end ← max(12, 14) = 14	[1, 6] [8, 14]

```
FUNCTION MergeIntervals(intervals)
    merged ← empty list                      // merged[−1] is its last interval
    FOR EACH [start, end] IN sorted(intervals)          // by start
        IF len(merged) > 0 AND start ≤ merged[−1][1]
            merged[−1][1] ← max(merged[−1][1], end)     // overlap: stretch it
        ELSE
            merged.append([start, end])                 // a gap: a new interval
    RETURN merged
```

The sort costs `O(n log n)` and the sweep `O(n)`, with `O(n)` space for the output. Reach for sort-then-sweep whenever a problem hands you ranges, bookings or segments and asks for their union, their overlaps or the gaps between them. When the list is already sorted and disjoint, as in Insert Interval (#65), skip the sort: copy the intervals that end before the new one starts, absorb every interval that overlaps it by taking the smaller start and the larger end, then copy the rest, all in `O(n)`. Interval List Intersections (#66) walks two sorted lists with two pointers instead, and Meeting Rooms needs only the comparison: after sorting, any start before the previous end is a conflict. The classic mistake is

stretching to `end` instead of `max(merged[-1][1], end)`: on the example, `[9, 10]` shrinks `[8, 12]` to `[8, 10]`, and `[12, 14]` is wrongly left on its own.

Practice: LC 56 Merge Intervals · LC 57 Insert Interval · LC 986 Interval List Intersections · LC 252 Meeting Rooms

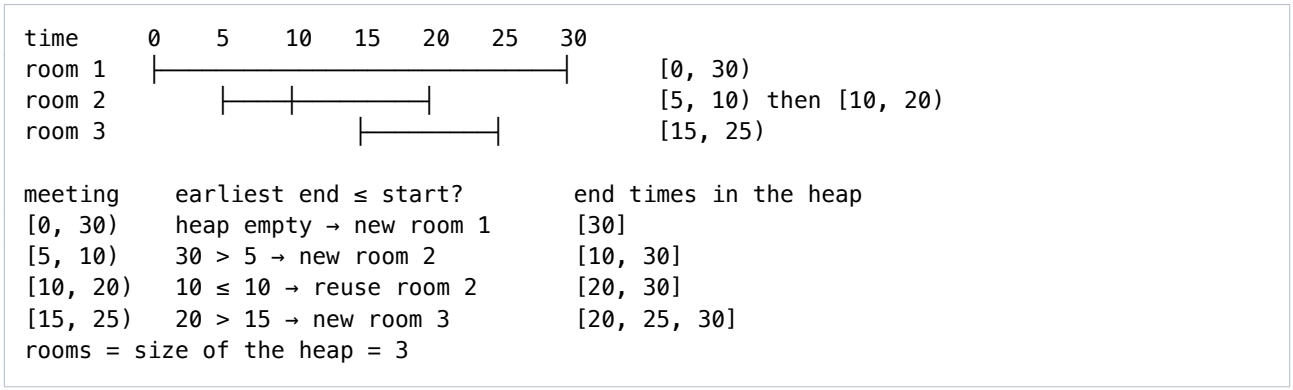
27. Meeting Rooms II

Minimum Rooms with a Heap of End Times · Intervals · Tier 1

Given meetings as half-open intervals `[start, end)`, return the fewest rooms that can host them all; a meeting may start the moment another one ends. For `[[0, 30], [5, 10], [10, 20], [15, 25]]` the answer is 3: at time 15 the meetings `[0, 30)`, `[10, 20)` and `[15, 25)` are all running, while `[10, 20)` can take over the room that `[5, 10)` frees at 10.

The brute force stands at every meeting's start, counts the meetings in progress and keeps the maximum, `O(n²)`. That peak overlap is the answer, and a heap finds it in `O(n log n)`. Take the meetings in order of start and keep one entry per room in a min-heap: the time that room becomes free. Only the top matters, the room that frees up first. If it is free by the new meeting's start, the meeting takes it over: pop the old end, push the new one. If even that room is busy, every room is busy, and the meeting opens a new room with a push alone. The heap never shrinks, so its final size is the number of rooms, and none of them is wasted: a room opens only when every existing room holds a meeting that began no later and ends after `start`, so all of them overlap the newcomer at that instant.

The same number falls out of a sweep line. Turn each meeting into two events, +1 at its start and −1 at its end, sort them by time and run a counter across them; the counter's peak is the answer. At equal times the ends must come first, since a meeting that ends at 10 hands its room to one that starts at 10, and sorting `(time, delta)` pairs does that because −1 sorts before +1.




```
FUNCTION MinMeetingRooms(intervals)
    ends ← empty min-heap                    // one end time per room
    FOR EACH [start, end] IN sorted(intervals)  // earliest start first
        IF len(ends) > 0 AND ends.top() ≤ start
            ends.pop()                       // the room that frees first is free
        ends.push(end)                       // this room is busy until end
    RETURN len(ends)
```

Sorting costs `O(n log n)` and each meeting makes at most one pop and one push, so the run is `O(n log n)` time and `O(n)` space. Reach for it whenever a problem asks how many things run at once: rooms, platforms, servers, bookings. Car Pooling is the sweep with passenger counts in place of +1; Divide Intervals Into Minimum Number of Groups is this problem with closed intervals, so a group frees up only when its end is strictly less than the new start; My Calendar III keeps the sweep in a sorted map. The classic mistake is testing < instead of ≤, which treats `[1, 5)` and `[5, 10)` as a clash and answers 2 instead of 1. Another is popping every finished meeting in a loop and returning the final heap size, which has shrunk by then: `[[1, 10], [2, 3], [11, 12]]` answers 1 instead of 2.

Practice: LC 253 Meeting Rooms II · LC 1094 Car Pooling · LC 2406 Divide Intervals Into Minimum Number of Groups · LC 732 My Calendar III

28. Kadane's Algorithm

Maximum Subarray: Extend or Restart · Greedy · Tier 1

Given a non-empty array of integers, find the contiguous subarray with the largest sum and report that sum with the subarray's first and last index. In `[−2, 1, −3, 4, −1, 2, 1, −5, 4]` the answer is 6, from `[4, −1, 2, 1]` at indices 3 to 6.

The brute force tries every subarray, and even with a running total per start that is $O(n^2)$ work. Kadane's insight is to ask a smaller question at each index: what is the best sum of a subarray that ends exactly here? That subarray either extends the best one ending one step earlier or starts fresh at the current number, so `ending_here = max(x, ending_here + x)`. Extending loses exactly when the run behind is negative, because a negative prefix can only drag `x` down, so the choice is a sign test: restart when `ending_here < 0`, extend otherwise. The answer is the best `ending_here` seen anywhere, and one pass carries just two numbers, the run that ends at the pointer and the best run so far.

Picture the prefix sums as a mountain profile, point `k` standing at the sum of the first `k` numbers; the sum of `nums[i..j]` is then the climb from point `i` to point `j + 1`. The run turns negative exactly when the profile dips to a new low, so every restart moves the base camp to the lowest point so far, and the best subarray is the biggest climb from a low to a later peak: the stock problem of #29 run on prefix sums, except that a trade is compulsory, because the subarray cannot be empty. The bounds come almost free. `start` jumps to `i` on every restart, and whenever `best` improves, the current `start` and `i` are copied into `best_start` and `best_end`.

```
i    x    extend or restart    ending_here   start   best   best bounds
0   −2    first element             −2           0     −2      [0, 0]
1    1    restart: −2 < 0            1           1      1      [1, 1]
2   −3    extend                    −2           1      1      [1, 1]
3    4    restart: −2 < 0            4           3      4      [3, 3]
4   −1    extend                     3           3      4      [3, 3]
5    2    extend                     5           3      5      [3, 5]
6    1    extend                     6           3      6      [3, 6]
7   −5    extend                     1           3      6      [3, 6]
8    4    extend                     5           3      6      [3, 6]

prefix sums   0  −2  −1  −4   0  −1   1   2  −3   1
                          ▲ new low       ▲ peak: 2 − (−4) = 6
```

```
FUNCTION MaxSubarray(nums)
    ending_here ← nums[0]                  // best sum of a subarray ending at i
    start ← 0                              // where that subarray begins
    best ← nums[0]                         // not 0: all may be negative
    best_start ← 0
    best_end ← 0
    FOR i FROM 1 TO len(nums) − 1
        IF ending_here < 0                 // a negative run only drags x down
            ending_here ← nums[i]          // restart at x
            start ← i
        ELSE
            ending_here ← ending_here + nums[i]        // extend the run
        IF ending_here > best              // improved: copy the bounds
            best ← ending_here
            best_start ← start
```

```
            best_end ← i
    RETURN (best, best_start, best_end)
```

One pass with constant work per element gives `O(n)` time and `O(1)` space. Reach for Kadane when a problem asks for the best contiguous stretch, a largest sum or gain over consecutive elements, and the best stretch ending at `i` can be built from the best one ending at `i − 1`. The variants change what is carried. A circular array (LC 918) takes the larger of the plain answer and the total minus the minimum subarray sum, unless every number is negative; a product (LC 152) carries both the largest and the smallest product ending here, because a negative factor swaps them; the largest absolute sum (LC 1749) is the larger of the maximum sum and the negated minimum sum. The classic mistake is starting `best` at `0`, which answers `0` for `[−3, −1, −2]` instead of `−1`: the empty subarray is not a choice.

Practice: LC 53 Maximum Subarray · LC 918 Maximum Sum Circular Subarray · LC 152 Maximum Product Subarray · LC 1749 Maximum Absolute Sum of Any Subarray

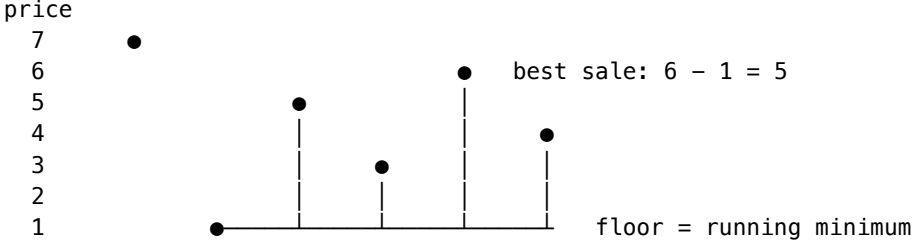
29. Best Time to Buy and Sell Stock

Track the Running Minimum · Greedy · Tier 1

Given a stock's price on each day, buy on one day and sell on a later day so that the profit is as large as possible, or return `0` when no trade makes money. For `[7, 1, 5, 3, 6, 4]` the answer is `5`: buy at `1` on day 1 and sell at `6` on day 4.

The brute force pairs every buy day with every later sell day, `O(n²)` pairs. Yet a sale on a given day does not care about all the earlier days, only about the cheapest of them: the best trade that sells today bought at the minimum price so far. So walk the days once, carry that running minimum, and price each day as a sale against it; the answer is the largest of those daily profits. The running minimum is the whole memory of the past. It is updated before today's sale, which at worst prices a same-day trade at `0`, the same as not trading.

Picture the price chart as a mountain range. A trade is a climb from one point to a later, higher point, and the answer is the tallest such climb. The running minimum is a floor that only ever drops, sitting at the lowest valley behind the walker, and each day's profit is the height of that day's point above the floor. A new low never hurts, because it lowers the floor for every later day. When transactions are unlimited (LC 122), the answer is instead the sum of every positive day-to-day rise: a long climb earns exactly the sum of its steps, and every step up can be taken on its own.



	d0	d1	d2	d3	d4	d5
min	7	1	1	1	1	1
profit	0	0	4	2	5	3
best	0	0	4	4	5	5

```
FUNCTION MaxProfit(prices)
    min_price ← ∞                          // cheapest price so far
    best ← 0                               // no trade at all earns 0
    FOR EACH price IN prices
        IF price < min_price               // a cheaper day to have bought on
            min_price ← price
        profit ← price − min_price         // sell today, bought at the minimum
        IF profit > best
            best ← profit
    RETURN best
```

One pass with constant work per day gives `O(n)` time and `O(1)` space. Reach for the running minimum whenever a problem asks for the best `a[j] − a[i]` with `i < j`: the best partner for every `j` is one number carried forward. Run on the day-to-day price differences, this is Kadane's algorithm (#28) with `best` started at `0`, the mistake there but correct here, because not trading is allowed. The fee and cooldown versions (LC 714, LC 309) carry a few running balances instead of one minimum, the best cash while holding a share and while not holding one, plus a resting state for the cooldown. The classic

mistake is answering `max(prices) − min(prices)`, which ignores the order of the days: on `[7, 6, 4, 3, 1]` it reports `6`, but the price only falls and the answer is `0`.

Practice: LC 121 Best Time to Buy and Sell Stock · LC 122 Best Time to Buy and Sell Stock II · LC 714 Best Time to Buy and Sell Stock with Transaction Fee · LC 309 Best Time to Buy and Sell Stock with Cooldown

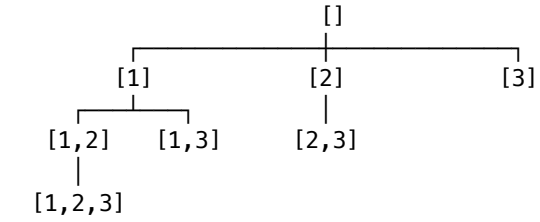
30. Subsets

Choose, Recurse, Un-Choose · Backtracking · Tier 1

Given an array of distinct integers, return all of its subsets, the power set, each exactly once and in any order. For `[1, 2, 3]` there are `2^3 = 8` of them: `[]`, `[1]`, `[1, 2]`, `[1, 2, 3]`, `[1, 3]`, `[2]`, `[2, 3]` and `[3]`.

Each element faces one yes-or-no decision, so there are `2^n` subsets, and any method that lists them pays at least that much. Backtracking grows them as a tree. The root is the empty subset, and each child appends one more element, always one that lies to the right of everything its parent holds. That rule is what makes every subset appear exactly once, in index order: `[1, 2]` is built, `[2, 1]` never is. The recursion carries the rule as a `start` index. At a node the loop tries each `i` from `start` on: append `nums[i]` (choose), recurse with `start = i + 1`, then pop it again (un-choose), so the next sibling begins from the same path its parent had.

Unlike permutations (#31), where only the leaves are answers, every node of this tree is a subset, so a call records its path the moment it begins, before the loop. One shared list, `path`, carries the whole search: it grows on the way down and shrinks on the way up, a stack that always spells the route from the root to the current node.




```
recorded in preorder: [] [1] [1,2] [1,2,3] [1,3] [2] [2,3] [3]
below [2] the loop starts at index 2, so only 3 may follow: no [2,1]
```

```
FUNCTION Backtrack(nums, start, path, result)
    result.append(path.copy())              // every node is a subset
    FOR i FROM start TO len(nums) - 1
        path.append(nums[i])                // choose
        Backtrack(nums, i + 1, path, result)   // only later indices below
        path.pop()                          // un-choose

FUNCTION Subsets(nums)
    result ← empty list
    Backtrack(nums, 0, empty list, result)
    RETURN result
```

The tree has `2^n` nodes and copying a path costs up to `O(n)`, so the time is `O(n · 2^n)`; beyond the output, the space is the `O(n)` path and recursion depth. Reach for this template when a problem asks for all subsets or all combinations of a small input (n up to about 20), or when a search has to try every selection. With duplicates in the input (LC 90), sort first and skip `nums[i]` when `i > start` and `nums[i] = nums[i − 1]`: equal values then sit side by side, and on one level only the first of a run may open a branch, which stops `[1, 2, 2]` from producing `[1, 2]` twice. The same `2^n` subsets also come out of a plain loop over bitmasks (#137). The classic mistake is `result.append(path)` without the

copy: every entry is then the same list object, and for `[1, 2, 3]` the search ends with eight references to one empty list.

Practice: LC 78 Subsets · LC 90 Subsets II · LC 1863 Sum of All Subset XOR Totals

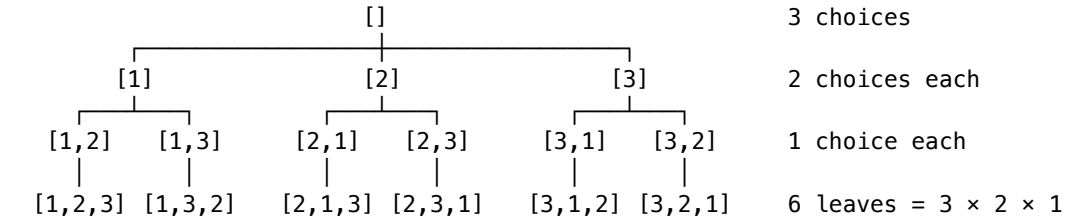
31. Permutations

Backtracking with a Used Array · Backtracking · Tier 1

Given an array of distinct integers, return every ordering of its elements, in any order. `[1, 2, 3]` has `3! = 6` permutations: `[1, 2, 3]`, `[1, 3, 2]`, `[2, 1, 3]`, `[2, 3, 1]`, `[3, 1, 2]` and `[3, 2, 1]`.

A permutation fills n slots from left to right: n choices for the first slot, `n − 1` for the second, down to one for the last. Backtracking walks exactly that decision tree. The `path` holds the slots filled so far, and a `used` array of booleans remembers which indices already sit in it, so every level loops over all the indices from 0 again and skips only the used ones. That is the difference from subsets (#30): there is no start index, because order matters here and 2 may come before 1, as long as it has not been placed already. Each choice is undone in reverse, pop the value and then clear its flag, so the next sibling sees exactly the state its parent saw.

Only a full path is an answer. The tree is a fan that narrows by one at every level, n branches at the root and `n − 1` under each child, and its `n!` leaves are the permutations, each recorded as a copy when the length of the path reaches n.




```
FUNCTION Backtrack(nums, used, path, result)
    IF len(path) = len(nums)
        result.append(path.copy())          // every slot is filled
        RETURN
    FOR i FROM 0 TO len(nums) − 1
        IF used[i]
            CONTINUE                        // already in the path
        used[i] ← true                      // choose
        path.append(nums[i])
        Backtrack(nums, used, path, result)
        path.pop()                          // un-choose, in reverse order
        used[i] ← false

FUNCTION Permutations(nums)
    used ← array of len(nums) copies of false
    result ← empty list
    Backtrack(nums, used, empty list, result)
    RETURN result
```

There are `n!` leaves and fewer than `e · n!` nodes in all, and each node scans at most n flags, so the time is `O(n · n!)`; the space beyond the output is `O(n)` for the path, the flags and the recursion. Reach for it when a problem wants every ordering or arrangement, which is only feasible for small n: `8!` is 40,320, while `12!` is already about 479 million. With duplicates in the input (LC 47), sort first and also skip `i` when `nums[i] = nums[i − 1]` and `used[i − 1]` is false: equal values must then enter the path from

left to right, so each distinct ordering is built once. When only the next ordering in lexicographic order is needed (LC 31), it can be found in place in `O(n)` without any tree. The classic mistake is forgetting `used[i] ← false` on the way back: the flags stay set after the first leaf, and `[1, 2, 3]` returns only `[[1, 2, 3]]`.

Practice: LC 46 Permutations · LC 47 Permutations II · LC 31 Next Permutation

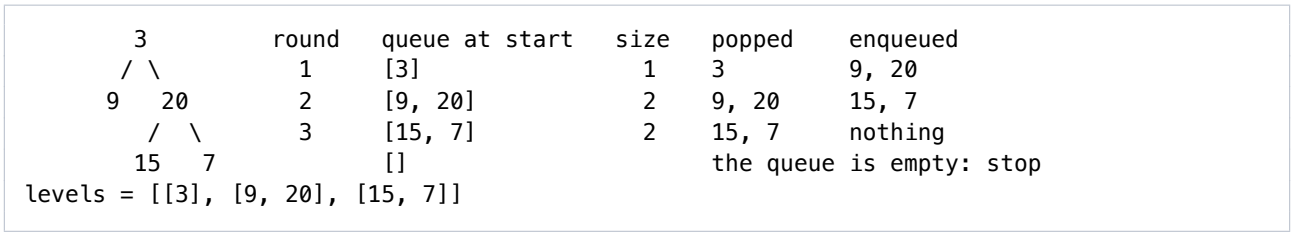
32. Binary Tree Level-Order Traversal

BFS One Level at a Time · BFS · Tier 1

Given the root of a binary tree, return its values level by level, each level read from left to right. Writing the tree in level order with `null` for a missing child, `[3, 9, 20, null, null, 15, 7]` is a root `3` with children `9` and `20`, and `20` has children `15` and `7`; the answer is `[[3], [9, 20], [15, 7]]`.

A queue already visits a tree in level order. The root goes in, and every node that comes out sends its children to the back, so all of depth `d` leaves the queue before anything of depth `d + 1`. What a plain breadth-first search loses is the boundary: it produces one flat stream and forgets where one level ends and the next begins. The fix is a snapshot. At the top of each round the queue holds exactly one complete level, nothing more and nothing less, so `size ← len(queue)` freezes that count and the inner loop pops exactly `size` nodes. The children those pops enqueue pile up behind them and become the next level, untouched until the next round takes its own snapshot.

Picture the tree peeled from the top, one horizontal layer per round: the front of the queue is the layer being stripped off, the back is the layer being laid down. Left-to-right order comes for free, because parents leave the queue left to right and each one enqueues its left child before its right.




```
FUNCTION LevelOrder(root)
    levels ← empty list
    IF root = null
        RETURN levels
    queue ← empty queue
    queue.enqueue(root)
    WHILE queue is not empty
        size ← len(queue)                  // snapshot: exactly this level
        level ← empty list
        FOR i FROM 0 TO size − 1
            node ← queue.dequeue()
            level.append(node.val)
            IF node.left ≠ null            // children join the back
                queue.enqueue(node.left)
            IF node.right ≠ null
                queue.enqueue(node.right)
        levels.append(level)               // the level is complete
    RETURN levels
```

Every node is enqueued and dequeued once, so the traversal takes `O(n)` time; the queue never holds more than one level plus part of the next, `O(w)` space for the widest level `w`, which reaches about `n / 2` in a complete tree. Reach for it whenever a problem says level, depth, row, layer or “seen from the side”. The variants differ only in what each finished level contributes: the right side view keeps its last value, zigzag order reverses every other `level` before appending it, and the average of each level divides its sum by `size`. On a general graph the same loop needs a visited set and becomes BFS shortest

path (#39). The classic mistake is looping against the live queue length instead of the snapshot: the children enqueued during the round grow the queue, and the next level leaks into this one.

Practice: LC 102 Binary Tree Level Order Traversal · LC 199 Binary Tree Right Side View · LC 103 Binary Tree Zigzag Level Order Traversal · LC 637 Average of Levels in Binary Tree

33. Diameter and Max Path Sum

Postorder DFS That Returns a Height · DFS · Tier 1

The diameter of a binary tree is the number of edges on its longest path between two nodes, and the maximum path sum is the largest total of node values along any path of at least one node. In `[1, 2, 3, 4, 5]` the diameter is 3, along `4 → 2 → 1 → 3`; in `[−10, 9, 20, null, null, 15, 7]` the best sum is 42, along `15 → 20 → 7`.

The brute force takes every pair of endpoints and walks the path between them: $O(n^2)$ pairs at up to $O(n)$ steps each, cubic time in all. The insight is that every path has exactly one highest node, where it bends: it climbs up one arm and comes down the other. So skip the endpoints and ask each node a single question, "what is the best path that bends here?", and answer it from two numbers its children already know, the best downward arm on each side. That makes the recursion postorder, children before parent, and gives it two jobs. It returns the best single arm upward, because a parent can extend only one arm. It writes the bent path, left arm plus right arm, into a shared `best`, because a path that bends here is finished and no ancestor can use it.

For the diameter the arm is a height: `Height(node)` counts the nodes on the longest downward path, so an empty subtree has height `0` and the path bending at a node has exactly `left + right` edges. For the path sum the arm is a gain, the node's value plus its better child gain, and a gain below zero is clipped to `0`: do not take that arm at all. Negatives are also why `best` starts at `root.val` instead of `0`: a path holds at least one node, so when every value is negative the answer is the largest single value.

```
      1          node  left  right  left + right  best  returns
     / \           4     0     0         0          0      1
    2   3          5     0     0         0          0      1
   / \            2     1     1         2          2      2
  4   5           3     0     0         0          2      1
                  1     2     1         3          3      3    diameter 3

    -10          node  left  right  val + left + right  best  returns
    /  \          9     0     0           9             9      9
   9    20        15     0     0          15            15     15
       /  \        7     0     0           7            15      7
      15   7      20    15     7          42            42     35
                 -10     9    35          34            42     25    sum 42
```

```
FUNCTION Diameter(root)
    best ← [0]                              // a box every call can update
    Height(root, best)
    RETURN best[0]

FUNCTION Height(node, best)
    IF node = null
        RETURN 0
    left ← Height(node.left, best)
    right ← Height(node.right, best)
    best[0] ← max(best[0], left + right)    // the path that bends at node
    RETURN 1 + max(left, right)             // only one arm continues upward

FUNCTION MaxPathSum(root)
    best ← [root.val]                       // one node alone is a path
```

```
    Gain(root, best)
    RETURN best[0]

FUNCTION Gain(node, best)
    IF node = null
        RETURN 0
    left ← max(Gain(node.left, best), 0)   // a negative arm is cut off
    right ← max(Gain(node.right, best), 0)
    best[0] ← max(best[0], node.val + left + right)
    RETURN node.val + max(left, right)
```

Each node is visited once with constant work, so both run in `O(n)` time and `O(h)` space for the recursion stack, where `h` is the tree's height, `O(n)` on a chain. Recognize the pattern when the answer is a path that may lie anywhere in the tree: return what a parent can extend, record what only this node can complete. The same skeleton checks balance by returning the height or `−1` once a subtree is lopsided, and finds the longest univalue path by letting an arm continue only into a child with the same value. The classic mistake is passing the bent path upward, `1 + left + right` instead of `1 + max(left, right)`: the parent then extends something that is no longer a path, and `[1, 2, 3, 4, 5]` reports a diameter of 4.

Practice: LC 543 Diameter of Binary Tree · LC 124 Binary Tree Maximum Path Sum · LC 110 Balanced Binary Tree · LC 687 Longest Univalue Path

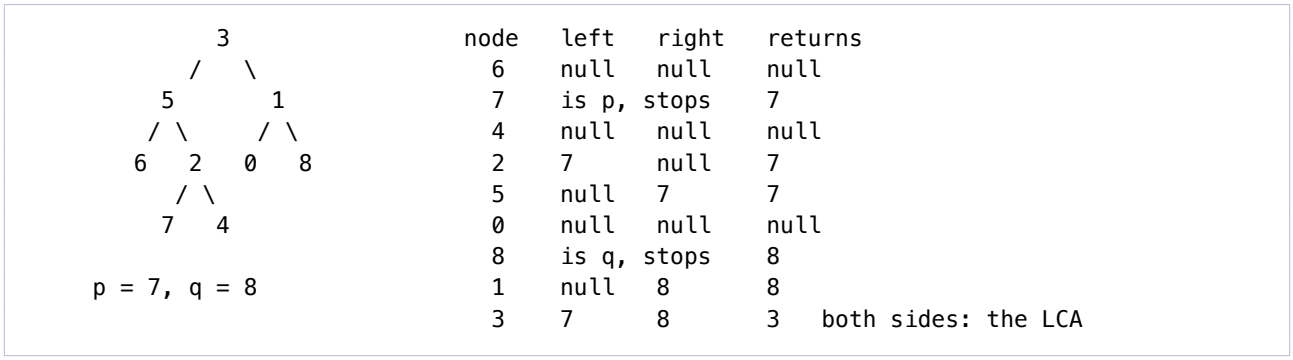
34. Lowest Common Ancestor

Postorder Search in a Binary Tree · DFS · Tier 1

Given a binary tree and two of its nodes `p` and `q`, both present, return their lowest common ancestor: the deepest node that has both of them in its subtree, where a node counts as an ancestor of itself. In `[3, 5, 1, 6, 2, 0, 8, null, null, 7, 4]` the LCA of 7 and 8 is the root 3, the LCA of 6 and 4 is 5, and the LCA of 5 and 4 is 5 itself.

One way is to record the path from the root to `p` and the path to `q` and keep the last node they share, which costs two searches and two stored paths. The postorder search does it in a single pass by letting every subtree send up one report: `null` if it holds neither target, the target itself if it holds one, and the meeting point once it holds both. A call that lands on `p` or `q` returns at once without looking below, and that is safe: if the other target sits underneath, this node is the answer anyway, and if it does not, the other target lives elsewhere and an ancestor will collect both reports. The first node that hears a report from its left subtree and another from its right is where the two root paths split, so it is the lowest common ancestor. From there up, every ancestor hears from one side only and simply forwards it.

Picture two flares fired from `p` and `q`. Each climbs toward the root through its ancestors, and the first node lit from both sides is the answer; it swallows both flares and sends up a single one, itself.




```
FUNCTION LowestCommonAncestor(root, p, q)
    IF root = null OR root = p OR root = q
        RETURN root                    // nothing here, or a target itself
    left ← LowestCommonAncestor(root.left, p, q)
    right ← LowestCommonAncestor(root.right, p, q)
    IF left ≠ null AND right ≠ null
        RETURN root                    // one target on each side: the LCA
    IF left ≠ null
        RETURN left                    // pass the single report upward
    RETURN right
```

Every node is visited at most once, so the search costs `O(n)` time and `O(h)` space for the recursion. Reach for it whenever a question involves two nodes and where their paths to the root meet; the distance between two nodes, for instance, is `depth(p) + depth(q) − 2 ∗ depth(lca)`. In a binary search tree the values do the steering: walk down from the root, going left while both values are smaller and right while both are larger, and the first node that separates them, or equals one of them, is the LCA, in `O(h)`. With parent pointers it turns into the intersection of two linked lists: climb from both nodes, restart each pointer at the other node when it passes the root, and the two meet at the LCA after equal distances. The classic mistake is forgetting the promise that both nodes exist: when only `p` is in the tree, the search still returns `p`, so a version without that promise must confirm that both targets were seen.

Practice: LC 236 Lowest Common Ancestor of a Binary Tree · LC 235 Lowest Common Ancestor of a Binary Search Tree · LC 1650 Lowest Common Ancestor of a Binary Tree III

35. Validate a Binary Search Tree

Carry the Allowed Range Down · BST · Tier 1

Given the root of a binary tree, decide whether it is a valid binary search tree: for every node, all values in its left subtree are strictly smaller than the node's value and all values in its right subtree are strictly larger. `[2, 1, 3]` is valid. `[5, 4, 6, null, null, 3, 7]` is not, even though every child sits on the correct side of its own parent, because `3` lives in the right subtree of `5`.

The tempting check compares each node only with its children, and that second tree is exactly why it fails: `3 < 6` is fine locally, yet the root promised that everything on its right exceeds `5`. The BST rule is a promise about whole subtrees, and promises pile up on the way down. Each time the path turns left at a node, everything below is capped by that node's value; each time it turns right, everything below is floored by it. So carry the promises as one open interval `(low, high)`. The root may hold anything, `(−∞, ∞)`; a left child keeps its parent's `low` and takes the parent's value as its `high`; a right child keeps `high` and takes the parent's value as its `low`. A node passes when its value lies strictly inside its interval, and the tree is valid when every node passes.

Geometrically, each node cuts the number line at its value, and stepping into a subtree keeps only one side of the cut. The window at a node is the gap between two ancestors: the nearest one it hangs to the right of and the nearest one it hangs to the left of. The search stops at the first node outside its window, so `7` below is never looked at.

```
          5                 node   allowed range   verdict
        /   \                 5      (−∞, ∞)         inside
       4     6                4      (−∞, 5)         inside
            / \               6      (5, ∞)          inside
           3   7              3      (5, 6)          3 ≤ 5: outside → false
                              7      never checked
each parent-child pair looks fine (4 < 5, 6 > 5, 3 < 6, 7 > 6), yet 3 < 5
```

```
FUNCTION IsValidBST(root)
    RETURN Fits(root, −∞, ∞)

FUNCTION Fits(node, low, high)
    IF node = null
        RETURN true
    IF node.val ≤ low OR node.val ≥ high
        RETURN false                       // breaks a promise made higher up
    IF NOT Fits(node.left, low, node.val)  // left side: below node.val
        RETURN false
    RETURN Fits(node.right, node.val, high)    // right side: above node.val
```

Each node is checked once, so validation costs `O(n)` time and `O(h)` space for the recursion. The equivalent test is an inorder traversal: a tree is a BST exactly when its inorder sequence is strictly increasing, so walk it while remembering the previous value and fail on the first value that is not larger. The same inorder walk, stopped at the `k`-th value, finds the `k`-th smallest element. Strict inequalities make duplicates invalid, as the definition demands. Reach for the interval whenever a constraint flows from ancestors to descendants rather than from parent to child. The classic mistake is the parent-versus-child check; the runner-up is seeding the bounds with the smallest and largest 32-bit integers, which rejects a lone root holding exactly one of those values, a trap that −∞ and ∞ avoid.

Practice: LC 98 Validate Binary Search Tree · LC 230 Kth Smallest Element in a BST · LC 99 Recover Binary Search Tree

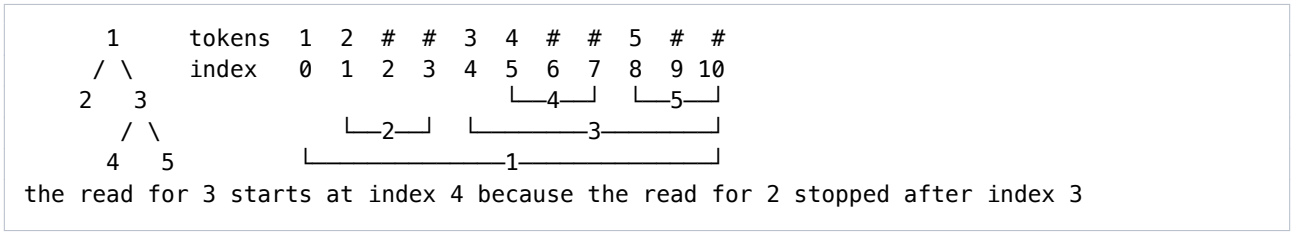
36. Serialize and Deserialize a Binary Tree

Preorder with Null Markers · DFS · Tier 1

Turn a binary tree into a string, and turn that string back into the very same tree. For `[1, 2, 3, null, null, 4, 5]`, a root 1 with children 2 and 3 where 3 has children 4 and 5, writing the values in preorder with # for every missing child gives `"1,2,#,#,3,4,#,#,5,#,#"`, and reading that string rebuilds the tree.

A bare preorder listing is ambiguous: `1, 2` could be a root with a left child 2 or a root with a right child 2, and nothing in the list tells them apart. What is lost is where each subtree ends, and the null marker puts it back. With a # for every empty child, a value token is always followed by exactly two complete subtrees and a # is a complete subtree on its own, so the two trees above become the different strings `1,2,#,#,#` and `1,#,2,#,#`. A tree with `n` nodes has `n + 1` empty children, so the string holds `2n + 1` tokens.

Decoding replays the encoding. A shared index moves left to right through the tokens; each call reads one token, returns null for #, and otherwise builds the node and calls itself twice, first for the left subtree, then for the right. The left call eats exactly the tokens of the left subtree and stops, leaving the index on the first token of the right subtree, and that is the whole trick: in preorder every subtree is one unbroken block of tokens, its value first, then its left block, then its right block.




```
FUNCTION Serialize(root)
    tokens ← empty list
    Write(root, tokens)
    RETURN join(tokens, ",")

FUNCTION Write(node, tokens)
    IF node = null
        tokens.append("#")                // the marker closes a branch
        RETURN
    tokens.append(str(node.val))
    Write(node.left, tokens)
    Write(node.right, tokens)

FUNCTION Deserialize(data)
    tokens ← split(data, ",")
    index ← [0]                           // one box all the calls share
    RETURN Read(tokens, index)

FUNCTION Read(tokens, index)
    token ← tokens[index[0]]
    index[0] ← index[0] + 1
    IF token = "#"
        RETURN null
    node ← new TreeNode(int(token))
```

```
    node.left ← Read(tokens, index)        // the left subtree eats its tokens
    node.right ← Read(tokens, index)       // the right one starts after them
    RETURN node
```

Both directions handle each node and each marker once, so they run in `O(n)` time, with an `O(n)` string and `O(h)` recursion depth. The commas matter: they keep `12` apart from `1` and `2` and let negative values parse with no extra rule. Reach for this whenever a tree must leave memory, to a file, across a network, or into a hash map that spots duplicate subtrees. A binary search tree needs no markers, because its values already say where the left subtree ends, at the first value larger than the root. Without markers a general tree needs a second traversal, which is how it is rebuilt from its preorder and inorder sequences together. The classic mistake is passing the index as a plain integer: each call advances its own copy, the caller never learns how many tokens its left subtree consumed, and the right subtree reads them all over again.

Practice: LC 297 Serialize and Deserialize Binary Tree · LC 449 Serialize and Deserialize BST · LC 105 Construct Binary Tree from Preorder and Inorder Traversal

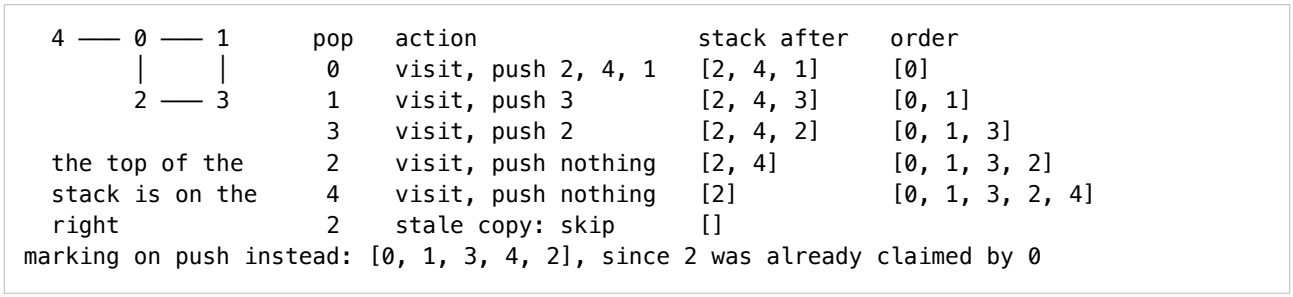
37. Depth-First Search on a Graph

Recursive and Iterative · DFS · Tier 1

Given an undirected graph with nodes `0` to `n - 1` and a list of edges, list the nodes in the order a depth-first search from `start` first reaches them, trying each node's neighbors in the order its edges appear. With `n = 5`, edges `[[0, 1], [0, 4], [0, 2], [1, 3], [3, 2]]` and `start = 0` the order is `[0, 1, 3, 2, 4]`.

Depth-first search is the maze walker's rule: always step to an unvisited neighbor of the node you stand on, and only when there is none, back up to the previous junction and try its next corridor. The recursion is that rule written down. Entering a node marks it visited, records it, and calls itself on every neighbor not yet visited; the call stack is the trail back to the start, and returning from a call is the step back. Marking on entry is what makes cycles harmless: the square `0, 1, 3, 2` leads back to `0`, but `0` is already marked, so the walk turns away instead of circling forever.

The iterative version swaps the call stack for an explicit one, and where it marks decides whether it is still depth-first. Marking a node when it is popped, and skipping the stale copies of nodes pushed twice, reproduces the recursion exactly, as long as the neighbors go on in reverse so the first one ends on top. Marking a node when it is pushed looks tidier, since nothing is pushed twice, but it is a different search: whoever sees a node first claims it, even when a deeper node would have stepped into it sooner. Here `0` claims 2, so 3 cannot step to it and the walk retreats to 4, giving `[0, 1, 3, 4, 2]`, an order no depth-first search can produce.




```
FUNCTION BuildGraph(n, edges)
    graph ← empty list
    FOR i FROM 0 TO n - 1
        graph.append(empty list)
    FOR EACH (u, v) IN edges
        graph[u].append(v)
        graph[v].append(u)                  // undirected: both directions
    RETURN graph

FUNCTION DfsRecursive(n, edges, start)
    graph ← BuildGraph(n, edges)
    visited ← empty set
    order ← empty list
    Visit(graph, start, visited, order)
    RETURN order

FUNCTION Visit(graph, node, visited, order)
    visited.add(node)                       // mark first: cycles cannot re-enter
    order.append(node)
    FOR EACH neighbor IN graph[node]
```

```
        IF neighbor NOT IN visited
            Visit(graph, neighbor, visited, order)

FUNCTION DfsIterative(n, edges, start)
    graph ← BuildGraph(n, edges)
    visited ← empty set
    order ← empty list
    stack ← empty stack
    stack.push(start)
    WHILE stack is not empty
        node ← stack.pop()
        IF node IN visited
            CONTINUE                        // a stale copy, reached another way
        visited.add(node)                   // mark when popped
        order.append(node)
        FOR EACH neighbor IN reversed(graph[node])   // first one ends on top
            IF neighbor NOT IN visited
                stack.push(neighbor)
    RETURN order
```

Every node is entered once and every edge is examined from both ends, so both versions run in `O(V + E)` time; the recursion can be `V` frames deep, and the explicit stack can hold one entry per edge end, `O(E)`. Reach for it whenever the question is about reachability, connectivity or exploring paths: counting components (#84), flood fill on a grid (#38), cycle detection (#81) and topological order (#82) are all this walk with something recorded on the way in or out. A directed graph drops the second `append`. In Python the recursive form dies on deep graphs, because the default recursion limit is 1000 frames and a path of a thousand nodes raises `RecursionError`, so the iterative version is the safe default. The classic mistake is marking a node only after its neighbors are explored, which lets a cycle walk back into it until the stack overflows.

Practice: LC 547 Number of Provinces · LC 841 Keys and Rooms · LC 797 All Paths From Source to Target · LC 133 Clone Graph

38. Number of Islands

Grid Flood Fill · DFS · Tier 1

Given a grid of `"1"` for land and `"0"` for water, count the islands: groups of land cells joined up, down, left or right, never diagonally. The grid with rows `11000`, `11000`, `00100` and `00011` has `3` islands, a two-by-two block at the top left, a single cell in the middle and a pair at the bottom right.

Read the grid as a graph whose nodes are land cells and whose edges join horizontal and vertical neighbors. An island is a connected component, so counting islands is counting components. A scan walks every cell in reading order; whenever it meets a `"1"`, that cell belongs to an island nobody has counted yet, so the count goes up by one. Then a flood fill sinks the whole island, turning every land cell reachable from that start into `"0"`. Sinking is the visited mark, written straight into the grid: when the scan later reaches another cell of the same island it finds water and moves on, so each island is counted exactly once, at its first cell in reading order.

The flood fill is the depth-first search of #37 on a graph that is never built. The four neighbors of `(row, col)` are one step down, up, right and left, computed on the fly. This version pushes all four without looking and lets the pop decide: a position off the map, on water or on land already sunk is thrown away, and a land cell is sunk and pushes its own four neighbors. Only the set of cells reached matters here, not their order, so this simplest form is enough.

```
the grid      sink order     the scan
1 1 0 0 0     1 2 . . .      (0, 0) is land: count 1, sink 4 cells in 17 pops
1 1 0 0 0     4 3 . . .
0 0 1 0 0     . . 1 . .      (2, 2) is land: count 2, sink 1 cell in 5 pops
0 0 0 1 1     . . . 1 2      (3, 3) is land: count 3, sink 2 cells in 9 pops
every other cell is water by the time the scan reaches it → 3 islands
```

```
FUNCTION NumIslands(grid)
    count ← 0
    FOR row FROM 0 TO len(grid) − 1
        FOR col FROM 0 TO len(grid[0]) − 1
            IF grid[row][col] = "1"
                count ← count + 1          // a new island starts here
                Sink(grid, row, col)       // so it is never counted again
    RETURN count

FUNCTION Sink(grid, start_row, start_col)
    stack ← empty stack
    stack.push((start_row, start_col))
    WHILE stack is not empty
        (row, col) ← stack.pop()
        IF row < 0 OR row ≥ len(grid) OR col < 0 OR col ≥ len(grid[0])
            CONTINUE                       // stepped off the map
        IF grid[row][col] ≠ "1"
            CONTINUE                       // water, or land already sunk
        grid[row][col] ← "0"               // sinking is the visited mark
        stack.push((row + 1, col))         // down, up, right, left
        stack.push((row − 1, col))
        stack.push((row, col + 1))
        stack.push((row, col − 1))
```

Every cell is scanned once and sunk at most once, and each sinking pushes four positions, so the count costs `O(R × C)` time for `R` rows and `C` columns, and the stack can reach `O(R × C)` on solid land. The recursive form reads the same, with `Sink` calling itself on the four neighbors instead of pushing them, but on a 300 by 300 grid of land it snakes through every cell and goes 90,000 calls deep, far past Python's recursion limit; the explicit stack has no such ceiling. When the caller's grid must survive, keep a visited set of positions instead of sinking. Signals are a grid and "how many groups"; the same fill measures the largest island by counting cells as it sinks them, recolors a region in flood fill, and solves surrounded regions by first marking everything connected to the border. The classic mistake in Python is dropping the `row < 0` or `col < 0` test: `grid[-1]` silently reads the last row, so an island on the top edge leaks into one on the bottom edge.

Practice: LC 200 Number of Islands · LC 695 Max Area of Island · LC 733 Flood Fill · LC 130 Surrounded Regions

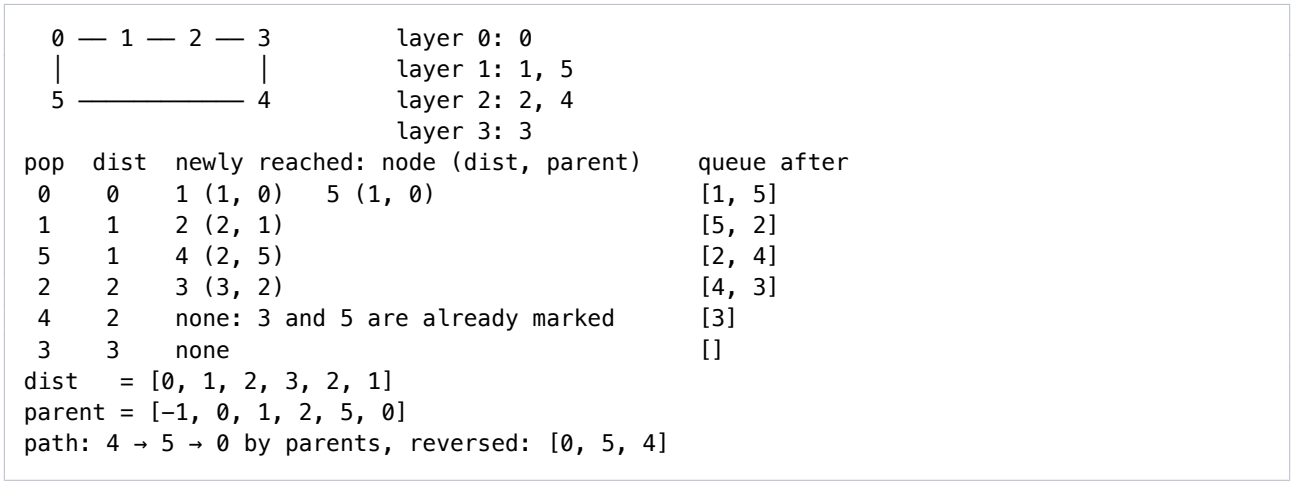
39. BFS Shortest Path

Unweighted Graphs, Layer by Layer · BFS · Tier 1

Given an unweighted graph, find a shortest path from a source node to a target node, where the length of a path is its number of edges. On six nodes with the undirected edges `0–1`, `1–2`, `2–3`, `3–4`, `0–5` and `5–4`, the shortest path from `0` to `4` is `[0, 5, 4]`, two edges, while the long way round, `0–1–2–3–4`, takes four.

The brute force lets a depth-first search enumerate every simple path and keeps the shortest, and the number of simple paths can grow exponentially with the graph. Breadth-first search never compares routes, because it explores in order of distance. The queue is first in, first out, so every node at distance `d` leaves it before any node at distance `d + 1`, and a node is first discovered by a neighbor from the lowest possible layer: the distance it receives at that moment is final. Two arrays hold the whole answer. `dist[v]` doubles as the visited mark, with `−1` for not reached yet, and `parent[v]` records which node discovered `v`. Following parents back from the target walks a shortest path in reverse.

Picture a stone dropped on the source. The ripples are the layers, `{0}`, then `{1, 5}`, then `{2, 4}`, then `{3}`, and each node belongs to the first ripple that touches it. Node `4` is touched at distance 2 by the ripple through `5`, while the wave along the top is still two steps away, and its parent pointer remembers the route it came by.




```
FUNCTION ShortestPath(n, edges, source, target)
    graph ← array of n empty lists
    FOR EACH (u, v) IN edges
        graph[u].append(v)                // undirected: both directions
        graph[v].append(u)
    dist ← array of n copies of −1        // −1 means not reached yet
    parent ← array of n copies of −1      // the node that first reached it
    dist[source] ← 0
    queue ← empty queue
    queue.enqueue(source)
    WHILE queue is not empty
        node ← queue.dequeue()
        FOR EACH neighbor IN graph[node]
            IF dist[neighbor] = −1        // first visit is a shortest one
                dist[neighbor] ← dist[node] + 1     // mark when enqueued
                parent[neighbor] ← node
```

```
                queue.enqueue(neighbor)
    IF dist[target] = -1
        RETURN empty list                   // target is in another component
    RETURN BuildPath(parent, target)

FUNCTION BuildPath(parent, target)
    path ← empty list
    node ← target
    WHILE node ≠ -1                         // only the source has no parent
        path.append(node)
        node ← parent[node]
    RETURN reverse(path)
```

Every node enters the queue at most once and every edge is examined once from each end, so the search costs `O(V + E)` time and `O(V)` space beyond the graph. Reach for it whenever a problem asks for the fewest steps, moves, hops or edits and every step costs the same; weighted edges need Dijkstra (#42). A grid is the same graph in disguise: in LC 1091 the nodes are the open cells, the neighbors are the eight cells around each one, and the loop runs unchanged on `(row, col)` pairs, though that problem counts the cells on the path, one more than its edges. When only one target matters, the loop may stop the moment the target is marked. The classic mistake is marking a node when it leaves the queue instead of when it enters. Until it leaves, every frontier node that touches it enqueues it again and rewrites its `dist`, so a longer route can overwrite the right answer: searching from `0` over the edges `0–1`, `0–2`, `1–3`, `2–4`, `3–4`, node `4` ends at distance 3 instead of 2.

Practice: LC 1091 Shortest Path in Binary Matrix · LC 1971 Find if Path Exists in Graph · LC 433 Minimum Genetic Mutation · LC 286 Walls and Gates

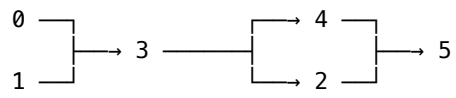
40. Kahn's Algorithm

Topological Sort by Peeling Indegree-Zero Nodes · Topological Sort · Tier 1

There are `n` courses numbered `0` to `n - 1`, and a prerequisite pair `[a, b]` means course `b` must be taken before course `a`. Return an order that takes every course, or an empty list if no order exists. With six courses and the pairs `[3, 0]`, `[3, 1]`, `[4, 3]`, `[2, 3]`, `[5, 4]`, `[5, 2]`, one valid order is `[0, 1, 3, 4, 2, 5]`.

Read each pair as an arrow `b → a`. An order is valid when every arrow points forward, which makes it a topological order of the graph. The brute force tries all `n!` orders and checks each one. Kahn's insight is that a graph without a cycle always has a free course, one with no arrow coming in. Take a free course, erase its outgoing arrows, and what remains is a smaller copy of the same problem. An indegree array makes the erasing cheap: `indegree[c]` counts the prerequisites of `c` not yet taken, taking a course subtracts one from each course it unlocks, and a course whose count reaches zero joins the queue of free courses. The queue holds exactly the courses that are free and not yet taken.

Picture the drawing eaten from the left. The courses with nothing pointing at them form its left edge; taking one erases its arrows, and any course that loses its last incoming arrow joins the new left edge. Course `3` sits behind two arrows, so it surfaces only after both `0` and `1` are taken, and `5` only after `4` and `2`. If the queue runs dry with courses left over, each of them still has an arrow coming in from another leftover course, and walking those arrows backward must eventually revisit a course. So a cycle exists, and `len(order) < num_courses` is the entire cycle test.




```
take   indegree of 0..5     queue     order
 -     [0, 0, 1, 2, 1, 2]   [0, 1]    []
 0     [0, 0, 1, 1, 1, 2]   [1]       [0]
 1     [0, 0, 1, 0, 1, 2]   [3]       [0, 1]
 3     [0, 0, 0, 0, 0, 2]   [4, 2]    [0, 1, 3]
 4     [0, 0, 0, 0, 0, 1]   [2]       [0, 1, 3, 4]
 2     [0, 0, 0, 0, 0, 0]   [5]       [0, 1, 3, 4, 2]
 5     [0, 0, 0, 0, 0, 0]   []        [0, 1, 3, 4, 2, 5]
```

```
FUNCTION FindOrder(num_courses, prerequisites)
    graph ← array of num_courses empty lists
    indegree ← array of num_courses zeros  // prerequisites not yet taken
    FOR EACH (course, before) IN prerequisites
        graph[before].append(course)       // [a, b] is the edge b → a
        indegree[course] ← indegree[course] + 1
    queue ← empty queue
    FOR course FROM 0 TO num_courses - 1
        IF indegree[course] = 0            // free to take right away
            queue.enqueue(course)
    order ← empty list
    WHILE queue is not empty
        course ← queue.dequeue()
        order.append(course)
        FOR EACH next_course IN graph[course]
            indegree[next_course] ← indegree[next_course] - 1    // one done
```

```
            IF indegree[next_course] = 0
                queue.enqueue(next_course)
    IF len(order) < num_courses            // the rest wait on each other
        RETURN empty list
    RETURN order
```

Each course enters the queue at most once and each arrow is erased at most once, so Kahn's algorithm runs in `O(V + E)` time and `O(V + E)` space for the graph. Reach for it whenever a problem speaks of prerequisites, dependencies, build steps or "must come before", or only asks whether an order exists at all, as Course Schedule (LC 207) does. It bends easily: processing the queue one level at a time counts the semesters in Parallel Courses (LC 1136), and in Alien Dictionary (#83) each arrow comes from the first differing letter of two adjacent words. Topological Sort with DFS (#82) builds an order from the other end. The classic mistake is reversing the pair: `[a, b]` is the arrow `b → a`, and building `a → b` instead returns an order that breaks every prerequisite.

Practice: LC 207 Course Schedule · LC 210 Course Schedule II · LC 269 Alien Dictionary · LC 1136 Parallel Courses

41. Union-Find

Path Compression + Union by Size · Connected Components · Tier 1

Keep `n` nodes in disjoint groups under two operations: `union(a, b)` merges the groups of `a` and `b`, and `find(x)` returns the representative of the group of `x`, so two nodes are connected exactly when they share a representative. Feed it the edges `[0, 1]`, `[2, 3]`, `[0, 2]`, `[3, 1]` on five nodes. The first three each merge two groups; the fourth joins two nodes that are already connected, closing the cycle `0–1–3–2–0`, so it is redundant, and two groups remain, `{0, 1, 2, 3}` and `{4}`.

The brute force stores a group label per node and relabels a whole group on every merge, `O(n)` per union. Union-find stores each group as a tree of parent pointers instead. Every node points toward its root, and the root points at itself and names the group; `find` climbs to the root, and `union` hangs one root under the other with a single pointer write. Two rules keep the trees flat. Union by size hangs the smaller tree under the root of the bigger one, so a node sinks one level deeper only when its tree at least doubles, and no tree grows taller than `log2 n`. Path compression makes `find` repoint every node it climbs past straight at the root, so the next climb takes one step.

Picture a forest of small trees that only ever merge. In the trace, the last union finds `3` two levels below root `0`, and the recursion flattens that path on its way back. The union then returns `false`, because both ends already share root `0`: that `false` is the redundant edge of LC 684, and `count`, which drops by one on every successful union, is the number of components.

```
edge     roots   what happens                       parent            count
 –                                                  [0, 1, 2, 3, 4]     5
[0, 1]   0, 1    1 hangs under 0, size[0] = 2       [0, 0, 2, 3, 4]     4
[2, 3]   2, 3    3 hangs under 2, size[2] = 2       [0, 0, 2, 2, 4]     3
[0, 2]   0, 2    tie: 2 hangs under 0, size 4       [0, 0, 0, 2, 4]     2
[3, 1]   0, 0    same root: false, redundant        [0, 0, 0, 0, 4]     2

the find(3) inside the last union:
       0                        0
      / \                      / | \
     1   2        ──→         1  2  3         4 stays alone
         |
         3
```

```
CLASS UnionFind
    FUNCTION Init(n)
        self.parent ← [0, 1, …, n − 1]     // every node starts as its own root
        self.size ← array of n copies of 1 // tree sizes, read only at roots
        self.count ← n                     // number of separate groups

    FUNCTION Find(x)
        IF self.parent[x] ≠ x
            self.parent[x] ← self.Find(self.parent[x])    // path compression
        RETURN self.parent[x]

    FUNCTION Union(a, b)
        root_a ← self.Find(a)
        root_b ← self.Find(b)
        IF root_a = root_b
            RETURN false                   // already connected: a cycle edge
```

```
        IF self.size[root_a] < self.size[root_b]
            swap(root_a, root_b)           // root_a is the bigger tree
        self.parent[root_b] ← root_a
        self.size[root_a] ← self.size[root_a] + self.size[root_b]
        self.count ← self.count − 1
        RETURN true

FUNCTION CountComponents(n, edges)
    uf ← UnionFind(n)
    FOR EACH (a, b) IN edges
        uf.Union(a, b)
    RETURN uf.count

FUNCTION FindRedundantConnection(edges)
    uf ← UnionFind(len(edges) + 1)         // nodes are 1..n; slot 0 unused
    FOR EACH (a, b) IN edges
        IF NOT uf.Union(a, b)              // first edge inside one group
            RETURN [a, b]
    RETURN empty list
```

With both rules, `m` operations on `n` nodes cost `O(m · alpha(n))`, where `alpha` is the inverse Ackermann function, at most 4 for any `n` a computer can hold: constant time in practice, and `O(n)` space. Reach for union-find when edges or equivalences arrive one at a time and the questions are "are these two connected?" and "how many groups are there now?": redundant connections, accounts that share an email, islands that appear one cell at a time, and the cycle test inside Kruskal's minimum spanning tree (#118). A flood fill (#84) answers the same questions for a graph given all at once but must search again after every new edge; union-find absorbs each edge in near-constant time, though it can never split a group. The classic mistake is comparing parents instead of roots: before the last union `parent[3]` is 2 and `parent[1]` is `0`, so the redundant edge `[3, 1]` would pass for a new connection.

Practice: LC 684 Redundant Connection · LC 547 Number of Provinces · LC 323 Number of Connected Components in an Undirected Graph · LC 721 Accounts Merge

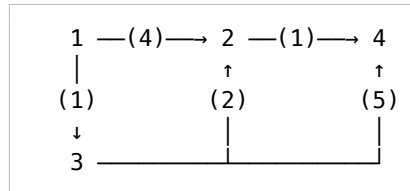
42. Dijkstra's Algorithm

Shortest Paths with Non-Negative Weights · Shortest Path · Tier 1

A signal leaves node `k` of a network with nodes `1` to `n`, and each directed edge `[u, v, w]` takes `w ≥ 0` units of time to cross. Return the time until every node has the signal, or `−1` if some node never gets it. Each node hears the signal at its shortest distance from `k`, so the answer is the largest of those distances. With `k = 1` and the edges `1→2 (4)`, `1→3 (1)`, `3→2 (2)`, `2→4 (1)` and `3→4 (5)`, the distances to nodes `1` to `4` are `0, 3, 1, 4`, and the answer is `4`.

Relaxing every edge again and again until nothing improves works, and it is Bellman-Ford (#117), at `O(V · E)`. Dijkstra relaxes each edge once, by settling nodes in order of distance, the way BFS (#39) settles them in order of hop count, with a min-heap in place of the queue. The heap holds candidate distances `(d, node)`, and the first entry it pops for a node is final. Any other route to that node must leave the settled region somewhere, and by the time it does it already costs at least `d`, since every node just outside the region waits in the heap at a distance no smaller than `d`; non-negative weights can only add to that. Settling a node relaxes its edges: when `d + w` beats the best known distance to a neighbor, the neighbor's distance drops and a new entry goes on the heap. A plain binary heap cannot lower an entry in place, so the old entry stays behind, and when it surfaces later its `d` is larger than `dist[node]` and it is skipped. That is lazy deletion, and its whole cost is a heap that can hold one entry per edge.

Picture a frontier that only ever moves outward, always at its nearest point. Node 2 first hears of the direct route at distance 4, then the detour through 3 lowers it to 3 and the entry `(4, 2)` goes stale. A negative edge is exactly what breaks this picture, because a step outside the frontier can now lead back to a shorter route. With edges `1→2 (2)`, `1→3 (3)`, `3→2 (−2)` and `2→4 (1)`, node 2 is settled at 2 and hands 3 on to node 4 before the route through 3 shows that 2 is really at 1 and 4 at 2. A version that never reopens a settled node answers 3 for node 4. The lazy version below reopens 2 and recovers, but nodes can then be popped many times, the time bound is gone, and a negative cycle keeps improving forever: negative weights need Bellman-Ford.



pop	what happens	dist of 1..4	heap after, sorted
(0, 1)	settle 1: 2 ← 4, 3 ← 1	[0, 4, 1, ∞]	(1,3) (4,2)
(1, 3)	settle 3: 2 ← 3, 4 ← 6	[0, 3, 1, 6]	(3,2) (4,2) (6,4)
(3, 2)	settle 2: 4 ← 4	[0, 3, 1, 4]	(4,2) (4,4) (6,4)
(4, 2)	stale: 4 > dist[2] = 3, skip	[0, 3, 1, 4]	(4,4) (6,4)
(4, 4)	settle 4: no edges out	[0, 3, 1, 4]	(6,4)
(6, 4)	stale: 6 > dist[4] = 4, skip	[0, 3, 1, 4]	empty

answer: the largest distance, 4

```
FUNCTION Dijkstra(graph, source)
    dist ← array of len(graph) copies of ∞
    dist[source] ← 0
    heap ← empty min-heap
    heap.push((0, source))                // (distance so far, node)
    WHILE heap is not empty
        (d, node) ← heap.pop()
```

```
        IF d > dist[node]
            CONTINUE                        // stale entry: node settled closer
        FOR EACH (neighbor, weight) IN graph[node]
            IF d + weight < dist[neighbor]          // relax: a shorter route
                dist[neighbor] ← d + weight
                heap.push((dist[neighbor], neighbor))
    RETURN dist

FUNCTION NetworkDelayTime(times, n, k)
    graph ← array of n + 1 empty lists      // nodes are 1..n; slot 0 unused
    FOR EACH (u, v, w) IN times
        graph[u].append((v, w))
    dist ← Dijkstra(graph, k)
    slowest ← 0
    FOR node FROM 1 TO n
        IF dist[node] = ∞
            RETURN -1                       // this node never gets the signal
        slowest ← max(slowest, dist[node])
    RETURN slowest
```

Each node is settled once, so each edge pushes at most one entry; at most `E + 1` pushes and pops at `O(log E) = O(log V)` each give `O((V + E) log V)` time and `O(V + E)` space. Reach for Dijkstra when edges carry non-negative costs, times or distances and the question is the cheapest route. The variants change what a route costs: Path With Minimum Effort minimizes the largest single step, Path with Maximum Probability multiplies probabilities and pops the largest first, and Number of Ways to Arrive at Destination carries a count of shortest routes beside each distance. The classic mistake is borrowing BFS's habit of finalizing a node when it is first pushed: here node 2 would keep distance 4 instead of 3, and node 4 would get 6.

Practice: LC 743 Network Delay Time · LC 1514 Path with Maximum Probability · LC 1631 Path With Minimum Effort · LC 1976 Number of Ways to Arrive at Destination

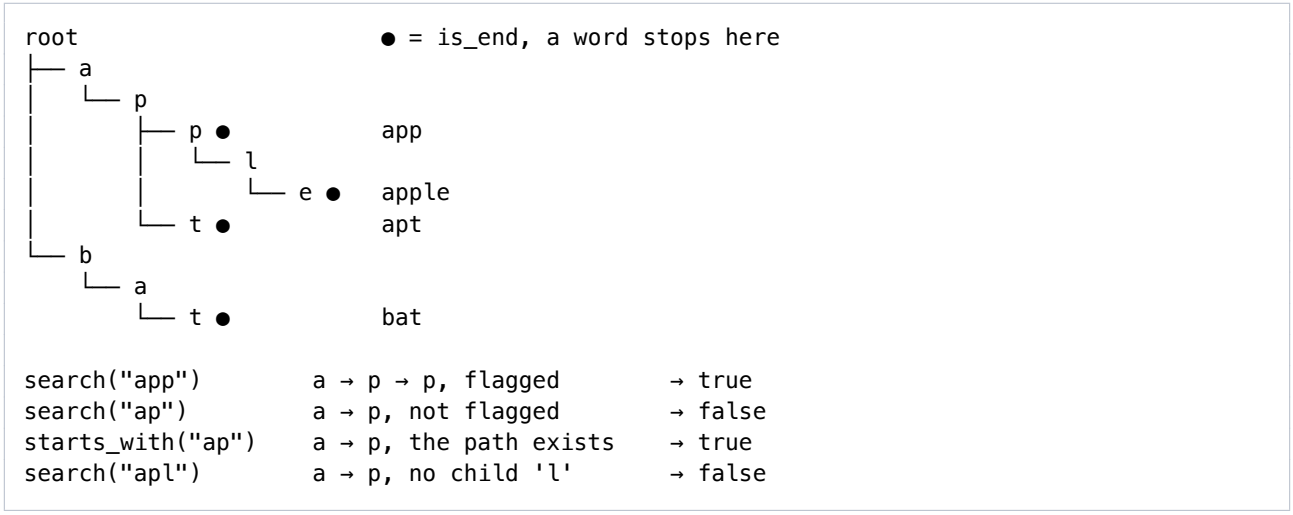
43. Trie

Insert, Search, Starts With · Tries · Tier 1

Build a prefix tree with three operations: `insert(word)`; `search(word)`, which asks whether that exact word was inserted; and `starts_with(prefix)`, which asks whether any inserted word begins with the prefix. After inserting `apple`, `app`, `apt` and `bat`, `search("ap")` is false but `starts_with("ap")` is true.

A hash set of the words answers `search` in `O(L)` for a word of length L, but a prefix question would have to compare against all n words, `O(n · L)`. A trie stores the words letter by letter along paths from the root, so words that share a prefix share the path that spells it: `app`, `apple` and `apt` all hang below the single path `a → p`. Each node holds a map from letter to child and one flag, `is_end`, that marks where an inserted word stops. Inserting walks down the letters and creates any missing child on the way; both queries walk down the same way and give up the moment a letter has no child.

The path is the prefix, and the subtree below a node holds every word that starts with it. That makes `starts_with` a pure walk: if the letters can be followed, some inserted word starts with them, because every node lies on the path of a word that was inserted. `search` asks one thing more, that the walk ends on a flagged node, and that flag is the whole difference between a prefix and a word.




```
CLASS TrieNode
    FUNCTION Init()
        self.children ← empty map            // letter → TrieNode
        self.is_end ← false                  // true if a word ends here

CLASS Trie
    FUNCTION Init()
        self.root ← new TrieNode()

    FUNCTION Insert(word)
        node ← self.root
        FOR EACH ch IN word
            IF ch NOT IN node.children
                node.children[ch] ← new TrieNode()
            node ← node.children[ch]
        node.is_end ← true                   // the word stops here
```

```
    FUNCTION Walk(prefix)
        node ← self.root
        FOR EACH ch IN prefix
            IF ch NOT IN node.children
                RETURN null                  // the path breaks off
            node ← node.children[ch]
        RETURN node

    FUNCTION Search(word)
        node ← self.Walk(word)
        RETURN node ≠ null AND node.is_end

    FUNCTION StartsWith(prefix)
        RETURN self.Walk(prefix) ≠ null
```

Each operation touches one node per letter, so all three run in `O(L)` no matter how many words are stored, and the trie holds at most one node per inserted letter, `O(total letters)` space. Reach for a trie when a problem asks many questions about the prefixes of a fixed set of strings. Autocomplete (#97) walks to the prefix node, then runs a DFS below it in alphabetical order to collect suggestions (LC 1268); Replace Words swaps each word for the shortest root met on its walk down (LC 648); a grid search for many words at once follows the trie instead of each word in turn (#85). A wildcard `.` that matches any letter (LC 211) turns the walk into a DFS that tries every child at each dot. The classic mistake is a `search` that returns true as soon as the path exists, which finds `ap` in a trie holding only `apple`.

Practice: LC 208 Implement Trie (Prefix Tree) · LC 211 Design Add and Search Words Data Structure · LC 1268 Search Suggestions System · LC 648 Replace Words

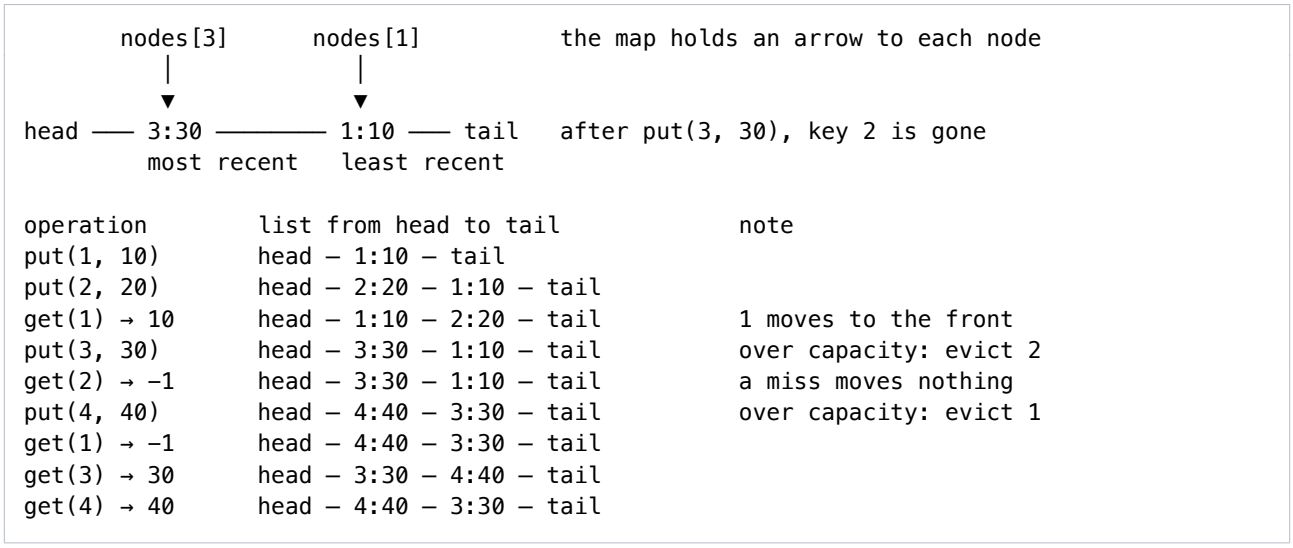
44. LRU Cache

Hash Map + Doubly Linked List · Design · Tier 1

Design a cache with a fixed capacity. `get(key)` returns the value or -1, `put(key, value)` inserts or updates, and when the cache grows past its capacity it evicts the least recently used key, the one whose last `get` or `put` is oldest. Both operations must run in `O(1)`. With capacity 2, after `put(1, 10)` and `put(2, 20)`, `get(1)` returns 10 and makes key 1 the most recent, so `put(3, 30)` evicts key 2 and `get(2)` returns -1.

The brute force keeps the keys in a list ordered by recency, and every operation scans the list to find a key and shift it to the front, `O(n)` each time. The cache needs two abilities at once. Finding a key in `O(1)` is a hash map's job. Keeping the keys in order of recency, moving any one of them to the front and dropping the oldest, each in `O(1)`, is a doubly linked list's job, because a node that knows both of its neighbors can unlink itself without a search. So use both. The list orders the nodes from the most recent, next to `head`, to the least recent, next to `tail`, and the map sends each key straight to its node: its values are arrows into the chain.

Every use of a key is the same two moves: unlink its node and splice it back in right after `head`. Eviction unlinks the node just before `tail` and deletes its key from the map, which is why a node stores its key as well as its value. The two sentinels sit at the ends forever, so the list is never empty, every real node has two neighbors, and the unlink and splice code needs no special cases.




```
CLASS Node
    FUNCTION Init(key, value)
        self.key ← key                  // kept so eviction can clean the map
        self.value ← value
        self.prev ← null
        self.next ← null

CLASS LRUCache
    FUNCTION Init(capacity)
        self.capacity ← capacity
        self.nodes ← empty map          // key → node
        self.head ← Node(0, 0)          // sentinel: head.next is most recent
        self.tail ← Node(0, 0)          // sentinel: tail.prev is least recent
```

```
        self.head.next ← self.tail
        self.tail.prev ← self.head

    FUNCTION Remove(node)
        node.prev.next ← node.next
        node.next.prev ← node.prev

    FUNCTION AddFront(node)
        node.prev ← self.head
        node.next ← self.head.next
        self.head.next.prev ← node
        self.head.next ← node

    FUNCTION Get(key)
        IF key NOT IN self.nodes
            RETURN -1
        node ← self.nodes[key]
        self.Remove(node)               // a read counts as a use
        self.AddFront(node)
        RETURN node.value

    FUNCTION Put(key, value)
        IF key IN self.nodes
            node ← self.nodes[key]
            node.value ← value
            self.Remove(node)           // it moves to the front below
        ELSE
            node ← Node(key, value)
            self.nodes[key] ← node
        self.AddFront(node)
        IF len(self.nodes) > self.capacity
            lru ← self.tail.prev        // least recently used
            self.Remove(lru)
            self.nodes.remove(lru.key)
```

Each operation is one map lookup plus a fixed number of pointer changes, so `get` and `put` run in `O(1)`, and the cache holds at most `capacity` nodes. Reach for this pairing whenever a problem needs fast lookup plus an order that changes on every access: caches, recently used lists, an eviction policy inside a larger design. Python's `OrderedDict` is this very structure, a hash map threaded with a doubly linked list, and `move_to_end` with `popitem(last=False)` gives a dozen-line solution, but interviewers usually want the list built by hand. LFU Cache (#93) extends the idea to one list per use count plus the smallest count in play. The classic mistake is creating a second node when `put` updates an existing key: the map points at the new node, the old one stays in the list, and when that ghost reaches the tail its eviction deletes the live key. Forgetting that `get` also moves its key to the front is the other one.

Practice: LC 146 LRU Cache · LC 460 LFU Cache · LC 707 Design Linked List

45. Design Hit Counter

Circular Buckets over the Last 300 Seconds · Design · Tier 1

Design a counter of recent traffic: `hit(timestamp)` records a hit at a time given in whole seconds, and `get_hits(timestamp)` returns how many hits arrived in the past 300 seconds, at times `s` with `timestamp − 300 < s ≤ timestamp`. Calls come in time order, and many hits may share a second. After hits at seconds 1, 2, 2, 3 and 300, `get_hits(300)` is 5 and `get_hits(301)` is 4: the hit at second 1 is now exactly 300 seconds old and has left the window.

The first design to come to mind is the sliding log of the rate limiter (#138), a queue of every hit's time with the expired times dropped from the front. It is `O(1)` amortized per call, but its memory grows with the traffic: a million hits in one second are a million entries, which is LeetCode's follow-up question. The window never needs more than 300 numbers, one count per second, so keep 300 buckets bent into a ring. Second `t` lives in bucket `t mod 300`; the 300 seconds of any window fall into 300 different buckets, and second `t + 300` returns to the bucket of second `t` at the very moment `t` leaves the window.

A reused bucket must never mix two seconds, so each bucket stores the second it counts next to its count. A hit at second `t` goes to bucket `t mod 300`: if the bucket's time is `t` its count goes up by one, and otherwise the bucket is still empty or holds a second at least 300 seconds old, outside every future window, and is reset to `(t, 0)` first. `get_hits` walks all 300 buckets and adds the counts whose age, `timestamp − time`, is under 300; a stale bucket that no hit has reclaimed fails the same test, so nothing is ever cleaned up eagerly. The trace follows the four buckets this sequence touches.

```
bucket = t mod 300 holds (time, count); buckets 4 to 299 stay (0, 0)
call            bucket 0    bucket 1    bucket 2    bucket 3    returns
hit(1)          (0, 0)      (1, 1)      (0, 0)      (0, 0)
hit(2), hit(2)  (0, 0)      (1, 1)      (2, 2)      (0, 0)
hit(3)          (0, 0)      (1, 1)      (2, 2)      (3, 1)
get_hits(4)       +0          +1          +2          +1         4
hit(300)        (300, 1)    (1, 1)      (2, 2)      (3, 1)
get_hits(300)     +1          +1          +2          +1         5
get_hits(301)     +1        ✗ age 300     +2          +1         4
hit(301)        (300, 1)    (301, 1)    (2, 2)      (3, 1)      bucket 1 reset
get_hits(301)     +1          +1          +2          +1         5
get_hits(600)   ✗ age 300     +1        ✗ age 598   ✗ age 597    1
get_hits(601)   ✗ age 301   ✗ age 300   ✗ age 599   ✗ age 598    0
+c: t − time < 300, count c added     ✗: 300 or more seconds old, skipped
```

```
CLASS HitCounter
    FUNCTION Init()
        self.window ← 300                       // seconds get_hits looks back
        self.times ← array of self.window zeros     // bucket i: its second
        self.counts ← array of self.window zeros    // bucket i: hits in it

    FUNCTION Hit(timestamp)
        slot ← timestamp mod self.window
        IF self.times[slot] ≠ timestamp         // empty, or 300+ seconds old
            self.times[slot] ← timestamp
            self.counts[slot] ← 0
        self.counts[slot] ← self.counts[slot] + 1
```

```
    FUNCTION GetHits(timestamp)
        total ← 0
        FOR slot FROM 0 TO self.window − 1
            IF timestamp − self.times[slot] < self.window    // in the window
                total ← total + self.counts[slot]
        RETURN total
```

`hit` runs in `O(1)` and `get_hits` in `O(300)`, a constant for a fixed window, and the memory is 600 numbers whatever the traffic; a window of `W` seconds needs `O(W)` per query and `O(W)` memory. The price is resolution, and coarser buckets trade more of it for longer windows: 60 one-minute buckets count the last hour with an edge accurate to a minute, the way monitoring systems keep their counters. Counts add, so a fleet of servers can each keep a ring and sum them at query time. Reach for buckets when the question is how many events fell in the last N units of time under heavy traffic; Number of Recent Calls (LC 933) keeps the queue, because its times strictly increase and its queue never holds more than 3001 entries, one per millisecond of the window. The classic mistake is reusing a bucket without zeroing it: the hit at second 301 lands on the leftover hit from second 1, and the last `get_hits(301)` returns 6 instead of 5. Its twin is ≤ in the age test, which still counts the hit at second 1 in the first `get_hits(301)` and returns 5 instead of 4.

Practice: LC 362 Design Hit Counter · LC 359 Logger Rate Limiter · LC 933 Number of Recent Calls

46. Design Twitter

Merge the Followees' Feeds with a Heap · Design · Tier 1

Design a small Twitter with four calls: `post_tweet(user, tweet_id)`, `follow(follower, followee)`, `unfollow(follower, followee)` and `get_news_feed(user)`, which returns the ids of the 10 most recent tweets posted by the user or by anyone the user follows, newest first. Users always see their own tweets and cannot unfollow themselves. If users 1, 2, 3, 2, 1 and 3 post tweets 11, 21, 31, 22, 12 and 32 in that order, the first digit naming the author, and user 1 follows users 2 and 3, the feed of user 1 is `[32, 12, 22, 31, 21, 11]`; after `unfollow(1, 3)` it is `[12, 22, 21, 11]`.

The brute force keeps one global list of every tweet and walks it from the newest end, keeping the tweets by the user or a followee until it has ten, so one small feed can read the history of the whole platform. Instead, give every user a private list of `(time, tweet_id)` pairs, stamped by one global clock that ticks once per tweet. Appending keeps each list sorted by time, so a feed is the newest end of k sorted lists merged together: the k-way merge of #20, run backward in time. A max-heap holds one entry per source, the newest tweet not yet taken from it. Each pop yields the newest tweet left anywhere, and the popped source pushes its next older tweet in its place. Ten pops fill the feed.

Picture the users as rows on a shared time axis. The feed reads the axis from right to left, and the heap keeps one finger on each row; every pop moves one finger a step to the left, and a row whose finger falls off its oldest tweet drops out of the merge.

```
time      0     1     2     3     4     5     tweets[user], oldest first
user 1    11                      12          [(0, 11), (4, 12)]
user 2          21          22                [(1, 21), (3, 22)]
user 3                31                32    [(2, 31), (5, 32)]

get_news_feed(1), following 2 and 3: heap starts (5, 32) (4, 12) (3, 22)
pop (time, id)   push its older tweet    heap after, newest first
(5, 32)          (2, 31) of user 3       (4, 12) (3, 22) (2, 31)
(4, 12)          (0, 11) of user 1       (3, 22) (2, 31) (0, 11)
(3, 22)          (1, 21) of user 2       (2, 31) (1, 21) (0, 11)
(2, 31)          user 3 is done          (1, 21) (0, 11)
(1, 21)          user 2 is done          (0, 11)
(0, 11)          user 1 is done          empty
feed = the pops in order = [32, 12, 22, 31, 21, 11]
after unfollow(1, 3) only rows 1 and 2 merge: [12, 22, 21, 11]
```

```
CLASS Twitter
    FUNCTION Init()
        self.clock ← 0                      // stamps tweets: higher is newer
        self.tweets ← empty map             // user → list of (time, tweet_id)
        self.following ← empty map          // user → set of users they follow

    FUNCTION PostTweet(user, tweet_id)
        IF user NOT IN self.tweets
            self.tweets[user] ← empty list
        self.tweets[user].append((self.clock, tweet_id))     // oldest first
        self.clock ← self.clock + 1

    FUNCTION Follow(follower, followee)
        IF follower = followee
```

```
            RETURN                          // your own tweets always count
        IF follower NOT IN self.following
            self.following[follower] ← empty set
        self.following[follower].add(followee)

    FUNCTION Unfollow(follower, followee)
        IF follower IN self.following
            self.following[follower].remove(followee)    // no-op if absent

    FUNCTION GetNewsFeed(user)
        sources ← [user]
        IF user IN self.following
            FOR EACH followee IN self.following[user]
                sources.append(followee)
        heap ← empty max-heap        // of (time, tweet_id, source, index)
        FOR EACH source IN sources
            IF source IN self.tweets
                index ← len(self.tweets[source]) - 1        // its newest tweet
                (time, tweet_id) ← self.tweets[source][index]
                heap.push((time, tweet_id, source, index))
        feed ← empty list
        WHILE len(heap) > 0 AND len(feed) < 10
            (time, tweet_id, source, index) ← heap.pop()
            feed.append(tweet_id)           // the newest tweet not yet taken
            IF index > 0                    // that source has an older tweet
                (time, older_id) ← self.tweets[source][index - 1]
                heap.push((time, older_id, source, index - 1))
        RETURN feed
```

`post_tweet`, `follow` and `unfollow` cost `O(1)`. `get_news_feed` costs `O(k log k)` to seed the heap with the newest tweet of each of its `k` sources (`O(k)` with heapify), plus `O(log k)` for each of at most 10 pops, so it reads at most `k + 10` tweets however long the histories are. Memory is one pair per tweet and one set entry per follow. Reach for this merge whenever a result is assembled from many sources that are each already sorted and only its first few items are wanted: feeds, merged logs, the k smallest pair sums. The global clock is what makes “most recent” well defined, and the classic mistake is ordering by tweet id instead: ids are labels, not times, and nothing promises that a later tweet has a larger id. The other slip is the user’s own tweets, either forgotten or merged twice after a self-follow; here `follow` ignores a self-follow and the user is always the first source. In system-design terms this read-time merge is fan-out on read. The usual design at scale fans out on write instead, pushing each new tweet into a cached timeline per follower so that a read is one lookup, and merges at read time only for celebrities, whose millions of followers would turn each post into millions of writes.

Practice: LC 355 Design Twitter · LC 23 Merge k Sorted Lists · LC 378 Kth Smallest Element in a Sorted Matrix

47. Snapshot Array

Per-Index History + Binary Search · Design · Tier 1

Design an array of `length` zeros that can take snapshots: `set(index, val)` writes a value, `snap()` takes a snapshot and returns its id, counting up from 0, and `get(index, snap_id)` returns the value the index held when that snapshot was taken. With length 3, after `set(0, 5)` the first `snap()` returns 0; after `set(0, 6)` and `set(0, 8)`, `get(0, 0)` still returns 5.

The brute force copies the whole array on every `snap()`, which costs `O(length)` time per snapshot and `O(length × snaps)` memory, even when only a few cells changed since the last one. Store the changes instead. Each index keeps its own history, a list of `(snap_id, value)` pairs. `set` records the value under the id of the snapshot that is still open, the one the next `snap()` will return, and `snap()` does nothing but move that counter forward. Writes since the last snapshot carry an id that no query may ask for yet, so a late write can never leak into an older snapshot. A second write to the same index before the next snap rewrites the newest pair instead of appending, which keeps one pair per index per snapshot.

Snap ids only grow, so every history is sorted for free, and `get(index, snap_id)` wants the last pair at or before `snap_id`: binary search for the first pair after it, the upper bound of #14, and step one slot to the left. If no pair comes before it, the index was never written by then and still holds 0. Picture each index as a row of snapshot columns with a value wherever that snapshot changed it; a query looks at its column and walks left to the nearest value. It is the Time-Based Key-Value Store (#92) with one timeline per index and snapshot ids for times.

```
calls: set(0, 5)  snap() → 0  set(0, 6)  set(1, 4)  set(0, 8)  get(0, 0) → 5
       snap() → 1  snap() → 2  set(1, 9)  snap() → 3

snap id        0      1      2      3      pairs after the calls
index 0        5      8      ·      ·      [(0, 5), (1, 8)]    8 rewrote 6
index 1        ·      4      ·      9      [(1, 4), (3, 9)]
index 2        ·      ·      ·      ·      []

get(i, s) looks at row i, column s, and walks left to the nearest value
get(0, 2): pairs [(0, 5), (1, 8)], left = 0, right = 2
  mid = 1: snap 1 ≤ 2 → left = 2; the answer is pairs[1] → 8
get(1, 2): pairs [(1, 4), (3, 9)], left = 0, right = 2
  mid = 1: snap 3 > 2 → right = 1;  mid = 0: snap 1 ≤ 2 → left = 1 → 4
get(1, 0): mid = 1: snap 3 > 0 → right = 1;  mid = 0: snap 1 > 0 → right = 0
  left = 0: index 1 was not written by snapshot 0 → 0
```

```
CLASS SnapshotArray
    FUNCTION Init(length)
        self.history ← empty list         // index → list of (snap_id, value)
        FOR i FROM 0 TO length − 1
            self.history.append(empty list)
        self.snap_id ← 0                  // the id the next Snap returns

    FUNCTION Set(index, val)
        pairs ← self.history[index]
        last ← len(pairs) − 1             // the newest pair, if any
        IF last ≥ 0 AND pairs[last][0] = self.snap_id
            pairs[last] ← (self.snap_id, val)    // same snapshot: rewrite it
        ELSE
```

```
            pairs.append((self.snap_id, val))

    FUNCTION Snap()
        self.snap_id ← self.snap_id + 1
        RETURN self.snap_id - 1

    FUNCTION Get(index, snap_id)
        pairs ← self.history[index]
        left ← 0
        right ← len(pairs)
        WHILE left < right                // find the first pair after snap_id
            mid ← floor((left + right) / 2)
            IF pairs[mid][0] ≤ snap_id
                left ← mid + 1            // mid was written by then
            ELSE
                right ← mid               // mid is later: maybe the first
        IF left = 0
            RETURN 0                      // not written by that snapshot
        RETURN pairs[left - 1][1]
```

`set` and `snap` cost `O(1)`, and `get` costs `O(log w)` for an index written in `w` different snapshots. Memory is `O(length + writes)`: it grows with the writes, never with snapshots times length, so a million snapshots of an untouched array cost one counter. Reach for per-index histories when a problem asks for versions, snapshots or a value as of some moment, and copying everything on each version would be too slow. Databases run the same idea at scale as multiversion concurrency control: every write adds a version stamped with a transaction id, and a reader sees the newest version at or before its own snapshot. The classic mistake is a lower-bound search for the first pair whose id is at least `snap_id`: when the index was not written in that very snapshot, it lands on a later write, and `get(1, 2)` above returns 9, written after snapshot 2, instead of 4. The other one is `snap()` returning the incremented counter, which shifts every id by one.

Practice: LC 1146 Snapshot Array · LC 981 Time Based Key-Value Store · LC 911 Online Election

48. Design Tic-Tac-Toe

Row, Column and Diagonal Counters · Design · Tier 1

Design the referee for tic-tac-toe on an n × n board. Two players alternate valid moves on empty cells, and `move(row, col, player)` returns the player who has just filled a whole row, column or diagonal with their marks, or 0 if nobody has. On a 3 × 3 board the moves (0, 0) by player 1, (0, 2) by 2, (2, 2) by 1, (1, 1) by 2, (2, 0) by 1 and (1, 0) by 2 return 0, and the seventh, player 1 at (2, 1), returns 1: the bottom row is full of X.

The brute force stores the board and after each move scans the new mark's row, column and diagonals, `O(n)` per move and `O(n²)` memory. But a line wins only when all n of its cells hold one player's mark, and that question fits in one number per line, a signed counter. Player 1 adds +1 and player 2 adds -1 to every line through the cell they take, so each counter is the difference between the two players' marks on its line. A line of n cells reaches n only when every cell holds player 1's mark and −n only when every cell holds player 2's; once both players have marked a line it can reach neither, which is exactly the rule that a mixed line is dead.

A cell lies on one row and one column, on the main diagonal when `row = col`, and on the anti-diagonal when `row + col = n − 1`. Only those two diagonals have n cells, so the shorter diagonals that N-Queens (#113) names by `r − c` and `r + c` can never win here. A move can complete only lines through its own cell, and only for the player who made it, so checking its row, its column and the two diagonals is the whole win check.

```
player 1 = X adds +1, player 2 = O adds −1 to each line through the cell
move (r, c, p)    rows          cols          diagonal  anti  returns
(0, 0, 1)         [1, 0, 0]     [1, 0, 0]         1      0      0
(0, 2, 2)         [0, 0, 0]     [1, 0, −1]        1     −1      0
(2, 2, 1)         [0, 0, 1]     [1, 0, 0]         2     −1      0
(1, 1, 2)         [0, −1, 1]    [1, −1, 0]        1     −2      0
(2, 0, 1)         [0, −1, 2]    [2, −1, 0]        1     −1      0
(1, 0, 2)         [0, −2, 2]    [1, −1, 0]        1     −1      0
(2, 1, 1)         [0, −2, 3]    [1, 0, 0]         1     −1      1

          col 0  col 1  col 2    rows[r]
row 0       X      .      O         0     X and O: dead, never 3 or −3
row 1       O      O      .        −2
row 2       X      X      X         3     = n: player 1 wins
cols[c]     1      0      0      diagonal X O X = 1, anti O O X = −1
```

```
CLASS TicTacToe
    FUNCTION Init(n)
        self.n ← n
        self.rows ← array of n zeros      // player 1 adds +1, player 2 adds −1
        self.cols ← array of n zeros
        self.diagonal ← 0                 // cells with row = col
        self.anti_diagonal ← 0            // cells with row + col = n − 1

    FUNCTION Move(row, col, player)
        sign ← 1
        IF player = 2
            sign ← −1
        self.rows[row] ← self.rows[row] + sign
```

```
        self.cols[col] ← self.cols[col] + sign
        IF row = col
            self.diagonal ← self.diagonal + sign
        IF row + col = self.n − 1         // not ELSE IF: the center is on both
            self.anti_diagonal ← self.anti_diagonal + sign
        IF abs(self.rows[row]) = self.n OR abs(self.cols[col]) = self.n
            RETURN player
        IF abs(self.diagonal) = self.n OR abs(self.anti_diagonal) = self.n
            RETURN player
        RETURN 0
```

Each move updates and tests at most four counters, so it costs `O(1)`, and the referee keeps `2n + 2` numbers instead of the `n²` cells of a board. Reach for signed counters whenever the question is whether a whole row, column or group has become uniform while cells only ever fill up. Find Winner on a Tic Tac Toe Game (LC 1275) replays a list of moves through the same counters and reports a draw when all nine cells fill without a winner, and Valid Tic-Tac-Toe State (LC 794) asks whether a board could arise in a real game, which comes down to the mark counts and who moved last. For k in a row on a bigger board, as in Gomoku, the counters stop working, because a winning run can sit anywhere along a longer line; walk outward from the new mark in the four directions instead, `O(k)` per move. The classic mistake is `ELSE IF` for the anti-diagonal. The center of an odd board lies on both diagonals, so when player 2 takes (0, 2), (1, 1) and (2, 0) the anti-diagonal counter misses the center, stops at -2, and the win goes unreported.

Practice: LC 348 Design Tic-Tac-Toe · LC 1275 Find Winner on a Tic Tac Toe Game · LC 794 Valid Tic-Tac-Toe State

49. Flatten Nested List Iterator

A Stack of List Positions · Design · Tier 1

Design an iterator over a nested list, in which every item is either an integer or another nested list: `next()` returns the next integer from left to right, and `has_next()` says whether any integer is left. Plain Python lists stand in for LeetCode's `NestedInteger`, so "the item is a list" plays the part of `not item.isInteger()`. For `[1, [4, [6]], [], 8]` the calls to `next()` return 1, 4, 6 and 8, and then `has_next()` is false.

The easy answer flattens everything up front with a recursive walk and serves the values from a list, which stores every integer and walks the whole input before the first call, even if the caller stops after one value. A lazy iterator pauses that walk between calls instead, and what a paused walk must remember is its place in every list it is inside, one position per level. Keep those places as a stack of frames `[list, index]`, the outermost list at the bottom, where `index` is the next item of that list still to visit. The stack is exactly the call stack the recursive walk would hold, made explicit so that it can stop and resume.

`has_next()` does all the work: it moves the top frame forward until the frame points at an integer. If the top frame has run off the end of its list, that list is used up, so pop it and carry on with the frame below. If it points at a sub-list, step the frame past the sub-list first and only then push a new frame for it, so that the parent resumes at the right item once the child is done. If it points at an integer, stop. Empty lists are entered and popped by the same loop, so they never surface. `next()` calls `has_next()` to settle on an integer, reads it and steps past it.

```
A = [1, B, D, 8]   B = [4, C]   C = [6]   D = []    a frame is (list, index)
call        returns  stack, top at right   steps inside has_next
start                (A, 0)
next()      1        (A, 1)
has_next()  true     (A, 2) (B, 0)         enter B
next()      4        (A, 2) (B, 1)
next()      6        (A, 2) (B, 2) (C, 1)  enter C
next()      8        (A, 4)                pop C, pop B, enter D, pop D
has_next()  false    empty                 pop A
next() runs has_next() first, then returns the integer on top and steps on
```

```
CLASS NestedIterator
    FUNCTION Init(nested)
        self.stack ← empty stack
        self.stack.push([nested, 0])      // frame: a list and its next index

    FUNCTION HasNext()
        WHILE len(self.stack) > 0
            frame ← self.stack.top()
            items ← frame[0]
            index ← frame[1]
            IF index = len(items)
                self.stack.pop()          // this list is used up
            ELSE IF items[index] is a list
                frame[1] ← index + 1      // step past the sub-list first
                self.stack.push([items[index], 0])    // then enter it
            ELSE
                RETURN true               // the top points at an integer
```

```
        RETURN false

    FUNCTION Next()
        self.HasNext()
        frame ← self.stack.top()
        value ← frame[0][frame[1]]
        frame[1] ← frame[1] + 1
        RETURN value
```

Each list is pushed once and popped once and each integer is returned once, so draining the iterator costs `O(N + L)` for `N` integers and `L` lists, `O(1)` amortized per call unless the lists far outnumber the integers, and the stack holds one frame per level of nesting, `O(depth)` memory. It is the pause-and-resume trick of the BST iterator (#77) and the frames of Decode String (#25). Signals: "design an iterator", "lazy", "flatten", or an input too large to copy or too deep to recurse. Two cousins come up as follow-ups. Peeking Iterator (LC 284) caches one value, so that `peek()` can show the next element without consuming it, and Zigzag Iterator (LC 281) keeps a queue of iterators, takes one value from the front one and rotates it to the back until it runs dry. The classic mistake is a `has_next()` that only checks that the stack is not empty: for `[[], [[]]]` it promises an integer that does not exist. The other is pushing the child frame without first stepping the parent past it; on the example the iterator then returns 1, 4 and 6, and 6 again forever, because every time `C` is used up its parent still points at it.

Practice: LC 341 Flatten Nested List Iterator · LC 284 Peeking Iterator · LC 281 Zigzag Iterator · LC 173 Binary Search Tree Iterator

50. Key-Value Store with Transactions

A Stack of Undo Logs · Design · Tier 1

Design a key-value store with `set(key, value)`, `get(key)` and `delete(key)`, plus transactions that nest. `begin()` opens a transaction, `rollback()` undoes every change made since the innermost open `begin` and closes that transaction, and `commit()` makes the changes of every open transaction permanent and closes them all. `get` returns `null` for an absent key, and `rollback` and `commit` return `OK`, or `NO TRANSACTION` when none is open. After `set(a, 10)`, `begin()`, `set(a, 20)`, `begin()` and `delete(a)`, the first rollback brings `a` back as `20` and the second as `10`.

The obvious design copies the whole dictionary at every `begin` and puts the copy back on rollback. It is correct, but it costs `O(n)` time and memory per `begin` for a store of `n` keys even when the transaction touches a single key, and every nested level pays again. A transaction only needs to remember what it changed. Keep one live dictionary that always holds the current state, and give each open transaction an undo log: a small map from every key that transaction changed to the value the key had before, with `null` for a key that did not exist. `begin` pushes an empty log onto a stack. Every `set` and `delete` first records the key in the innermost log and only then changes the live dictionary. Only the first change of a key within a level is recorded, because that is the value the level started with; later changes would save values the level itself wrote.

Rollback pops the innermost log and writes each saved value back, deleting the keys that were absent. Each key appears in a log once, so the order of the replay does not matter, and afterwards the live dictionary is exactly what it was at the matching `begin`, the state the outer logs were recorded against. Commit, with the semantics above, throws every log away, because the live dictionary already holds the committed state. Picture the logs as a stack of trays, one per open level, each holding the before-values of the keys its level touched; reads never look at the trays.

```
operation            data                logs, innermost last
set(a, 10)           {a: 10}             []
begin()              {a: 10}             [{}]
set(a, 20)           {a: 20}             [{a: 10}]
set(b, 30)           {a: 20, b: 30}      [{a: 10, b: null}]
begin()              {a: 20, b: 30}      [{a: 10, b: null}, {}]
delete(a)            {b: 30}             [{a: 10, b: null}, {a: 20}]
set(b, 40)           {b: 40}             [{a: 10, b: null}, {a: 20, b: 30}]
set(b, 45)           {b: 45}             [{a: 10, b: null}, {a: 20, b: 30}]
rollback() → OK      {a: 20, b: 30}      [{a: 10, b: null}]
rollback() → OK      {a: 10}             []
rollback() → NO TRANSACTION
begin(), set(a, 50)  {a: 50}             [{a: 10}]
begin(), set(c, 70)  {a: 50, c: 70}      [{a: 10}, {c: null}]
commit() → OK        {a: 50, c: 70}      []         both levels are permanent
rollback() → NO TRANSACTION
```

```
CLASS KeyValueStore
    FUNCTION Init()
        self.data ← empty map              // key → value: the live state
        self.logs ← empty stack            // one undo log per open level

    FUNCTION Record(key)
        IF len(self.logs) = 0
```

```
        RETURN                          // no transaction: nothing to undo
    log ← self.logs.top()               // the innermost transaction
    IF key NOT IN log                   // only the first change is kept
        log[key] ← self.Get(key)        // null: the key was absent

FUNCTION Set(key, value)
    self.Record(key)
    self.data[key] ← value

FUNCTION Get(key)
    IF key IN self.data
        RETURN self.data[key]
    RETURN null

FUNCTION Delete(key)
    IF key IN self.data
        self.Record(key)
        self.data.remove(key)

FUNCTION Begin()
    self.logs.push(empty map)

FUNCTION Rollback()
    IF len(self.logs) = 0
        RETURN "NO TRANSACTION"
    log ← self.logs.pop()               // undo the innermost level only
    FOR EACH key IN log
        IF log[key] = null              // absent before the level began
            IF key IN self.data
                self.data.remove(key)
        ELSE
            self.data[key] ← log[key]   // the value before the level
    RETURN "OK"

FUNCTION Commit()
    IF len(self.logs) = 0
        RETURN "NO TRANSACTION"
    self.logs ← empty stack             // nothing can be undone any more
    RETURN "OK"
```

`set`, `get`, `delete`, `begin` and `commit` cost `O(1)`, and `rollback` costs `O(k)` for the `k` keys its level changed; a log holds at most one entry per key, so memory follows the changes, never the size of the store. Since `null` marks an absent key, `null` is never stored as a value. The signals are undo, nested transactions, savepoints and "restore the state as of the last begin". Many databases roll back the same way, from undo records that hold the before-image of each changed row. The mirror design buffers each level's writes in its own map and applies them on commit: rollback becomes a single pop, but every read must search the levels from the innermost outward. If `commit` closed only the innermost level, its log would merge into the parent's, each key the parent lacks moving down so the parent can still undo it. Min Stack (#24) is a small cousin: every entry remembers the minimum below it, so a pop restores the old minimum for free. Snapshot Array (#47) keeps old versions instead of undo records. The classic mistake is recording a key on every change: after `set(b, 40)` and `set(b, 45)` the log would hold `40`, and the rollback would land on `40` instead of `30`. Forgetting to log `null` for a new key is the other one, since rollback then keeps a key the transaction created.

Practice: LC 1146 Snapshot Array · LC 981 Time Based Key-Value Store · LC 155 Min Stack

Tier 2 · Should Know Well

The next fifty deepen every topic of the first tier and add the classic harder variants: partition-based selection, patience sorting, the histogram stack, expression evaluation, the greedy exchange arguments, grid backtracking, BFS on implicit graphs, topological order by DFS, KMP, and a second wave of designs: LFU, expiring caches, autocomplete, calendars, the snake game and the URL shortener. They come up often enough that each one deserves a careful read and a run of its code.

51. Hash Map with Separate Chaining

Collisions Live in Buckets · Hash Map Internals · Tier 2

Design a hash map without a built-in dictionary: `put(key, value)` inserts a key or overwrites its value, `get(key)` returns the value or −1 when the key is missing, and `remove(key)` deletes the key. With 8 buckets and integers hashing to themselves, `put(1, 10)` and `put(9, 90)` send both keys to bucket 1, because `1 mod 8 = 9 mod 8 = 1`, and `get(9)` must still return `90`.

A plain list of pairs works, but every call scans all of it, `O(n)` per operation. A hash map splits that one long list into many short ones. It keeps an array of `capacity` buckets and sends each key to bucket `hash(key) mod capacity`, so an operation computes one index and touches only that bucket. Different keys can land in the same bucket; that is a collision, and separate chaining simply lets it happen: each bucket is a small list, a chain, of `[key, value]` pairs. The hash chooses the chain and comparing keys inside the chain chooses the pair, so a `put` of an existing key overwrites in place, a new key is appended, and `remove` unlinks its pair.

The load factor `size / capacity` is the average chain length. With a hash that spreads keys evenly, a lookup scans about that many pairs, so keeping the load factor at most 0.75 makes every operation `O(1)` on average. When a `put` pushes it past 0.75, the map doubles the bucket array and rehashes every pair, because the bucket index depends on the capacity: each old chain splits between bucket `b` and bucket `b + old capacity`.

```
8 buckets, bucket = key mod 8; put keys 1, 9, 17, 2, 10, 4 with value 10 × key
  bucket 0     ·
  bucket 1     1:10 → 9:90 → 17:170      1, 9 and 17 collide
  bucket 2     2:20 → 10:100             2 and 10 collide
  bucket 3     ·
  bucket 4     4:40
  bucket 5–7   ·                         load = 6 / 8 = 0.75, no resize yet

get(17): bucket 17 mod 8 = 1; compare keys 1 ✗, 9 ✗, 17 ✓ → 170

put(25): bucket 1 again; load 7 / 8 = 0.875 > 0.75 → 16 buckets, rehash
  bucket 1     1:10 → 17:170             the old chain 1 splits
  bucket 9     9:90 → 25:250             between buckets 1 and 9
  bucket 2     2:20                      and chain 2 between
  bucket 10    10:100                    buckets 2 and 10
  bucket 4     4:40                      every other bucket is empty
```

```
CLASS ChainedHashMap
    FUNCTION Init(capacity)
        self.capacity ← capacity
        self.size ← 0                          // number of keys stored
        self.buckets ← empty list
        FOR i FROM 0 TO capacity − 1
            self.buckets.append(empty list)    // one empty chain per bucket

    FUNCTION BucketOf(key)                     // the chain that holds key
        RETURN self.buckets[hash(key) mod self.capacity]

    FUNCTION Put(key, value)
        bucket ← self.BucketOf(key)
```

```
        FOR EACH pair IN bucket
            IF pair[0] = key                    // same key: overwrite the value
                pair[1] ← value
                RETURN
        bucket.append([key, value])             // new key: the chain grows
        self.size ← self.size + 1
        IF self.size / self.capacity > 0.75
            self.Resize()                       // load factor too high: grow

    FUNCTION Get(key)
        bucket ← self.BucketOf(key)
        FOR EACH pair IN bucket
            IF pair[0] = key                    // compare keys, not hashes
                RETURN pair[1]
        RETURN -1

    FUNCTION Remove(key)
        bucket ← self.BucketOf(key)
        FOR EACH pair IN bucket
            IF pair[0] = key
                bucket.remove(pair)
                self.size ← self.size - 1
                RETURN

    FUNCTION Resize()
        old_buckets ← self.buckets
        self.capacity ← 2 * self.capacity
        self.buckets ← empty list
        FOR i FROM 0 TO self.capacity - 1
            self.buckets.append(empty list)
        FOR EACH bucket IN old_buckets
            FOR EACH pair IN bucket
                new_bucket ← self.BucketOf(pair[0])   // uses the new capacity
                new_bucket.append(pair)
```

Every operation costs `O(1)` on average and `O(n)` in the worst case, when all keys hash to the same bucket and the map degenerates into one long chain; Java's `HashMap` turns a long chain into a balanced tree for exactly that reason. A resize costs `O(n)`, but the next one waits until the key count has nearly doubled, so the rehashing over `n` puts adds up to `O(n)`, amortized `O(1)` per put. The space is `O(n + capacity)`. The signals are a request to design a hash map or hash set, or a question about how a dictionary works inside; a good answer names the bucket index, collisions, the load factor and resizing. The classic mistake is copying each old chain to the same index of the bigger array instead of rehashing: key 9 stays in bucket 1, while `get(9)` now looks in bucket `9 mod 16 = 9` and misses it.

Practice: LC 706 Design HashMap · LC 705 Design HashSet

52. Dutch National Flag

Three-Way Partition in One Pass · Two Pointers · Tier 2

Given an array that holds only `0`, `1` and `2`, the three colors of the Dutch flag, sort it in place in a single pass with `O(1)` extra space. `[2, 0, 2, 1, 1, 0]` becomes `[0, 0, 1, 1, 2, 2]`.

Counting sort already does it in two passes: count the three values, then overwrite the array. The one-pass idea is to keep the array split into four regions at all times, with three pointers on the borders: `nums[0..low−1]` holds 0s, `nums[low..mid−1]` holds 1s, `nums[mid..high]` is still unseen, and `nums[high+1..n−1]` holds 2s. Each step classifies `nums[mid]`, the first unseen value. A `0` swaps with the first 1 at `low`, which grows the 0s and slides the block of 1s one step right, so `low` and `mid` both advance. A `1` is already in place, so only `mid` advances. A `2` swaps with the last unseen value at `high`, and `high` retreats.

The asymmetry is the whole subtlety. After a `0` swap, the value that comes back from `low` is a 1 that `mid` has already passed, or the 0 itself when there are no 1s yet, so `mid` may move on. After a `2` swap, the value that comes back from `high` has never been looked at, so `mid` must stay and examine it. The unseen middle shrinks by exactly one every step, and when it is empty the flag is sorted.

```
 0s  | 1s  | unseen      | 2s    low mid high   action
     |     | 2 0 2 1 1 0 |        0   0    5    2: swap to high, high − 1
     |     | 0 0 2 1 1   | 2      0   0    4    0: swap to low, low + 1, mid + 1
 0   |     | 0 2 1 1     | 2      1   1    4    0: swap to low, low + 1, mid + 1
 0 0 |     | 2 1 1       | 2      2   2    4    2: swap to high, high − 1
 0 0 |     | 1 1         | 2 2    2   2    3    1: mid + 1
 0 0 | 1   | 1           | 2 2    2   3    3    1: mid + 1
 0 0 | 1 1 |             | 2 2    2   4    3    mid > high: done
```

```
FUNCTION SortColors(nums)
    low ← 0                                  // nums[0..low−1] are 0s
    mid ← 0                                  // nums[low..mid−1] are 1s
    high ← len(nums) − 1                     // nums[high+1..n−1] are 2s
    WHILE mid ≤ high                         // nums[mid..high] is still unseen
        IF nums[mid] = 0
            swap(nums[low], nums[mid])
            low ← low + 1
            mid ← mid + 1                    // the block of 1s shifts right
        ELSE IF nums[mid] = 1
            mid ← mid + 1
        ELSE
            swap(nums[mid], nums[high])
            high ← high − 1                  // mid stays: this value is unseen
```

The unseen region loses one element per step, so the sort takes exactly `n` steps, `O(n)` time with `O(1)` space and at most one swap per step. This is the three-way partition that rescues quicksort (#18) from repeated pivots: values less than, equal to and greater than the pivot play 0, 1 and 2, and the equal block never needs sorting again. Reach for it whenever an array must be split in place into two or three classes whose inner order does not matter, by parity, by sign or around a pivot; Partition Array According to Given Pivot keeps each class in its original order, which these swaps would scramble, so it fills the regions in separate passes instead. The classic mistake is advancing `mid` after swapping a `2` to the back: on `[1, 2, 0]` the `0` that comes forward is never examined, and the result is `[1, 0, 2]`.

Practice: LC 75 Sort Colors · LC 905 Sort Array By Parity · LC 2161 Partition Array According to Given Pivot

53. Longest Repeating Character Replacement

Window Length Minus Top Count at Most k · Sliding Window · Tier 2

Given a string of uppercase letters and an integer `k`, return the length of the longest substring that can be turned into one repeated letter by replacing at most `k` of its characters. For `"AABABBA"` with `k = 1` the answer is `4`: replacing the `B` in `"AABA"` gives `"AAAA"`.

The cheapest way to make a window uniform keeps its most frequent letter and replaces all the others, so a window needs exactly `length − top count` replacements, and it qualifies when that is at most `k`. The brute force tries every start with every end, $O(n^2)$ windows. The condition survives shrinking, since a window inside a fixable window is fixable, so the variable window of #11 applies: extend `right` one letter at a time, and while the window needs more than `k` replacements, drop `s[left]` and advance `left`. The 26 letter counts of the window update in `O(1)` per step.

The subtle part is `max_count`, the largest count any letter has reached in any window so far. When `left` moves, the true top count of the window can fall, yet `max_count` is never lowered, and it never needs to be. Once the window has slid for the first time, its length is exactly `max_count + k`, so it can grow again only when the entering letter pushes `max_count` higher, and that higher count is real, inside the current window. A stale `max_count` therefore never inflates the answer; at worst it keeps the frame sliding at its current width, as at `right = 4` and `right = 6` below, where `"ABAB"` and `"ABBA"` would really need two replacements. And because `max_count` never drops, the `while` runs at most once per step: the window either grows by one or slides by one, and its final length is the answer.

```
k = 1; the test runs as s[right] arrives, the window is drawn after the step
right  in  max_count  length − max_count  action  window after     counts  best
  0    A       1      1 − 1 = 0           grow    [A]A B A B B A    A1       1
  1    A       2      2 − 2 = 0           grow    [A A]B A B B A    A2       2
  2    B       2      3 − 2 = 1           grow    [A A B]A B B A    A2 B1    3
  3    A       3      4 − 3 = 1           grow    [A A B A]B B A    A3 B1    4
  4    B       3      5 − 3 = 2 > 1       slide    A[A B A B]B A    A2 B2    4
  5    B       3      5 − 3 = 2 > 1       slide    A A[B A B B]A    A1 B3    4
  6    A       3      5 − 3 = 2 > 1       slide    A A B[A B B A]   A2 B2    4
right 4 and 6: ABAB and ABBA need 2 replacements; max_count 3 is stale
```

```
FUNCTION CharacterReplacement(s, k)
    count ← array of 26 zeros              // letter counts inside the window
    max_count ← 0                          // top single-letter count so far
    left ← 0
    best ← 0
    FOR right FROM 0 TO len(s) − 1
        slot ← ord(s[right]) − ord('A')
        count[slot] ← count[slot] + 1
        max_count ← max(max_count, count[slot])
        WHILE right − left + 1 − max_count > k     // too many to replace
            drop ← ord(s[left]) − ord('A')
            count[drop] ← count[drop] − 1
            left ← left + 1                // runs at most once per step
        best ← max(best, right − left + 1)
    RETURN best
```

Each index enters the window once and leaves it at most once, so the scan is `O(n)` time with 26 counts of space. Reach for this window whenever a problem asks for the longest stretch that a budget of `k`

changes can make uniform: Max Consecutive Ones III is the same loop with the zeros as the letters to replace, and Maximize the Confusion of an Exam is this problem over the two letters `T` and `F`. The classic mistake is measuring the window against the count of the letter that just entered instead of the top count: on `"AAAB"` with `k = 1`, the arriving `B` shrinks the window to `"AB"`, and the answer comes out `3` instead of `4`.

Practice: LC 424 Longest Repeating Character Replacement · LC 1004 Max Consecutive Ones III · LC 2024 Maximize the Confusion of an Exam

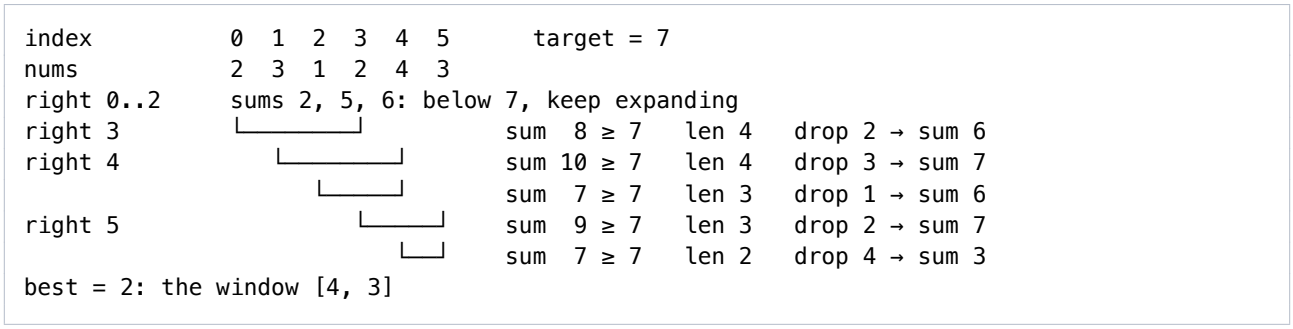
54. Minimum Size Subarray Sum

Shrink While the Sum Is Enough · Sliding Window · Tier 2

Given an array of positive integers and a positive target, return the length of the shortest contiguous subarray whose sum is at least the target, or `0` if no subarray gets there. In `[2, 3, 1, 2, 4, 3]` with target `7` the answer is `2`, the subarray `[4, 3]`.

The brute force sums forward from every start until it reaches the target, `O(n²)` in the worst case. The insight is that positive values make the window sum move in a known direction: pushing the right edge forward can only raise it, and pushing the left edge forward can only lower it. So one window slides with two edges. `right` takes one more value per step; whenever the sum is enough, record the window's length and drop `nums[left]` to try a shorter one, and keep dropping while the sum stays enough. The first drop that leaves the sum short hands control back to `right`. This is the expand-right, shrink-left loop of Minimum Window Substring (#13), with a running sum in place of character counts.

Positivity is also what makes it safe to drop a left index for good. When `left` leaves, `nums[left..right]` is the first window starting at `left` that reaches the target: one step earlier, a window that started at or before `left` and ended at `right − 1` fell short, and a window inside it sums to no more. So it is the shortest valid window starting at `left`, and later right ends only make it longer. A negative value breaks this. On `[3, −2, 4]` with target `4` the window reaches sum `5`, records length `3`, drops the `3`, sees `2` and stops, never learning that dropping the `−2` too climbs back to `4` with the single value `[4]`.




```
FUNCTION MinSubarrayLen(nums, target)
    left ← 0
    window_sum ← 0                          // sum of nums[left..right]
    best ← len(nums) + 1                    // longer than any real window
    FOR right FROM 0 TO len(nums) − 1
        window_sum ← window_sum + nums[right]   // expand: take one more value
        WHILE window_sum ≥ target               // enough: record, then shrink
            best ← min(best, right − left + 1)
            window_sum ← window_sum − nums[left]
            left ← left + 1
    IF best > len(nums)
        RETURN 0                            // no window ever reached the target
    RETURN best
```

Each index enters and leaves the window at most once, so the scan is `O(n)` time even when one step drops several values, with `O(1)` extra space. Reach for it when a problem asks for the shortest or longest stretch under a condition that moves one way as the window grows, such as a sum or product of positive numbers. Subarray Product Less Than K counts instead: shrink while the product is at least `k`, and then all `right − left + 1` windows that end at `right` qualify. With negatives, as in Shortest Subarray with

Sum at Least K, keep the prefix sums of #2 in an increasing monotonic deque (#64): a new prefix pops the front while it beats it by at least `k`, recording lengths, and pops the back while the back is no smaller before joining. The classic mistake is shrinking with `IF` instead of `WHILE`: the window then gives up one value per step, so on `[1, 1, 1, 1, 7]` with target `7` it records length 5 and never shrinks to the single `7`.

Practice: LC 209 Minimum Size Subarray Sum · LC 713 Subarray Product Less Than K · LC 862 Shortest Subarray with Sum at Least K

55. Median of Two Sorted Arrays

Binary Search the Partition · Binary Search · Tier 2

Given two sorted arrays `a` and `b` of lengths `m` and `n`, with at least one value between them, return the median of all `m + n` values in `O(log(m + n))` time. For `[1, 3, 8, 9, 15]` and `[7, 11, 18, 19, 21, 25]` the eleven values in order are `1 3 7 8 9 11 15 18 19 21 25`, so the median is `11`; with an even total it is the average of the two middle values.

The brute force merges the two arrays with two pointers up to the middle, `O(m + n)`. The logarithmic idea stops thinking about positions in the merged array and thinks about one cut through both arrays. The median splits all values into a left part of `half = ⌊(m + n + 1) / 2⌋` values and a right part, with every left value at most every right value. If the cut takes `i` values from `a`, it must take `j = half − i` from `b`, so the single number `i` describes the whole split. Each array is sorted on its own, so the cut is valid exactly when the two cross pairs agree: `a[i−1] ≤ b[j]` and `b[j−1] ≤ a[i]`, reading a missing neighbor at an edge as −∞ or ∞. The same tests say which way to move. When `a[i−1] > b[j]`, `a` gave too many values to the left and `i` must shrink; when `b[j−1] > a[i]`, it gave too few and `i` must grow. That is a binary search over `i` in `[0, m]`.

Search the shorter array: then `j = half − i` stays inside `[0, n]` for every `i`, which a cut through the longer array cannot promise. At the valid cut the median is the largest left value when the total is odd, because the left part holds the extra value, and the average of the largest left value and the smallest right value when the total is even.

```
m = 5, n = 6, half = ⌊(5 + 6 + 1) / 2⌋ = 6 values in the left part
                    left │ right
step 1   lo = 0, hi = 5, i = 2, j = 4
   a:              1   3 │  8   9  15
   b:     7  11  18  19 │ 21  25
   a_left 3 ≤ b_right 21 ✓    b_left 19 ≤ a_right 8 ✗    → lo ← 3
step 2   lo = 3, hi = 5, i = 4, j = 2
   a:      1   3   8   9 │ 15
   b:              7  11 │ 18  19  21  25
   a_left 9 ≤ b_right 18 ✓    b_left 11 ≤ a_right 15 ✓   → valid cut
   11 values, odd: median = max(a_left, b_left) = max(9, 11) = 11
```

```
FUNCTION ValueAt(arr, k)                // −∞ before the start, ∞ past the end
    IF k < 0
        RETURN −∞
    IF k ≥ len(arr)
        RETURN ∞
    RETURN arr[k]

FUNCTION FindMedian(a, b)
    IF len(a) > len(b)
        swap(a, b)                      // search the shorter array
    m ← len(a)
    n ← len(b)
    half ← floor((m + n + 1) / 2)       // size of the left part
    lo ← 0
    hi ← m
    WHILE lo ≤ hi
        i ← floor((lo + hi) / 2)        // a[0..i−1] goes to the left part
```

```
        j ← half − i                    // b[0..j−1] fills the rest of it
        a_left ← ValueAt(a, i − 1)
        a_right ← ValueAt(a, i)
        b_left ← ValueAt(b, j − 1)
        b_right ← ValueAt(b, j)
        IF a_left ≤ b_right AND b_left ≤ a_right    // every left ≤ right
            IF (m + n) mod 2 = 1
                RETURN max(a_left, b_left)
            RETURN (max(a_left, b_left) + min(a_right, b_right)) / 2
        IF a_left > b_right
            hi ← i − 1                  // too much of a on the left
        ELSE
            lo ← i + 1                  // too little of a on the left
```

The range of `i` halves at every step, so the search costs `O(log min(m, n))` time and `O(1)` space, even better than the `O(log(m + n))` the problem demands. The signal is a median or a k-th smallest value across two sorted arrays under a logarithmic bound; across many sorted arrays, binary search on the value instead (#105). The classic mistakes are searching the longer array, which pushes `j` below `0` or past `n`, and dropping the infinities, which crashes or misjudges every cut at an edge, such as `i = 0` when all of `a` belongs on the right.

Practice: LC 4 Median of Two Sorted Arrays · LC 88 Merge Sorted Array · LC 295 Find Median from Data Stream

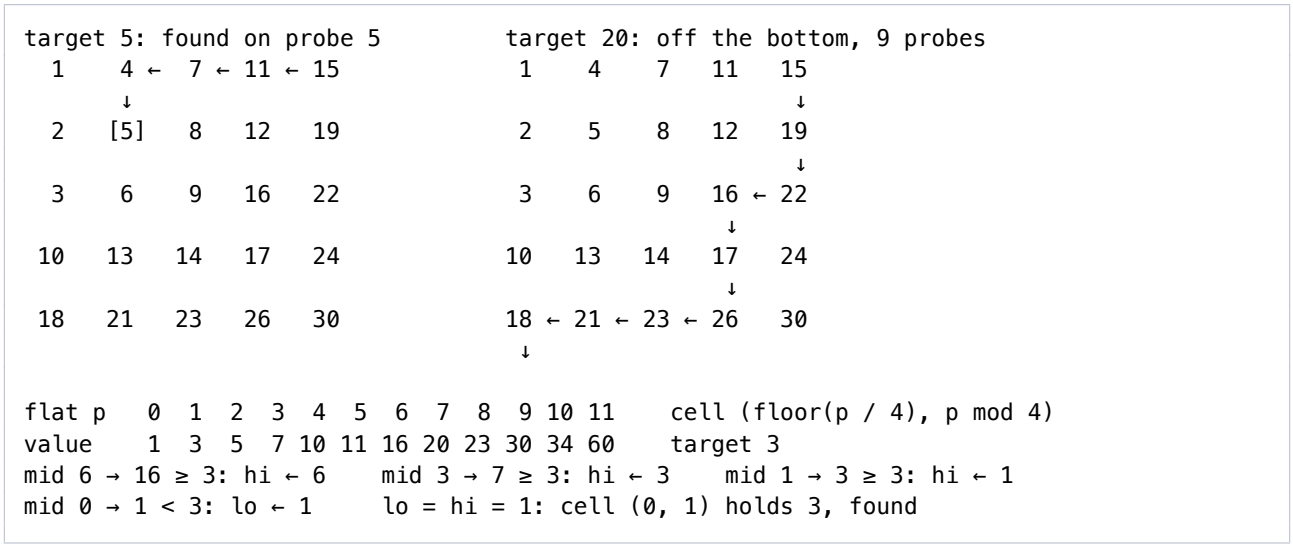
56. Search a 2D Matrix

Staircase from the Top-Right Corner · Binary Search · Tier 2

Given a matrix whose rows are sorted from left to right and whose columns are sorted from top to bottom, report whether a target is present. In the matrix with rows `[1, 4, 7, 11, 15]`, `[2, 5, 8, 12, 19]`, `[3, 6, 9, 16, 22]`, `[10, 13, 14, 17, 24]` and `[18, 21, 23, 26, 30]`, the target `5` is present and `20` is not.

Scanning every cell costs `O(m n)`; a binary search per row costs `O(m log n)` and ignores the order down the columns. The trick is to stand in the top-right corner. The top-right value is the largest in its row and the smallest in its column, so one comparison with the target throws away a whole line. If it is bigger than the target, everything below it is bigger still: the column is out, and the walk steps left. If it is smaller, everything to its left is smaller still: the row is out, and the walk steps down. What remains is a smaller matrix with the same kind of corner, so the walk traces a staircase down and to the left until it lands on the target or steps off the board. The bottom-left corner works too, with the moves mirrored; at the top-left or the bottom-right both moves push the value the same way, and one comparison cannot choose between them.

A stronger order allows a faster search. When each row starts above the end of the row before, the rows laid end to end form one sorted array of `m × n` values, and the lower-bound search of #14 runs on it unchanged; only the read changes. Flat position `p` lives in row `floor(p / n)` and column `p mod n` for rows of length `n`, as below on the rows `[1, 3, 5, 7]`, `[10, 11, 16, 20]` and `[23, 30, 34, 60]`.




```
FUNCTION StaircaseSearch(matrix, target)
    IF len(matrix) = 0
        RETURN false
    row ← 0
    col ← len(matrix[0]) − 1        // top-right: row's max, column's min
    WHILE row < len(matrix) AND col ≥ 0
        value ← matrix[row][col]
        IF value = target
            RETURN true
        IF value > target
            col ← col − 1           // the rest of this column is bigger still
        ELSE
```

```
            row ← row + 1           // the rest of this row is smaller still
    RETURN false

FUNCTION FlattenedSearch(matrix, target)
    IF len(matrix) = 0
        RETURN false
    cols ← len(matrix[0])
    lo ← 0
    hi ← len(matrix) * cols         // half-open [lo, hi) of flat positions
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)
        value ← matrix[floor(mid / cols)][mid mod cols]   // fold into a cell
        IF value < target
            lo ← mid + 1
        ELSE
            hi ← mid
    IF lo = len(matrix) * cols
        RETURN false                // every value is smaller than the target
    RETURN matrix[floor(lo / cols)][lo mod cols] = target
```

Each staircase probe discards a row or a column, so there are at most `m + n` of them: `O(m + n)` time and `O(1)` space. The flat search costs `O(log(m n))`. Rows and columns sorted separately call for the staircase; rows that continue one another call for one flat search. The staircase also counts. Count Negative Numbers in a Sorted Matrix walks it from the bottom-left along the border of the negatives, adding a row's whole negative tail at each step up, and Kth Smallest Element in a Sorted Matrix binary searches on the value (#15) with a staircase count of the cells at most `mid` as its check. The classic mistake in the flat search is dividing by the number of rows instead of the length of a row: in the 3 by 4 example position 3 holds the 7, but dividing by 3 sends the read to row 1, column 0, the `10`, and the 7 is never found.

Practice: LC 240 Search a 2D Matrix II · LC 74 Search a 2D Matrix · LC 1351 Count Negative Numbers in a Sorted Matrix · LC 378 Kth Smallest Element in a Sorted Matrix

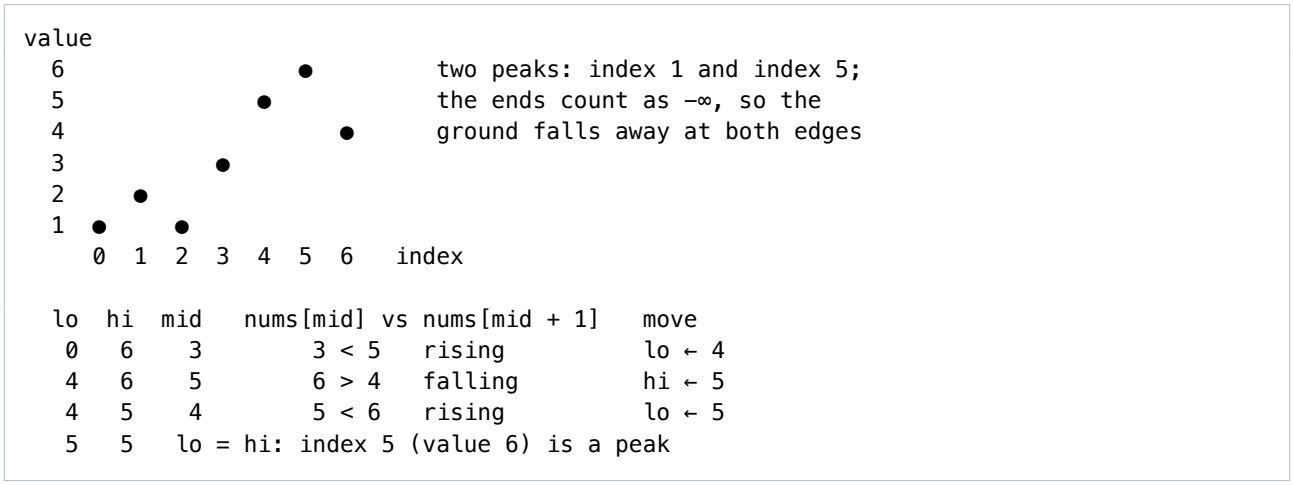
57. Find Peak Element

Binary Search Uphill · Binary Search · Tier 2

Given an array in which no two neighbors are equal, return the index of a peak, a value larger than both of its neighbors, where the positions just outside the array count as −∞. Any peak is accepted. In `[1, 2, 1, 3, 5, 6, 4]` both index 1 (value 2) and index 5 (value 6) are peaks, and the search below returns 5.

A linear scan for the first value whose right neighbor is smaller costs `O(n)`. Binary search beats it even though nothing is sorted, because all it needs is a reason to throw half of the range away, and the slope at `mid` supplies one. Stand at `mid` and look at `mid + 1`. If the ground rises, walk right for as long as it keeps rising. The walk cannot climb forever, because the right end drops to −∞; it stops at the first index whose right neighbor is lower, and since it arrived there by climbing, that index is higher than both neighbors: a peak, to the right of `mid`. So `lo ← mid + 1` gives up only ground the walk never needed. If the ground falls, the same walk run leftward from `mid` stops at a peak at `mid` or to its left, so `hi ← mid`.

The window keeps the promise that the −∞ ends make for the whole array: the ground climbs into `lo` from the left and falls away from `hi` to the right. Every move preserves it, since `lo ← mid + 1` happens only when `nums[mid] < nums[mid + 1]` and `hi ← mid` only when `nums[mid] > nums[mid + 1]`. A window entered uphill and left downhill has its highest point at a peak of the whole array, so when it shrinks to the single index `lo = hi`, that index is a peak. The window is closed, `hi = len(nums) − 1`, because `mid < hi` keeps the read of `nums[mid + 1]` inside the array.




```
FUNCTION FindPeakElement(nums)
    lo ← 0
    hi ← len(nums) − 1                  // closed window [lo, hi] holds a peak
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)      // mid < hi, so nums[mid + 1] exists
        IF nums[mid] < nums[mid + 1]
            lo ← mid + 1                // rising: a peak lies right of mid
        ELSE
            hi ← mid                    // falling: a peak at mid or left of it
    RETURN lo
```

Each probe halves the window, so the search costs `O(log n)` time and `O(1)` space. Reach for it when a problem asks for any local maximum or minimum, or for the top of a mountain array: Peak Index in

a Mountain Array is this loop unchanged, and Find in Mountain Array finds the peak first and then binary searches each slope. On a grid, Find a Peak Element II binary searches the columns: the largest value in the middle column beats its neighbors above and below, and the search steps toward a left or right neighbor that beats it, stopping when neither does, for `O(m log n)`. The classic mistake is borrowing the half-open window of #14: with `hi ← len(nums)`, the first probe on `[1, 2]` is `mid = 1`, and `nums[mid + 1]` reads past the end of the array.

Practice: LC 162 Find Peak Element · LC 852 Peak Index in a Mountain Array · LC 1901 Find a Peak Element II · LC 1095 Find in Mountain Array

58. Longest Increasing Subsequence

Patience Sorting with Binary Search · Binary Search · Tier 2

Given an array of integers, return the length of its longest strictly increasing subsequence: values kept in their original order, but not necessarily adjacent. In `[10, 9, 2, 5, 3, 7, 101, 18]` the answer is `4`, for example `2, 3, 7, 18`.

The quadratic method gives each index the length of the longest increasing subsequence ending there, one more than the best over earlier smaller values, $O(n^2)$. The fast method keeps only the best way to reach each length. Let `tails[k]` be the smallest value that ends an increasing subsequence of length `k + 1` so far; a smaller tail is the better state, since whatever extends a larger tail extends a smaller one too. The tails increase strictly, because cutting the last value off a subsequence of length `k + 2` leaves one of length `k + 1` that ends lower. So a new value `x` binary searches them with the lower bound of #14. The tails below `x` are lengths it can extend, and the first tail at least `x`, at index `i`, is where it stops: `x` ends a subsequence of length `i + 1` and is no larger than the old tail, so it takes that slot. If no tail is at least `x`, then `x` extends the longest subsequence and `tails` grows.

This is patience sorting: deal the values like cards, each onto the leftmost pile whose top is at least the card, or onto a new pile at the right. The pile tops are `tails`, and the number of piles is the answer. The tops are not themselves a subsequence: on `[4, 5, 6, 1]` they end as `[1, 5, 6]`, out of order in the array, yet the length 3 is right, because each slot only records the best tail for its length. To rebuild a real subsequence, keep indices in `tails`, link each value to the index in the slot to its left when it arrived, and follow the links back from the last slot.

```
   x    first tail ≥ x         tails after
  10    none: append           [10]
   9    10 at 0: replace       [9]
   2    9 at 0: replace        [2]
   5    none: append           [2, 5]
   3    5 at 1: replace        [2, 3]
   7    none: append           [2, 3, 7]
 101    none: append           [2, 3, 7, 101]
  18    101 at 3: replace      [2, 3, 7, 18]
answer = len(tails) = 4

piles      10    5    7  101     each card lands on the leftmost pile
            9    3        18     whose top is ≥ it; the four tops
            2                    2, 3, 7, 18 are the tails
```

```
FUNCTION LowerBound(values, target)     // first i with values[i] ≥ target
    lo ← 0
    hi ← len(values)                    // half-open [lo, hi)
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)
        IF values[mid] < target
            lo ← mid + 1
        ELSE
            hi ← mid
    RETURN lo

FUNCTION LengthOfLIS(nums)
    tails ← empty list              // tails[k]: smallest tail of length k + 1
```

```
    FOR EACH x IN nums
        i ← LowerBound(tails, x)        // how many tails are smaller than x
        IF i = len(tails)
            tails.append(x)             // x extends the longest subsequence
        ELSE
            tails[i] ← x                // a smaller tail for length i + 1
    RETURN len(tails)
```

Each value costs one binary search over at most `n` tails, so the method runs in `O(n log n)` time and `O(n)` space. The signals are “longest increasing”, “longest chain” and “fits inside”, including two-dimensional orders that a sort reduces to one. Russian Doll Envelopes sorts by width ascending and, for equal widths, by height descending, then runs this on the heights; the descending tie-break keeps two envelopes of the same width out of one chain, since their heights now appear in decreasing order. Find the Longest Valid Obstacle Course at Each Position wants non-decreasing courses, so it searches for the first tail greater than `x` and reports `i + 1` at every position. That swap is the classic mistake here: with the first tail greater than `x`, an equal value extends its twin, and `[7, 7, 7, 7]` reports 4 instead of 1.

Practice: LC 300 Longest Increasing Subsequence · LC 354 Russian Doll Envelopes · LC 1964 Find the Longest Valid Obstacle Course at Each Position

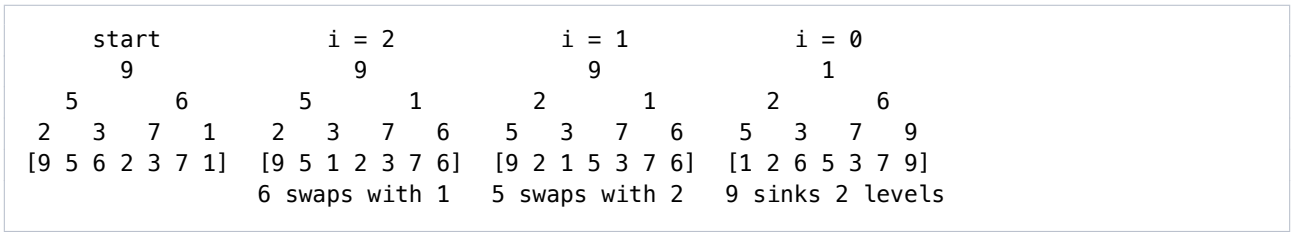
59. Heapify

Build a Heap in O(n) with Sift-Down · Heaps · Tier 2

Rearrange an array in place into a min-heap: a complete binary tree stored level by level, in which every parent is at most its children. The children of index `i` sit at `2i + 1` and `2i + 2`, and its parent at `floor((i − 1) / 2)`. `[9, 5, 6, 2, 3, 7, 1]` becomes `[1, 2, 6, 5, 3, 7, 9]`. Then push and pop on that heap by hand.

Pushing the n values one at a time builds a heap in `O(n log n)`. Heapify builds it in `O(n)` by working from the bottom up with sift-down. A node whose two subtrees are already heaps is repaired by swapping it with its smaller child until neither child is smaller, and then its whole subtree is a heap. Leaves are heaps of one value, so start at the last parent, `floor(n / 2) − 1`, and walk back to the root: every node is sifted after both of its subtrees, so they are heaps by the time its turn comes.

A sift-down costs at most the height of its node, the distance to the bottom, and most nodes live near the bottom. Half of them are leaves and cost nothing, a quarter sit one level up and cost at most one swap, an eighth cost at most two, and the sum `n/4 · 1 + n/8 · 2 + n/16 · 3 + …` stays below n. Building by pushes uses sift-up, whose cost is the depth instead, the distance to the top, and the deep nodes are the many: that is where the extra `log n` comes from.




```
FUNCTION SiftDown(heap, i, n)              // heap[0..n−1] is the heap
    WHILE true
        smallest ← i
        left ← 2 * i + 1
        right ← 2 * i + 2
        IF left < n AND heap[left] < heap[smallest]
            smallest ← left
        IF right < n AND heap[right] < heap[smallest]
            smallest ← right
        IF smallest = i                    // no child is smaller: done
            RETURN
        swap(heap[i], heap[smallest])
        i ← smallest

FUNCTION Heapify(heap)
    n ← len(heap)
    FOR i FROM floor(n / 2) − 1 DOWNTO 0   // last parent, back to the root
        SiftDown(heap, i, n)

FUNCTION SiftUp(heap, i)
    WHILE i > 0
        parent ← floor((i − 1) / 2)
        IF heap[parent] ≤ heap[i]          // the parent is no larger: done
            RETURN
        swap(heap[i], heap[parent])
```

```
        i ← parent

FUNCTION HeapPush(heap, value)
    heap.append(value)
    SiftUp(heap, len(heap) - 1)

FUNCTION HeapPop(heap)
    last ← len(heap) - 1
    swap(heap[0], heap[last])
    top ← heap.removeLast()                // remove the last slot: the old root
    SiftDown(heap, 0, len(heap))
    RETURN top
```

Heapify takes `O(n)` time and `O(1)` extra space; push and pop take `O(log n)`, the height of the tree. Push appends the value as the last leaf and sifts it up, swapping with its parent while it is smaller. Pop swaps the root with the last leaf, removes that last slot, which now holds the old minimum, and sifts the new root down. Every heap in this book rests on these two moves, from Top-K (#19) to Heap Sort (#109), which runs the same build as a max-heap. Reach for heapify when all the values are present at the start: Last Stone Weight heapifies the stones once and then only pops and pushes. The classic mistake is running `i` upward from the root instead of backward from the last parent: the root is sifted before its subtrees are heaps, and the example ends as `[5, 2, 1, 9, 3, 7, 6]`, with 5 sitting above 1.

Practice: LC 912 Sort an Array · LC 215 Kth Largest Element in an Array · LC 1046 Last Stone Weight

60. Quickselect

Kth Largest in Expected O(n) · Heaps · Tier 2

Find the kth largest value of an unsorted array, where duplicates count as separate values. In `[3, 2, 1, 5, 6, 4]` with `k = 2` the answer is `5`.

Sorting finds it in `O(n log n)`, and the size-k heap of #19 in `O(n log k)`. Quickselect reaches expected `O(n)` with one fact about the partition of Quick Sort (#18): after partitioning, the pivot sits at its final sorted index `p`, with only smaller values to its left. In ascending order the kth largest lives at index `target = n − k`. If `p = target`, the pivot is the answer. If `p < target`, the answer lies to the right of the pivot and everything from `lo` to `p` can be forgotten; if `p > target`, it lies to the left. Quick sort recurses into both sides; quickselect keeps only one, so it needs no recursion at all, just a loop that narrows the window `[lo, hi]`.

Picture the window closing in on index `target`, which never leaves it. With distinct values, a random pivot lands in the middle half of the window half of the time, and such a pivot cuts the window to at most three quarters of its size. So it takes two rounds on average to shrink the window by a quarter, and the expected work is at most twice the geometric series `n + 3n/4 + 9n/16 + …`, that is at most 8n.

```
k = 2, n = 6, so the answer sits at index target = 6 − 2 = 4
one run; the pivots are random picks
  window   pivot   nums after partitioning the window   p   next
  [0, 5]     2     [1 2 3 5 6 4]                        1   1 < 4: lo ← 2
  [2, 5]     6      1 2 [3 5 4 6]                       5   5 > 4: hi ← 4
  [2, 4]     5      1 2 [3 4 5] 6                       4   4 = 4: return 5
```

```
FUNCTION Partition(nums, lo, hi)
    pick ← RandomInt(lo, hi)            // lo ≤ pick ≤ hi
    swap(nums[pick], nums[hi])          // park the pivot at the end
    pivot ← nums[hi]
    store ← lo                          // invariant: nums[lo..store−1] < pivot
    FOR i FROM lo TO hi − 1
        IF nums[i] < pivot              // a small value joins the left region
            swap(nums[store], nums[i])
            store ← store + 1
    swap(nums[store], nums[hi])         // pivot between the regions
    RETURN store

FUNCTION KthLargest(nums, k)
    target ← len(nums) − k              // the answer's index in sorted order
    lo ← 0
    hi ← len(nums) − 1
    WHILE true
        p ← Partition(nums, lo, hi)
        IF p = target
            RETURN nums[p]
        ELSE IF p < target
            lo ← p + 1                  // the answer is right of the pivot
        ELSE
            hi ← p − 1                  // the answer is left of the pivot
```

On distinct values the expected time is `O(n)` and the extra space `O(1)`; a run of unlucky pivots still costs `O(n²)`. Against the heap of #19, quickselect is faster on average but reorders the input, needs all of it in memory and keeps that quadratic tail; the heap works on a stream at `O(n log k)` every time. Signals: kth largest or smallest, k closest, top k in any order, since when the loop stops `nums[target..n−1]` holds the k largest values, unsorted, just as aiming at index `k − 1` leaves the k smallest in `nums[0..k−1]`; that is how it answers K Closest Points to Origin on squared distances and Top K Frequent Elements on counts. Many equal values are its weak spot: they all land on one side of the `<` test, so an array of identical values goes quadratic unless k is close to n; the three-way partition of #52 groups them in the middle and stops as soon as the target falls among them. The classic mistake is aiming at index `k − 1`, which finds the kth smallest: on the example it returns 2 instead of 5.

Practice: LC 215 Kth Largest Element in an Array · LC 973 K Closest Points to Origin · LC 347 Top K Frequent Elements

61. Bucket Sort

Top K Frequent by Frequency Buckets · Sorting · Tier 2

Given an array and an integer k, return the k most frequent values in any order; the answer is guaranteed to be unique. In `[1, 1, 1, 2, 2, 3]` with `k = 2` the answer is `[1, 2]`.

Counting the frequencies with a map is the easy part. Ordering the distinct values by count with a sort costs `O(d log d)` for d distinct values, and the size-k heap of #19 costs `O(d log k)`. Bucket sort removes the comparisons altogether: a frequency is a whole number between 1 and n, so it can serve directly as an array index. Make `n + 1` empty buckets, drop each distinct value into `buckets[count[value]]`, and walk the buckets from n down to 1, collecting values until k of them are in hand. The bucket index already is the order, so nothing inside a bucket needs sorting.

Picture a shelf with one slot per possible frequency. Every distinct value lands in the slot of its own count, and reading the shelf from the right end reads the values from most to least frequent.

```
nums = [1, 1, 1, 2, 2, 3], k = 2      count = {1: 3, 2: 2, 3: 1}
  freq      0     1     2     3     4     5     6
  bucket   [ ]   [3]   [2]   [1]   [ ]   [ ]   [ ]
           ←──────────────────────────────────────── read from freq 6 down
  freq 6, 5, 4: empty
  freq 3: take 1      result [1]
  freq 2: take 2      result [1, 2]      k values: return
```

```
FUNCTION TopKFrequent(nums, k)
    count ← empty map                       // value → frequency
    FOR EACH value IN nums
        count[value] ← count.get(value, 0) + 1
    buckets ← empty list                    // index = frequency
    FOR freq FROM 0 TO len(nums)            // a frequency is at most n
        buckets.append(empty list)
    FOR EACH value IN count
        buckets[count[value]].append(value)
    result ← empty list
    FOR freq FROM len(nums) DOWNTO 1        // most frequent first
        FOR EACH value IN buckets[freq]
            result.append(value)
            IF len(result) = k
                RETURN result
    RETURN result
```

Counting takes `O(n)`, filling the buckets `O(d)`, and the walk visits at most `n + 1` buckets and takes k values, so the whole job is `O(n)` time and `O(n)` space. Signals: top k frequent, sort by frequency, and in general any sort key that is a small whole number bounded by n. General bucket sort spreads values over equal-width ranges instead of exact keys: n numbers spread evenly over `[low, high)` go into n buckets of width `(high − low) / n`, each bucket catches about one value, a tiny sort inside each bucket is cheap, and reading the buckets in order gives the sorted array in expected `O(n)`. Maximum Gap (LC 164) uses such buckets for a pigeonhole argument: the largest gap between sorted neighbors is at least the average gap `(max − min) / (n − 1)`, so with buckets no wider than that it never falls inside one bucket, and only each bucket's minimum and maximum matter. The classic mistake is making n

buckets instead of `n + 1`: when one value fills the whole array, as in `[7, 7, 7]`, its frequency is 3 and `buckets[3]` does not exist.

Practice: LC 347 Top K Frequent Elements · LC 451 Sort Characters By Frequency · LC 692 Top K Frequent Words · LC 164 Maximum Gap

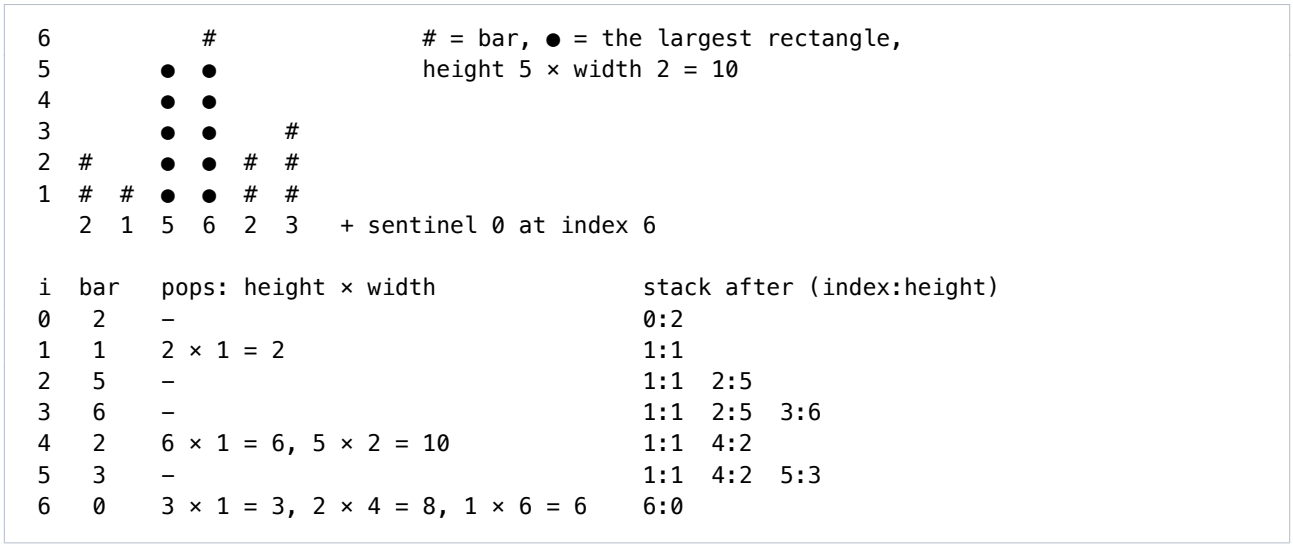
62. Largest Rectangle in Histogram

Monotonic Increasing Stack · Stacks · Tier 2

Given the heights of a histogram whose bars all have width 1, return the area of the largest rectangle that fits inside it. For `[2, 1, 5, 6, 2, 3]` the answer is 10: the bars 5 and 6 together hold a rectangle of height 5 and width 2.

The brute force tries every pair of left and right edges while carrying the minimum height between them, $O(n^2)$. Look at it per bar instead. The widest rectangle at exactly one bar's height spreads left and right until it meets a lower bar on each side, so every bar needs its nearest lower neighbor on both sides, and the monotonic stack of #23 finds both in a single pass. Keep a stack of indices whose heights never fall from bottom to top. When bar `i` arrives lower than the top, the top's rectangle stops at `i − 1`, since every bar between them is at least as tall or it would have popped the top sooner. Pop it and measure. Its left edge sits just right of the new top, because every bar between those two was popped earlier, and only by a lower bar, so all of them are taller than the bar being measured. The area is the height times `i − left`, and the best area wins.

Two details complete the loop. A 0 appended to the heights is lower than every bar of positive height, so at the end it pops each of them still on the stack and measures it against the right edge of the histogram. When a pop empties the stack, nothing lower lies to the left, and the rectangle reaches index 0. Equal heights may stand side by side; the leftmost of them is popped last and measures the full width.



i	bar	pops: height × width	stack after (index:height)
0	2	–	0:2
1	1	2 × 1 = 2	1:1
2	5	–	1:1 2:5
3	6	–	1:1 2:5 3:6
4	2	6 × 1 = 6, 5 × 2 = 10	1:1 4:2
5	3	–	1:1 4:2 5:3
6	0	3 × 1 = 3, 2 × 4 = 8, 1 × 6 = 6	6:0

```
FUNCTION LargestRectangleArea(heights)
    bars ← heights + [0]                // the 0 sentinel flushes the stack
    stack ← empty stack                 // indices; heights never fall
    best ← 0
    FOR i FROM 0 TO len(bars) − 1
        WHILE len(stack) > 0 AND bars[stack.top()] > bars[i]
            height ← bars[stack.pop()]          // its rectangle ends at i − 1
            IF len(stack) > 0
                left ← stack.top() + 1          // just right of the new top
            ELSE
                left ← 0                        // empty stack: reaches index 0
            width ← i − left
            best ← max(best, height * width)
```

```
        stack.push(i)
    RETURN best
```

Each index is pushed once and popped at most once, so the run is `O(n)` time with `O(n)` space for the stack. The signal is a largest rectangle, or any "minimum times width" score, over contiguous bars or subarrays. Maximal Rectangle in a binary matrix reduces to this: treat each row as the floor of a histogram whose heights count the consecutive 1s running up from it, and run this loop once per row, `O(rows × cols)` in all. Maximum Score of a Good Subarray is the same rectangle with the extra rule that it must cover index `k`. The classic mistakes are measuring the width as `i − j` from the popped index `j` instead of from the new top, which ignores the taller bars popped earlier on the left and returns 3 instead of 4 for `[1, 3, 2]`, and forgetting the sentinel, which leaves a rising histogram such as `[1, 2, 3]` unmeasured on the stack and returns 0 instead of 4.

Practice: LC 84 Largest Rectangle in Histogram · LC 85 Maximal Rectangle · LC 1793 Maximum Score of a Good Subarray

63. Basic Calculator

Evaluate + - * / and Parentheses with a Stack · Stacks · Tier 2

Evaluate a valid expression made of non-negative integers, the operators `+ - * /`, parentheses and spaces, with the usual precedence. Division truncates toward zero, and a minus sign may also open the expression or a group, as in `-(2 + 3)`. For `"2*(3+4)-10/3"` the answer is 11: the group gives 7, `2 * 7 = 14`, and `10 / 3` truncates to 3.

Precedence looks like it needs a parser, but one level of an expression without parentheses is just a sum of signed terms, and a stack of those terms evaluates it in one pass. Read each number and remember the operator in front of it. When the next operator, a `)` or the end of the string finishes the number, fold it in by that remembered operator: after + push it, after − push its negation, after * or / pop the top term, combine it with the number and push the result back. Multiplication and division reach back exactly one term, because the terms on the stack are separated by + and −, which bind more loosely. When the level ends, its value is the sum of the stack.

Parentheses are recursion. Seen from outside, a group is a single number, so on `(` the function calls itself on the text that follows. The call evaluates that level with its own stack and returns the group's value and the index just past its `)`, where the caller resumes as if it had read one long number. A leading minus needs no special case: `op` starts as + and `num` as 0, so the minus in `-(2 + 3)` finishes an invisible 0, and the group's 5 then arrives behind a −, giving `0 - 5`.

```
s = "2*(3+4)-10/3"     (indices 0 to 11)

level 0   '*' ends 2     op in front +   push 2              stack [2]
level 0   '(' at 2       recurse from index 3
level 1     '+' ends 3   op in front +   push 3              stack [3]
level 1     ')' ends 4   op in front +   push 4              stack [3, 4]
level 1     return (3 + 4, 7): value 7, resume at index 7
level 0   '-' ends 7     op in front *   top 2 * 7 = 14      stack [14]
level 0   '/' ends 10    op in front -   push -10            stack [14, -10]
level 0   end ends 3     op in front /   top -10 / 3 = -3    stack [14, -3]
level 0   return 14 + (-3) = 11
```

```
FUNCTION Calculate(s)
    value, end ← Evaluate(s, 0)
    RETURN value

FUNCTION Evaluate(s, i)             // returns (value, next index to read)
    stack ← empty stack             // signed terms of this level; value = sum
    num ← 0
    op ← '+'                        // the operator in front of num
    WHILE i < len(s)
        ch ← s[i]
        i ← i + 1
        IF ch is a digit
            num ← num * 10 + int(ch)
        ELSE IF ch = '('
            num, i ← Evaluate(s, i)         // a whole group acts as one number
        ELSE IF ch IN "+-*/)"
            ApplyOp(stack, op, num)
            IF ch = ')'
```

```
                RETURN (sum(stack), i)
            op ← ch
            num ← 0
    ApplyOp(stack, op, num)
    RETURN (sum(stack), i)

FUNCTION ApplyOp(stack, op, num)
    IF op = '+'
        stack.push(num)
    ELSE IF op = '-'
        stack.push(-num)
    ELSE IF op = '*'
        stack.push(stack.pop() * num)       // * reaches back one term only
    ELSE
        stack.push(truncate(stack.pop() / num))     // toward zero
```

Every character is read once, by the level that owns it, so the run is `O(n)` time, and the stacks plus the recursion hold `O(n)` at worst. Reach for this whenever a string encodes arithmetic or other nesting to resolve from the inside out. Basic Calculator II is one level without parentheses, the stack alone; the original Basic Calculator has only `+ - ( )`, and Basic Calculator III has everything, as here. The recursion is as deep as the nesting, so the original problem is often solved iteratively instead, pushing the running total and sign at `(` and popping them at `)`. Evaluate Reverse Polish Notation is the same stack with precedence already settled by the notation: push numbers and let each operator pop two. The classic mistake is Python's `//`, which rounds toward negative infinity: `-10 // 3` is -4 and turns the example's 11 into 10, while `int(-10 / 3)` truncates to -3. The other is folding a number only when an operator arrives, which loses the last number of the string and the one just before each `)`.

Practice: LC 224 Basic Calculator · LC 227 Basic Calculator II · LC 772 Basic Calculator III · LC 150 Evaluate Reverse Polish Notation

64. Sliding Window Maximum

Monotonic Deque · Queues · Tier 2

Given an array and a window size `k`, return the maximum of every window of `k` consecutive values as the window slides one step at a time from left to right. For `[9, 7, 8, 3, 2, 5, 1]` and `k = 3` the windows are `[9, 7, 8]`, `[7, 8, 3]`, `[8, 3, 2]`, `[3, 2, 5]` and `[2, 5, 1]`, so the answer is `[9, 8, 8, 5, 5]`.

The brute force rescans every window, `O(n · k)`, and a max-heap that lazily discards expired indices still pays `O(n log n)`. The linear solution rests on one observation: when a new value arrives, every older value in the window that it matches or beats is never needed again, because the newcomer is at least as large and stays in the window longer. Throw those losers away and the survivors, read from oldest to newest, form a strictly decreasing staircase of candidates. Keep their indices in a deque. The newcomer enters at the back after popping every value at the back that is no larger than itself. The oldest survivor sits at the front and is the largest, so it is the window's maximum, unless its index has slid out of the window, in which case it leaves through the front.

Each index is pushed once and popped at most once, from one end or the other, so the total work is `O(n)` even though a single step may pop several values. The deque stores indices rather than values because the index is what tells the front that it has expired.

```
k = 3; the deque holds index:value, front on the left
  i   nums[i]   popped from back   popped from front   deque after     max
  0      9                                            [0:9]
  1      7                                            [0:9 1:7]
  2      8      1:7                                   [0:9 2:8]        9
  3      3                         0:9 (expired)      [2:8 3:3]        8
  4      2                                            [2:8 3:3 4:2]    8
  5      5      4:2 3:3            2:8 (expired)      [5:5]            5
  6      1                                            [5:5 6:1]        5
```

```
FUNCTION MaxSlidingWindow(nums, k)
    window ← empty deque                // indices of the candidates
    result ← empty list
    FOR i FROM 0 TO len(nums) − 1
        WHILE len(window) > 0 AND nums[window.back()] ≤ nums[i]
            window.pop_back()           // older and no larger: never needed
        window.push_back(i)
        IF window.front() ≤ i − k
            window.pop_front()          // the front slid out of the window
        IF i ≥ k − 1
            result.append(nums[window.front()])   // front is the window max
    RETURN result
```

The cost is `O(n)` time and `O(k)` space for the deque, plus the output. The signals are a window that slides while its maximum or minimum is needed, and dynamic programming whose next state takes the best of the last `k` states, as in Jump Game VI; for a minimum, flip the comparison so the staircase climbs instead. It is the monotonic stack of #23 with a second exit at the front. The classic mistakes are storing values instead of indices, which leaves the front no way to know that it has expired, and popping the back with `IF` instead of `WHILE`: a large newcomer must clear every smaller value, or on `[3, 1, 2, 5]` with `k = 4` the `3` stays in front of the larger `5` and is reported instead of it.

Practice: LC 239 Sliding Window Maximum · LC 1438 Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit · LC 862 Shortest Subarray with Sum at Least K · LC 1696 Jump Game VI

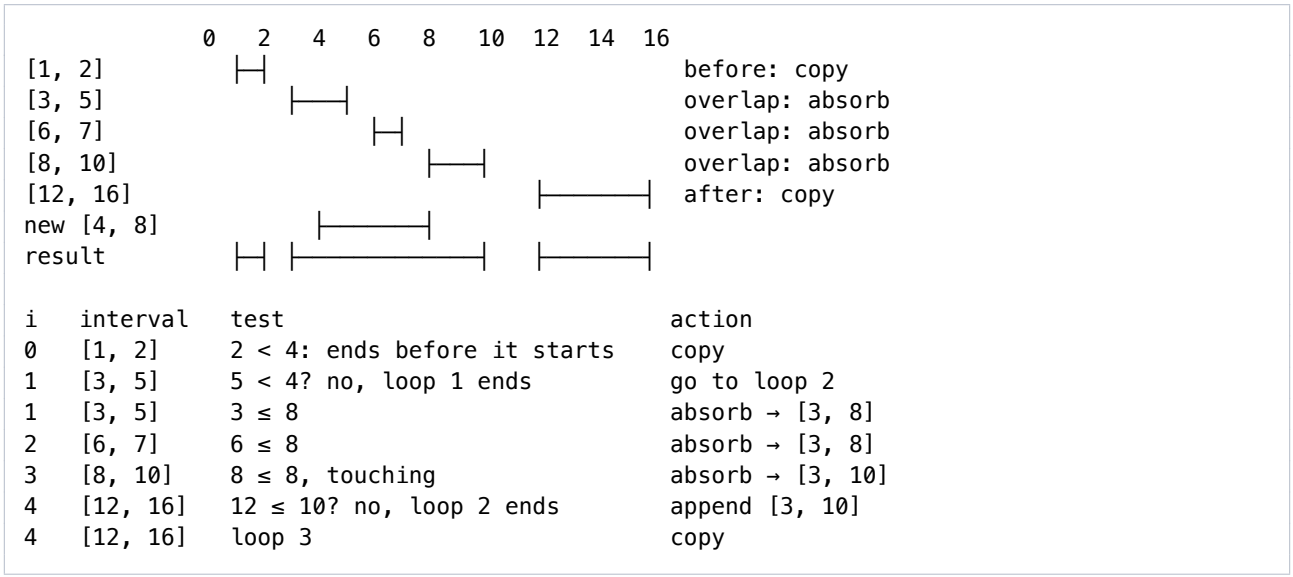
65. Insert Interval

Before, Overlapping, After · Intervals · Tier 2

Given closed intervals `[start, end]` sorted by start and pairwise disjoint, insert one more, merging it with every interval it overlaps or touches, and return the list still sorted and disjoint. Inserting `[4, 8]` into `[[1, 2], [3, 5], [6, 7], [8, 10], [12, 16]]` gives `[[1, 2], [3, 10], [12, 16]]`: the new interval swallows `[3, 5]`, `[6, 7]` and `[8, 10]`, which it touches at 8.

The brute force appends the new interval and runs Merge Intervals (#26), paying `O(n log n)` to sort a list that was sorted all along. Because the intervals are sorted and disjoint, their starts and their ends both increase from left to right, so the new interval splits the line into three runs: the intervals that end before it starts, the ones that overlap or touch it, and the ones that start after it ends. Each run is contiguous, since once an interval ends at or after the new start every later one does too, and once one starts beyond the new end every later one starts further right. So a single index walks three loops. The first copies while `end < new_start`. The second absorbs while `start ≤ new_end`, stretching the new interval to the smaller start and the larger end. After the grown interval is appended, the third copies the rest.

Picture the new interval as a bar laid across the line: whatever it touches melts into it, and whatever lies strictly left or right stays as it was. Only the first absorbed interval can lower the start and only the last can raise the end, but `min` and `max` at every step are simpler and just as right. Touching counts as overlap for closed intervals, which is why the first loop runs on a strict `<` and the second on `≤`.



i	interval	test	action
0	[1, 2]	2 < 4: ends before it starts	copy
1	[3, 5]	5 < 4? no, loop 1 ends	go to loop 2
1	[3, 5]	3 ≤ 8	absorb → [3, 8]
2	[6, 7]	6 ≤ 8	absorb → [3, 8]
3	[8, 10]	8 ≤ 8, touching	absorb → [3, 10]
4	[12, 16]	12 ≤ 10? no, loop 2 ends	append [3, 10]
4	[12, 16]	loop 3	copy

```
FUNCTION InsertInterval(intervals, new_interval)
    result ← empty list
    new_start ← new_interval[0]
    new_end ← new_interval[1]
    i ← 0
    n ← len(intervals)
    WHILE i < n AND intervals[i][1] < new_start      // ends before it starts
        result.append(intervals[i])
        i ← i + 1
    WHILE i < n AND intervals[i][0] ≤ new_end        // overlaps or touches it
        new_start ← min(new_start, intervals[i][0])  // the new interval grows
```

```
        new_end ← max(new_end, intervals[i][1])
        i ← i + 1
    result.append([new_start, new_end])              // added exactly once
    WHILE i < n                                      // starts after it ends
        result.append(intervals[i])
        i ← i + 1
    RETURN result
```

Each interval is examined once and copied at most once, so the insert takes `O(n)` time and `O(n)` space for the new list, with no sort. Binary search could find where the overlapping run begins in `O(log n)`, but writing the output costs `O(n)` anyway. Reach for the three loops whenever a list is already sorted and disjoint and one more interval arrives; Range Module (#142) keeps such a list, and its `add_range` is this insert. The classic mistake is ≤ in the first loop, which copies an interval that merely touches the new one, so `[[1, 4]]` with `[4, 6]` becomes `[[1, 4], [4, 6]]` instead of `[[1, 6]]`. A one-loop version has its own trap: it appends the new interval when the first interval after it shows up, so without one more append after the loop it loses the new interval whenever that belongs at the very end.

Practice: LC 57 Insert Interval · LC 56 Merge Intervals · LC 715 Range Module · LC 352 Data Stream as Disjoint Intervals

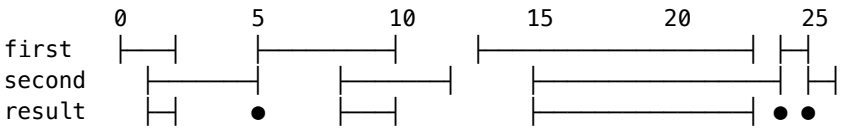
66. Interval List Intersections

Two Pointers over Two Sorted Lists · Intervals · Tier 2

Given two lists of closed intervals, each sorted and pairwise disjoint, return every stretch the two lists share. For `[[0, 2], [5, 10], [13, 23], [24, 25]]` and `[[1, 5], [8, 12], [15, 24], [25, 26]]` the answer is `[[1, 2], [5, 5], [8, 10], [15, 23], [24, 24], [25, 25]]`; the single point `[5, 5]` is shared because `[1, 5]` ends exactly where `[5, 10]` begins.

Comparing every pair costs `O(m n)`. Both lists are sorted, though, so one sweep from left to right handles them, like the merge step of merge sort (#17): one pointer per list, each resting on its current interval. Two closed intervals share exactly the stretch from the later start to the earlier end, `[max(starts), min(ends)]`, and they share anything at all only when that start is at most that end. After recording the overlap, if any, drop the interval that ends first. It can meet nothing further on in the other list: the next interval there starts after the current one ends, which is no earlier than the end of the interval being dropped. The interval that ends later may still reach into its partner's next interval, so it stays.

Picture both lists as rows of bars over one number line, with a vertical line sweeping to the right. The result is wherever both rows are covered at once, and the pointers advance in the order in which the bars end. A tie needs no special case: when both intervals end at the same point, either one can go, and the code drops the one from `second`. On the fourth step below, `[13, 23]` and `[8, 12]` do not overlap at all, since `13 > 12`, and the step simply drops `[8, 12]`, which ends first.



first[i]	second[j]	start	end	overlap	ends first
[0, 2]	[1, 5]	1	2	[1, 2]	first: i ← 1
[5, 10]	[1, 5]	5	5	[5, 5]	second: j ← 1
[5, 10]	[8, 12]	8	10	[8, 10]	first: i ← 2
[13, 23]	[8, 12]	13	12	none	second: j ← 2
[13, 23]	[15, 24]	15	23	[15, 23]	first: i ← 3
[24, 25]	[15, 24]	24	24	[24, 24]	second: j ← 3
[24, 25]	[25, 26]	25	25	[25, 25]	first: i ← 4

```
FUNCTION IntervalIntersection(first, second)
    result ← empty list
    i ← 0
    j ← 0
    WHILE i < len(first) AND j < len(second)
        (first_start, first_end) ← first[i]
        (second_start, second_end) ← second[j]
        overlap_start ← max(first_start, second_start)    // the later start
        overlap_end ← min(first_end, second_end)          // the earlier end
        IF overlap_start ≤ overlap_end
            result.append([overlap_start, overlap_end])   // a point counts too
        IF first_end < second_end
            i ← i + 1              // first[i] can meet nothing further on
        ELSE
            j ← j + 1              // second[j] can meet nothing further on
    RETURN result
```

Each step advances one pointer, so the sweep takes at most `m + n` steps: `O(m + n)` time and `O(1)` extra space beyond the output. The signals are two sorted schedules, calendars or sets of busy or free times, and a question about when both hold at once. Meeting Scheduler sorts both people's free slots and runs this sweep, stopping at the first overlap that is at least the meeting's duration long. A single unsorted list of intervals calls for the sort and sweep of Merge Intervals (#26) instead. The classic mistake is advancing the pointer whose interval starts first: with `[[0, 10]]` against `[[1, 2], [3, 4]]` it drops `[0, 10]` after the first overlap and never finds `[3, 4]`. Testing the overlap with `<` in place of `≤` is the other one; it silently loses the single points `[5, 5]`, `[24, 24]` and `[25, 25]` above.

Practice: LC 986 Interval List Intersections · LC 56 Merge Intervals · LC 1229 Meeting Scheduler

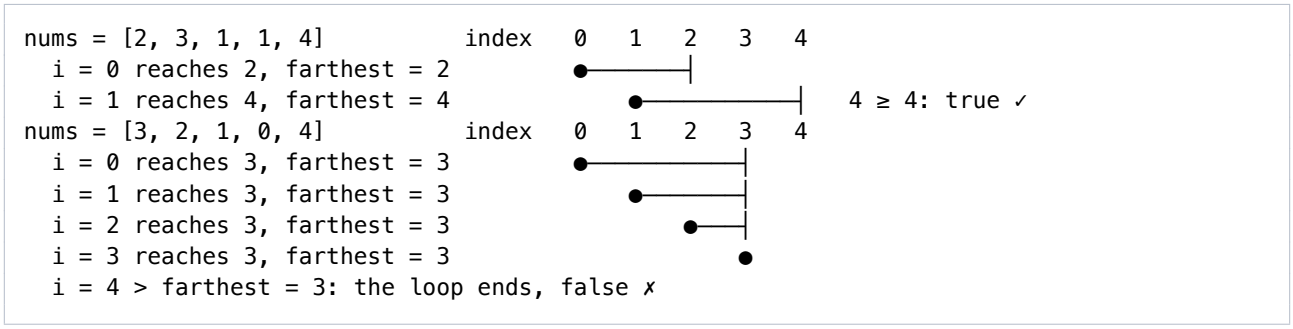
67. Jump Game

Track the Farthest Reach · Greedy · Tier 2

Each `nums[i]` is the longest jump allowed from index `i`, and any shorter jump is allowed too. Starting at index `0`, decide whether the last index can be reached. `[2, 3, 1, 1, 4]` can be crossed, one step to index 1 and then three to the end, while `[3, 2, 1, 0, 4]` cannot, because every path lands on the `0` at index 3.

The brute force explores jump sequences, exponentially many by plain recursion, or $O(n^2)$ with a reachable flag per index. The insight is that the reachable indices always form one solid block `[0, farthest]`: whoever can stand on `i` can also stand on every index from `i` to `i + nums[i]`, and those stretches overlap into a single interval. So the whole state is one number. Scan left to right while `i` stays inside the block, stretch the block to `i + nums[i]`, and answer yes the moment it covers the last index. If the scan steps past `farthest`, index `i` is unreachable, and so is everything beyond it, since no reachable index jumps that far.

Picture every index casting a shadow forward over the spots it can jump to. The walk begins under the shadow of index 0, and every index under the shadow extends it with its own, so the reachable region grows as one bar. A zero inside the bar is harmless while some earlier shadow passes over it; it becomes a wall only when no shadow reaches beyond it, which is exactly what happens at index 3 below.




```
FUNCTION CanJump(nums)
    farthest ← 0                          // indices 0..farthest are reachable
    i ← 0
    WHILE i ≤ farthest                    // i is a spot we can really stand on
        farthest ← max(farthest, i + nums[i])
        IF farthest ≥ len(nums) − 1       // the reach covers the last index
            RETURN true
        i ← i + 1
    RETURN false                          // walked past the reach: a wall
```

Each index is visited at most once with constant work, so the scan costs $O(n)$ time and $O(1)$ space. Reach for the farthest reach whenever moves go forward along a line and every spot up to a move's length is a legal landing: reachability collapses to one number. When jumps are exact, as in Jump Game III (LC 1306) where index `i` leads only to `i + arr[i]` and `i − arr[i]`, the block argument fails and the problem becomes a graph search over indices, the BFS of #39. Counting the fewest jumps is #68, which reads this same scan level by level. The classic mistake is dropping the `i ≤ farthest` guard and scanning every index: on `[3, 2, 1, 0, 4]` the scan then reads index 4, where it can never stand, stretches the reach to 8 and wrongly answers true.

Practice: LC 55 Jump Game · LC 45 Jump Game II · LC 1306 Jump Game III

68. Jump Game II

Greedy BFS over Reach Levels · Greedy · Tier 2

`nums[i]` is again the longest jump allowed from index `i`, and this time the last index is known to be reachable. Return the fewest jumps from index 0 to the last index. For `[2, 3, 1, 1, 4]` the answer is `2`: one step to index 1, then three steps to the end.

Fewest moves in an unweighted graph is BFS (#39), and the brute force runs it literally, enqueuing every index each jump can land on, $O(n^2)$ edges in the worst case. The insight is that every BFS level here is a contiguous interval. Level 0 is `[0, 0]`, and if level `k` ends at `b`, level `k + 1` is `[b + 1, farthest]`, where `farthest` is the largest `i + nums[i]` seen so far, because every spot up to a reach is a legal landing. So the queue collapses to two numbers: `current_end`, the right edge of the level being scanned, and `farthest`, the right edge of the next one. Walk `i` forward, pushing `farthest` out; when `i` reaches `current_end`, the level is used up, so count one more jump and move `current_end` to `farthest`.

Picture the indices tiled into consecutive bands, each band one jump farther from the start than the band before it. The scan reads each band once, measuring how far its best launcher reaches, and that reach is where the next band ends. The answer is the level number of the band that holds the last index. The loop stops just before the last index, because standing there needs no further jump.

```
index        0    1    2    3    4
nums         2    3    1    1    4
level 0      ●                          [0, 0]  the start
level 1           ●———●                 [1, 2]  up to 0 + 2 = 2
level 2                ●———●            [3, 4]  up to max(1 + 3, 2 + 1) = 4

  i   i + nums[i]   farthest   i = current_end?   jumps   current_end
  0        2            2         yes, jump         1          2
  1        4            4         no                1          2
  2        3            4         yes, jump         2          4
  3        4            4         no                2          4
  the loop stops before index 4: 2 jumps
```

```
FUNCTION MinJumps(nums)
    jumps ← 0
    current_end ← 0                       // right edge of the current level
    farthest ← 0                          // right edge of the next level
    FOR i FROM 0 TO len(nums) − 2         // never jump from the last index
        farthest ← max(farthest, i + nums[i])
        IF i = current_end                // this level is used up
            jumps ← jumps + 1
            current_end ← farthest        // the next level ends here
    RETURN jumps
```

Every index is read once, so the cost is $O(n)$ time and $O(1)$ space, against $O(n^2)$ for the explicit BFS. Reach for this level-by-interval BFS whenever the positions reachable in `k` steps always form a contiguous range: covering a line with jumps, with video clips (LC 1024) or with garden taps (LC 1326), once each clip or tap is recorded as a reach from its left end. When the end might be unreachable, check at the end of each level whether `farthest` is still `i`; if so, nothing beyond can be reached, so return `−1`.

The classic mistake is letting the loop run over the last index too: when the last index closes a level, the scan counts a jump it never takes, and `[2, 3, 1, 1, 4]` reports `3`.

Practice: LC 45 Jump Game II · LC 1024 Video Stitching · LC 1326 Minimum Number of Taps to Open to Water a Garden

69. Gas Station

Restart After Every Deficit · Greedy · Tier 2

Stations sit on a circle. Station `i` gives `gas[i]` fuel, and the drive on to the next station burns `cost[i]`. Starting with an empty tank, return the station from which a full lap is possible, or `−1` if there is none; the input promises at most one such station. With `gas = [4, 1, 2, 5, 1]` and `cost = [3, 3, 3, 1, 2]` the answer is `3`.

The brute force simulates a lap from every station, $O(n^2)$. Two facts make one pass enough. First, a lap exists exactly when the total gas covers the total cost, so `sum(gas) < sum(cost)` answers `−1` at once. Second, a failed attempt rules out a whole stretch, not just its first station. Say the tank, started empty at `start`, first runs dry on the drive out of station `i`. Every station `k` in between was reached from `start` with a non-negative tank, because no deficit had happened yet. Starting fresh at `k` means arriving there with an empty tank instead, which is never better, so that attempt also runs dry by station `i`. Every station from `start` to `i` is out, and the next candidate is `i + 1`, with the tank reset to zero.

Picture the running sum of `gas[i] − cost[i]` as a profile over the stations, the bottom row of the picture. A car that starts at station `s` carries exactly the profile's rise since the point just before `s`, so starting right after the lowest point keeps the tank non-negative all the way to the end of the array. Past the wrap, the profile repeats lifted by the total, which is non-negative, so the tank stays non-negative there too. Each deficit in the scan is the profile hitting a new low, and moving `start` to `i + 1` simply follows the low point down.

```
gas  = [4, 1, 2, 5, 1]   sum 13
cost = [3, 3, 3, 1, 2]   sum 12   13 ≥ 12, so a lap exists

  i   gas − cost   tank   start
  0        1         1     0
  1       −2        −1     0 → 2     deficit: stations 0 and 1 are out
  2       −1        −1     2 → 3     deficit: station 2 is out
  3        4         4     3
  4       −1         3     3         answer 3

running sum   1  −1  −2   2   1
                    ▲ lowest, after station 2: start at 3
```

```
FUNCTION CanCompleteCircuit(gas, cost)
    IF sum(gas) < sum(cost)                // not enough fuel for any lap
        RETURN −1
    start ← 0
    tank ← 0                               // fuel left since leaving start
    FOR i FROM 0 TO len(gas) − 1
        tank ← tank + gas[i] − cost[i]
        IF tank < 0                        // stranded: start..i are all out
            start ← i + 1
            tank ← 0
    RETURN start
```

One pass plus the two sums gives `O(n)` time and `O(1)` space. Reach for this pattern when a running balance around a loop or along a route must never go negative and the question is where to begin: the answer sits right after the lowest point of the prefix sums. It is the restart of Kadane's algorithm

(#28) in another costume, where a run that turned negative is dropped whole. The classic mistake is trusting the scan without the total check: for `gas = [2, 3, 4]` and `cost = [3, 4, 3]` the scan ends on station 2 with fuel to spare, yet no lap is possible, because the total gas `9` is less than the total cost `10`.

Practice: LC 134 Gas Station · LC 1413 Minimum Value to Get Positive Step by Step Sum · LC 918 Maximum Sum Circular Subarray

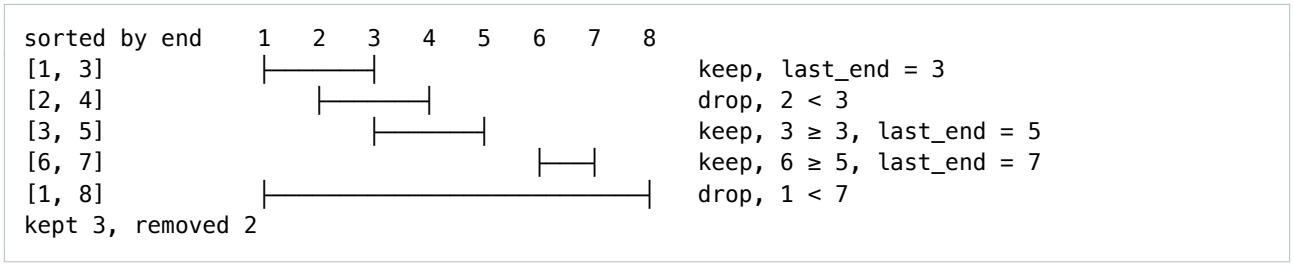
70. Interval Scheduling

Sort by End, Keep What Fits · Greedy · Tier 2

Given intervals `[start, end]` with `start < end`, keep as many as possible so that no two overlap; ends that only touch, such as `[1, 3]` and `[3, 5]`, do not count as overlapping. The fewest removals that leave the rest non-overlapping is then everything not kept. From `[[1, 3], [2, 4], [3, 5], [1, 8], [6, 7]]` three can stay, `[1, 3]`, `[3, 5]` and `[6, 7]`, so two must go.

The brute force tries every subset, `O(2^n)` of them. The greedy question is which interval to keep first, and the answer is the one that ends earliest, because it leaves the most room for everything after it. Sort by end, then sweep once: keep an interval if it starts at or after `last_end`, the end of the last kept one, and skip it otherwise. Merge Intervals (#26) sorts by start because it wants to glue overlaps together; scheduling sorts by end because it wants to leave room.

The exchange argument shows that the earliest end is a safe first choice. Take any optimal selection and its first interval by end. The earliest-ending interval overall ends no later, so swapping it in keeps every other chosen interval compatible: each of them starts at or after the old first one's end, hence at or after the new one's end too. The optimum keeps its size and now begins with the greedy choice, and the same argument repeats on the intervals that start after it. On the timeline, `last_end` is a finish line that only moves right, and each kept interval pushes it as little as possible.




```
FUNCTION MaxNonOverlapping(intervals)
    by_end ← empty list
    FOR EACH (start, end) IN intervals
        by_end.append((end, start))        // end first: the sort orders by end
    by_end.sort()
    kept ← 0
    last_end ← −∞                          // end of the last kept interval
    FOR EACH (end, start) IN by_end
        IF start ≥ last_end                // fits after the last kept one
            kept ← kept + 1
            last_end ← end
    RETURN kept

FUNCTION MinRemovals(intervals)            // LC 435
    RETURN len(intervals) − MaxNonOverlapping(intervals)
```

Sorting costs `O(n log n)` and the sweep `O(n)`, plus `O(n)` space for the sorted copy. Reach for the earliest end when a problem asks for the most non-overlapping intervals, the fewest removals to get there (LC 435), the longest chain of pairs (LC 646) or the fewest arrows that burst every balloon (LC 452), whose count equals the number kept when touching counts as overlapping. Attending one event per day (LC 1353) runs the same earliest-deadline idea day by day with a min-heap of end days. Read the touching rule carefully: `start ≥ last_end` keeps touching intervals, while balloons and pair chains need `start`

`> last_end`. The classic mistake is sorting by start: on `[[1, 10], [2, 3], [4, 5]]` the long `[1, 10]` comes first, is kept, and blocks both short intervals inside it, keeping one where two fit.

Practice: LC 435 Non-overlapping Intervals · LC 452 Minimum Number of Arrows to Burst Balloons · LC 646 Maximum Length of Pair Chain · LC 1353 Maximum Number of Events That Can Be Attended

71. Task Scheduler

Frame the Most Frequent Task · Greedy · Tier 2

Each task takes one unit of time, and two runs of the same task need at least `n` units between them; the CPU may sit idle. Return the fewest units that run every task. For `["A", "A", "A", "B", "B", "B"]` with `n = 2` the answer is 8: `A B idle A B idle A B`.

Simulating unit by unit works, but the answer is a formula, because the most frequent task is the bottleneck. Its `max_count` copies need a gap of `n` units after every copy but the last, so they frame a grid: `max_count − 1` full rows of width `n + 1`, each starting with that task, and then a last row. Every other task drops into the gaps column by column, most frequent first, an order that keeps each task's copies at least a full row, `n + 1` units, apart. The tasks tied at `max_count` also need a slot in the last row, so `frame = (max_count − 1) ∗ (n + 1) + tied`. Nothing beats the frame, because the last copies of the tied tasks cannot start before the last row and each needs a unit of its own.

Picture the grid filling up. While gaps remain, the leftover gaps become idles and the answer is the frame. Once the other tasks overflow the gaps, the rows simply widen past `n + 1`; the copies of each task are still a full row apart, so no idle is needed at all and the answer is `len(tasks)`, which no schedule can beat either. Both cases fold into `max(frame, len(tasks))`.

```
tasks A A A B B B, n = 2      counts A 3, B 3: max_count = 3, tied = 2
            ┌─ n + 1 = 3 ─┐
   row 1    A    B    idle
   row 2    A    B    idle
   row 3    A    B         last row: only the tied tasks
   frame = (3 − 1) ∗ 3 + 2 = 8 ≥ len(tasks) = 6: answer 8

add C C D D                   counts A 3, B 3, C 2, D 2: frame still 8
   row 1    A    B    C    D  rows widen past n + 1
   row 2    A    B    C    D
   row 3    A    B
   frame = 8 < len(tasks) = 10: answer 10, no idles
```

```
FUNCTION LeastInterval(tasks, n)
    counts ← empty map
    FOR EACH task IN tasks
        counts[task] ← counts.get(task, 0) + 1
    max_count ← 0
    FOR EACH task IN counts
        max_count ← max(max_count, counts[task])
    tied ← 0                                  // tasks that also reach max_count
    FOR EACH task IN counts
        IF counts[task] = max_count
            tied ← tied + 1
    frame ← (max_count − 1) ∗ (n + 1) + tied       // full rows, then a last row
    RETURN max(frame, len(tasks))          // overflow: rows widen, no idles
```

Counting takes `O(len(tasks))` time, and the map holds one entry per distinct task, `O(1)` for a fixed alphabet such as the 26 letters of LC 621. Reach for the frame when a problem spaces out repeats with a cooldown and asks only for a length or a yes-or-no answer. When the actual order is needed, simulate instead: a max-heap of remaining counts picks the most frequent ready task each unit, and a

queue holds the tasks that are cooling down. Reorganize String (LC 767) is the case `n = 1` with no idle allowed, so an arrangement exists exactly when `frame ≤ len(s)`; LC 358 is the same with cooldown `k − 1`. The classic mistake is forgetting `tied` and adding just `1` for the last row, which answers `7` on the example, while `B` needs a slot in that row too and the answer is `8`.

Practice: LC 621 Task Scheduler · LC 767 Reorganize String · LC 358 Rearrange String k Distance Apart

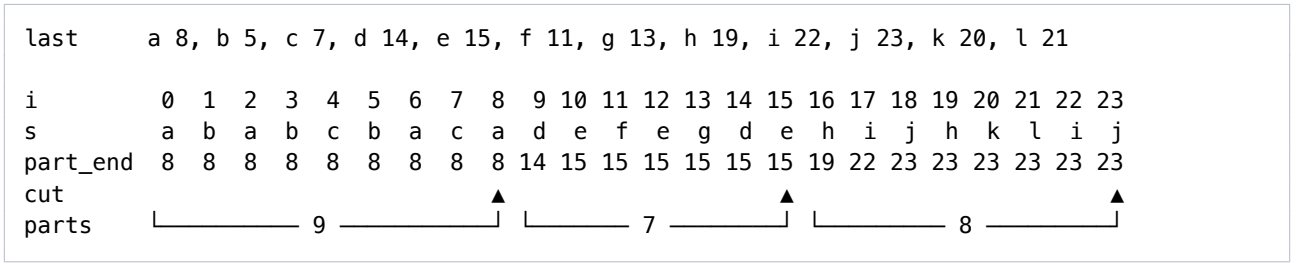
72. Partition Labels

Extend to the Last Occurrence · Greedy · Tier 2

Cut a string into as many parts as possible so that each letter appears in at most one part, and return the part sizes in order. `"ababcbacadefegdehijhklij"` splits into `"ababcbaca"`, `"defegde"` and `"hijhklij"`, so the answer is `[9, 7, 8]`.

A part that contains a letter must stretch to that letter's last occurrence, and every letter swept up on the way can stretch it further. The brute force grows a part one character at a time and rescans the rest of the string for its letters, $O(n^2)$. Instead, one pass records `last[c]`, the final index of every letter. A second pass carries `part_end`, the farthest last occurrence among the letters of the current part: each character pushes `part_end` to `max(part_end, last[c])`, and when `i` catches up with `part_end`, no letter inside reaches further, so the part closes and the next one starts at `i + 1`. Cutting at the first such moment is what makes the count maximal, because every earlier position inside the part has some letter crossing it, so no valid cut is ever skipped.

Picture each letter as an interval from its first copy to its last. The parts are exactly the groups of overlapping intervals, so this is Merge Intervals (#26) without the sort: scanning the string meets the intervals in order of their starts for free, and `part_end` is the right edge of the group being merged. In the picture, `a` alone drags the first part to index 8, while `e` and `j` set the ends of the other two.




```
FUNCTION PartitionLabels(s)
    last ← empty map                          // letter → index of its last copy
    FOR i FROM 0 TO len(s) − 1
        last[s[i]] ← i
    sizes ← empty list
    start ← 0                                 // first index of the current part
    part_end ← 0                              // the part must reach at least here
    FOR i FROM 0 TO len(s) − 1
        part_end ← max(part_end, last[s[i]])      // this letter drags it along
        IF i = part_end                       // nothing inside reaches past i
            sizes.append(i − start + 1)
            start ← i + 1
    RETURN sizes
```

Two passes give $O(n)$ time, and `last` holds one entry per distinct letter, $O(1)$ for the 26 lowercase letters. Reach for this pattern when a sequence must be cut so that no item, a letter, a value or an id, crosses a cut: record how far each item reaches, and cut wherever the running maximum reach meets the scan. Max Chunks To Make Sorted (LC 769) is the same scan, with the running maximum of the values as the reach. The classic mistake is `part_end = last[s[i]]` without the `max`, which lets a short-lived letter pull the end back: on `"abac"` it cuts after `"ab"` although `a` returns at index 2, answering `[2, 1, 1]` instead of `[3, 1]`.

Practice: LC 763 Partition Labels · LC 56 Merge Intervals · LC 769 Max Chunks To Make Sorted

73. Combination Sum

Unlimited Reuse with a Start Index · Backtracking · Tier 2

Given an array of distinct positive integers `candidates` and a `target`, return every combination of candidates that sums to the target. A candidate may be used any number of times, and order does not matter, so `[2, 2, 3]` and `[3, 2, 2]` are the same combination. For `[2, 3, 6, 7]` and target 7 the answer is `[[2, 2, 3], [7]]`.

This is the subsets tree of #30 with two changes. First, a chosen candidate may be chosen again, so the recursion passes `start = i` instead of `i + 1`: the next pick may repeat the current candidate or move right, but it never moves left. Every path is therefore non-decreasing, one canonical order per combination, so each combination is built exactly once. Second, each call also carries what is still missing, `remaining`, and a node records its path only when `remaining` hits 0 exactly.

Sorting first turns overshooting into a wall. Scanning left to right, the first candidate larger than `remaining` proves that every later one is larger too, so the loop breaks instead of continuing, and a whole fan of hopeless branches is never entered. In the tree below each node shows its path and what is still missing; the whole search takes ten calls.

```
[]  7 left                  candidates sorted: [2, 3, 6, 7]
├── [2]  5 left
│   ├── [2,2]  3 left
│   │   ├── [2,2,2]  1 left   then 2 > 1: break
│   │   ├── [2,2,3]  0 left   ✓ record
│   │   └── 6 > 3: break
│   ├── [2,3]  2 left         loop starts at 3 (start = 1); 3 > 2: break
│   └── 6 > 5: break
├── [3]  4 left
│   ├── [3,3]  1 left         then 3 > 1: break
│   └── 6 > 4: break
├── [6]  1 left               then 6 > 1: break
└── [7]  0 left               ✓ record
```

```
FUNCTION Backtrack(candidates, start, remaining, path, result)
    IF remaining = 0
        result.append(path.copy())          // exact hit: one combination
        RETURN
    FOR i FROM start TO len(candidates) - 1
        IF candidates[i] > remaining
            BREAK                           // sorted: the rest are bigger
        path.append(candidates[i])          // choose
        // pass i, not i + 1: the same candidate may be chosen again
        Backtrack(candidates, i, remaining - candidates[i], path, result)
        path.pop()                          // un-choose

FUNCTION CombinationSum(candidates, target)
    candidates ← sorted(candidates)         // makes the BREAK valid
    result ← empty list
    Backtrack(candidates, 0, target, empty list, result)
    RETURN result
```

The tree is at most `t / m` levels deep, for target `t` and smallest candidate `m`, and branches at most `n` ways, so it has `O(n^(t/m + 1))` nodes: exponential, which the small constraints keep in check; the extra space is the `O(t / m)` depth. Signals: all combinations that reach a sum, an unlimited supply of each item, and a request to list them rather than count them. Two neighbors change a line or two. LC 40 uses each candidate at most once and may contain duplicates, so it recurses with `i + 1` and skips `candidates[i]` when `i > start` and `candidates[i] = candidates[i − 1]`, the rule from Subsets II; LC 216 picks exactly `k` distinct digits from 1 to 9. LC 377 only looks similar: it counts ordered sequences, which is a counting DP rather than a listing. Recursing with `i + 1` here loses `[2, 2, 3]`. The classic mistake goes the other way, recursing with `start = 0`: the path may then move left, and `[2, 2, 3]`, `[2, 3, 2]` and `[3, 2, 2]` come back as three different answers.

Practice: LC 39 Combination Sum · LC 40 Combination Sum II · LC 216 Combination Sum III · LC 377 Combination Sum IV

74. Word Search

Grid Backtracking with In-Place Marks · Backtracking · Tier 2

Given a grid of letters and a word, decide whether the word can be spelled along a path of horizontally or vertically adjacent cells, using each cell at most once. In the grid with rows `ABCE`, `SFCS` and `ADEE`, the word `ABCCED` is there, while `ABCB` is not, because its second `B` would have to reuse a cell.

A path is a sequence of choices, so this is backtracking on the grid. Every cell is tried as the start. From a cell that matches `word[k]`, the search steps to each of the four neighbors looking for `word[k + 1]`, and it succeeds when k reaches the length of the word. The path must not cross itself, so before stepping on, the cell is overwritten with `#`, a character no word contains, and once the search from that cell is over, the letter is written back. The mark replaces a visited set, and it folds the visited test into the letter test, because `#` never equals `word[k]`.

Picture a snake growing through the grid. Its body is the marked cells, its head tries the neighbors in a fixed order (down, up, right, left), and when the head is stuck the snake pulls back one cell and unmarks it, so that a different path may use that cell later.

```
board       "ABCCED": k on its path       "ABCB": the board when k = 3
A B C E     0 1 2 .                       # # # E    the C at (0,2) needs a B,
S F C S     . . 3 .                       S F C S    and its only B neighbor
A D E E     . 5 4 .                       A D E E    is '#': dead end

ABCCED at k = 5: down is off the grid, up is '#', right is E, left is D ✓
ABCB: unmark C, B, A in turn; the other A, at (2,0), fails at once: false
```

```
FUNCTION Dfs(board, word, row, col, k)
    IF k = len(word)
        RETURN true                          // every letter matched
    IF row < 0 OR row ≥ len(board) OR col < 0 OR col ≥ len(board[0])
        RETURN false                         // off the grid
    IF board[row][col] ≠ word[k]
        RETURN false                         // wrong letter, or a '#'
    board[row][col] ← '#'                    // mark: on the path now
    found ← false
    FOR EACH (d_row, d_col) IN [(1, 0), (-1, 0), (0, 1), (0, -1)]
        IF Dfs(board, word, row + d_row, col + d_col, k + 1)
            found ← true
            BREAK
    board[row][col] ← word[k]                // unmark on the way back
    RETURN found

FUNCTION Exist(board, word)
    FOR row FROM 0 TO len(board) - 1
        FOR col FROM 0 TO len(board[0]) - 1
            IF Dfs(board, word, row, col, 0)
                RETURN true
    RETURN false
```

Each of the `m · n` starts grows paths of up to L cells, and after the first step the cell behind the head is always marked, so at most three ways lead on: the worst case is `O(m · n · 3^L)`, usually quoted as the looser `O(m · n · 4^L)`, with `O(L)` recursion depth. Two cheap prunes help in practice. Count the

letters of the grid first and answer false if the word needs more of some letter than the grid holds; and if the word's last letter is rarer in the grid than its first, search for the reversed word instead, so fewer starts pass the first test. Signals: a grid, a path through adjacent cells, no reuse, one word. For many words at once, the trie-guided version is #85. The classic mistake is forgetting to write the letter back: the `#` stays behind and blocks later paths, and on this grid a failed search for `ABCB` would leave every `A` marked, so a following search for `ABCCED` would wrongly say false.

Practice: LC 79 Word Search · LC 212 Word Search II

75. Construct Binary Tree from Preorder and Inorder

Preorder Picks the Root, Inorder Splits the Rest · Trees · Tier 2

Given the preorder and inorder traversals of a binary tree whose values are all different, rebuild the tree. Preorder `[3, 9, 20, 15, 7]` and inorder `[9, 3, 15, 20, 7]` describe a root `3` with a left child `9` and a right child `20`, whose own children are `15` and `7`; in level order the tree is `[3, 9, 20, null, null, 15, 7]`.

Neither traversal pins the tree down alone, but together they do, because they answer different questions. Preorder lists every root before everything below it, so its first value is the root of the whole tree. Inorder lists the left subtree, then the root, then the right subtree, so the root's position there splits everything else into the two subtrees. The direct recursion finds the root with a linear scan of inorder and slices both arrays for the two recursive calls, which costs `O(n²)` on a chain-shaped tree. Two changes make it linear: a map from value to inorder index finds the root in `O(1)`, and each call receives the bounds `lo..hi` of its inorder range instead of copied slices.

The preorder side needs no bounds at all. `Build` creates a root, then finishes the entire left subtree, then the right one, so it asks for roots in exactly the order preorder lists them: root, left block, right block. One index moving left to right through preorder, kept in a one-element box that every call shares, hands each call its root. The left call consumes exactly the `mid − lo` values of the left block and stops, leaving the index on the first value of the right block, which is why the left subtree must be built first.

```
preorder   3 | 9 | 20 15 7        root | left block | right block
inorder    9 | 3 | 15 20 7        left block | root | right block
position   {9: 0, 3: 1, 15: 2, 20: 3, 7: 4}

call          pre_index   root   mid   left range    right range
Build(0, 4)       0         3     1    0..0          2..4
Build(0, 0)       1         9     0    0..-1 empty   1..0 empty
Build(2, 4)       2        20     3    2..2          4..4
Build(2, 2)       3        15     2    2..1 empty    3..2 empty
Build(4, 4)       4         7     4    4..3 empty    5..4 empty

      3           level order: [3, 9, 20, null, null, 15, 7]
     / \
    9   20
       /  \
      15   7
```

```
FUNCTION ConstructTree(preorder, inorder)
    position ← empty map                    // value → index in inorder
    FOR i FROM 0 TO len(inorder) − 1
        position[inorder[i]] ← i
    pre_index ← [0]                         // one box every call shares
    RETURN Build(preorder, position, pre_index, 0, len(inorder) − 1)

FUNCTION Build(preorder, position, pre_index, lo, hi)
    IF lo > hi
        RETURN null                         // empty range: no subtree here
    value ← preorder[pre_index[0]]          // roots come in preorder
    pre_index[0] ← pre_index[0] + 1
    root ← new TreeNode(value)
```

```
    mid ← position[value]                  // the root splits inorder[lo..hi]
    root.left ← Build(preorder, position, pre_index, lo, mid − 1)
    root.right ← Build(preorder, position, pre_index, mid + 1, hi)
    RETURN root
```

Every node is created once after an `O(1)` lookup, so the build runs in `O(n)` time, with `O(n)` space for the map and recursion as deep as the tree is tall; a chain of more than about a thousand nodes passes Python's default recursion limit. Reach for this whenever two traversals must become a tree again. Inorder with postorder is the mirror image: walk postorder from its end, where the root sits, and build the right subtree before the left. Preorder with postorder fixes the tree only when no node has a single child, since a lone child could hang on either side. A binary search tree needs its preorder alone, because its inorder is the sorted order, and null markers make a single preorder enough for any tree (#36). The classic mistake is building the right subtree first: the right range `2..4` then receives 9, the root of the left block, and from there every range is wrong until the build reads past the end of preorder.

Practice: LC 105 Construct Binary Tree from Preorder and Inorder Traversal · LC 106 Construct Binary Tree from Inorder and Postorder Traversal · LC 889 Construct Binary Tree from Preorder and Postorder Traversal · LC 1008 Construct Binary Search Tree from Preorder Traversal

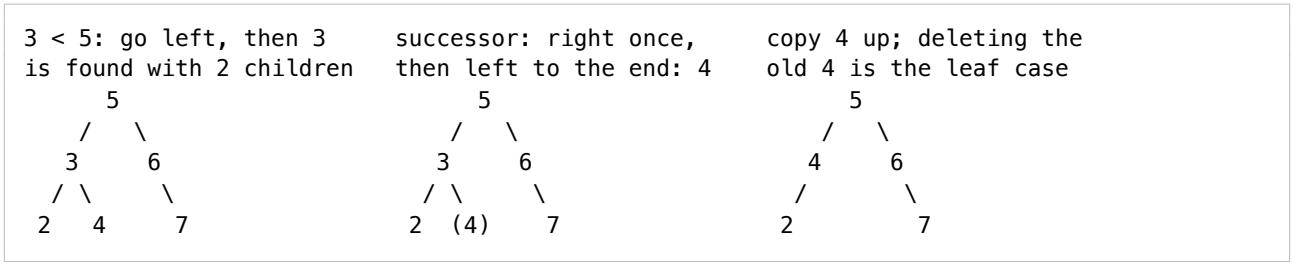
76. BST Search, Insert, Delete

Delete with the In-Order Successor · BST · Tier 2

A binary search tree keeps every value in a node's left subtree smaller than the node and every value in its right subtree larger. Search for a value, insert a new one and delete an existing one, keeping that order intact. Deleting 3 from `[5, 3, 6, 2, 4, null, 7]` gives `[5, 4, 6, 2, null, null, 7]`: the successor 4 moves up into the gap.

All three operations ride on one move. At each node a single comparison throws away an entire subtree, so the walk follows one path from the root toward a leaf. Search loops down that path until it meets the value or falls off the tree. Insert follows the same path to the empty spot where the value would have been and hangs a new leaf there. Written recursively, each call returns the root of its subtree and the parent reattaches whatever comes back, so the one new link is made without any special case, not even for an empty tree.

Delete is the operation with cases, sorted by the number of children the doomed node has. A leaf simply disappears, and its parent receives null. A node with one child is spliced out and the child takes its place; the order survives because that whole subtree already sat on the correct side of the parent. A node with two children cannot leave without orphaning one of them, so it keeps its place and changes its value instead. The standard replacement is the in-order successor, the smallest value of the right subtree, found by stepping right once and then left as far as possible: it is larger than everything on the left and smaller than everything else on the right. Copy it up, then delete it from the right subtree, where it has no left child and so falls into an easy case.




```
FUNCTION Search(root, val)
    node ← root
    WHILE node ≠ null AND node.val ≠ val
        IF val < node.val
            node ← node.left
        ELSE
            node ← node.right
    RETURN node                              // the node holding val, or null

FUNCTION Insert(root, val)
    IF root = null
        RETURN new TreeNode(val)             // the empty spot: a new leaf
    IF val < root.val
        root.left ← Insert(root.left, val)
    ELSE IF val > root.val
        root.right ← Insert(root.right, val)
    RETURN root                              // equal: already present

FUNCTION Delete(root, val)
    IF root = null
```

```
        RETURN null                         // val is not in the tree
    IF val < root.val
        root.left ← Delete(root.left, val)
    ELSE IF val > root.val
        root.right ← Delete(root.right, val)
    ELSE
        IF root.left = null                 // leaf, or right child only
            RETURN root.right
        IF root.right = null                // left child only
            RETURN root.left
        successor ← root.right              // two children: leftmost node
        WHILE successor.left ≠ null         // of the right subtree
            successor ← successor.left
        root.val ← successor.val            // copy it up...
        root.right ← Delete(root.right, successor.val)   // ...and remove it
    RETURN root
```

Each operation walks a single path, so all three cost `O(h)`: `O(log n)` while the tree stays balanced, `O(n)` once sorted insertions have stretched it into a chain, which is why the ordered maps of standard libraries are self-balancing trees. Reach for a BST when data must stay sorted while it changes: the next larger value, every value in a range, the `k`-th smallest (#77 walks a BST in order). The in-order predecessor, the largest value of the left subtree, works just as well as the replacement. The classic mistake is deleting the original value from the right subtree after copying the successor up, instead of the successor's value: the original is no longer there, nothing is removed, and the tree ends up holding the successor twice.

Practice: LC 700 Search in a Binary Search Tree · LC 701 Insert into a Binary Search Tree · LC 450 Delete Node in a BST

77. BST Iterator

Controlled In-Order with a Stack · BST · Tier 2

Design an iterator over a binary search tree that hands out its values in ascending order: `next()` returns the next smallest value and `has_next()` says whether any are left. For `[7, 3, 15, null, null, 9, 20]`, five calls to `next()` return `3`, `7`, `9`, `15` and `20`, and `has_next()` turns false after the last one.

The easy answer runs an inorder traversal up front and stores all `n` values in a list, paying `O(n)` memory before the first call. A real iterator does the traversal lazily, one step per call, which means pausing an inorder walk in the middle. A recursive walk knows where it is through the ancestors whose values are still owed, the ones it left by going down into their left subtrees; keep exactly those on an explicit stack and pausing is free. Start by pushing the root and its whole left spine, so the top of the stack is the leftmost node, the smallest value. Each `next()` pops the top, the smallest value not yet returned, and before handing it out pushes the left spine of the popped node's right child, because the values just above the popped one live in that right subtree, smallest first along its spine.

The stack always holds a chain of nodes, each in the left subtree of the one below it. Everything inside the right subtree being explored is smaller than every ancestor still waiting underneath, which is why the order never breaks.

```
       7          call          returns   stack after (top on the right)
      / \         init                    [7, 3]
     3   15       next()          3       [7]
        /  \      next()          7       [15, 9]
       9    20    has_next()      true    [15, 9]
                  next()          9       [15]
                  next()          15      [20]
                  next()          20      []
                  has_next()      false   []
```

```
CLASS BSTIterator
    FUNCTION Init(root)
        self.stack ← empty stack
        self.PushLeft(root)

    FUNCTION PushLeft(node)
        WHILE node ≠ null                  // walk down the left spine
            self.stack.push(node)
            node ← node.left

    FUNCTION Next()
        node ← self.stack.pop()            // the smallest value still waiting
        self.PushLeft(node.right)          // its successors live on the right
        RETURN node.val

    FUNCTION HasNext()
        RETURN len(self.stack) > 0

FUNCTION KthSmallest(root, k)
    iterator ← new BSTIterator(root)
    value ← null
    FOR i FROM 0 TO k − 1
        value ← iterator.Next()
```

```
    RETURN value
```

A single `next()` can push a long spine, but over the whole iteration every node is pushed once and popped once, so `n` calls cost `O(n)` in total, `O(1)` amortized each, and `has_next()` is always `O(1)`. The stack holds at most one node per level, `O(h)` space instead of `O(n)`. The `k`-th smallest value is just `k` calls to `next()`, `O(h + k)` time. Reach for this whenever a tree must be consumed in sorted order one value at a time: merging two BSTs by advancing two iterators like two sorted lists, finding a pair with a given sum with one ascending and one descending iterator, or finding an in-order successor. The classic mistake is pushing only the popped node's right child instead of its whole left spine; on this tree the iterator then returns `3, 7, 15, 20` and never sees `9`.

Practice: LC 173 Binary Search Tree Iterator · LC 230 Kth Smallest Element in a BST · LC 285 Inorder Successor in BST · LC 1305 All Elements in Two Binary Search Trees

78. Multi-Source BFS

Rotting Oranges · BFS · Tier 2

In a grid, `0` is an empty cell, `1` a fresh orange and `2` a rotten one. Every minute, each fresh orange next to a rotten one, up, down, left or right, turns rotten. Return the number of minutes until no fresh orange is left, or `-1` if some orange can never rot. In `[[2, 1, 1, 1], [1, 0, 1, 1], [1, 1, 1, 2]]` the two rotten corners infect everything in 2 minutes.

Rot that spreads one step a minute is BFS (#39) on a graph whose nodes are the orange cells: the minute an orange rots is its distance to the nearest rotten orange, and the answer is the largest of those distances. The brute force runs a separate BFS from each of the `k` rotten oranges and keeps the minimum per cell, `O(k · rows · cols)`. One BFS does it all if every rotten orange enters the queue at minute 0. Imagine an extra node joined to every rotten orange: a single BFS from it reaches each cell through the nearest rotten orange, and seeding the queue with all of them is that same BFS with its first step already taken. The layer loop of level-order traversal (#32) counts the minutes, because each pass pops exactly the oranges that rotted in the minute before.

Picture several stones dropped into a pond at the same moment. Their ripples spread in lockstep, and wherever two meet, a cell belongs to whichever ripple arrived first. In the example each corner alone would need 5 minutes, but the two waves meet in the middle, and the combined map is the cell-by-cell minimum of the two single-source maps. Counting `fresh` up front gives both the stop condition and the `-1` test: the loop ends the moment the last fresh orange rots, and an orange still fresh when the queue runs dry is walled off from every rotten one.

```
the minute each cell rots (· is empty)

grid        both at once     only (0,0)     only (2,3)
2 1 1 1     0 1 2 2          0 1 2 3        5 4 3 2
1 · 1 1     1 · 2 1          1 · 3 4        4 · 2 1
1 1 1 2     2 2 1 0          2 3 4 5        3 2 1 0
            answer 2         answer 5       answer 5

minute 0: the queue holds (0,0) (2,3)             fresh 9
minute 1: rots (1,0) (0,1) (1,3) (2,2)            fresh 5
minute 2: rots (2,0) (0,2) (0,3) (1,2) (2,1)      fresh 0: stop, 2 minutes
```

```
DIRECTIONS ← [(1, 0), (-1, 0), (0, 1), (0, -1)]

FUNCTION OrangesRotting(grid)
    rows ← len(grid)
    cols ← len(grid[0])
    queue ← empty queue
    fresh ← 0
    FOR row FROM 0 TO rows - 1
        FOR col FROM 0 TO cols - 1
            IF grid[row][col] = 2
                queue.enqueue((row, col))  // every source starts at minute 0
            ELSE IF grid[row][col] = 1
                fresh ← fresh + 1
    minutes ← 0
    WHILE queue is not empty AND fresh > 0
        FOR i FROM 1 TO len(queue)         // len read once: this layer only
```

```
            (row, col) ← queue.dequeue()
            FOR EACH (d_row, d_col) IN DIRECTIONS
                next_row ← row + d_row
                next_col ← col + d_col
                inside ← 0 ≤ next_row < rows AND 0 ≤ next_col < cols
                IF inside AND grid[next_row][next_col] = 1
                    grid[next_row][next_col] ← 2   // rot now: enqueued once
                    fresh ← fresh − 1
                    queue.enqueue((next_row, next_col))
        minutes ← minutes + 1
    IF fresh > 0
        RETURN −1                          // walled off from every rotten one
    RETURN minutes
```

Each cell enters the queue at most once and is looked at from at most four neighbors, so the search takes `O(rows · cols)` time and space. Reach for multi-source BFS whenever a problem asks for each cell's distance to the nearest of many sources, or how long something takes to spread from several starting points at once: the distance to the nearest `0` (LC 542), to the nearest gate (LC 286), or the water cell farthest from any land (LC 1162). The classic mistake is looping while the queue is non-empty and counting the final pass, in which the last oranges to rot infect nobody: that answers 3 here instead of 2. The `fresh > 0` condition stops the clock on the right minute.

Practice: LC 994 Rotting Oranges · LC 542 01 Matrix · LC 286 Walls and Gates · LC 1162 As Far from Land as Possible

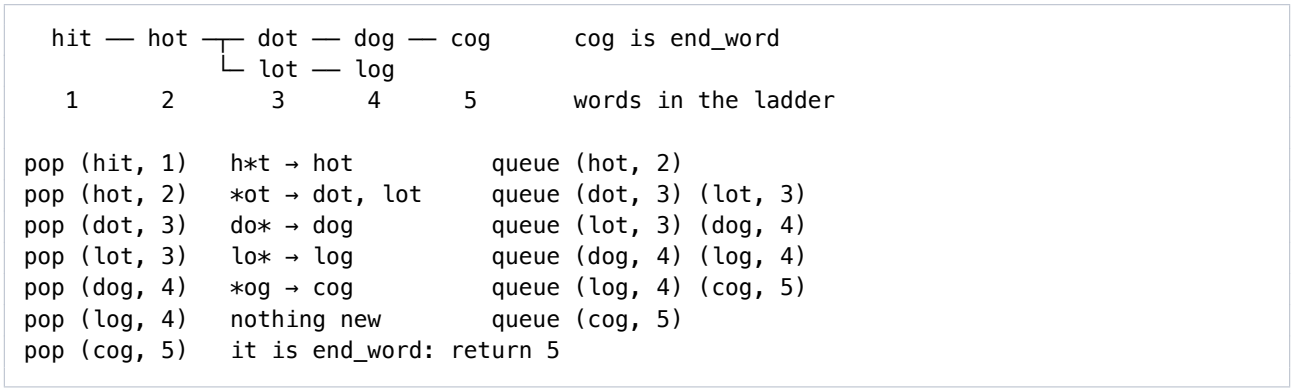
79. Word Ladder

BFS on an Implicit Graph · BFS · Tier 2

Turn `begin_word` into `end_word` by changing one letter at a time, where every word after the first must come from `word_list`. Return the number of words in the shortest such ladder, or `0` if there is none. From `hit` to `cog` with the list `hot, dot, dog, lot, log, cog`, one shortest ladder is `hit → hot → dot → dog → cog`, so the answer is `5`.

The words are the nodes of a graph that is never written down: two words are joined when they differ in exactly one letter, and the question is a shortest path in that graph, which is BFS (#39). Building the graph by comparing every pair of words costs $O(n^2 \cdot L)$ for n words of length L. Generating neighbors on demand is cheaper for a large dictionary: change each of the L positions to each of the 26 letters and keep the candidates that are unvisited dictionary words. The set of unvisited words does double duty. It is the dictionary, so a string that is not a word never becomes a neighbor, and it is the visited mark, because a word is removed the moment it is enqueued and can never enter the queue twice. Each queue entry carries the length of the ladder that reached it, and the first time `end_word` leaves the queue, that length is the answer.

Picture BFS rings around `hit`, one ring per changed letter. The pattern beside each step names the position that changed: `hot` and `dot` and `lot` all match `*ot`, which is why they sit one step apart. Grouping the dictionary by such patterns ahead of time, `*ot → [hot, dot, lot]`, is the other standard way to find neighbors.




```
FUNCTION LadderLength(begin_word, end_word, word_list)
    unvisited ← set(word_list)              // dictionary words not reached yet
    IF end_word NOT IN unvisited
        RETURN 0
    IF begin_word IN unvisited
        unvisited.remove(begin_word)        // the start counts as reached
    queue ← empty queue
    queue.enqueue((begin_word, 1))          // (word, words in the ladder)
    WHILE queue is not empty
        (word, length) ← queue.dequeue()
        IF word = end_word
            RETURN length
        FOR i FROM 0 TO len(word) − 1
            FOR EACH letter IN "abcdefghijklmnopqrstuvwxyz"
                candidate ← word with position i replaced by letter
                IF candidate IN unvisited          // a word not reached yet
                    unvisited.remove(candidate)    // mark when enqueued
```

```
                queue.enqueue((candidate, length + 1))
    RETURN 0
```

Each word is enqueued at most once and tries `26 · L` candidates that each cost `O(L)` to build and look up, so the search takes `O(n · 26 · L²)` time and `O(n · L)` space. Reach for BFS on an implicit graph whenever the states are strings, lock combinations or board positions and one move changes a state a little: Open the Lock turns one wheel per move, and Minimum Genetic Mutation changes one gene letter. Bidirectional BFS grows one frontier from each end, always expanding the smaller, and stops when they touch; with `b` neighbors per word and a ladder of `d` steps it explores about `2 · b^(d/2)` words instead of `b^d`. Word Ladder II wants every shortest ladder, so it records each word's parents from the previous layer and walks them back from `end_word`. The classic mistake is counting changes instead of words: starting the length at 0 answers 4.

Practice: LC 127 Word Ladder · LC 126 Word Ladder II · LC 752 Open the Lock · LC 433 Minimum Genetic Mutation

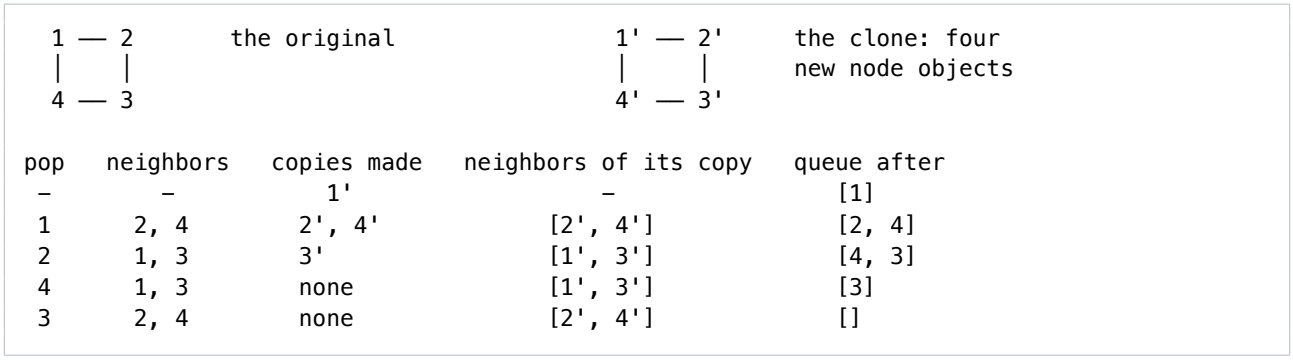
80. Clone Graph

The Old-to-New Map Doubles as Visited · Graphs · Tier 2

Given one node of a connected undirected graph, where every node holds a value and a list of its neighbors, return a deep copy of the whole graph: new nodes with the same values, wired the same way, without a single pointer into the original. The square 1–2–3–4–1, written as the adjacency list `[[2, 4], [1, 3], [2, 4], [1, 3]]` where row `i` lists the neighbors of node `i + 1`, must come back as the same list built from four brand-new nodes.

A copy that simply copies every neighbor it meets goes wrong in two ways. It never stops, because every undirected edge is a cycle of pointers: copying 1 copies its neighbor 2, whose list leads straight back to 1. And even a copy that somehow stopped would give a node reached along two paths two copies, as 3 is reached from both 2 and 4, so the clone would not have the original's shape. Both failures come from forgetting what was already copied, and one map from each original node to its copy cures both. The map answers "have I met this node before?" exactly as a visited set would, and it also hands over the copy that a new edge must point at. So the map is the visited set, and a breadth-first search over the original (#39) does the rest.

The search follows one rule. A node's copy is made the moment the node is first seen as somebody's neighbor, and the node is queued then. When a node is popped, every one of its edges is copied by appending the neighbor's copy to the popped node's copy. An edge to a node that already has a copy is appended too, and those edges are what close the cycles in the clone. Each node is popped exactly once, so each copy receives its neighbor list exactly once, in the original's order.




```
CLASS Node
    FUNCTION Init(val)
        self.val ← val
        self.neighbors ← empty list

FUNCTION CloneGraph(node)
    IF node = null
        RETURN null
    copies ← empty map                      // old node → its copy
    copies[node] ← new Node(node.val)
    queue ← empty queue
    queue.enqueue(node)
    WHILE queue is not empty
        current ← queue.dequeue()
        FOR EACH neighbor IN current.neighbors
            IF neighbor NOT IN copies       // first sighting: copy, visit later
                copies[neighbor] ← new Node(neighbor.val)
```

```
            queue.enqueue(neighbor)
        copies[current].neighbors.append(copies[neighbor])  // copy to copy
RETURN copies[node]
```

Every node is copied, queued and popped once, and every neighbor list is walked once, so the clone costs `O(V + E)` time and `O(V)` extra space for the map and the queue, beyond the clone itself. Reach for the old-to-new map whenever a structure with shared or cyclic references must be deep-copied. Copy List with Random Pointer is the same idea on a list: one pass puts a copy of every node in the map, and a second pass wires each copy's `next` and `random` through it. A recursive DFS works just as well, provided a node's copy enters the map before the recursion reaches its neighbors; otherwise a cycle recurses forever. The classic mistake is appending only the neighbors seen for the first time: the edges of the search tree survive, every edge back to an already copied node is lost, and the square comes back as `[[2, 4], [3], [], []]`.

Practice: LC 133 Clone Graph · LC 138 Copy List with Random Pointer · LC 1490 Clone N-ary Tree

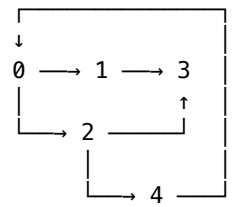
81. Cycle Detection in a Directed Graph

Three Colors: White, Gray, Black · Cycle Detection · Tier 2

Given a directed graph on nodes `0` to `n − 1`, decide whether it contains a cycle, a path that follows the arrows back to where it started. With the edges `0→1`, `0→2`, `1→3`, `2→3`, `2→4` and `4→0` the answer is yes, `0 → 2 → 4 → 0`. Without `4→0` the answer is no, even though node `3` can be reached two ways.

In an undirected graph, running into an already visited node signals a cycle, as long as it is not the parent just left (#115). In a directed graph it signals nothing: the diamond `0→1→3`, `0→2→3` reaches `3` twice and has no cycle. What matters is whether that node is still on the current path. The brute force asks every node whether it can reach itself, one search per node, `O(V · (V + E))`. A single DFS with three colors does better. White means not visited yet. Gray means entered but not finished: exactly the current recursion path. Black means finished: everything reachable from the node has been explored and holds no cycle. An edge into a gray node points back at an ancestor on the current path, and that back edge plus the path down from the ancestor is a cycle. An edge into a black node is harmless and never followed, which keeps the search linear.

Picture the gray nodes as a rope paid out behind the search: entering a node adds it to the rope, and finishing it reels it back in and paints it black. A cycle is an edge that lands back on the rope. In the trace, `2→3` lands on black `3`, finished and off the rope, where a plain visited set would wrongly report a cycle. Then `4→0` lands on `0`, still on the rope, and the rope from `0` to `4` plus that edge is the cycle.



event	gray: the current path	black
enter 0	0	–
enter 1	0 1	–
enter 3	0 1 3	–
finish 3	0 1	3
finish 1	0	3 1
enter 2	0 2	3 1
edge 2→3	3 is black: skip it	
enter 4	0 2 4	3 1
edge 4→0	0 is gray: cycle 0 → 2 → 4 → 0 ✓	

```
WHITE ← 0                                   // not visited yet
GRAY ← 1                                    // on the current DFS path
BLACK ← 2                                   // finished: no cycle reachable

FUNCTION HasCycle(n, edges)
    graph ← array of n empty lists
    FOR EACH (u, v) IN edges
        graph[u].append(v)
    color ← array of n copies of WHITE
    FOR node FROM 0 TO n − 1
        IF color[node] = WHITE AND DFS(graph, node, color)
            RETURN true
    RETURN false

FUNCTION DFS(graph, node, color)
    color[node] ← GRAY                      // node joins the current path
    FOR EACH neighbor IN graph[node]
        IF color[neighbor] = GRAY
```

```
            RETURN true                    // back edge to a node on the path
        IF color[neighbor] = WHITE AND DFS(graph, neighbor, color)
            RETURN true
    color[node] ← BLACK                    // every path out of node is safe
    RETURN false
```

Every node and every edge is handled once, so the search costs `O(V + E)` time and `O(V)` space beyond the graph, for the colors and the recursion. Reach for three colors whenever dependencies, prerequisites or state transitions must be checked for a loop, or when the nodes that can never reach a cycle are wanted: in Find Eventual Safe States those are exactly the nodes that end black. Kahn's algorithm (#40) answers the same question without recursion, since a cycle exists exactly when fewer than `n` nodes get peeled; that matters in Python, where a chain of about a thousand nodes overflows the default recursion limit. The classic mistake is the plain visited set, which calls the diamond a cycle. Never painting finished nodes black is the same mistake in disguise: they stay gray, and any later visit looks like a cycle.

Practice: LC 207 Course Schedule · LC 802 Find Eventual Safe States · LC 1059 All Paths from Source Lead to Destination

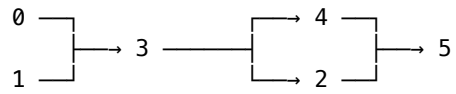
82. Topological Sort with DFS

Reverse Postorder · Topological Sort · Tier 2

Given `n` nodes `0` to `n − 1` and directed edges `[u, v]` meaning `u` must come before `v`, return an order of all the nodes that respects every edge, or an empty list if a cycle makes that impossible. On the graph of Kahn's example (#40), with edges `0→3`, `1→3`, `3→4`, `3→2`, `4→5` and `2→5`, one valid order is `[1, 0, 3, 2, 4, 5]`.

Kahn's algorithm builds the order from the front, by repeatedly taking a node that nothing points to. Depth-first search builds it from the back. In a graph without cycles, a DFS finishes a node only after everything reachable from it has finished, so the order in which nodes finish, the postorder, lists every node after all of its descendants, and reversing it lists every node before them. The proof takes one edge at a time. When the search examines `u → v`, the node `v` is white, and is then entered and finished before `u` is; or black, and finished already; or gray, which makes `v` an ancestor of `u` on the current path and the edge part of a cycle. In every acyclic case `v` finishes before `u`, so in the reversed list `u` comes first. The colors are those of cycle detection (#81), and the same gray test triggers the `[]`.

Picture the answer being written from right to left. The first node to finish is a dead end, `5`, which belongs at the far right; each later finisher goes to the left of everything placed so far, because everything it leads to is already placed. Node `1` finishes last; its only edge leads into the black `3`. The result differs from Kahn's `[0, 1, 3, 4, 2, 5]` and is just as valid, since a graph can have many topological orders.




```
enter 0, 3, 4, 5
finish 5                                finished [5]
finish 4                                finished [5, 4]
enter 2; edge 2→5: 5 is black, skip
finish 2                                finished [5, 4, 2]
finish 3                                finished [5, 4, 2, 3]
finish 0                                finished [5, 4, 2, 3, 0]
enter 1; edge 1→3: 3 is black, skip
finish 1                                finished [5, 4, 2, 3, 0, 1]
reverse                                 order    [1, 0, 3, 2, 4, 5]
```

```
WHITE ← 0                                   // not visited yet
GRAY ← 1                                    // on the current DFS path
BLACK ← 2                                   // finished: after its descendants

FUNCTION TopologicalSort(n, edges)
    graph ← array of n empty lists
    FOR EACH (u, v) IN edges
        graph[u].append(v)
    color ← array of n copies of WHITE
    finished ← empty list                   // nodes in the order they finish
    FOR node FROM 0 TO n − 1
        IF color[node] = WHITE AND NOT DFS(graph, node, color, finished)
            RETURN empty list               // a cycle: no order exists
    RETURN reverse(finished)
```

```
FUNCTION DFS(graph, node, color, finished)
    color[node] ← GRAY                      // node joins the current path
    FOR EACH neighbor IN graph[node]
        IF color[neighbor] = GRAY
            RETURN false                    // back edge: a cycle
        IF color[neighbor] = WHITE
            IF NOT DFS(graph, neighbor, color, finished)
                RETURN false
    color[node] ← BLACK
    finished.append(node)                   // its descendants are all in already
    RETURN true
```

Every node and every edge is handled once, so the sort costs `O(V + E)` time and `O(V)` extra space. Choose between the two sorts by the shape of the problem. Kahn's needs no recursion, which matters in Python on long chains, hands out the order front to back, and peels the graph level by level, which counting semesters needs. The DFS version fits when a DFS is already running, as in Longest Increasing Path in a Matrix, which computes each cell only after the cells it can step to: exactly this postorder. Sort Items by Groups Respecting Dependencies needs either sort twice, once for the groups and once for the items inside each group. The classic mistake is recording a node when the search enters it instead of when it finishes: the entry order here is `[0, 3, 4, 5, 2, 1]`, which puts `3` before its prerequisite `1`.

Practice: LC 210 Course Schedule II · LC 269 Alien Dictionary · LC 1203 Sort Items by Groups Respecting Dependencies · LC 329 Longest Increasing Path in a Matrix

83. Alien Dictionary

Letter Edges from Adjacent Words, Then Topological Sort · Topological Sort · Tier 2

A list of words is sorted by the rules of an unknown alphabet. Return the letters that appear in the words, arranged in an alphabet order that sorts the list, or the empty string if no alphabet can. From `["wrt", "wrf", "er", "ett", "rftt"]` the only possible order is `"wertf"`.

The brute force tries every arrangement of the letters and checks whether it sorts the list, `U!` arrangements for `U` distinct letters. Everything the list says about the alphabet sits between neighbors. Two adjacent words agree up to some position, and the first letters that differ are in alphabet order: `wrt` before `wrf` says only that `t` comes before `f`. The letters after that position say nothing, and words further apart add nothing the adjacent pairs do not already imply, because sorted order is transitive. So each adjacent pair yields at most one edge `before → after` between two letters, and the alphabet is a topological order of that letter graph, which Kahn's algorithm (#40) produces by repeatedly placing a letter that has nothing left in front of it.

Two kinds of input have no answer. A cycle among the letters, as in `["z", "x", "z"]`, leaves letters whose indegree never reaches zero, so fewer letters come out than went in. A word followed by its own proper prefix, `abc` before `ab`, breaks the rule that a prefix sorts first, and since no edge can express that, the pair check rejects it directly. When the queue holds several letters at once, any of them may go next, so many alphabets can be right: `["ab", "ac", "d"]` only says that `b` precedes `c` and `a` precedes `d`, and six orders satisfy both. Starting the queue from the letters in sorted order makes the answer repeatable.

```
adjacent pair   first difference    edge
wrt   wrf       index 2: t ≠ f      t → f
wrf   er        index 0: w ≠ e      w → e
er    ett       index 1: r ≠ t      r → t
ett   rftt      index 0: e ≠ r      e → r

graph  w → e → r → t → f       indegree  w 0, r 1, t 1, f 1, e 1

pop   order    queue after
 –    ""       [w]          only w has nothing in front of it
 w    w        [e]
 e    we       [r]
 r    wer      [t]
 t    wert     [f]
 f    wertf    []           all 5 letters placed: no cycle
```

```
FUNCTION AlienOrder(words)
    graph ← empty map                       // letter → letters after it
    indegree ← empty map                    // letter → letters still before it
    FOR EACH word IN words
        FOR EACH letter IN word
            IF letter NOT IN graph
                graph[letter] ← empty list
                indegree[letter] ← 0
    FOR i FROM 0 TO len(words) − 2
        IF NOT AddEdge(words[i], words[i + 1], graph, indegree)
            RETURN ""                       // a word came before its own prefix
    RETURN TopologicalOrder(graph, indegree)
```

```
FUNCTION AddEdge(first, second, graph, indegree)
    FOR j FROM 0 TO min(len(first), len(second)) - 1
        before ← first[j]
        after ← second[j]
        IF before ≠ after
            IF after NOT IN graph[before]  // count each edge once
                graph[before].append(after)
                indegree[after] ← indegree[after] + 1
            RETURN true                    // later letters say nothing
    RETURN len(first) ≤ len(second)        // "abc" before "ab" is impossible

FUNCTION TopologicalOrder(graph, indegree)
    queue ← empty queue
    FOR EACH letter IN sorted(keys of indegree)    // a repeatable answer
        IF indegree[letter] = 0
            queue.enqueue(letter)
    order ← empty list
    WHILE queue is not empty
        letter ← queue.dequeue()
        order.append(letter)
        FOR EACH after IN graph[letter]
            indegree[after] ← indegree[after] - 1  // one fewer before it
            IF indegree[after] = 0
                queue.enqueue(after)
    IF len(order) < len(indegree)
        RETURN ""                          // the rest sit on a cycle
    RETURN join(order, "")
```

Comparing neighbors reads each letter of the input at most twice, and the graph holds at most 26 letters, so the whole run is `O(C)` for `C` letters in all the words. Reach for this whenever a sorted list must reveal the order that sorted it, or whenever "comes before" facts must be assembled into one sequence. Checking a proposed alphabet is the easy half, LC 953, which compares each adjacent pair by letter ranks. The classic mistake is reading past the first difference: it turns `["ab", "ba"]` into the false cycle `a → b → a` and reports no alphabet, when `ab` is the answer. Forgetting the prefix rule is the other one, and it returns `abc` for `["abc", "ab"]` instead of the empty string.

Practice: LC 269 Alien Dictionary · LC 953 Verifying an Alien Dictionary · LC 210 Course Schedule II · LC 444 Sequence Reconstruction

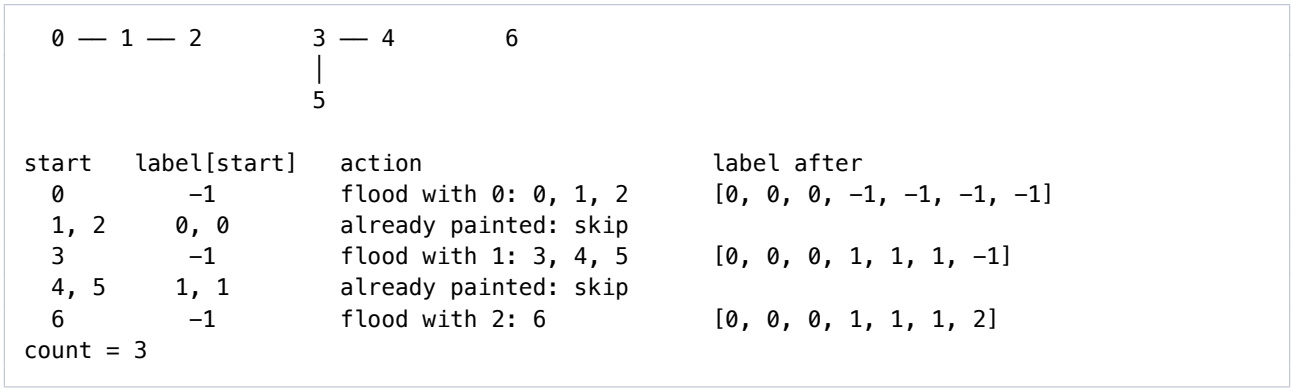
84. Connected Components

Count and Label by Flood Fill · Connected Components · Tier 2

Given `n` nodes `0` to `n − 1` and a list of undirected edges, count the connected components, the groups of nodes that can reach one another, and label each node with the number of its component. With seven nodes and the edges `0–1`, `1–2`, `3–4` and `3–5` there are 3 components, `{0, 1, 2}`, `{3, 4, 5}` and the lone `{6}`, so the labels are `[0, 0, 0, 1, 1, 1, 2]`.

The brute force asks for every pair of nodes whether a path joins them, a search per pair, `O(V² · (V + E))`. The insight is that one search from a node reaches exactly its own component, all of it and nothing else. So scan the nodes in order: each node that is still unlabeled starts a flood fill that paints its entire component with a fresh label, and every node the flood paints is skipped when the scan reaches it. The number of floods is the number of components. The flood keeps an explicit stack, like the iterative DFS of #37, but paints each node the moment it is pushed, so no node enters the stack twice. As #37 points out, that order is no longer strictly depth-first, and here it does not matter: BFS would serve equally well, because all that counts is that every reachable node gets painted. `label` doubles as the visited mark, with `−1` for unpainted.

Picture paint poured on a node. It runs along the edges and stops at the border of the component, because no edge leaves it. The scan then walks past the painted nodes until it finds a dry one and pours the next color. On a grid the same loop counts islands (#38), with the four neighboring land cells as the edges.




```
FUNCTION ConnectedComponents(n, edges)
    graph ← array of n empty lists
    FOR EACH (u, v) IN edges
        graph[u].append(v)                  // undirected: both directions
        graph[v].append(u)
    label ← array of n copies of −1         // −1 means not reached yet
    count ← 0
    FOR start FROM 0 TO n − 1
        IF label[start] = −1                // not reached by any earlier flood
            Flood(graph, start, count, label)
            count ← count + 1
    RETURN (count, label)

FUNCTION Flood(graph, start, mark, label)
    label[start] ← mark
    stack ← empty stack
    stack.push(start)
```

```
    WHILE stack is not empty
        node ← stack.pop()
        FOR EACH neighbor IN graph[node]
            IF label[neighbor] = −1
                label[neighbor] ← mark     // label when pushed: pushed once
                stack.push(neighbor)
```

Each node is painted once and each edge is examined from both ends, so the whole labeling costs `O(V + E)` time and space. Reach for it when a problem asks how many groups, networks, provinces or islands there are, whether two nodes are connected, or how big each group is; Count Unreachable Pairs needs only the component sizes, adding `size × (n − size)` over the components and halving the total. When the edges arrive one at a time and the count must stay current, union-find (#41) absorbs each new edge in near-constant time instead of flooding again. The classic mistake is storing each edge in one direction only: a flood from `0` can then never cross an edge written `[1, 0]`, and two nodes joined by that edge are counted as two components.

Practice: LC 323 Number of Connected Components in an Undirected Graph · LC 547 Number of Provinces · LC 1971 Find if Path Exists in Graph · LC 2316 Count Unreachable Pairs of Nodes in an Undirected Graph

85. Word Search II

Trie-Guided Grid Backtracking · Tries · Tier 2

Given a grid of letters and a list of words, return every word that can be spelled along a path of horizontally or vertically adjacent cells, each cell used at most once per word. On the grid with rows `oaan`, `etae`, `ihkr` and `iflv`, of the words `oath`, `pea`, `eat` and `rain`, the answer is `oath` and `eat`.

Running Word Search (#74) once per word repeats work: every search starts again at every cell, and words that share a prefix, like `oat` and `oath`, walk the same cells once each. Put all the words into a trie (#43) instead and search once. The DFS carries a trie node along with the cell, steps onto a neighbor only if the node has a child for that neighbor's letter, and dies the moment its letters stop being a prefix of any word. Each grid path is matched against every word at the same time, a path never grows longer than the longest word, and the number of words no longer multiplies the work.

The node at the end of a word stores the word itself, so a hit is recorded without rebuilding the string, and the word is then cleared, so a second path that spells it adds nothing. After a node's subtree has been searched, a node left with no children is deleted from its parent: every word below it has been found, and later starts skip that branch in one lookup. The trie shrinks as the answer grows. The `#` mark from #74 needs no test of its own here, because `#` is never a child in the trie.

```
board       trie of the words (● keeps a word)
o a a n     root ─┬─ o ─ a ─ t ─ h ●    oath
e t a e           ├─ p ─ e ─ a ●        pea
i h k r           ├─ e ─ a ─ t ●        eat
i f l v           └─ r ─ a ─ i ─ n ●    rain

start (0,0) o: o → a(0,1) → t(1,1) → h(2,1) ●  record oath, prune o-a-t-h
start (1,0) e: no neighbor is an a, dead after one cell
start (1,3) e: e → a(1,2) → t(1,1) ●  record eat, prune e-a-t
start (2,3) r: no neighbor is an a, dead after one cell
every other start: its letter is not a child of the root (no p on the board)
```

```
FUNCTION BuildTrie(words)
    root ← new TrieNode()                    // no children yet, word null
    FOR EACH word IN words
        node ← root
        FOR EACH ch IN word
            IF ch NOT IN node.children
                node.children[ch] ← new TrieNode()
            node ← node.children[ch]
        node.word ← word                     // the last node keeps the word
    RETURN root

FUNCTION Dfs(board, row, col, parent, found)
    IF row < 0 OR row ≥ len(board) OR col < 0 OR col ≥ len(board[0])
        RETURN
    ch ← board[row][col]
    IF ch NOT IN parent.children
        RETURN                               // no word goes this way
    node ← parent.children[ch]
    IF node.word ≠ null
        found.append(node.word)
```

```
        node.word ← null                    // record each word only once
    board[row][col] ← '#'                   // mark: on the current path
    FOR EACH (d_row, d_col) IN [(1, 0), (-1, 0), (0, 1), (0, -1)]
        Dfs(board, row + d_row, col + d_col, node, found)
    board[row][col] ← ch                    // unmark on the way back
    IF len(node.children) = 0
        parent.children.remove(ch)          // prune: no word left below

FUNCTION FindWords(board, words)
    root ← BuildTrie(words)
    found ← empty list
    FOR row FROM 0 TO len(board) - 1
        FOR col FROM 0 TO len(board[0]) - 1
            Dfs(board, row, col, root, found)
    RETURN found
```

Building the trie costs `O(total letters)`. The search is `O(m · n · 3^L)` in the worst case, for L the longest word, because each start grows paths of at most L cells with at most three ways on after the first step, and that bound does not grow with the number of words; the extra space is the trie plus `O(L)` recursion. Reach for this whenever one grid, or one text, must be searched for many words: the trie merges k searches into one walk that shares every common prefix. The classic mistake, carried over from #74, is returning as soon as a word is found. Here the search must go on, because `oat` and `oath` lie on the same path, and stopping at `oat` loses `oath`.

Practice: LC 212 Word Search II · LC 79 Word Search · LC 208 Implement Trie (Prefix Tree)

86. KMP String Matching

The Prefix Function Never Re-Reads the Text · Strings · Tier 2

Given a text and a pattern, return every index where the pattern starts in the text, overlapping matches included. In the text `abababc` the pattern `ababc` starts only at index 2, so the answer is `[2]`.

The brute force tries every start and compares up to `m` characters there, `O(n · m)` time; on `aaa…ab` against `aaab` it re-reads almost the whole window at every start. Its waste is forgetting what it has already read. When a mismatch comes after `j` characters matched, those `j` text characters are known: they are `pattern[0..j − 1]`. The next start that can still succeed therefore depends on the pattern alone. Slide the pattern right until the longest border of the matched part, its longest proper prefix that is also a suffix, lines up with that suffix in the text, and keep comparing at the same text position. `lps[i]` stores that border length for every prefix `pattern[0..i]`, so a mismatch at length `j` falls back to `j ← lps[j − 1]` and tries the same text character again, until it matches or `j` reaches 0.

Picture the text pointer `i` as a ratchet that only turns forward while the pattern jumps right underneath it. The table is built by the same loop run on the pattern against itself: `j` is the border being extended, a match grows it by one, a mismatch falls back to the next shorter border. After a full match, `j ← lps[j − 1]` keeps the overlap, which is how `aa` is found at 0, 1, 2 and 3 in `aaaaa`.

```
pattern   a b a b c
lps       0 0 1 2 0    lps[3] = 2: "ab" is a prefix and a suffix of "abab"

index     0 1 2 3 4 5 6
text      a b a b a b c
start 0   a b a b c        i = 4: text 'a' ≠ pattern 'c' with j = 4
start 2       a b a b c    j ← lps[3] = 2: "ab" stays matched, i stays 4
                  ↑
matched   1 2 3 4 3 4 5    j = 5 = len(pattern) at i = 6 → match at 2
```

```
FUNCTION BuildLPS(pattern)
    lps ← array of len(pattern) zeros
    j ← 0                                   // length of the border so far
    FOR i FROM 1 TO len(pattern) − 1        // lps[0] = 0: proper prefixes only
        WHILE j > 0 AND pattern[i] ≠ pattern[j]
            j ← lps[j − 1]                  // fall back to a shorter border
        IF pattern[i] = pattern[j]
            j ← j + 1
        lps[i] ← j
    RETURN lps

FUNCTION KMPSearch(text, pattern)
    IF len(pattern) = 0
        RETURN empty list
    lps ← BuildLPS(pattern)
    matches ← empty list
    j ← 0                                   // pattern characters matched
    FOR i FROM 0 TO len(text) − 1
        WHILE j > 0 AND text[i] ≠ pattern[j]
            j ← lps[j − 1]                  // slide the pattern, retry text[i]
        IF text[i] = pattern[j]
            j ← j + 1
```

```
        IF j = len(pattern)
            matches.append(i − j + 1)       // the match ends at i
            j ← lps[j − 1]                  // keep the border for overlaps
    RETURN matches
```

Every step of `i` raises `j` by at most one and every fallback lowers it, so the fallbacks never outnumber the steps: `O(n)` for the search and `O(m)` for the table, `O(n + m)` time and `O(m)` space in all. Reach for KMP when exact matching needs a worst-case guarantee, or when the question is about borders themselves: the longest prefix that is also a suffix is `lps[m − 1]`, a string of length `m` is one block repeated exactly when `lps[m − 1] > 0` and `m` is divisible by `m − lps[m − 1]`, and the table of `s + "#" + reverse(s)` gives the longest palindromic prefix behind the shortest palindrome. Rabin-Karp (#120) reaches expected linear time with a rolling hash instead. The classic mistakes are an `IF` where the fallback needs a `WHILE` (the shorter border can mismatch too), starting the table loop at `i = 0`, which lets every prefix count itself as its own border, and resetting `j` to 0 after a match, which misses overlapping matches.

Practice: LC 28 Find the Index of the First Occurrence in a String · LC 1392 Longest Happy Prefix · LC 459 Repeated Substring Pattern · LC 214 Shortest Palindrome

87. Longest Palindromic Substring

Expand Around Every Center · Strings · Tier 2

Given a string, return its longest substring that reads the same forwards and backwards, the leftmost one on a tie. For `babad` the answer is `bab`; `aba` is just as long but starts later.

The brute force tests every substring, $O(n^2)$ of them at $O(n)$ each, for cubic time. Turn the test inside out: a palindrome is symmetric about its middle, so grow palindromes outward from every possible middle instead of checking substrings one by one. A middle is either a character (odd length, like `aba`) or the gap between two neighbors (even length, like `abba`), which makes `n + (n − 1) = 2n − 1` centers. Number them so that even centers sit on characters and odd centers on gaps: center `c` starts with `left = floor(c / 2)` and `right = left + c mod 2`.

From each center, two arms swing outward in step while they land on equal characters. The moment they differ, or one runs off the string, the window just inside the arms is the longest palindrome around that center, and nothing wider around it can be one, since a wider window would contain the mismatched pair at mirror positions. Keep the longest window seen; a strict comparison leaves ties with the earlier center.

```
s          b   a   b   a   d
index      0   1   2   3   4
center     0 1 2 3 4 5 6 7 8     even center: a char, odd center: a gap

center   (left, right)   grows to   stops at      best so far
  0         (0, 0)         "b"        left edge     "b"
  1         (0, 1)         ""         b ≠ a         "b"
  2         (1, 1)         "bab"      left edge     "bab"
  3         (1, 2)         ""         a ≠ b         "bab"
  4         (2, 2)         "aba"      b ≠ d         "bab"  (tie keeps first)
  5         (2, 3)         ""         b ≠ a         "bab"
  6         (3, 3)         "a"        b ≠ d         "bab"
  7         (3, 4)         ""         a ≠ d         "bab"
  8         (4, 4)         "d"        right edge    "bab"
```

```
FUNCTION LongestPalindrome(s)
    best_start ← 0
    best_len ← 0
    FOR center FROM 0 TO 2 * len(s) − 2
        left ← floor(center / 2)
        right ← left + center mod 2          // even: a char, odd: a gap
        WHILE left ≥ 0 AND right < len(s) AND s[left] = s[right]
            left ← left − 1                  // ends match: grow both ways
            right ← right + 1
        length ← right − left − 1            // one step past each end
        IF length > best_len                 // strict: ties keep the first
            best_start ← left + 1
            best_len ← length
    RETURN s[best_start..best_start + best_len − 1]
```

Each of the `2n − 1` centers expands at most about `n / 2` times, so the scan is $O(n^2)$ time in the worst case (one letter repeated) and $O(1)$ extra space. Reach for it when a problem says palindromic substring: contiguous, with `n` in the low thousands. The same loop counts every palindromic substring if it adds

one per successful expansion step, because each step uncovers exactly one new palindrome. Manacher's algorithm gets to `O(n)` by starting each expansion part of the way out, using the mirror image of a palindrome already found; it is rarely expected in an interview. A palindromic subsequence is not contiguous and needs dynamic programming instead. The classic mistake is expanding only around characters, which misses every even-length palindrome and returns `c` for `cbbd` instead of `bb`; the other is an off-by-one when turning the overshot arms back into a window.

Practice: LC 5 Longest Palindromic Substring · LC 647 Palindromic Substrings · LC 516 Longest Palindromic Subsequence

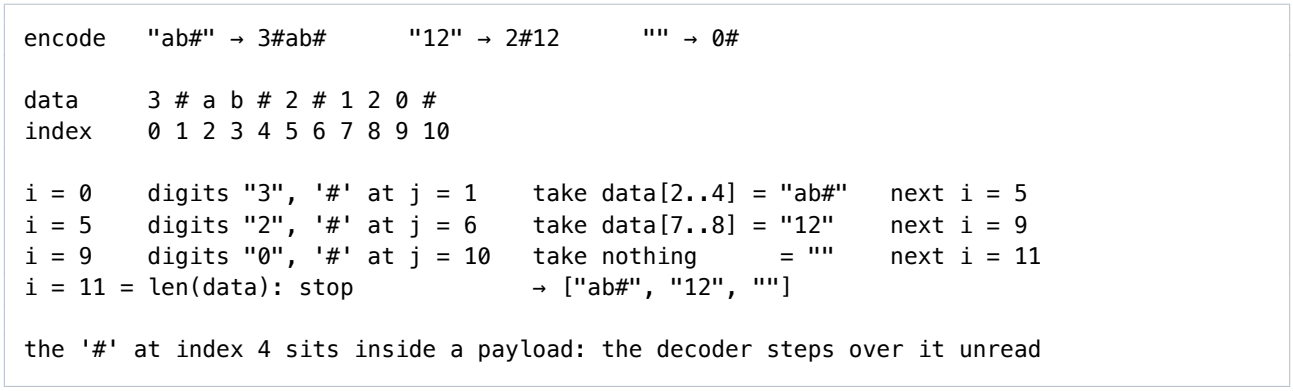
88. Encode and Decode Strings

Length-Prefix Framing · Strings · Tier 2

Design `encode`, which packs a list of strings into a single string, and `decode`, which recovers exactly the same list. The strings may contain any character, `#` and digits included. `["ab#", "12", ""]` encodes to `3#ab#2#12#0#` and decodes back to the same three strings.

The obvious scheme puts a delimiter between the strings and splits on it, and it breaks the moment a string contains the delimiter: joining with `#` turns `["ab#", "12", ""]` into `ab##12#`, which splits back into four strings. It cannot even tell `[]` from `[""]`, since both join to the empty string. Escaping can patch it, at the price of inspecting and rewriting every character on both sides. Framing sidesteps the question. Write each string's length in front of it, then a `#` that ends the length field, then the string itself. The decoder reads digits up to the first `#`, which is unambiguous because a length is made only of digits, and then takes exactly that many characters without looking at them.

Picture the encoded string as a train of boxes, each with a label on its front that says how long the box is. The decoder reads a label, steps over the whole box, and lands on the next label. A `#` or a digit inside a payload is never read as structure, because the decoder jumps over payloads instead of scanning them.



```
encode   "ab#" → 3#ab#      "12" → 2#12      "" → 0#

data     3 # a b # 2 # 1 2 0 #
index    0 1 2 3 4 5 6 7 8 9 10

i = 0    digits "3", '#' at j = 1    take data[2..4] = "ab#"   next i = 5
i = 5    digits "2", '#' at j = 6    take data[7..8] = "12"    next i = 9
i = 9    digits "0", '#' at j = 10   take nothing      = ""    next i = 11
i = 11 = len(data): stop             → ["ab#", "12", ""]

the '#' at index 4 sits inside a payload: the decoder steps over it unread
```

```
FUNCTION Encode(strs)
    parts ← empty list
    FOR EACH s IN strs
        parts.append(str(len(s)) + "#" + s)   // length, '#', then s itself
    RETURN join(parts)                        // concatenate once at the end

FUNCTION Decode(data)
    strs ← empty list
    i ← 0
    WHILE i < len(data)
        j ← i
        WHILE data[j] ≠ "#"                   // the length ends at a '#'
            j ← j + 1
        length ← int(data[i..j − 1])
        strs.append(data[j + 1..j + length])  // exactly length characters
        i ← j + 1 + length                    // jump over the payload unread
    RETURN strs
```

Both directions touch every character a constant number of times, so encoding and decoding take `O(total length)` time and space; collect the pieces in a list and join once, because `+=` in a loop can

recopy the growing string every time. Reach for length-prefix framing whenever several values must travel through one string or one stream and come back intact: serializing a list or a record, or messages on a network socket. Serializing a binary tree (#36) is the same contract with the shape recorded by null markers, and the TinyURL design (#100) meets it with a counter and a map. The classic mistake is a pure delimiter, which passes the examples until a string contains it; the subtler one is reading a single digit as the length, which breaks on the first string of ten or more characters.

Practice: LC 271 Encode and Decode Strings · LC 535 Encode and Decode TinyURL · LC 297 Serialize and Deserialize Binary Tree

89. Game of Life

Update in Place with Two-Bit States · Matrix · Tier 2

Each cell of a board is live (`1`) or dead (`0`), and all cells change at once. A live cell with two or three live neighbors among its eight survives, a dead cell with exactly three live neighbors comes alive, and every other cell is dead in the next generation. Compute the next board in place. The board with rows `010`, `001`, `111`, `000` becomes `000`, `101`, `011`, `010`.

The easy solution reads neighbors from a copy of the board and writes into the original, for `O(m n)` extra space. Writing into the board directly fails, because the rules are simultaneous: once a cell has been overwritten, a neighbor scanned later counts its new state instead of its old one. The fix is to give every cell room for both generations. Only bit 0 of each cell is in use, so bit 1 is free. During the first pass, bit 0 keeps the current state for every reader, and each cell's next state goes into bit 1: `cell | 2` marks it live next, and a cell left alone is dead next. Every neighbor count reads `cell & 1`, which ignores bit 1 whether or not that neighbor has been visited yet. A second pass then shifts each value right by one, so bit 1 drops into bit 0 and the present is forgotten.

Read each value as two binary digits with the future on the left: `2` is a dead cell coming alive, `1` a live cell dying. The rules collapse into one test, since a cell is live next exactly when it has three live neighbors, or when it is live and has two.

```
cell = 2 × next + current:  0 dead → dead    1 live → dead
                            2 dead → live    3 live → live

board     live neighbors    after pass 1    after pass 2 (>> 1)
0 1 0     1 1 2             0 1 0           0 0 0
0 0 1     3 5 3             2 0 3           1 0 1
1 1 1     1 3 2             1 3 3           0 1 1
0 0 0     2 3 2             0 2 0           0 1 0
```

```
FUNCTION CountLiveNeighbors(board, row, col)
    live ← 0
    FOR near_row FROM max(0, row − 1) TO min(len(board) − 1, row + 1)
        FOR near_col FROM max(0, col − 1) TO min(len(board[0]) − 1, col + 1)
            IF near_row = row AND near_col = col
                CONTINUE                    // a cell is not its own neighbor
            live ← live + (board[near_row][near_col] & 1)   // bit 0: now
    RETURN live

FUNCTION GameOfLife(board)
    FOR row FROM 0 TO len(board) − 1
        FOR col FROM 0 TO len(board[0]) − 1
            live ← CountLiveNeighbors(board, row, col)
            alive ← board[row][col] & 1
            IF live = 3 OR (alive = 1 AND live = 2)
                board[row][col] ← board[row][col] | 2    // bit 1: live next
    FOR row FROM 0 TO len(board) − 1
        FOR col FROM 0 TO len(board[0]) − 1
            board[row][col] ← board[row][col] >> 1       // keep only the next
```

Each cell reads at most eight neighbors, so the two passes cost `O(m n)` time with `O(1)` extra space. The signal is a grid that must change all at once and in place, with values small enough to leave spare

bits. The same move, keeping extra information inside the input instead of in a copy, solves Set Matrix Zeroes, which marks the rows and columns to clear in the matrix's own first row and column. On an infinite board, keep a set of live cells instead: each live cell adds one to a count for each of its eight neighbors in a map, and the next set holds the cells counted three times plus the live cells counted twice, so a generation costs time in proportion to the live cells. The classic mistake is testing `cell = 1` instead of reading bit 0 when counting neighbors: a cell already marked `3`, live now and live next, stops counting as live, and the 2 by 2 block of live cells, which never changes, loses its bottom row.

Practice: LC 289 Game of Life · LC 73 Set Matrix Zeroes

90. Fibonacci

Recursion, Memo, Iteration, Fast Doubling · Math · Tier 2

The Fibonacci numbers start with `F(0) = 0` and `F(1) = 1`, and every later term is the sum of the two before it: `0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55`. Given `n`, compute `F(n)`; for example `F(5) = 5` and `F(10) = 55`.

The definition is already an algorithm, and a famously slow one. Each call makes two more calls, so the calls form a binary tree with `2F(n + 1) − 1` nodes, a count that grows like `1.618^n`, and the same subproblems sit in it over and over: computing `F(5)` solves `F(3)` twice and `F(2)` three times. Memoization stores each answer the first time it is computed, and every later request becomes a lookup. The tree collapses to its left edge, with each `F(k)` computed once, at the cost of an `O(n)` map and a call stack `n` frames deep.

The loop keeps only what the recurrence still needs. Every new term reads just the two before it, so a window of two numbers slides along the sequence: `(prev, curr) = (F(i), F(i + 1))` becomes `(F(i + 1), F(i + 2))` in each step, and after `n` steps `prev` is `F(n)`. Fast doubling stops walking and starts jumping. From the pair `(F(k), F(k + 1))` the identities `F(2k) = F(k)(2F(k + 1) − F(k))` and `F(2k + 1) = F(k)² + F(k + 1)²` land directly on position `2k` or `2k + 1`, so the recursion halves `n` on the way down and rebuilds one bit of `n` per level on the way back up. Both identities fall out of squaring a power of the matrix `[[1, 1], [1, 0]]`, whose powers hold consecutive Fibonacci numbers, which makes this the same halving trick as fast exponentiation (#130).

```
plain recursion for F(5): 15 calls, F(3) twice, F(2) three times
                          F(5)
                ┌──────────┴──────────┐
              F(4)                  F(3)
          ┌─────┴─────┐         ┌─────┴─────┐
        F(3)        F(2)      F(2)        F(1)
       ┌──┴──┐     ┌──┴──┐   ┌──┴──┐
     F(2)  F(1)  F(1)  F(0) F(1)  F(0)
    ┌──┴──┐
  F(1)  F(0)

memo      9 calls: 5 4 3 2 1 0 1 2 3 (the second 2 and 3 are lookups)
loop      (prev, curr) = (0,1) (1,1) (1,2) (2,3) (3,5) (5,8) → F(5) = 5
doubling  n = 10 → 5 → 2 → 1 → 0, then (F(k), F(k + 1)) climbs back:
          (0,1) → (1,1) → (1,2) → (5,8) → (55,89) → F(10) = 55
```

```
FUNCTION FibNaive(n)
    IF n < 2
        RETURN n
    RETURN FibNaive(n − 1) + FibNaive(n − 2)  // two calls per call

FUNCTION FibMemo(n, memo)
    IF n < 2
        RETURN n
    IF n IN memo
        RETURN memo[n]                  // solved before: skip the whole subtree
    memo[n] ← FibMemo(n − 1, memo) + FibMemo(n − 2, memo)
    RETURN memo[n]
```

```
FUNCTION FibIter(n)
    prev ← 0
    curr ← 1
    FOR i FROM 0 TO n − 1              // prev = F(i), curr = F(i + 1)
        nxt ← prev + curr
        prev ← curr
        curr ← nxt
    RETURN prev

FUNCTION FibPair(n)                    // returns (F(n), F(n + 1))
    IF n = 0
        RETURN (0, 1)
    (a, b) ← FibPair(floor(n / 2))     // a = F(k), b = F(k + 1)
    even ← a * (2 * b − a)             // F(2k)
    odd ← a * a + b * b                // F(2k + 1)
    IF n mod 2 = 0
        RETURN (even, odd)
    RETURN (odd, even + odd)           // n = 2k + 1: one step further

FUNCTION FibFast(n)
    pair ← FibPair(n)
    RETURN pair[0]
```

The plain recursion costs exponential time, memoization `O(n)` time and space, the loop `O(n)` time and `O(1)` space, and fast doubling `O(log n)` arithmetic steps, one per bit of `n`; `F(n)` has about `0.69n` bits, so for huge `n` the big-integer multiplications set the real cost. Reach for the two-variable loop whenever each state depends only on the previous two: Climbing Stairs counts the ways to reach step `n` as the ways to reach `n − 1` plus the ways to reach `n − 2`, which is `F(n + 1)`, and Tribonacci slides a window of three. The classic mistake is presenting the plain recursion as the answer in an interview, where `F(50)` already takes about 40 billion calls; the second is memoizing in Python and hitting the default recursion limit near `n = 1000`, which the loop never touches.

Practice: LC 509 Fibonacci Number · LC 70 Climbing Stairs · LC 1137 N-th Tribonacci Number

91. Insert Delete GetRandom O(1)

Array + Index Map, Swap with Last · Design · Tier 2

Design a set of integers with three operations, `insert(val)`, `remove(val)` and `get_random()`, each in average `O(1)` time. Insert and remove return whether the set changed, and `get_random` returns every stored value with the same probability. After inserting `10`, `20` and `30` and removing `10`, the set holds `30` and `20`, and `get_random` returns each of them half the time.

Neither familiar structure delivers all three. A hash set inserts and removes in `O(1)` but cannot pick a random member without walking to it, `O(n)`; a plain list picks a random index in `O(1)` but needs an `O(n)` search to refuse a duplicate or to find a value, and deleting from its middle shifts the whole tail left, `O(n)` again. The design keeps both. A list `values` stays packed from index `0` with no holes, so a uniformly random index is a uniformly random value, and a map `index_of` sends each value to its slot, so finding a value is one lookup. Insertion appends and records the new slot. Removal is the one clever step: deleting from the middle would open a hole, and closing it by shifting is exactly the cost to avoid, so the last value jumps into the hole, its map entry is corrected, and the tail slot is popped. A set has no order to preserve, which is what makes the jump legal.

Picture the list as a row of tiles. A removal never slides the row; the rightmost tile fills the gap and the row shrinks by one from the right. After every operation `index_of[values[i]] = i` holds for every slot `i`, and there is exactly one slot per key.

```
                        values          index_of
insert 10, 20, 30       [10, 20, 30]    {10: 0, 20: 1, 30: 2}
remove(10)              i = 0 (the slot of 10), last = 30
  last into slot 0      [30, 20, 30]    {10: 0, 20: 1, 30: 0}
  pop tail, delete 10   [30, 20]        {20: 1, 30: 0}
get_random              RandomInt(0, 1) → slot 0 or 1 → 30 or 20, half each
```

```
CLASS RandomizedSet
    FUNCTION Init()
        self.values ← empty list              // packed from index 0, no holes
        self.index_of ← empty map             // value → its slot in values

    FUNCTION Insert(val)
        IF val IN self.index_of
            RETURN false
        self.index_of[val] ← len(self.values)
        self.values.append(val)
        RETURN true

    FUNCTION Remove(val)
        IF val NOT IN self.index_of
            RETURN false
        i ← self.index_of[val]
        last ← self.values[len(self.values) − 1]
        self.values[i] ← last                 // the last value fills the hole
        self.index_of[last] ← i               // and learns its new slot
        self.values.pop()                     // drop the tail slot
        self.index_of.remove(val)             // only now: last may be val
        RETURN true
```

```
    FUNCTION GetRandom()
        i ← RandomInt(0, len(self.values) − 1)    // uniform, both ends included
        RETURN self.values[i]
```

Every operation is a constant number of hash-map steps plus work at the end of the list, so each costs `O(1)` on average (hashing gives the average; appending and popping the end are amortized `O(1)`), and the structure takes `O(n)` space. Reach for it whenever a problem wants uniform random sampling from a collection that changes, or an `O(1)` delete from an array whose order does not matter; it is the same pairing of a map with a structure the map points into that powers the LRU cache of #44. With duplicates allowed (LC 381) the map holds a set of slots for each value and removal takes any one of them, with the same jump of the last tile. The classic mistake is deleting `val` from the map before correcting the moved value's slot: when `val` is itself the last value, the correction writes it straight back, leaving a ghost key that points past the end of the list, and a later `insert(val)` wrongly returns false.

Practice: LC 380 Insert Delete GetRandom O(1) · LC 381 Insert Delete GetRandom O(1) - Duplicates allowed · LC 710 Random Pick with Blacklist

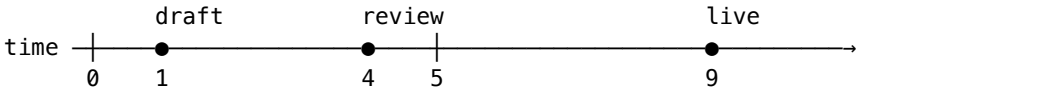
92. Time-Based Key-Value Store

Per-Key Timestamp Lists + Binary Search · Design · Tier 2

Design a store that keeps every version of a key. `set(key, value, timestamp)` records a value at a time, with the timestamps of each key arriving in increasing order, and `get(key, timestamp)` returns the value set at the largest time ≤ `timestamp`, or the empty string if the key had no value yet. After setting `status` to `draft` at time `1`, `review` at `4` and `live` at `9`, `get("status", 5)` is `"review"` and `get("status", 0)` is `""`.

The brute force keeps one log of every `set` call and scans all of it on each `get`, `O(n)` per query. The order in which versions arrive is the gift: because the timestamps of a key only grow, appending to that key's list keeps it sorted for free, and a sorted list answers "the last time at or before `t`" by binary search. Each key therefore owns two parallel lists, the times and the values set at those times; `set` appends to both and `get` searches the times. The search finds the first time strictly after `t`, the upper bound, and steps one slot to the left, where the newest version that already existed at time `t` sits. If the upper bound is slot `0`, every version is newer than `t` and the answer is empty.

Picture each key as a timeline with a dot at every version. A query drops a pin at time `t` and walks left to the nearest dot. The binary search keeps every slot before `left` at or before `t` and every slot from `right` on after it, and halves the unknown gap between them until it closes.




```
get("status", 5): left = 0, right = 3
  mid = 1: time 4 ≤ 5 → left = 2
  mid = 2: time 9 > 5 → right = 2
  the first time after 5 is slot 2, so the answer is slot 1 → "review"
get("status", 0): every time is after 0, left stays 0 → ""
```

```
CLASS TimeMap
    FUNCTION Init()
        self.times ← empty map                  // key → increasing set times
        self.values ← empty map                 // key → values, in the same order

    FUNCTION Set(key, value, timestamp)
        IF key NOT IN self.times
            self.times[key] ← empty list
            self.values[key] ← empty list
        self.times[key].append(timestamp)       // times only grow: stays sorted
        self.values[key].append(value)

    FUNCTION Get(key, timestamp)
        IF key NOT IN self.times
            RETURN ""
        times ← self.times[key]
        left ← 0
        right ← len(times)
        WHILE left < right                      // find the first time > timestamp
            mid ← floor((left + right) / 2)
            IF times[mid] ≤ timestamp
```

```
            left ← mid + 1              // mid is not after timestamp
        ELSE
            right ← mid                 // mid is after: maybe the first
    IF left = 0
        RETURN ""                       // every version is newer
    RETURN self.values[key][left − 1]
```

`set` costs amortized `O(1)` and `get` costs `O(log n)` for a key with `n` versions, because each step halves the gap; the store keeps every version, so space is linear in the number of `set` calls. The signals are values that change over time and questions asked "as of" a moment: versioned configs, snapshots, prices or election leaders at a time. The search is the lower-bound template of #14 with ≤ in place of <, which turns lower bound into upper bound, and Snapshot Array (#47, LC 1146) is the same structure with one timeline per index and snapshot ids as times. If timestamps could arrive out of order, appending would break the sorting and each key would need a balanced search tree (a `TreeMap` in Java) instead of a list. The classic mistake is forgetting the `left = 0` case: Python then reads `values[−1]` and silently returns the newest version for a time before the first one.

Practice: LC 981 Time Based Key-Value Store · LC 1146 Snapshot Array · LC 911 Online Election

93. LFU Cache

Frequency Buckets That Keep LRU Order · Design · Tier 2

Design a cache with a fixed capacity whose `get(key)` and `put(key, value)` both run in `O(1)`. `get` returns the value or `−1`, and every `get` that finds its key and every `put` counts as one use of that key. When a new key arrives and the cache is full, it evicts the least frequently used key, and among keys with the same count the least recently used one. With capacity 2, after `put(1, 10)`, `get(1)` and `put(2, 20)`, the call `put(3, 30)` evicts key 2, used once, and keeps key 1, used twice, even though key 1 is the less recent of the two.

The brute force stores a use count and a last-use time with every key and scans all of them at each eviction, `O(n)`. That scan answers two nested questions, the smallest count in the cache and the oldest key with that count, and grouping the keys makes both answers immediate. Put the keys into buckets by use count, and keep each bucket in recency order with the doubly linked list of the LRU Cache (#44): newest after `head`, oldest before `tail`. One map sends each key to its node, which carries the key, the value and the count `freq`; a second map sends each count to its bucket. The oldest key with a given count is now `bucket.tail.prev`. The smallest count is one integer, `min_freq`, because a count only ever grows by one, so the minimum can move in just two ways. A use moves a node from bucket `f` to the front of bucket `f + 1`, and when that empties bucket `f` and `f` was the minimum, the minimum becomes `f + 1`, where the node just landed. A new key enters bucket 1 and sets `min_freq` back to 1, since nothing is used less than once.

Picture the buckets as the floors of a building, each floor a queue ordered by recency. A use lifts a key one floor and puts it at the newest end; eviction takes the oldest key on the lowest occupied floor. Evict before inserting: a cache that inserts first and then evicts from bucket 1 throws out the newcomer itself whenever every older key has been used at least twice.

```
after get(3) → 30, both keys sit in bucket 2:
    freq 2   head — 3:30 — 1:10 — tail      put(4, 40) evicts tail.prev = 1
                   newest  oldest

operation       freq: keys, newest     min_freq  note
put(1, 10)      1: [1]                 1
get(1) → 10     2: [1]                 2         bucket 1 empties: min rises
put(2, 20)      1: [2]  2: [1]         1         a new key resets min to 1
put(3, 30)      1: [3]  2: [1]         1         full: evict 2, used once
get(2) → −1     1: [3]  2: [1]         1         a miss changes nothing
get(3) → 30     2: [3, 1]              2         a tie at 2 uses
put(4, 40)      1: [4]  2: [3]         1         full: evict 1, the older
get(1) → −1     1: [4]  2: [3]         1
get(3) → 30     1: [4]  3: [3]         1
get(4) → 40     2: [4]  3: [3]         2         bucket 1 empties: min rises
```

```
CLASS Node
    FUNCTION Init(key, value)
        self.key ← key                  // kept so eviction can clean the map
        self.value ← value
        self.freq ← 1                   // how many times the key was used
        self.prev ← null
        self.next ← null
```

```
CLASS Bucket                            // the list of #44, one per freq
    FUNCTION Init()
        self.head ← Node(0, 0)          // sentinel: head.next is the newest
        self.tail ← Node(0, 0)          // sentinel: tail.prev is the oldest
        self.head.next ← self.tail
        self.tail.prev ← self.head
    // AddFront(node) and Remove(node): exactly as in LRU Cache (#44)

    FUNCTION IsEmpty()
        RETURN self.head.next = self.tail

CLASS LFUCache
    FUNCTION Init(capacity)
        self.capacity ← capacity
        self.nodes ← empty map          // key → node
        self.buckets ← empty map        // freq → bucket of keys used freq times
        self.min_freq ← 0               // the smallest freq that has a key

    FUNCTION Add(node)
        IF node.freq NOT IN self.buckets
            self.buckets[node.freq] ← Bucket()
        self.buckets[node.freq].AddFront(node)

    FUNCTION Touch(node)
        bucket ← self.buckets[node.freq]
        bucket.Remove(node)
        IF bucket.IsEmpty()
            self.buckets.remove(node.freq)
            IF self.min_freq = node.freq
                self.min_freq ← self.min_freq + 1   // node moves up to it
        node.freq ← node.freq + 1
        self.Add(node)

    FUNCTION Get(key)
        IF key NOT IN self.nodes
            RETURN -1
        node ← self.nodes[key]
        self.Touch(node)
        RETURN node.value

    FUNCTION Put(key, value)
        IF self.capacity = 0
            RETURN
        IF key IN self.nodes
            node ← self.nodes[key]
            node.value ← value
            self.Touch(node)
            RETURN
        IF len(self.nodes) = self.capacity
            self.Evict()                // before the new key is added
        node ← Node(key, value)
        self.nodes[key] ← node
        self.Add(node)
        self.min_freq ← 1               // a new key is the least used
```

```
    FUNCTION Evict()
        bucket ← self.buckets[self.min_freq]
        victim ← bucket.tail.prev       // least used, then least recent
        bucket.Remove(victim)
        IF bucket.IsEmpty()
            self.buckets.remove(self.min_freq)
        self.nodes.remove(victim.key)
```

Every operation is a constant number of map lookups and pointer changes, so `get` and `put` run in `O(1)`, and the cache holds at most `capacity` nodes. After an eviction `min_freq` may name a bucket that is gone, but the insertion that follows resets it to 1, so no search is ever needed. LRU evicts a key used a thousand times as soon as `capacity` other keys have been used after it; LFU protects popular keys but lets a key that was hot long ago linger, which is why real caches age their counts or blend the two policies. A Python dict keeps insertion order too, but finding its first key walks over the empty slots that deletions leave at the front, so a bucket that churns like a queue slows down as it grows; an `OrderedDict` or the hand-built list keeps every step `O(1)`. All O(1) Data Structure (#141) reuses the buckets, chained into a linked list of counts, and Maximum Frequency Stack turns them upside down: it keeps the largest count and pops the newest entry of that bucket. The classic mistakes are forgetting that a `put` to an existing key is a use, and dropping the capacity-0 guard, so that the first `put` tries to evict from a `buckets[0]` that does not exist.

Practice: LC 460 LFU Cache · LC 146 LRU Cache · LC 895 Maximum Frequency Stack

94. Cache with Expiring Keys

Lazy Expiry + a Min-Heap of Deadlines · Design · Tier 2

Design a cache whose keys expire. `put(key, value, ttl, now)` stores a value that lives until the deadline `now + ttl`, and putting the key again replaces both its value and its deadline. `get(key, now)` returns the value only while `now` is earlier than the deadline, and `null` otherwise, and `size(now)` counts the keys still alive. Every call receives the current time from the caller, so the cache never reads a clock and any sequence of calls can be replayed exactly. After `put(b, 2, 3, 1)` the key `b` lives until time 4: `get(b, 3)` returns `2` and `get(b, 4)` returns `null`.

A timer per key is the heavy design, and a `size` that scans every key for dead ones costs `O(n)` per call. Expiry splits into two jobs, and each has a cheap answer. A read checks a single deadline, so `get` expires its own key lazily: when the key it finds is already dead, it deletes the key and reports a miss. Keys that are never read again would pile up, so `size` also expires keys in bulk, and to find them without a scan it keeps every deadline in a min-heap of `(expire_at, key)` pairs. The earliest deadline sits on top, so popping while the top is `≤ now` visits exactly the pairs whose time has come, in time order, and stops at the first one still in the future.

A plain heap cannot change or remove a pair in the middle, so a rewrite simply pushes a new pair and leaves the old one behind, stale, and a lazy delete in `get` leaves its pair behind as well. The map is the source of truth: a popped pair deletes its key only when the map still holds that key with that very deadline, and every other pair is dropped without a word. This is the lazy deletion of Dijkstra's algorithm (#42). Picture two layers: the map holds one current deadline per key, and the heap holds the deadlines promised and not yet popped, in time order, some of them already broken.

```
call              entries key(value, expire_at)   heap, earliest first
put(a, 1, 5, 0)   a(1, 5)                         (5, a)
put(b, 2, 3, 1)   a(1, 5) b(2, 4)                 (4, b) (5, a)
put(c, 3, 10, 2)  a(1, 5) b(2, 4) c(3, 12)        (4, b) (5, a) (12, c)
get(b, 3) → 2     3 < 4: b is live
get(b, 4) → null  a(1, 5) c(3, 12)                (4, b) is left behind
put(a, 7, 5, 4)   a(7, 9) c(3, 12)                (4, b) (5, a) (9, a) (12, c)
size(6) → 2       pop (4, b): b is gone, skip
                  pop (5, a): a expires at 9, stale, skip
                  a(7, 9) c(3, 12)                (9, a) (12, c)
get(a, 8) → 7     8 < 9: a is live
get(a, 9) → null  c(3, 12)                        (9, a) is left behind
size(11) → 1      pop (9, a): a is gone, skip     (12, c)
size(12) → 0      pop (12, c): current, delete c  empty
```

```
CLASS TTLCache
    FUNCTION Init()
        self.entries ← empty map            // key → (value, expire_at)
        self.deadlines ← empty min-heap     // (expire_at, key), some stale

    FUNCTION Put(key, value, ttl, now)
        expire_at ← now + ttl
        self.entries[key] ← (value, expire_at)
        self.deadlines.push((expire_at, key))   // old pair goes stale

    FUNCTION Get(key, now)
        IF key NOT IN self.entries
```

```
            RETURN null
        (value, expire_at) ← self.entries[key]
        IF now ≥ expire_at
            self.entries.remove(key)        // lazy expiry: found dead on a read
            RETURN null
        RETURN value

    FUNCTION Purge(now)
        WHILE len(self.deadlines) > 0 AND self.deadlines.top()[0] ≤ now
            (expire_at, key) ← self.deadlines.pop()      // earliest first
            IF key IN self.entries AND self.entries[key][1] = expire_at
                self.entries.remove(key)    // current pair: the key expired
            // any other pair is stale: the key was rewritten or removed

    FUNCTION Size(now)
        self.Purge(now)
        RETURN len(self.entries)
```

`get` costs `O(1)`, `put` costs `O(log n)` for the push, and `size` costs `O(k log n)` for the `k` pairs it pops; every pair is pushed once and popped at most once, so the purging is paid for by the puts. Stale pairs make the heap larger than the map, and each one lingers until a purge reaches its deadline, so a key rewritten often with a long TTL leaves a trail of them. Both checks call a key dead at its deadline, `now ≥ expire_at` in `get` and `top ≤ now` in the purge; they must agree, and time must never run backwards between calls. A capacity bound turns this into an LRU cache with expiry: the map points at the list nodes of #44, and a full `put` can purge first and evict the least recently used key only when that freed no room. Redis expires keys the same two ways, passively when a client touches a key that has timed out, and periodically by testing a few random keys among those with a TTL and deleting the expired ones. The signals are sessions, tokens, DNS records and anything with a time to live; Stock Price Tracker (#140) uses the same check of a heap entry against the map. The classic mistake is deleting the key of every popped pair without comparing deadlines: in the trace, the stale `(5, a)` would delete `a` at time 6, although its rewrite keeps it alive until 9.

Practice: LC 1797 Design Authentication Manager · LC 2034 Stock Price Fluctuation · LC 146 LRU Cache

95. Design Browser History

One Array and a Cursor · Design · Tier 2

Design the history of one browser tab. It starts on a homepage; `visit(url)` opens a new page and erases every page that `forward` could still have reached, and `back(steps)` and `forward(steps)` move up to `steps` pages through the history, stopping at either end, and return the page they land on. Starting at `home` and visiting `news`, `mail` and `maps`, `back(2)` lands on `news`; visiting `docs` from there erases `mail` and `maps`, so `forward(1)` stays on `docs`.

The natural model is two stacks, the pages behind the current one and the pages ahead of it. It works, but `back(steps)` carries pages from one stack to the other one at a time, `O(steps)`, and `forward` does the same in reverse. The history is a straight line, though, and a position on a line is just an index. Keep every page in one array with a cursor `current` on the page on screen, and `back` and `forward` become arithmetic: subtract or add `steps`, clamp to the ends, and read one slot, `O(1)` however far the jump. The only delicate call is `visit`, which must cut off the forward history. Truncating the array works, but a single visit can then pay for thousands of dropped pages, so a second index, `last`, marks where the live history ends instead. `visit` moves the cursor one slot forward, writes the new page there, overwriting a dead page or appending when the array has no slot left, and sets `last` to that slot. The slots past `last` still hold old pages, but no call can reach them, because `forward` clamps at `last`.

Picture the array as a ruler, with the cursor sliding along it and a fence at `last`. `back` and `forward` slide the cursor between slot 0 and the fence; `visit` steps the cursor one slot right, writes there and moves the fence right behind it, leaving everything beyond the fence dead.

```
slot                0     1     2     3
after visit(docs)   home  news  docs  (maps)     maps lies past last: dead
                                  ▲
                        current = last = 2

operation           pages                   current   last
start               home                    0         0
visit(news)         home news               1         1
visit(mail)         home news mail          2         2
visit(maps)         home news mail maps     3         3
back(2) → news      home news mail maps     1         3
visit(docs)         home news docs (maps)   2         2     overwrite slot 2
forward(1) → docs   home news docs (maps)   2         2     clamped at last
back(5) → home      home news docs (maps)   0         2     clamped at 0
forward(9) → docs   home news docs (maps)   2         2     clamped at last
```

```
CLASS BrowserHistory
    FUNCTION Init(homepage)
        self.pages ← [homepage]             // pages[0..last] is live history
        self.current ← 0                    // index of the page on screen
        self.last ← 0                       // the newest page forward can reach

    FUNCTION Visit(url)
        self.current ← self.current + 1
        IF self.current = len(self.pages)
            self.pages.append(url)          // the array grows by one
        ELSE
            self.pages[self.current] ← url  // overwrite a dead slot
```

```
        self.last ← self.current            // forward history ends here

    FUNCTION Back(steps)
        self.current ← max(0, self.current − steps)
        RETURN self.pages[self.current]

    FUNCTION Forward(steps)
        self.current ← min(self.last, self.current + steps)
        RETURN self.pages[self.current]
```

`back` and `forward` cost `O(1)`, `visit` costs amortized `O(1)` because the array sometimes grows, and the memory is one slot per page of the longest history so far. The signals are back and forward, undo and redo, and any cursor on a sequence where a new action throws away the redo side. The two-stack version is this array cut in two at the cursor; it fits when every move is a single step, as in undo and redo, and Design a Text Editor (#144) uses it to hold the text on either side of the cursor. A doubly linked list of pages is the third common answer, again with `O(steps)` moves. The classic mistake is clamping `forward` at `len(pages) − 1` instead of at `last`: after `visit(docs)` in the trace, `forward(9)` would bring back the dead `maps`. Updating `last` only when the array grows does the same damage.

Practice: LC 1472 Design Browser History · LC 2296 Design a Text Editor · LC 1381 Design a Stack With Increment Operation

96. Design Underground System

Open Trips and Route Totals in Two Maps · Design · Tier 2

Design the tracker behind a metro's fare gates. `check_in(card_id, station, t)` records a card entering, `check_out(card_id, station, t)` records the same card leaving, and `get_average_time(start, end)` returns the average duration of every finished trip that went from `start` to `end`. A card has at most one open trip, every trip ends later than it began, and a query always names a route with at least one finished trip. A card that enters at Leyton at time 3 and leaves at Waterloo at 15 rides for 12 minutes; with a second Leyton-to-Waterloo trip of 10 minutes, the average is `11.0`.

The brute force logs every finished trip and scans the log on each query, `O(trips)` per average, and every scan re-adds numbers that can never change again. An average is a total divided by a count, and both grow by a single addition when a trip ends, so the tracker can do the work at write time and keep just those two numbers per route. That is the core pattern of tracker designs: decide what the reads will ask, aggregate it as the writes arrive, and make every read a lookup. Two maps hold the whole state, one for each lifetime of the data. `open_trips` sends each card in the system to where and when it entered, and an entry lives only from check-in to check-out. `routes` sends each ordered pair `(start, end)` to its `[total_time, count]`, and an entry lives forever, in two numbers. A check-out moves a trip from the first map into the second: it removes the card's entry, computes the duration and folds it into its route's totals, so a trip is forgotten the moment it is counted.

Picture the open trips as a waiting room and the routes as a grid of counters, one cell per ordered pair of stations. A check-in walks into the waiting room, a check-out walks out and adds one trip to one cell, and a query divides the two numbers in one cell.

```
after the three check-ins, open_trips holds
    45 → (Leyton, 3)     32 → (Paradise, 8)     27 → (Leyton, 10)

call                          trip time      routes cell after the call
check_out(45, Waterloo, 15)   15 - 3 = 12    (Leyton, Waterloo) → [12, 1]
check_out(27, Waterloo, 20)   20 - 10 = 10   (Leyton, Waterloo) → [22, 2]
check_out(32, Cambridge, 22)  22 - 8 = 14    (Paradise, Cambridge) → [14, 1]
get_average_time(Paradise, Cambridge) = 14 / 1 = 14.0
get_average_time(Leyton, Waterloo)    = 22 / 2 = 11.0
check_in(10, Leyton, 24)      still open     no cell changes
get_average_time(Leyton, Waterloo)    = 22 / 2 = 11.0
check_out(10, Waterloo, 38)   38 - 24 = 14   (Leyton, Waterloo) → [36, 3]
get_average_time(Leyton, Waterloo)    = 36 / 3 = 12.0
open_trips is empty again
```

```
CLASS UndergroundSystem
    FUNCTION Init()
        self.open_trips ← empty map     // card_id → (start station, start time)
        self.routes ← empty map         // (start, end) → [total_time, count]

    FUNCTION CheckIn(card_id, station, t)
        self.open_trips[card_id] ← (station, t)

    FUNCTION CheckOut(card_id, station, t)
        (start, start_time) ← self.open_trips[card_id]
        self.open_trips.remove(card_id)     // the trip is over
        route ← (start, station)            // direction matters
```

```
        IF route NOT IN self.routes
            self.routes[route] ← [0, 0]     // [total_time, count]
        self.routes[route][0] ← self.routes[route][0] + t − start_time
        self.routes[route][1] ← self.routes[route][1] + 1

    FUNCTION GetAverageTime(start, end)
        (total_time, count) ← self.routes[(start, end)]
        RETURN total_time / count
```

Every call is a constant number of hash-map operations, so each runs in `O(1)`, and the memory is one entry per card in the system plus one per route ever completed, never one per trip. The signals are averages, counts, sums and totals asked again and again over a stream of events: the tracker answers in `O(1)` because it stores the ingredients of the answer instead of the events. Ask for a median or a percentile and two numbers no longer suffice; the values themselves must be kept, as in the two heaps of #21. The hit counter of #45 aggregates the same way, one counter per second instead of per route, and Design A Leaderboard keeps each player's running score as it arrives. The classic mistakes are keying the totals by the start station alone or by an unordered pair, although Leyton to Waterloo and Waterloo to Leyton are different routes, and forgetting to remove the card at check-out, so that `open_trips` keeps every card that ever rode and no longer tells who is inside. In a language whose `/` truncates integers, the average must be computed in floating point.

Practice: LC 1396 Design Underground System · LC 362 Design Hit Counter · LC 1244 Design A Leaderboard · LC 2034 Stock Price Fluctuation

97. Design Search Autocomplete System

Walk a Trie, Rank the Top 3 · Design · Tier 2

Design the suggestion box of a search engine. It starts from a history of past sentences and the number of times each was typed. `input(c)` receives one character and returns the three hottest past sentences that start with everything typed so far, most typed first and ties in ASCII order; `#` ends the sentence, records it as typed once more, and returns an empty list. With the history `i love you` 5, `island` 3, `iroman` 2 and `i love leetcode` 2, typing `i` returns `["i love you", "island", "i love leetcode"]`, and a space next narrows it to `["i love you", "i love leetcode"]`.

The brute force tests every sentence against the typed prefix on every keystroke and sorts the matches, `O(n · L)` per keystroke for the tests alone, with n sentences of length L. A trie (#43) indexes the sentences by prefix instead: each sentence is spelled along a path from the root, the node where it ends remembers the whole sentence, and a separate map holds the counts. The sentences that start with a prefix are exactly those that end in the subtree of that prefix's node. The design keeps a cursor on the node of the text typed so far, from one call to the next, so a keystroke is one step down to a child, never a fresh walk from the root. A DFS below the cursor collects the sentences, and sorting the pairs `(-count, sentence)` ranks by count and breaks ties in the same comparison. That is why `i love leetcode` beats `iroman` at count 2: a space sorts before every letter.

A missing child is a dead end. No past sentence starts with the typed text, and none can start with any longer text, so the cursor becomes null and stays null until `#`. The typed text is still kept, because `#` records it: a fresh path grows into the trie, its count rises by one, and the cursor returns to the root.

```
trie after the demo; _ is a space and a chain of one-child nodes is one label
root ── i ─┬─ _ ─┬─ love_ ─┬─ you ●        i love you         5
           │    │          └─ leetcode ●   i love leetcode    2
           │    └─ a ●                     i a               2 (typed twice)
           ├─ sland ●                      island            3
           └─ roman ●                      iroman            2

key  cursor     sentences below, by (-count, sentence); | marks the cut at 3
i    i          i love you 5, island 3, i love leetcode 2 | iroman 2
_    i_         i love you 5, i love leetcode 2
a    null       no child a under i_: dead end, [] until the #
#    root       record "i a" with count 1, clear the typed text
i    i          i love you 5, island 3, i love leetcode 2 | iroman 2, i a 1
_    i_         i love you 5, i love leetcode 2, i a 1
a    i_a        i a 1
#    root       record "i a" again: count 2
```

```
CLASS TrieNode
    FUNCTION Init()
        self.children ← empty map              // character → TrieNode
        self.sentence ← null                   // the sentence that ends here

CLASS AutocompleteSystem
    FUNCTION Init(sentences, times)
        self.root ← new TrieNode()
        self.counts ← empty map                // sentence → times typed
        FOR i FROM 0 TO len(sentences) - 1
            self.Record(sentences[i], times[i])
```

```
        self.typed ← ""                          // the sentence typed so far
        self.node ← self.root                    // node of typed; null: dead end

    FUNCTION Record(sentence, times)
        node ← self.root
        FOR EACH ch IN sentence
            IF ch NOT IN node.children
                node.children[ch] ← new TrieNode()
            node ← node.children[ch]
        node.sentence ← sentence                 // mark where the sentence ends
        self.counts[sentence] ← self.counts.get(sentence, 0) + times

    FUNCTION Input(c)
        IF c = "#"
            self.Record(self.typed, 1)           // it was typed once more
            self.typed ← ""
            self.node ← self.root
            RETURN empty list
        self.typed ← self.typed + c
        IF self.node = null OR c NOT IN self.node.children
            self.node ← null                     // dead end: stays dead until #
            RETURN empty list
        self.node ← self.node.children[c]        // one step down, no rescan
        RETURN self.TopThree(self.node)

    FUNCTION Collect(node, found)
        IF node.sentence ≠ null
            found.append(node.sentence)
        FOR EACH ch IN node.children
            self.Collect(node.children[ch], found)

    FUNCTION TopThree(node)
        found ← empty list
        self.Collect(node, found)
        ranked ← empty list
        FOR EACH sentence IN found
            ranked.append((-self.counts[sentence], sentence))
        ranked.sort()                            // top count first, then ASCII
        best ← empty list
        FOR i FROM 0 TO min(3, len(ranked)) - 1
            best.append(ranked[i][1])            // keep the sentence
        RETURN best
```

A keystroke costs `O(1)` for the step plus `O(k + m log m)` to collect and rank, for k nodes and m sentences under the cursor (a size-3 heap from #19 trims the ranking to `O(m)`); `#` costs `O(L)`, and the trie holds one node per distinct prefix. The collect is the expensive part, and the trade-off interviewers want to hear is caching the top three at every node. A keystroke then reads three references in `O(1)`, while `#` must update every node on the sentence's path and every node stores three more references. Because counts only grow, each update is an exact merge into a list of three: the recorded sentence is the only one whose rank changes, and it can only climb. Search engines take that side of the trade, computing the suggestions for each prefix offline from query logs and refreshing them in batches. The signals are suggestions as you type and ranked prefix queries. The classic mistake is treating a dead end as a restart: if the cursor falls back to the root, typing `x` and then `i` suggests `i love you` for the text `xi`.

Practice: LC 642 Design Search Autocomplete System · LC 1268 Search Suggestions System · LC 208 Implement Trie (Prefix Tree)

98. My Calendar I and II

Bookings Without Double or Triple Overlaps · Design · Tier 2

Build a calendar that accepts or refuses events one at a time. An event is the half-open interval `[start, end)`, so `[10, 20)` and `[20, 30)` only touch. `MyCalendar.book(start, end)` adds the event and returns `true` unless it overlaps a booking, in which case it returns `false` and changes nothing. `MyCalendarTwo.book` lets two events overlap but refuses one that would put any instant under three. The first calendar accepts `[10, 20)`, refuses `[15, 25)` and accepts `[20, 30)`. The second accepts `[10, 20)`, `[50, 60)` and `[10, 40)`, then refuses `[5, 15)`, because the time from 10 to 15 would be booked three times.

The brute force compares a new event with every booking, `O(n)` comparisons per call. Two half-open intervals overlap exactly when each starts before the other ends. In My Calendar I the accepted bookings never overlap, so sorted by start they are also sorted by end, and they lie on the time line like disjoint tiles. Binary search finds the slot where the new event would go, the first booking whose start is at least the new start (the lower bound of #14). Only the two tiles beside that slot can collide with the event: the next one if it starts before the new end, the previous one if it ends after the new start. Every tile further left ends no later than the previous one starts, and every tile further right starts after the next one does. If both neighbors are clear, the event is inserted at the slot and the tiles stay sorted.

My Calendar II cannot use tiles, because its bookings may overlap. It keeps two lists instead: all the bookings, and the overlaps, the stretches of time already booked twice. A new event that meets any overlap would create a triple booking, so it is refused before anything changes. Otherwise each intersection of the event with an existing booking, from the later start to the earlier end, is now booked twice and joins the overlaps, and the event joins the bookings.

```
MyCalendar: the slot is the first booking whose start is ≥ the new start
book       slot   previous   next       verdict
[10, 20)    0     none       none       ✓
[15, 25)    1     [10, 20)   none       ✗  the previous ends at 20 > 15
[20, 30)    1     [10, 20)   none       ✓  20 > 20 is false: touching is fine
[5, 10)     0     none       [10, 20)   ✓  10 < 10 is false
[0, 6)      0     none       [5, 10)    ✗  the next starts at 5 < 6
time   0    5    10   15   20   25   30
            ├────┼─────────┼─────────┤   starts [5, 10, 20], ends [10, 20, 30]

MyCalendarTwo: the overlaps hold the time already booked twice
book       meets an overlap?    new overlaps         verdict
[10, 20)   no                   none                 ✓
[50, 60)   no                   none                 ✓
[10, 40)   no                   [10, 20)             ✓
[5, 15)    yes, [10, 20)        none                 ✗  10 to 15 would be triple
[5, 10)    no, it only touches  none                 ✓
[25, 55)   no                   [50, 55), [25, 40)   ✓
```

```
CLASS MyCalendar
    FUNCTION Init()
        self.starts ← empty list              // starts of the bookings, sorted
        self.ends ← empty list                // ends[i] goes with starts[i]

    FUNCTION Book(start, end)
```

```
        left ← 0
        right ← len(self.starts)
        WHILE left < right                      // first booking starting ≥ start
            mid ← floor((left + right) / 2)
            IF self.starts[mid] < start
                left ← mid + 1
            ELSE
                right ← mid
        IF left < len(self.starts) AND self.starts[left] < end
            RETURN false                        // the next starts too soon
        IF left > 0 AND self.ends[left − 1] > start
            RETURN false                        // the previous ends too late
        self.starts.insert(left, start)
        self.ends.insert(left, end)
        RETURN true

CLASS MyCalendarTwo
    FUNCTION Init()
        self.bookings ← empty list              // every accepted (start, end)
        self.overlaps ← empty list              // the parts booked twice

    FUNCTION Book(start, end)
        FOR EACH (overlap_start, overlap_end) IN self.overlaps
            IF overlap_start < end AND start < overlap_end
                RETURN false                    // it would be booked 3 times
        FOR EACH (booked_start, booked_end) IN self.bookings
            IF booked_start < end AND start < booked_end
                low ← max(start, booked_start)
                high ← min(end, booked_end)
                self.overlaps.append((low, high))   // now booked twice
        self.bookings.append((start, end))
        RETURN true
```

My Calendar I spends only `O(log n)` comparisons on the search, but inserting into the middle of an array shifts every later entry, so a booking costs `O(n)`; a balanced search tree, such as Java's `TreeMap` with `floorKey` and `ceilingKey`, brings it to `O(log n)`. My Calendar II costs `O(n)` per booking, and because no instant is ever booked three times there are always fewer overlaps than bookings. My Calendar III asks for the largest number of events sharing an instant: a map of +1 at every start and -1 at every end, walked in time order, counts the open events at each moment, the sweep line that also solves Meeting Rooms II (#27). The signals are requests accepted or refused one at a time under a cap on how many events may share an instant. The classic mistakes are treating the intervals as closed, which refuses `[20, 30)` after `[10, 20)`, and, in the second calendar, counting how many bookings a new event meets: `[15, 25)` meets both `[10, 20)` and `[20, 30)`, yet no instant would be booked three times.

Practice: LC 729 My Calendar I · LC 731 My Calendar II · LC 732 My Calendar III

99. Design Snake Game

A Deque for the Body, a Set for Collisions · Design · Tier 2

Design the game Snake on a grid `width` cells wide and `height` cells high. The snake starts as one cell in the top-left corner, `(0, 0)`, and food appears at the given cells one piece at a time, each only after the previous one is eaten. `move(direction)`, with direction `U`, `D`, `L` or `R`, moves the head one cell and returns the score, the number of pieces eaten; eating makes the snake one cell longer. Once the head leaves the grid or runs into the body the game is over, and every move returns `-1`. On a grid 3 wide and 2 high with food at `(1, 2)` and then `(0, 1)`, the moves `R D R U L U` return `0 0 1 1 2 -1`.

A plain list of cells costs `O(length)` per move, both to shift the body and to test whether the head hits it. Yet a move changes only the ends: a new cell appears at the head and, unless the snake eats, the cell at the tail disappears; everything in between stays put. So the body is a deque of cells with the head at the front, and a move is one push at the front and at most one pop at the back, both `O(1)`. For the collision test a set holds the same cells, which turns "is this cell part of the body?" into one lookup instead of a scan; every cell that enters or leaves the deque enters or leaves the set in the same step.

The order of the steps is the whole difficulty. Compute the new head and test the walls first. Then settle the tail: if the head lands on the next piece of food, the score rises and the tail stays, which is exactly how the snake grows; otherwise the tail leaves both the deque and the set. Only then test the head against the set. Letting the tail go before the test is what allows the head to enter the cell the tail is just leaving, as when a snake that fills a 2×2 board chases its own tail around it.

```
3 wide, 2 high; H head, o body, * the food on the board, one piece at a time
start     R → 0     D → 0     R → 1     U → 1     L → 2     U → -1
H . .     . H .     . . .     . * .     . * H     . H o     the head
. . *     . . *     . H *     . o H     . . o     . . o     leaves the grid
                              eats:     the tail  eats:
                              the tail  leaves    the tail
                              stays     (1, 1)    stays
body after L, head first: [(0, 1), (0, 2), (1, 2)]; occupied holds the same

2 × 2, food (0, 1), (1, 1), (1, 0); t marks the tail
R D L → 1 2 3   U → 3                       D → -1
t o             H t                         the head goes to (1, 0), the
H o             o o                         neck, which stays: it bites
board full      the tail left (0, 0) first, itself
                so the head may move in
```

```
MOVES ← map with U → (-1, 0), D → (1, 0), L → (0, -1), R → (0, 1)

CLASS SnakeGame
    FUNCTION Init(width, height, food)
        self.width ← width
        self.height ← height
        self.food ← food                        // (row, col) cells, in order
        self.score ← 0                          // pieces eaten = next food index
        self.body ← empty deque                 // head at the front
        self.body.push_back((0, 0))
        self.occupied ← empty set               // the same cells as body
        self.occupied.add((0, 0))
        self.over ← false
```

```
    FUNCTION Move(direction)
        IF self.over
            RETURN -1
        (row, col) ← self.body.front()
        (step_row, step_col) ← MOVES[direction]
        row ← row + step_row
        col ← col + step_col
        IF row < 0 OR row ≥ self.height OR col < 0 OR col ≥ self.width
            self.over ← true                    // hit a wall
            RETURN -1
        head ← (row, col)
        IF self.score < len(self.food) AND head = self.food[self.score]
            self.score ← self.score + 1         // eat: the tail stays
        ELSE
            tail ← self.body.pop_back()         // the tail leaves first, so
            self.occupied.remove(tail)          // the head may enter its cell
        IF head IN self.occupied
            self.over ← true                    // ran into its own body
            RETURN -1
        self.body.push_front(head)
        self.occupied.add(head)
        RETURN self.score
```

Each move is a constant number of deque and set operations, so `move` runs in `O(1)` time, and the two containers hold `O(length)` cells, at most `width × height`. The food needs no search either: pieces appear in order, so an index into the list, which is also the score, names the only piece on the board. Pairing an ordered container with a set of its members works wherever a sequence must also answer "is x in it?" at once, just as the LRU cache (#44) pairs its list with a map. The signals are a grid simulation with a body that grows at one end and shrinks at the other, plus fast membership tests. The classic mistake is testing the collision before letting the tail go, which kills a snake that follows its own tail; the opposite mistake, dropping the tail even when the snake eats, keeps it from ever growing.

Practice: LC 353 Design Snake Game · LC 348 Design Tic-Tac-Toe · LC 641 Design Circular Deque

100. URL Shortener

Base62-Encode an Increasing ID · Design · Tier 2

Design a service that turns a long URL into a short code and back. `encode(long_url)` returns a code, `decode(short_code)` returns the original URL, or the empty string for a code never handed out, and a URL seen before gets its old code again. The design gives every new URL the next integer id and writes that id in base 62, whose digits are `0–9`, `a–z` and `A–Z`. If this server's ids start at 1,000,000, the first URL gets the code `4c92` and the next new one `4c93`, and decoding `4c93` returns the second URL.

Base 62 is positional notation with 62 digits instead of 10. `n mod 62` is the last digit, dividing by 62 drops it, and repeating until `n` reaches 0 produces the digits from right to left, so they are reversed at the end. Reading a code back runs the other way: start from 0 and, for each character, multiply by 62 and add the character's value. Every extra character multiplies the number of codes by 62, so 6 characters cover 56.8 billion ids and 7 cover 3.5 trillion. Two maps complete the design: code to URL for `decode`, and URL to code, so that a repeated URL gets its old code back instead of using up a new id.

The obvious alternative hashes the URL and keeps the first 7 characters of the hash written in base 62. The same URL then always gets the same code without a second map, but different URLs can share those 7 characters: by the birthday bound a collision becomes likely after about two million URLs, the square root of `62^7`, so every insert must look its code up and rehash on a clash. A counter cannot collide, since every id is new, and it hands out the shortest codes possible, because the ids are used in order with no gaps.

```
to_base62(1000000): the last digit is n mod 62, then n ← floor(n / 62)
   n          n mod 62   digit   floor(n / 62)
   1000000        2        2        16129
   16129          9        9          260
   260           12        c            4
   4              4        4            0
   the digits come out as 2 9 c 4; reversed they read 4c92

from_base62("4c92"): n ← n × 62 + the value of the next digit
   0 × 62 + 4 = 4    4 × 62 + 12 = 260    260 × 62 + 9 = 16129
   16129 × 62 + 2 = 1000000

after encode(long_url), encode(other_url), encode(long_url):
   next_id   1000002              the repeat took no new id
   url_of    4c92 → https://example.com/articles/2026/how-base62-works
             4c93 → https://example.com/docs
   code_of   the same two pairs the other way round
```

```
ALPHABET ← "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

FUNCTION ToBase62(n)
    IF n = 0
        RETURN "0"
    digits ← empty list
    WHILE n > 0
        digits.append(ALPHABET[n mod 62])      // the last digit comes out first
        n ← floor(n / 62)
    digits.reverse()                           // most significant digit first
    RETURN join(digits)
```

```
FUNCTION FromBase62(code)
    n ← 0
    FOR EACH ch IN code
        n ← n * 62 + ALPHABET.index(ch)        // index = the digit's value
    RETURN n

CLASS URLShortener
    FUNCTION Init(first_id)
        self.next_id ← first_id                // start of this server's ids
        self.url_of ← empty map                // code → long url
        self.code_of ← empty map               // long url → code

    FUNCTION Encode(long_url)
        IF long_url IN self.code_of
            RETURN self.code_of[long_url]      // seen before: same code
        code ← ToBase62(self.next_id)
        self.next_id ← self.next_id + 1
        self.url_of[code] ← long_url
        self.code_of[long_url] ← code
        RETURN code

    FUNCTION Decode(short_code)
        IF short_code NOT IN self.url_of
            RETURN ""
        RETURN self.url_of[short_code]
```

Encoding costs `O(log n)` for id n, one step per character, plus the map lookups, and the maps hold every pair. At scale the single counter is a bottleneck and a single point of failure, so a coordinator, such as a database sequence or ZooKeeper, hands each server a block of ids, a million at a time, which is what `first_id` stands for; each server counts through its block alone, and a crash wastes at most the rest of one block. Snowflake ids (#145) drop the central counter altogether. Sequential codes are guessable, so links that must not be enumerable scramble the id first or use the hash with its collision check. A custom alias is a code the user picks, accepted only if the code-to-URL table lacks it; the counter must then skip any code already taken, or the id that later reaches an alias would overwrite it. Reads far outnumber writes, so hot codes live in an LRU cache (#44) in front of a store split across machines by code (#139). Excel column titles are the same conversion in base 26 with no zero digit, so each step subtracts 1 before taking the remainder. The classic mistakes are skipping the reversal, which turns 1,000,000 into `29c4`, and forgetting that 0 needs the code `0`: the loop never runs and the code comes out empty.

Practice: LC 535 Encode and Decode TinyURL · LC 168 Excel Sheet Column Title · LC 171 Excel Sheet Column Number

Tier 3 · Good to Know

The last fifty round out the set: second implementations of ideas you already own (open addressing, heap sort, Prim), bridges in a graph, the segment tree, the string toolkit, number theory, geometry and bit tricks, and the system-design algorithms that anchor a distributed-systems discussion: rate limiting, consistent hashing, unique IDs, Bloom filters, count-min sketches, quadtrees, job scheduling and the inverted index. Read each one once, keep its picture in mind, and come back to it when a problem or a design discussion calls for it.

101. Hash Map with Open Addressing

Linear Probing and Tombstones · Hash Map Internals · Tier 3

Design a hash map without a built-in dictionary that stores every pair directly in one array of slots, with no chains: `put(key, value)`, `get(key)` returning `-1` for a missing key, and `remove(key)`. With 8 slots and integers hashing to themselves, key `3` lives in slot `3`; key `11` also hashes to slot `3`, finds it taken and settles in slot `4`, so after `put(3, 30)` and `put(11, 110)` the call `get(11)` returns `110`.

Separate chaining (#51) hangs a list off every bucket. Open addressing keeps one flat array and resolves a collision, a slot already holding a different key, by probing: try the home slot `hash(key) mod capacity`, then the next slot, and the next, wrapping around the end. A key therefore sits somewhere on the unbroken run of non-empty slots that starts at its home, and a search walks that run until it meets the key or an EMPTY slot. An EMPTY slot ends the search, because an insert of the key would have stopped there; the key cannot lie beyond it.

Deletion is the subtle part. Emptying a slot would cut a run in two and strand every key whose path crosses the gap, so `remove` writes a DELETED marker, a tombstone, that searches step over and inserts may reuse: once a full search has shown that a key is absent, `put` takes the first tombstone or EMPTY slot on its path. Tombstones lengthen searches just like live keys, so the load counts both, and when `(size + tombstones) / capacity` passes 0.5 the table doubles and re-puts only its live pairs, which sweeps the tombstones away. The table stays half empty because of primary clustering: occupied slots merge into runs, a key whose home falls anywhere inside a run lands at its end, and so long runs catch more keys and grow faster still. A miss inspects about 2.5 slots on average at load 0.5, but dozens at load 0.9.

```
8 slots, home = key mod 8                · EMPTY     X tombstone
               slot   0    1    2    3    4    5    6    7
put 3                 ·    ·    ·    3    ·    ·    ·    ·    home 3
put 11                ·    ·    ·    3   11    ·    ·    ·    3 taken → 4
put 19                ·    ·    ·    3   11   19    ·    ·    3, 4 taken → 5
put 4                 ·    ·    ·    3   11   19    4    ·    home 4 → 6
remove 11             ·    ·    ·    3    X   19    4    ·
get 19:    slot 3 holds 3, slot 4 is X so keep going, slot 5 holds 19 → 190
put 27:    the search passes 3, X, 19, 4 and stops at EMPTY slot 7: absent;
           the first free slot on its path is the tombstone in slot 4
put 27                ·    ·    ·    3   27   19    4    ·
put 12:    lands in slot 7; load (5 + 0) / 8 = 0.625 > 0.5 → 16 slots,
           live keys re-put: 3 → 3, 19 → 4, 4 → 5, 27 → 11, 12 → 12
```

```
CLASS ProbingHashMap                // a slot: EMPTY, DELETED or [key, value]
    FUNCTION Init(capacity)
        self.capacity ← capacity
        self.size ← 0                       // live keys
        self.tombstones ← 0                 // DELETED slots
        self.slots ← array of capacity copies of EMPTY

    FUNCTION Find(key)                      // index of key's slot, or -1
        index ← hash(key) mod self.capacity
        WHILE self.slots[index] ≠ EMPTY
            slot ← self.slots[index]
            IF slot ≠ DELETED AND slot[0] = key
                RETURN index
```

```
            index ← (index + 1) mod self.capacity    // taken: try the next
        RETURN −1

    FUNCTION Put(key, value)
        index ← self.Find(key)
        IF index ≠ −1
            self.slots[index][1] ← value             // already stored
            RETURN
        index ← hash(key) mod self.capacity
        WHILE self.slots[index] ≠ EMPTY AND self.slots[index] ≠ DELETED
            index ← (index + 1) mod self.capacity    // skip live pairs
        IF self.slots[index] = DELETED
            self.tombstones ← self.tombstones − 1    // reuse the tombstone
        self.slots[index] ← [key, value]
        self.size ← self.size + 1
        IF (self.size + self.tombstones) / self.capacity > 0.5
            self.Resize()

    FUNCTION Get(key)
        index ← self.Find(key)
        IF index = −1
            RETURN −1
        RETURN self.slots[index][1]

    FUNCTION Remove(key)
        index ← self.Find(key)
        IF index ≠ −1
            self.slots[index] ← DELETED     // later searches step over it
            self.size ← self.size − 1
            self.tombstones ← self.tombstones + 1

    FUNCTION Resize()
        old_slots ← self.slots
        self.capacity ← 2 * self.capacity
        self.slots ← array of self.capacity copies of EMPTY
        self.size ← 0
        self.tombstones ← 0
        FOR EACH slot IN old_slots
            IF slot ≠ EMPTY AND slot ≠ DELETED
                self.Put(slot[0], slot[1])  // tombstones are dropped
```

With a good hash and the load capped at one half, every operation takes `O(1)` expected time; the worst case is `O(n)`, one run through the whole table, and a resize costs `O(n)` but is amortized away as in #51. One flat array of pairs and no pointers keeps the table compact and friendly to the cache; CPython's `dict` is an open-addressing table too, though it scrambles the probe order instead of stepping by one. The signals are a design question about hash map internals, or a follow-up asking how a map works without linked lists. The classic mistake is a `remove` that writes EMPTY: after removing 11 from the cluster in the picture, `get(19)` stops at the hole in slot 4 and reports −1.

Practice: LC 706 Design HashMap · LC 705 Design HashSet

102. Boyer-Moore Majority Vote

Different Votes Cancel in Pairs · Arrays & Hashing · Tier 3

Given an array, return the value that appears more than `n / 2` times, or report that none does, using `O(n)` time and `O(1)` extra space. In `[2, 2, 1, 1, 1, 2, 2]` the answer is 2, with 4 of the 7 votes.

A hash map of counts finds it in `O(n)` time but `O(n)` space, and sorting puts it at the middle index in `O(n log n)`. Boyer-Moore needs two variables. Treat every element as a vote and let any two different votes cancel each other. A majority value has more votes than all the other values combined, so even if every other vote is spent cancelling one of its own, some of its votes survive. The scan keeps a `candidate` and the `count` of its uncancelled votes: when `count` is 0 the current value takes over as candidate, an equal value adds a vote, and a different value cancels one.

The count is a walk that returns to zero whenever the votes seen so far pair off completely. Cut the array at those points. In every closed piece the candidate holds exactly half the votes, so the true majority, which holds more than half of the whole array, must hold more than half of the final open piece, and there it is the candidate. The guarantee runs one way only: without a majority the survivor is just the last value standing, and `[1, 2, 3]` returns 3. So when the problem does not promise a majority, a second pass counts the candidate's votes and accepts it only above `n / 2`.

```
x           2  2  1  1  |  1  2  |  2
candidate   2  2  2  2  |  1  1  |  2
count       1  2  1  0  |  1  0  |  1
cut where count hits 0: each closed piece is half candidate, half others
survivor 2; second pass: 2 appears 4 times, 4 > 7 / 2, so 2 is the majority
```

```
FUNCTION MajorityCandidate(nums)
    candidate ← null
    count ← 0                              // candidate votes not yet cancelled
    FOR EACH x IN nums
        IF count = 0
            candidate ← x                  // nobody standing: x takes over
            count ← 1
        ELSE IF x = candidate
            count ← count + 1              // one more vote for the candidate
        ELSE
            count ← count − 1              // x cancels one candidate vote
    RETURN candidate

FUNCTION MajorityElement(nums)
    candidate ← MajorityCandidate(nums)
    votes ← 0
    FOR EACH x IN nums
        IF x = candidate
            votes ← votes + 1
    IF votes * 2 > len(nums)
        RETURN candidate
    RETURN null                            // no value holds a strict majority
```

Two linear passes give `O(n)` time with `O(1)` space. Reach for vote cancelling when a problem asks for a value that dominates an array or a stream in constant memory. The idea generalizes: at most two values can each appear more than `n / 3` times, so Majority Element II keeps two candidates with two

counts, lets a value that matches neither and finds no empty slot cancel one vote from each, and verifies both at the end. The classic mistake is trusting the candidate without the second pass when no majority is promised; the check must also demand strictly more than half, since in `[1, 2]` neither value is a majority.

Practice: LC 169 Majority Element · LC 229 Majority Element II · LC 1150 Check If a Number Is Majority Element in a Sorted Array

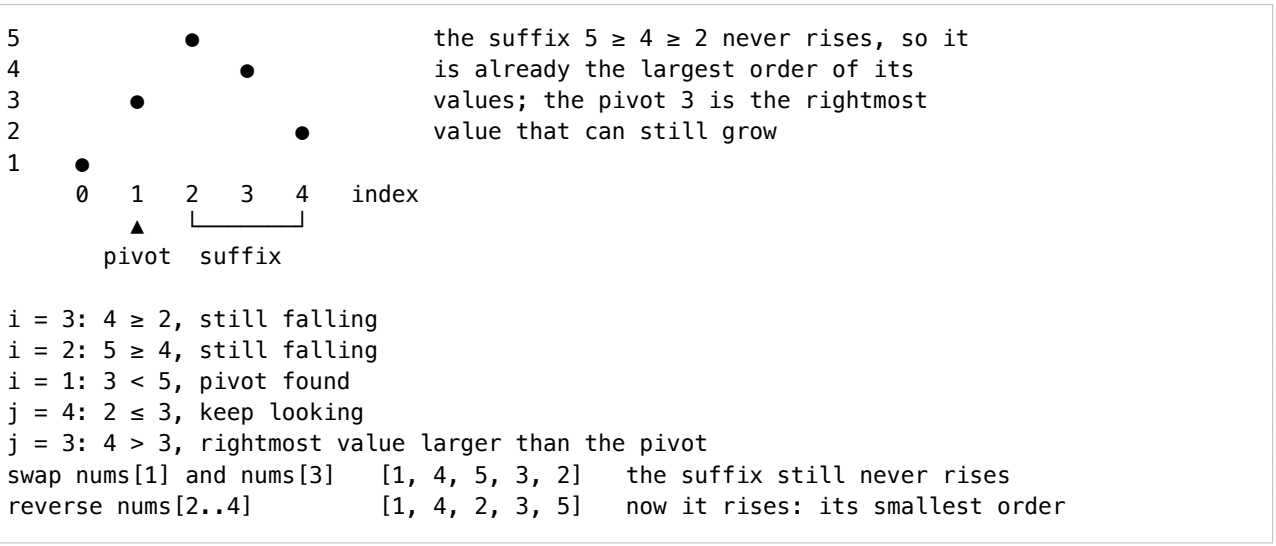
103. Next Permutation

Find the Pivot, Swap, Reverse the Suffix · Arrays & Hashing · Tier 3

Rearrange an array in place into its next permutation in lexicographic order, the next larger arrangement of the same values when they are compared like words in a dictionary; if the array is already the largest arrangement, wrap around to the smallest, the sorted one. Values may repeat. `[1, 3, 5, 4, 2]` becomes `[1, 4, 2, 3, 5]`, and `[3, 2, 1]` wraps to `[1, 2, 3]`.

The brute force lists the permutations in lexicographic order and returns the one after the input, `O(n · n!)`. The insight is to read the array from the right. A suffix that never rises, such as `5, 4, 2`, is already the largest arrangement of its values, so no reordering inside it can make the array larger, and the change has to reach further left. Walk left while each value is at least its right neighbor and stop at the first `nums[i] < nums[i + 1]`. That `i` is the pivot, the rightmost position that can grow, and because of where the walk stopped, everything after it is non-increasing. If the walk runs off the left end, the whole array never rises: it is the last permutation, and reversing it gives the first.

To grow the pivot by as little as possible, swap it with the smallest suffix value that is larger than it; since the suffix never rises, that is the rightmost value larger than the pivot. The swap leaves the suffix non-increasing, because the values left of `j` are at least the old `nums[j]` and the values right of `j` are at most the pivot. With the pivot raised by the minimum, everything after it must be as small as possible, which means sorted order, and reversing a non-increasing run produces exactly that in `O(n)`, with no sort.




```
FUNCTION NextPermutation(nums)
    i ← len(nums) − 2
    WHILE i ≥ 0 AND nums[i] ≥ nums[i + 1]       // walk past the falling suffix
        i ← i − 1
    IF i ≥ 0                                    // nums[i] is the pivot
        j ← len(nums) − 1
        WHILE nums[j] ≤ nums[i]                 // rightmost larger value
            j ← j − 1
        swap(nums[i], nums[j])
    Reverse(nums, i + 1, len(nums) − 1)         // falling suffix → rising

FUNCTION Reverse(nums, left, right)
```

```
    WHILE left < right
        swap(nums[left], nums[right])
        left ← left + 1
        right ← right − 1
```

Two scans from the right and one reversal make it `O(n)` time and `O(1)` space. Reach for it when a problem wants the next arrangement rather than all of them: the next larger number with the same digits is Next Greater Element III, which returns -1 when the digits never rise or the answer overflows 32 bits, and starting from the sorted array and calling it until it wraps lists every distinct permutation exactly once, duplicates included, without the used array of Permutations (#31). Flipping the two value comparisons, ≥ to ≤ and ≤ to ≥, gives the previous permutation. The classic mistake is a strict comparison where duplicates need equality. Scanning for `j` with `nums[j] < nums[i]` stops on a value equal to the pivot, so `[1, 5, 1]` becomes `[1, 1, 5]`, a smaller permutation, instead of `[5, 1, 1]`; a strict pivot scan picks the equal pair of `[1, 1]` as a pivot, and the search for `j` runs off the left end.

Practice: LC 31 Next Permutation · LC 556 Next Greater Element III · LC 47 Permutations II · LC 60 Permutation Sequence

104. Reorder List

Find the Middle, Reverse, Weave · Two Pointers · Tier 3

Given the head of a singly linked list `L0 → L1 → … → Ln`, reorder it in place into `L0 → Ln → L1 → Ln−1 → L2 → …`, relinking the nodes rather than copying their values. `1 → 2 → 3 → 4 → 5` becomes `1 → 5 → 2 → 4 → 3`.

The brute force copies the nodes into an array and relinks them with two indices closing in from both ends: `O(n)` time, but `O(n)` extra space. Without the array the order looks hard, because it reads the list from both ends at once and a singly linked list can only be read forward. Split it in two, though, and the answer takes the first half forward and the second half backward, one node from each in turn. Reverse the second half in place and both halves read forward, so the answer is a zipper of two forward lists. That is three steps, each one already known: slow and fast pointers find the end of the first half, a cut and a reversal (#6) turn the second half around, and a weave alternates the halves.

Picture folding the list at its middle like a strip of paper, so that the tail lies reversed under the head, and reading down the zigzag. Fast moves two nodes for every one that slow moves, so when fast has no double step left, slow stands on index `(n − 1) / 2` rounded down, the last node of the first half. With an odd count the first half keeps the extra node, which is right, since the middle node ends the woven list. The second half is never longer, so the weave runs while it has nodes, and each step saves both `next` pointers before it relinks `first → second → the rest of the first half`.

```
1       2       3        first half, read forward
  \   /   \   /          the woven list zigzags: 1 → 5 → 2 → 4 → 3
    5       4            second half, reversed

start     1 → 2 → 3 → 4 → 5
middle    slow 1, 2, 3   fast 1, 3, 5     fast.next is null: slow ends on 3
cut       first 1 → 2 → 3      second 4 → 5
reverse   first 1 → 2 → 3      second 5 → 4
weave 1   1 → 5, 5 → 2   list 1 → 5 → 2 → 3          second left 4
weave 2   2 → 4, 4 → 3   list 1 → 5 → 2 → 4 → 3      second left null
```

```
FUNCTION ReverseList(head)
    prev ← null
    curr ← head
    WHILE curr ≠ null
        next_node ← curr.next              // save the rest before the cut
        curr.next ← prev
        prev ← curr
        curr ← next_node
    RETURN prev

FUNCTION ReorderList(head)
    IF head = null
        RETURN
    slow ← head
    fast ← head
    WHILE fast.next ≠ null AND fast.next.next ≠ null
        slow ← slow.next                   // ends on the first half's last node
        fast ← fast.next.next
    second ← slow.next
```

```
    slow.next ← null                        // cut after the first half
    second ← ReverseList(second)
    first ← head
    WHILE second ≠ null                     // the second half is never longer
        next_first ← first.next             // save both rests before relinking
        next_second ← second.next
        first.next ← second
        second.next ← next_first
        first ← next_first
        second ← next_second
```

Each step walks the list at most once, so the reorder takes `O(n)` time with a handful of pointers, `O(1)` space. Reach for middle plus reversal whenever a list problem pairs its front with its back: Palindrome Linked List compares the first half with the reversed second, Maximum Twin Sum of a Linked List adds them pair by pair, and Middle of the Linked List is the first step alone, written `WHILE fast ≠ null AND fast.next ≠ null` because it wants the second middle of an even-length list. The classic mistake is skipping the cut. The first half's last node then still points into the second half, so on the example `3 → 4` survives while the weave adds `4 → 3`, and the list ends in a cycle, `1 → 5 → 2 → 4 → 3 → 4 → 3 → …`.

Practice: LC 143 Reorder List · LC 876 Middle of the Linked List · LC 234 Palindrome Linked List · LC 2130 Maximum Twin Sum of a Linked List

105. Median of K Sorted Arrays

Binary Search on the Value · Binary Search · Tier 3

Given `k` sorted arrays of integers holding `N ≥ 1` values in total, return the median of all of them: the middle value when `N` is odd, the average of the two middle values when `N` is even. For `[1, 4, 9]`, `[2, 3, 7, 10]` and `[5, 8]` the nine values in order are `1 2 3 4 5 7 8 9 10`, so the median is `5`.

The brute force merges the arrays with a min-heap of their front elements (#20) and pops until it reaches the middle, `O(N log k)`. The faster idea stops hunting for a position and searches for a value instead. Ask of any integer `v`: how many values are at most `v`? Each array answers with one upper-bound binary search (#14), and the sum over all arrays is a count that never shrinks as `v` grows. So the question "are at least `r` values at most `v`?" reads `✗ ✗ ✗` then `✓ ✓ ✓` along the number line, and the `r`-th smallest value is the first `✓`, found by binary search on the answer (#15) between the smallest first element and the largest last element.

That first `✓` is always a real element of some array, because the count changes only at values that appear in the arrays: if `v` were absent, `v − 1` would have the same count and would be an earlier `✓`. The median then takes one or two of these order statistics, counted from 1: rank `⌊N / 2⌋ + 1` when `N` is odd, and the average of ranks `⌊N / 2⌋` and `⌊N / 2⌋ + 1` when it is even.

```
[1, 4, 9]  [2, 3, 7, 10]  [5, 8]: N = 9, so the median is the 5th smallest
value v          1   2   3   4   5   6   7   8   9  10
count(≤ v)       1   2   3   4   5   5   6   7   8   9
count ≥ 5?       ✗   ✗   ✗   ✗   ✓   ✓   ✓   ✓   ✓   ✓
                                 ▲ the first ✓ is 5, a real element

   lo   hi   mid   count(≤ mid), per array   ≥ 5?   move
    1   10     5    2 + 2 + 1 = 5             yes   hi ← 5
    1    5     3    1 + 2 + 0 = 3             no    lo ← 4
    4    5     4    2 + 2 + 0 = 4             no    lo ← 5
    5    5   lo = hi: the 5th smallest value is 5
```

```
FUNCTION CountAtMost(arr, value)        // upper bound (#14): values ≤ value
    lo ← 0
    hi ← len(arr)
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)
        IF arr[mid] ≤ value
            lo ← mid + 1
        ELSE
            hi ← mid
    RETURN lo

FUNCTION KthSmallest(arrays, rank)      // smallest v with count(≤ v) ≥ rank
    lo ← ∞
    hi ← −∞
    FOR EACH arr IN arrays
        IF len(arr) > 0
            lo ← min(lo, arr[0])                // smallest first element
            hi ← max(hi, arr[len(arr) − 1])     // largest last element
    WHILE lo < hi
        mid ← floor((lo + hi) / 2)
```

```
        count ← 0
        FOR EACH arr IN arrays
            count ← count + CountAtMost(arr, mid)
        IF count ≥ rank
            hi ← mid                    // mid already covers rank values
        ELSE
            lo ← mid + 1                // too few values are ≤ mid
    RETURN lo

FUNCTION MedianKSorted(arrays)
    total ← 0
    FOR EACH arr IN arrays
        total ← total + len(arr)
    IF total mod 2 = 1
        RETURN KthSmallest(arrays, floor(total / 2) + 1)
    low ← KthSmallest(arrays, floor(total / 2))
    high ← KthSmallest(arrays, floor(total / 2) + 1)
    RETURN (low + high) / 2
```

Each probe of the value search runs `k` binary searches of `O(log L)`, where `L` is the longest array, and the search halves the value range `R` every time, so the cost is `O(k log L log R)` time and `O(1)` extra space, no matter how many values sit before the middle. With few long arrays it beats the heap merge by far; for exactly two arrays, the partition search of #55 is tighter still. The same count-then-search pattern works on any sorted structure that can count the values at most `v` quickly, such as a sorted matrix, a multiplication table or the pairwise distances of a sorted array. The classic mistake is an off-by-one in the ranks: they count from 1, so the median of an odd `N` is rank `⌊N / 2⌋ + 1`, and rank `⌊N / 2⌋` returns the value just before it.

Practice: LC 4 Median of Two Sorted Arrays · LC 378 Kth Smallest Element in a Sorted Matrix · LC 719 Find K-th Smallest Pair Distance · LC 668 Kth Smallest Number in Multiplication Table

106. Random Pick with Weight

Prefix Sums + Binary Search · Binary Search · Tier 3

Given an array of positive weights, design `pick_index()`, which returns index `i` with probability `w[i] / total`, where `total` is the sum of the weights. With `w = [1, 3, 2]` the total is `6`, so index `0` should come up one time in six, index `1` three times in six and index `2` two times in six.

The brute force fills a bag with `w[i]` copies of every index and draws a uniform slot from it, which costs memory proportional to the total weight, hopeless when a single weight is a billion. The bag never needs to exist. Lay the weights end to end on a number line from `1` to `total`, so that index `i` owns a segment of exactly `w[i]` consecutive integers: index `0` owns `1`, index `1` owns `2` to `4`, index `2` owns `5` and `6`. The prefix sums `[1, 4, 6]` are the right ends of the segments, and because every weight is positive they strictly increase. A pick draws an integer target uniformly from `1` to `total` and asks which segment contains it, which is the first index whose prefix sum is at least the target. That question is the lower bound of #14 asked of the prefix sums, so building costs one pass and every pick costs one binary search.

The probability argument is a count. Each of the `total` targets is drawn with chance `1 / total`, and index `i` owns `w[i]` of them, so it is returned with chance `w[i] / total`, exactly as asked. The draw and the comparison must agree on where a segment ends: with targets `1` to `total` and the test `prefix[i] ≥ target`, the target equal to `prefix[i]` is the last one index `i` owns.

```
w = [1, 3, 2]     prefix = [1, 4, 6]     total = 6
target      1 │  2   3   4 │  5   6     RandomInt(1, 6): each has chance 1/6
owner       0 │  1   1   1 │  2   2     index i owns w[i] targets in a row
chance    1/6 │     3/6    │   2/6      prefix[i] is the last target i owns

LowerBound(5): the first i with prefix[i] ≥ 5
   lo   hi   mid   prefix[mid]   < 5?   move
    0    3    1        4         yes    lo ← 2     segments 0 and 1 end below 5
    2    3    2        6         no     hi ← 2     segment 2 reaches 5: keep it
    2    2   window empty: return 2

60000 picks after random.seed(7):   0 → 9986    1 → 30002    2 → 20012
expected 60000 × w[i] / 6:             10000        30000        20000
```

```
CLASS WeightedPicker
    FUNCTION Init(w)
        self.prefix ← empty list              // prefix[i] = w[0] + … + w[i]
        running ← 0
        FOR EACH weight IN w
            running ← running + weight
            self.prefix.append(running)
        self.total ← running

    FUNCTION LowerBound(target)               // first i with prefix[i] ≥ target
        lo ← 0
        hi ← len(self.prefix)                 // half-open [lo, hi)
        WHILE lo < hi
            mid ← floor((lo + hi) / 2)
            IF self.prefix[mid] < target
                lo ← mid + 1                  // mid's whole range lies below
```

```
            ELSE
                hi ← mid                        // mid's range reaches target
        RETURN lo

    FUNCTION PickIndex()
        target ← RandomInt(1, self.total)       // uniform over 1..total
        RETURN self.LowerBound(target)
```

Building the prefix sums costs `O(n)` time and space, and each pick costs `O(log n)` for the binary search. Reach for it whenever items must be chosen in proportion to a weight, a count or a capacity: Random Point in Non-overlapping Rectangles weights each rectangle by the number of integer points it holds and then picks a point inside the chosen one uniformly, and Random Pick with Blacklist draws a uniform rank among the allowed numbers and finds the number with that rank, again by binary search. With real-valued weights, draw `random() × total` and search for the first prefix sum strictly greater than it. When the weights change between picks, a Fenwick tree or the segment tree of #107 keeps the prefix sums current in `O(log n)` per update; when they arrive as a stream, the weighted reservoir of #131 picks in one pass without storing them. The classic mistake is a draw that disagrees with the comparison: `RandomInt(0, total − 1)` with the test ≥ hands index `0` the extra target `0` and takes target `6` away from index `2`, so `[1, 3, 2]` comes out as `2/6`, `3/6`, `1/6`, which the frequency check in the Python book catches.

Practice: LC 528 Random Pick with Weight · LC 497 Random Point in Non-overlapping Rectangles · LC 710 Random Pick with Blacklist

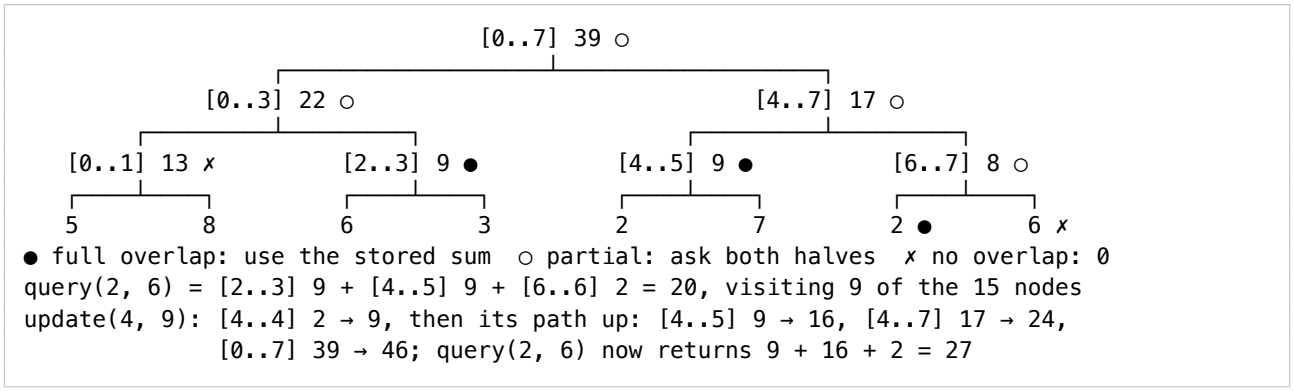
107. Segment Tree

Range Sums with Point Updates · Range Queries · Tier 3

Keep an array of numbers under two operations that arrive in any mix: `update(index, value)` sets one element, and `query(left, right)` returns the sum of `nums[left..right]`, both ends included. On `[5, 8, 6, 3, 2, 7, 2, 6]`, `query(2, 6)` is `6 + 3 + 2 + 7 + 2 = 20`; after `update(4, 9)` the same query returns 27.

The plain array updates in `O(1)` but sums a range in `O(n)`. Prefix sums flip the trade: a range sum is the difference of two prefixes, `O(1)`, but changing one element shifts every prefix after it, `O(n)`. A segment tree stores the sums of ranges at every scale, chosen so that both operations touch only `O(log n)` of them. The root owns the whole array, each node hands the halves of its segment to its two children, and each leaf owns a single element; every node stores the sum of its segment. Numbered like a heap, node `i` has the children `2i` and `2i + 1`, and an array of `4n` slots always holds them.

A query walks down from the root, and each node it meets is one of three cases. A segment entirely outside `left..right` contributes `0`. A segment entirely inside contributes its stored sum, and nothing below it is visited. A segment that overlaps partly asks both children and adds their answers. Only a segment that contains an end of the range can overlap partly, so each level holds at most two such nodes and the query visits at most four nodes per level. An update walks the one path from the root to its leaf and redoes each sum on the way back up, since those are the only segments containing the index.




```
CLASS SegmentTree
    FUNCTION Init(nums)
        self.n ← len(nums)
        self.tree ← array of 4 * self.n zeros  // 4n slots always suffice
        IF self.n > 0
            self.Build(nums, 1, 0, self.n - 1)

    FUNCTION Build(nums, node, lo, hi)
        IF lo = hi
            self.tree[node] ← nums[lo]         // a leaf owns one element
            RETURN
        mid ← floor((lo + hi) / 2)
        self.Build(nums, 2 * node, lo, mid)
        self.Build(nums, 2 * node + 1, mid + 1, hi)
        self.tree[node] ← self.tree[2 * node] + self.tree[2 * node + 1]

    FUNCTION Update(index, value)
```

```
        self.UpdateAt(1, 0, self.n - 1, index, value)

    FUNCTION UpdateAt(node, lo, hi, index, value)
        IF lo = hi
            self.tree[node] ← value
            RETURN
        mid ← floor((lo + hi) / 2)
        IF index ≤ mid                          // index lies in the left half
            self.UpdateAt(2 * node, lo, mid, index, value)
        ELSE
            self.UpdateAt(2 * node + 1, mid + 1, hi, index, value)
        self.tree[node] ← self.tree[2 * node] + self.tree[2 * node + 1]

    FUNCTION Query(left, right)
        RETURN self.QueryAt(1, 0, self.n - 1, left, right)

    FUNCTION QueryAt(node, lo, hi, left, right)
        IF right < lo OR hi < left
            RETURN 0                            // no overlap
        IF left ≤ lo AND hi ≤ right
            RETURN self.tree[node]              // full overlap: the stored sum
        mid ← floor((lo + hi) / 2)              // partial overlap: both halves
        left_sum ← self.QueryAt(2 * node, lo, mid, left, right)
        right_sum ← self.QueryAt(2 * node + 1, mid + 1, hi, left, right)
        RETURN left_sum + right_sum
```

The build fills each of the `2n − 1` nodes once, `O(n)`; `update` and `query` each cost `O(log n)`, and the tree takes `O(n)` space. Reach for a segment tree when range questions interleave with changes: sums, minimums or maximums of a changing array, or counts over ranges of values, as in Count of Smaller Numbers After Self, where the tree, indexed by value rank, counts the numbers already passed on a walk from the right. Swapping `+` for `min` or `max`, and the no-overlap `0` for ∞ or −∞, gives the minimum and maximum trees. A Fenwick tree, or binary indexed tree, answers prefix sums with point updates in about ten lines at the same `O(log n)`, but it turns prefixes into ranges by subtraction, so it does not carry over directly to minimum and maximum. Adding to a whole range at once needs lazy propagation, which parks pending updates on nodes. The classic mistake is sizing the array `2n` because the tree has `2n − 1` nodes: the heap numbering leaves gaps, and with six elements the deepest leaf is node 13, past the end of a 12-slot array.

Practice: LC 307 Range Sum Query - Mutable · LC 315 Count of Smaller Numbers After Self · LC 308 Range Sum Query 2D - Mutable · LC 303 Range Sum Query - Immutable

108. Counting Sort

Count, Prefix-Sum, Place · Sorting · Tier 3

Sort an array of integers whose values lie in a small range `[low, high]`, negatives included. `[2, −1, 3, 2, 0, −1]` becomes `[−1, −1, 0, 2, 2, 3]`.

Any sort that only compares pairs of values needs on the order of `n log n` comparisons in the worst case. Counting sort never compares: it uses each value as an array index. Count how often each value occurs in an array of `high − low + 1` slots, shifting by `low` so that the smallest value lands in slot 0 and negatives work. Then turn the counts into running totals: after the prefix sum, the slot of value v holds how many values are `≤ v`, which is exactly one past the last position v occupies in the sorted output. Finally walk the input from right to left; for each value, decrement its slot and write the value at that position.

Picture the output as a row of reserved seats. The prefix sums mark where each value's block of seats ends, and the copies of a value fill their block from the back. Because the input is read from the right, the last copy takes the last seat, so equal values keep their input order. That stability does not matter for bare integers, but it matters as soon as the integers are keys of larger records, and it is the whole reason radix sort works.

```
nums = [2, −1, 3, 2, 0, −1]; low = −1, high = 3, so slot = value + 1
  value          −1    0    1    2    3
  count           2    1    0    2    1
  prefix sums     2    3    3    5    6      how many values are ≤ value
place from the right: decrement the slot, then write at it
  i = 5   −1    2 → 1   result[1] = −1      · −1  ·  ·  ·  ·
  i = 4    0    3 → 2   result[2] =  0      · −1  0  ·  ·  ·
  i = 3    2    5 → 4   result[4] =  2      · −1  0  ·  2  ·
  i = 2    3    6 → 5   result[5] =  3      · −1  0  ·  2  3
  i = 1   −1    1 → 0   result[0] = −1     −1 −1  0  ·  2  3
  i = 0    2    4 → 3   result[3] =  2     −1 −1  0  2  2  3
the 2 from index 0 sits before the 2 from index 3: input order kept
```

```
FUNCTION CountingSort(nums)
    IF len(nums) = 0
        RETURN empty list
    low ← min(nums)
    high ← max(nums)
    count ← array of high − low + 1 zeros  // count[v − low] is for value v
    FOR EACH value IN nums
        count[value − low] ← count[value − low] + 1
    FOR slot FROM 1 TO len(count) − 1      // now: how many values are ≤ it
        count[slot] ← count[slot] + count[slot − 1]
    result ← array of len(nums) zeros
    FOR i FROM len(nums) − 1 DOWNTO 0      // right to left keeps it stable
        value ← nums[i]
        count[value − low] ← count[value − low] − 1   // the last free seat
        result[count[value − low]] ← value
    RETURN result
```

A few passes over the n values and one over the `k = high − low + 1` slots give `O(n + k)` time and `O(n + k)` space. That beats `O(n log n)` only when k is not much larger than n; values spread over a range

of a billion make the count array the bottleneck. Signals: small integer keys such as ages, grades or heights, or a tightly bounded value range in the constraints. Radix sort extends the idea to large keys: run a stable counting sort on the last digit, then on the next digit, up to the first. After the pass on digit d the numbers are sorted by their last d digits, because stability keeps the earlier order among numbers that tie on the new digit; with d digits in base b it costs `O(d(n + b))`, and it is the linear-time answer to Maximum Gap (LC 164). The classic mistake is forgetting the offset, sizing the count array `high + 1` and indexing it by the value itself: in Python a negative index does not fail, `count[−1]` silently counts into the last slot, and `[2, −1, 3, 2, 0, −1]` comes back as `[0, 2, 2, −1, 3, −1]`.

Practice: LC 912 Sort an Array · LC 1051 Height Checker · LC 274 H-Index · LC 164 Maximum Gap

109. Heap Sort

Build a Max-Heap, Swap the Max to the End · Sorting · Tier 3

Sort an array of integers in place, with `O(1)` extra space and an `O(n log n)` bound even in the worst case. `[4, 10, 3, 5, 1]` becomes `[1, 3, 4, 5, 10]`.

Selection sort finds the maximum, moves it to the end and repeats on the rest; each search is a linear scan, so it costs $O(n^2)$. Heap sort is selection sort with a max-heap doing the search. First turn the array into a max-heap in place with the bottom-up build of Heapify (#59), mirrored so that every parent is at least its children and the maximum sits at index 0. Then repeat: swap `nums[0]` with `nums[end]`, the last slot of the heap, which drops the maximum into its final place; shrink the heap to `nums[0..end-1]`; and sift the new root down to restore the heap.

Picture a wall inside the array. Left of it is a max-heap that loses one slot per step; right of it is the sorted tail, which gains one. Each swap carries the largest value still in the heap across the wall, so the tail fills from the right in decreasing order. The heap lives in the very array it sorts, which is why no extra space is needed.

```
build, i = 1 then 0:   4 10 3 5 1  →  10 5 3 4 1     max-heap, max at 0
  end   swap        after the swap     after the sift-down   (| = wall)
   4    10 and 1    1 5 3 4 | 10       5 4 3 1 | 10
   3     5 and 1    1 4 3 | 5 10       4 1 3 | 5 10
   2     4 and 3    3 1 | 4 5 10       3 1 | 4 5 10
   1     3 and 1    1 | 3 4 5 10       1 | 3 4 5 10
```

```
FUNCTION SiftDown(nums, i, n)              // max-heap on nums[0..n-1]
    WHILE true
        largest ← i
        left ← 2 * i + 1
        right ← 2 * i + 2
        IF left < n AND nums[left] > nums[largest]
            largest ← left
        IF right < n AND nums[right] > nums[largest]
            largest ← right
        IF largest = i                     // no child is larger: done
            RETURN
        swap(nums[i], nums[largest])
        i ← largest

FUNCTION HeapSort(nums)
    n ← len(nums)
    FOR i FROM floor(n / 2) - 1 DOWNTO 0   // build: last parent to root
        SiftDown(nums, i, n)
    FOR end FROM n - 1 DOWNTO 1
        swap(nums[0], nums[end])           // the max joins the sorted tail
        SiftDown(nums, 0, end)             // the heap is nums[0..end-1]
    RETURN nums
```

The build is `O(n)` and each of the n - 1 sift-downs costs `O(log n)`, so heap sort is `O(n log n)` in the worst case, with `O(1)` extra space because sift-down is a loop. It is not stable: the swap from the root to the end carries a value past others equal to it. In practice it loses to quick sort (#18), because sift-down jumps from `i` to `2i + 1` across memory and caches dislike that; its niche is the guarantee, which is

why introsort, behind most C++ `std::sort` implementations, switches to heap sort when quick sort recurses too deep. Signals: an in-place sort with a worst-case bound, or only the top of the order, since after k swaps the tail holds the k largest values in sorted order. The classic mistake is sifting with `n` instead of `end` after the swap: the sift sees the maximum just parked at `nums[end]` as a child and pulls it back into the heap, and the example comes back as `[10, 5, 3, 4, 1]`, still a heap and not sorted.

Practice: LC 912 Sort an Array · LC 215 Kth Largest Element in an Array

110. Circular Queue

Ring Buffer with Head and Count · Queues · Tier 3

Design a first-in, first-out queue of fixed capacity on a single array: `enqueue(value)` and `dequeue()` return `true` or `false` for success, `front()` and `rear()` return the oldest and the newest value or `-1` when the queue is empty, and `is_empty()` and `is_full()` report the state. With capacity 3, after enqueueing 1, 2 and 3 the queue is full and enqueueing 4 fails; after one dequeue the 4 fits, and then `front()` is 2 and `rear()` is 4.

An array queue that shifts every value left on each dequeue pays `O(n)` per dequeue, and one that only moves its start forward runs off the end of the array while the slots behind it sit empty. The ring buffer bends the array into a circle: slot `capacity - 1` is followed by slot `0`, and every index is taken modulo the capacity. Two numbers describe the whole queue: `head`, the slot of the oldest value, and `count`, how many values are stored. The values fill `count` consecutive slots of the circle starting at `head`, so the next free slot is `tail = (head + count) mod capacity` and the newest value sits just before it. Enqueue writes at the tail and increments `count`. Dequeue steps `head` forward one slot and decrements `count`, leaving the old value behind to be overwritten later. Nothing ever moves.

Storing `count` instead of a second index is the heart of this design. With only `head` and `tail`, an empty queue and a full queue look identical, `head = tail`, as the start and the third enqueue below show; the usual alternatives are exactly this count or one slot left permanently unused.

```
capacity 3       slot  0    1    2    head  count   tail
start                  ·    ·    ·       0      0      0    empty: head = tail
enqueue 1 → true       1    ·    ·       0      1      1
enqueue 2 → true       1    2    ·       0      2      2
enqueue 3 → true       1    2    3       0      3      0    full: head = tail
enqueue 4 → false      1    2    3       0      3      0    count = capacity
dequeue   → true      (1)   2    3       1      2      0    head steps on
enqueue 4 → true       4    2    3       1      3      1    4 wrapped to slot 0
front = items[1] = 2    rear = items[(1 + 3 - 1) mod 3] = items[0] = 4
· never written    (1) an old value left behind, free to overwrite
```

```
CLASS CircularQueue
    FUNCTION Init(capacity)
        self.items ← array of capacity zeros
        self.capacity ← capacity
        self.head ← 0                       // slot of the front value
        self.count ← 0                      // how many values are stored

    FUNCTION Enqueue(value)
        IF self.IsFull()
            RETURN false
        tail ← (self.head + self.count) mod self.capacity    // wraps to 0
        self.items[tail] ← value
        self.count ← self.count + 1
        RETURN true

    FUNCTION Dequeue()
        IF self.IsEmpty()
            RETURN false
        self.head ← (self.head + 1) mod self.capacity    // old value stays
```

```
        self.count ← self.count − 1
        RETURN true

    FUNCTION Front()
        IF self.IsEmpty()
            RETURN −1
        RETURN self.items[self.head]

    FUNCTION Rear()
        IF self.IsEmpty()
            RETURN −1
        RETURN self.items[(self.head + self.count − 1) mod self.capacity]

    FUNCTION IsEmpty()
        RETURN self.count = 0

    FUNCTION IsFull()
        RETURN self.count = self.capacity
```

Each operation is a constant amount of arithmetic, so all six run in `O(1)` time, and the queue takes `O(capacity)` space allocated once. The signals are a fixed-size buffer, a stream where only the last `k` items matter, such as a moving average, or a request to build a queue or deque without linked nodes. A circular deque adds insertion at the front by stepping `head` backward to `(head − 1 + capacity) mod capacity`, and Java's `ArrayDeque` is a ring buffer of the same kind that grows when it fills. The classic mistake is dropping a modulo: `tail = head + count` runs off the end of the array exactly when the ring should wrap, as in the last enqueue above, where `1 + 2 = 3` must become slot `0`.

Practice: LC 622 Design Circular Queue · LC 641 Design Circular Deque · LC 346 Moving Average from Data Stream

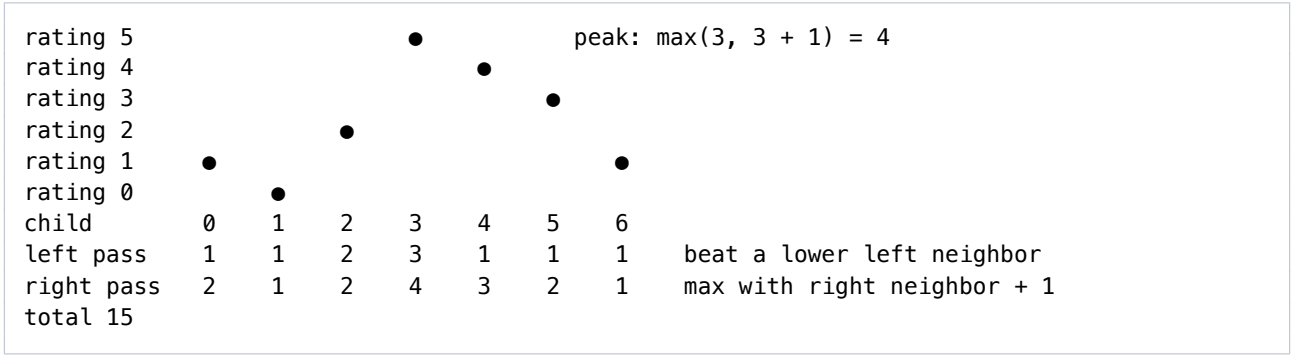
111. Candy

Two Passes: Left, Then Right · Greedy · Tier 3

Children stand in a line, each with a rating. Every child gets at least one candy, and a child rated higher than a neighbor must get more candies than that neighbor. Return the fewest candies in total. For ratings `[1, 0, 2, 5, 4, 3, 1]` the answer is 15, handed out as `[2, 1, 2, 4, 3, 2, 1]`.

Every rule looks at one side: a child against a lower neighbor on its left, or against a lower neighbor on its right. The brute force starts everyone at one candy and keeps sweeping, raising any child who breaks a rule, until a sweep changes nothing, which takes $O(n^2)$ time on a long slope. Two directed passes do it in linear time. Left to right, a child rated above its left neighbor gets one more candy than that neighbor; the neighbor was settled a step earlier, so every left rule holds after one pass. Right to left, a child rated above its right neighbor needs `candies[i + 1] + 1`, but it must keep what the left pass demanded, so it takes the larger of the two.

Why does the right pass not undo the left one? It raises `candies[i]` only when child `i` outranks its right neighbor. Raising `i` can only help the left rule at `i`, and the left rule at `i + 1` is silent, because that child is the lower one. The one rule a raise can disturb, between `i − 1` and `i`, is a right rule, repaired on the very next step of the same pass. Geometrically, read the ratings as a mountain profile: a child's count is the longer of the two strictly rising runs that climb to it, one from each side, so a peak takes the longer of its slopes. Every valid assignment needs at least those run lengths, so the sum is the minimum.




```
FUNCTION Candy(ratings)
    n ← len(ratings)
    candies ← array of n copies of 1       // everyone gets at least one
    FOR i FROM 1 TO n − 1                  // left to right
        IF ratings[i] > ratings[i − 1]     // beat a lower left neighbor
            candies[i] ← candies[i − 1] + 1
    FOR i FROM n − 2 DOWNTO 0              // right to left
        IF ratings[i] > ratings[i + 1]     // also beat a lower right neighbor
            candies[i] ← max(candies[i], candies[i + 1] + 1)
    RETURN sum(candies)
```

Two linear passes give $O(n)$ time, with $O(n)$ space for the counts. Reach for two opposite passes whenever each element's condition splits into a left half and a right half, each settled by a sweep in its own direction: the two wall arrays of Trapping Rain Water (#9), the prefix and suffix products of LC 238 and the run lengths of LC 2100 all have this shape. Equal neighbors impose nothing, so `[1, 2, 2]` needs only 4 candies, handed out as `[1, 2, 1]`. The classic mistake is a right pass that overwrites with `candies[i + 1] + 1` instead of taking the maximum: on `[1, 2, 3, 1]` it cuts the top child from 3 candies to 2, level with its lower left neighbor, and reports 6 instead of 7.

Practice: LC 135 Candy · LC 42 Trapping Rain Water · LC 238 Product of Array Except Self · LC 2100 Find Good Days to Rob the Bank

112. Hand of Straights

Start Every Group at the Smallest Card · Greedy · Tier 3

Given a hand of cards and a group size, decide whether the cards can be split into groups of `group_size` consecutive values, every card used exactly once. `[1, 2, 3, 6, 2, 3, 4, 7, 8]` with `group_size = 3` can, as `[1, 2, 3]`, `[2, 3, 4]` and `[6, 7, 8]`; `[1, 2, 4, 5, 6, 7]` cannot, because the group that `1` must start needs a `3` that is not there.

The brute force deals the cards into groups every possible way, an exponential search. One fact collapses it: the smallest card left in the hand cannot sit in the middle or at the end of a group, because nothing smaller remains to come before it, so it must start one. If it has `starts` copies, all of them start groups, and each of the next `group_size − 1` values must supply `starts` copies too. So count the cards, walk the distinct values in increasing order, and whenever a value still has copies left, peel off its `starts` groups at once; a value that runs short is a gap, and the answer is false.

Picture the counts as a histogram over the number line. A group is a horizontal strip `group_size` columns wide, and the question is whether such strips tile the histogram exactly. The leftmost column can only be covered by strips that start there, so its height says how many strips to lay down, and laying them lowers the columns to its right. When a later column is too short for the strips that must cross it, no tiling exists. Checking `len(hand) mod group_size` first rejects the hopeless hands before any counting.

```
count 2              ●   ●
count 1          ●   ●   ●   ●       ●   ●   ●
card             1   2   3   4   5   6   7   8

count            1   2   2   1   –   1   1   1
1 starts 1       0   1   1   1   –   1   1   1   takes 1, 2, 3
2 starts 1       0   0   0   0   –   1   1   1   takes 2, 3, 4
  3 and 4 left at 0, skipped
6 starts 1       0   0   0   0   –   0   0   0   takes 6, 7, 8
  7 and 8 left at 0, skipped
every count reaches 0: true ✓
```

```
FUNCTION CanFormStraights(hand, group_size)
    IF len(hand) mod group_size ≠ 0
        RETURN false
    count ← empty map
    FOR EACH card IN hand
        count[card] ← count.get(card, 0) + 1
    FOR EACH card IN sorted(count)        // distinct cards, smallest first
        starts ← count[card]              // groups that must start at card
        IF starts > 0
            FOR next_card FROM card TO card + group_size − 1
                IF count.get(next_card, 0) < starts
                    RETURN false          // a group would have a gap
                count[next_card] ← count[next_card] − starts
    RETURN true
```

Sorting the distinct values costs `O(n log n)`, while the peeling is only `O(n)`: every value that starts groups starts at least one, so at most `n / group_size` values do, and each costs `group_size` steps.

The map takes `O(n)` space. Reach for smallest-first peeling when a multiset must be split into runs of consecutive values. LC 1296 is the same problem, and Split Array into Consecutive Subsequences (LC 659) is its cousin with runs of length at least 3, where each card first extends a run that ends just below it and only otherwise opens a new one. The classic mistake is starting a group at a card that is not the smallest left: walking the hand in input order, `[2, 1, 3]` with `group_size = 3` tries `2, 3, 4` first and fails, although `[1, 2, 3]` is a valid group.

Practice: LC 846 Hand of Straights · LC 1296 Divide Array in Sets of K Consecutive Numbers · LC 659 Split Array into Consecutive Subsequences

113. N-Queens

Columns and Diagonals as Sets · Backtracking · Tier 3

Place n queens on an n × n chessboard so that no two attack each other along a row, a column or a diagonal, and return every such board as a list of strings, `Q` for a queen and `.` for an empty square. For `n = 4` there are two boards, `[".Q..", "...Q", "Q...", "..Q."]` and its mirror image `["..Q.", "Q...", "...Q", ".Q.."]`.

No two queens may share a row, so place exactly one queen per row, from top to bottom; the search only chooses a column for each row, and the row rule holds by construction. A square is safe when no earlier queen sits in its column, on its diagonal or on its anti-diagonal. Comparing against the earlier queens one by one costs `O(n)` per square; the trick is that every diagonal has a name. Along a down-right diagonal `r − c` stays constant, and along a down-left anti-diagonal `r + c` stays constant, so three sets, `cols`, `diags` and `anti_diags`, answer "is this square attacked?" with three lookups. Placing a queen adds its three names to the sets; backtracking removes them again.

The two grids below are the whole geometry of the problem. Every square carries two diagonal names, and two queens share a diagonal exactly when they share a name. Under them, the first board is built row by row, with the sets as the code holds them after each placement.

```
r − c: one name per \ diagonal    r + c: one name per / diagonal
  c= 0  1  2  3                     c= 0  1  2  3
r=0  0 −1 −2 −3                   r=0  0  1  2  3
r=1  1  0 −1 −2                   r=1  1  2  3  4
r=2  2  1  0 −1                   r=2  2  3  4  5
r=3  3  2  1  0                   r=3  3  4  5  6

n = 4: row 0 col 0 dead-ends twice, then row 0 col 1 leads to a board
. Q . .   row 0 col 1   cols {1}           r−c {−1}            r+c {1}
. . . Q   row 1 col 3   cols {1, 3}        r−c {−2, −1}        r+c {1, 4}
Q . . .   row 2 col 0   cols {0, 1, 3}     r−c {−2, −1, 2}     r+c {1, 2, 4}
. . Q .   row 3 col 2   cols {0, 1, 2, 3}  r−c {−2, −1, 1, 2}  r+c {1, 2, 4, 5}
```

```
FUNCTION Draw(queens)
    n ← len(queens)
    board ← empty list
    FOR EACH col IN queens
        board.append("." × col + "Q" + "." × (n − 1 − col))
    RETURN board

FUNCTION Backtrack(n, row, queens, cols, diags, anti_diags, boards)
    IF row = n
        boards.append(Draw(queens))          // every row holds a queen
        RETURN
    FOR col FROM 0 TO n − 1
        IF col IN cols OR (row − col) IN diags OR (row + col) IN anti_diags
            CONTINUE                         // attacked
        queens.append(col)                   // place the queen
        cols.add(col)
        diags.add(row − col)
        anti_diags.add(row + col)
        Backtrack(n, row + 1, queens, cols, diags, anti_diags, boards)
```

```
        queens.pop()                          // remove it, undo all four marks
        cols.remove(col)
        diags.remove(row - col)
        anti_diags.remove(row + col)

FUNCTION SolveNQueens(n)
    boards ← empty list
    Backtrack(n, 0, empty list, empty set, empty set, empty set, boards)
    RETURN boards
```

With the column set alone, row r has at most `n - r` free columns, so the tree has at most `n!` leaves and the search is bounded by `O(n · n!)`, plus `O(n²)` to draw each board found; the diagonal sets prune far below that bound, and for n = 8 the search makes only 2,057 calls to find all 92 boards. The extra space is `O(n)` besides the output. When only the number of boards is asked (LC 52), keep the recursion and add 1 at `row = n` instead of drawing. The same idea of constraint sets, one per row, column and 3 × 3 box, drives Sudoku Solver (LC 37), and the three sets can also be three integers used as bitmasks (#137). The classic mistake is naming both diagonal directions with `r - c`: the anti-diagonals then go unchecked, and n = 4 returns 7 boards instead of 2.

Practice: LC 51 N-Queens · LC 52 N-Queens II · LC 37 Sudoku Solver

114. Bipartite Check

BFS Two-Coloring · BFS · Tier 3

Given an undirected graph as an adjacency list, `graph[u]` holding the neighbors of `u`, decide whether its nodes can be split into two sides so that every edge runs between the sides and none runs inside one. A square is bipartite, with `{0, 2}` on one side and `{1, 3}` on the other; a triangle is not, because its three corners cannot alternate between two sides. So the graph made of a square `0`–`1`–`2`–`3` and a triangle `4`–`5`–`6`, `[[1, 3], [0, 2], [1, 3], [0, 2], [5, 6], [4, 6], [4, 5]]`, is not bipartite.

The brute force tries all `2^n` ways to assign the sides. But within a connected piece one choice forces all the others: once a node is put on side 0, each neighbor must go on side 1, and each of their neighbors on side 0. BFS spreads that forced choice layer by layer, giving every newly reached node `1 - color[node]`, the opposite of the node that reached it. An edge to a node already on the other side is consistent. An edge to a node on the same side is a contradiction that no recoloring can fix, because every color in the component was forced by the first one. The graph may be disconnected, so the outer loop starts a fresh BFS from every node still uncolored, and each component gets its own free first choice.

Picture the BFS layers as rings around the start, colored alternately. In the square the rings are `0`, then `1` and `3`, then `2`, and every edge joins two neighboring rings. In the triangle, `5` and `6` land on the same ring, and the edge between them stays inside it. That edge betrays an odd cycle, and bipartite means exactly no odd cycle: walking around any cycle switches sides at every step, so it returns home only after an even number of steps, while a same-colored edge plus the two BFS paths back to the start closes a walk of odd length, which always contains an odd cycle.

```
  0 ── 1          4
  │    │         / \
  3 ── 2        5 ─ 6

start 0: color[0] ← 0
  pop 0   1 ← 1   3 ← 1
  pop 1   0 is 0 ✓   2 ← 0
  pop 3   0 is 0 ✓   2 is 0 ✓
  pop 2   1 is 1 ✓   3 is 1 ✓          the square splits {0, 2} | {1, 3}
start 4: color[4] ← 0
  pop 4   5 ← 1   6 ← 1
  pop 5   4 is 0 ✓   6 is 1, the same as 5 ✗   return false
```

```
FUNCTION IsBipartite(graph)
    n ← len(graph)
    color ← array of n copies of −1        // −1 uncolored, else side 0 or 1
    FOR start FROM 0 TO n − 1
        IF color[start] ≠ −1
            CONTINUE                       // colored by an earlier search
        color[start] ← 0                   // new component: either side
        queue ← empty queue
        queue.enqueue(start)
        WHILE queue is not empty
            node ← queue.dequeue()
            FOR EACH neighbor IN graph[node]
                IF color[neighbor] = −1
                    color[neighbor] ← 1 − color[node]   // the other side
                    queue.enqueue(neighbor)
```

```
            ELSE IF color[neighbor] = color[node]
                RETURN false          // an edge inside one side
    RETURN true
```

Each node is colored once and each edge is examined from both ends, so the check costs `O(V + E)` time and `O(V)` space. Reach for two-coloring whenever a problem splits things into two groups under "these two must differ" constraints, such as the dislikes of Possible Bipartition, or asks whether a graph has an odd cycle. Flower Planting With No Adjacent is the many-color cousin: every garden has at most three neighbors, so choosing any flower its neighbors do not use always succeeds with four. The classic mistake is searching only from node `0`: that colors the square, never visits the triangle, and returns `true` for this graph.

Practice: LC 785 Is Graph Bipartite? · LC 886 Possible Bipartition · LC 1042 Flower Planting With No Adjacent

115. Cycle Detection in an Undirected Graph

DFS That Skips the Parent Edge · Cycle Detection · Tier 3

Given `n` nodes and a list of undirected edges with no edge repeated, decide whether the graph contains a cycle. The graph may be disconnected. With 6 nodes and edges `0–1, 2–3, 3–4, 4–5, 5–2` the answer is `true`: nodes 2, 3, 4 and 5 form a square, even though the component `0–1` is a plain tree.

A DFS marks each node the moment it discovers it and remembers the node it came from, its parent. The discovery edges form a spanning tree of each component, and a tree has no cycle, so a cycle must use some edge outside that tree. Such an edge joins two nodes that the tree already connects, and together with the tree path between them it closes a loop. So while a node scans its neighbors, there are three cases. An unmarked neighbor is discovered through a new tree edge. The parent is reached through the tree edge the search walked in on, seen again from the other end, because an undirected edge sits in both endpoints' lists. Any other neighbor that is already marked is an extra edge, and that is a cycle.

Picture unrolling a thread through the graph. Each edge out of the current node leads either to fresh ground, where the thread goes next, or to a node the thread already reaches. The edge you arrived on always leads back, so it proves nothing; any other edge to a marked node means the graph loops back on itself. An outer loop starts a fresh search from every unmarked node, because the cycle can sit in any component.

```
  0 — 1          2 — 3
                 |   |
                 5 — 4

start   pop (node, parent)   neighbors   action
  0         (0, -1)          1           discover 1
            (1, 0)           0           0 is the parent: skip
  1     already visited
  2         (2, -1)          3, 5        discover 3, discover 5
            (5, 2)           4, 2        discover 4; 2 is the parent: skip
            (4, 5)           3, 5        3 is marked and is not the parent:
                                         edge 4–3 closes the square → true
```

```
FUNCTION HasCycle(n, edges)
    graph ← array of n empty lists
    FOR EACH [u, v] IN edges
        graph[u].append(v)
        graph[v].append(u)                    // undirected: both directions
    visited ← array of n copies of false
    FOR start FROM 0 TO n − 1                 // a cycle can be in any component
        IF visited[start]
            CONTINUE
        visited[start] ← true
        stack ← empty stack
        stack.push((start, −1))               // (node, the node we came from)
        WHILE stack is not empty
            (node, parent) ← stack.pop()
            FOR EACH neighbor IN graph[node]
                IF NOT visited[neighbor]
                    visited[neighbor] ← true  // mark when discovered
```

```
                stack.push((neighbor, node))
            ELSE IF neighbor ≠ parent
                RETURN true              // reached again, not via parent
    RETURN false
```

Each node is pushed once and each edge is scanned twice, once from each end, so the check runs in `O(V + E)` time and `O(V + E)` space for the lists. The explicit stack replaces recursion, whose depth reaches `n` on a long path and overflows Python's default limit; the recursive DFS makes exactly the same test. Union-find (#41) answers the question edge by edge instead: an edge whose two ends already share a root closes a cycle, and that edge is the answer to Redundant Connection. A graph is a valid tree exactly when it has `n − 1` edges and no cycle, or equivalently `n − 1` edges and a single component. In a directed graph a marked neighbor does not prove a cycle, which is why #81 needs three colors. The classic mistake is forgetting to skip the parent, which reports a cycle on the first tree edge seen from its child; the other is searching only from node 0 and missing a cycle in another component.

Practice: LC 261 Graph Valid Tree · LC 684 Redundant Connection · LC 1559 Detect Cycles in 2D Grid

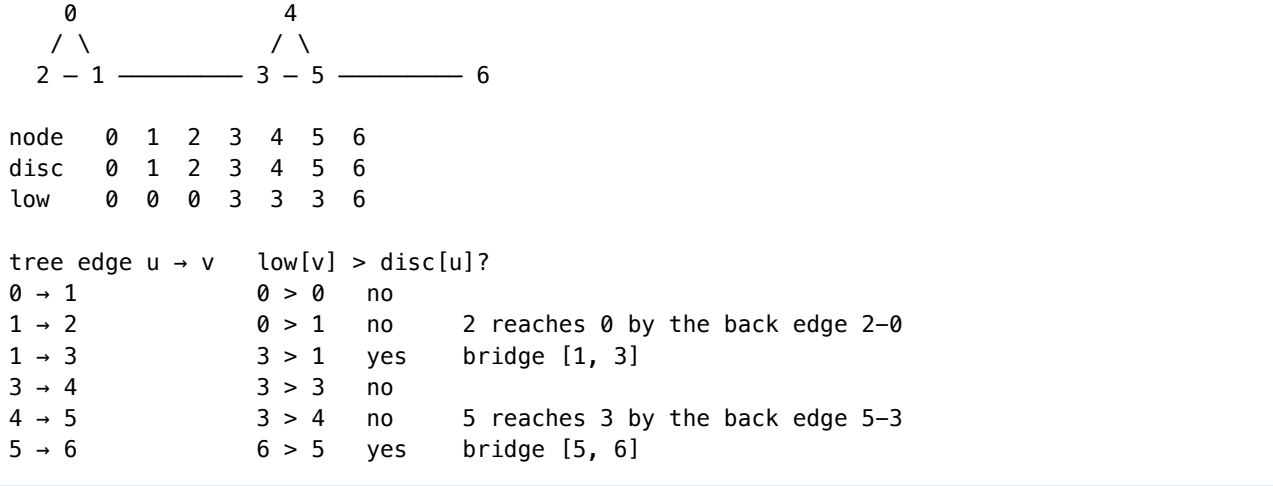
116. Critical Connections

Tarjan's Bridges: Discovery Time vs Low-Link · Graphs · Tier 3

A network has `n` nodes joined by undirected connections, none repeated. A connection is critical, a bridge, when removing it cuts some pair of nodes off from each other. Return every bridge, each pair sorted and the list sorted. Seven nodes forming two triangles, `0–1–2` and `3–4–5`, joined by the edge `1–3`, with one more edge `5–6` hanging off node 5, have the bridges `[[1, 3], [5, 6]]`.

The brute force removes each edge in turn and checks whether the graph still hangs together, `O(E · (V + E))`. One depth-first search finds every bridge at once. It stamps `disc[u]`, the time node `u` is first reached, and its tree edges form a spanning tree in which every other edge is a back edge, joining a node to one of its ancestors. A back edge lies on a cycle with the tree path between its ends, so it is never a bridge. Removing a tree edge `u → v` leaves the subtree of `v` attached exactly when some back edge leaves that subtree and lands on `u` or above it, closing a cycle through the edge. So let `low[u]` be the smallest discovery time reachable from `u`'s subtree using one back edge. The tree edge `u → v` is a bridge exactly when `low[v] > disc[u]`: nothing below `v` reaches back to `u` or higher, so cutting the edge strands the whole subtree.

Low-links are settled on the way back up. On entry `low[u]` starts at `disc[u]`; a back edge to an ancestor `v` can lower it to `disc[v]`, and each finished child `v` can lower it to its own `low[v]`. Only the edge to the parent is skipped, because it is the tree edge just walked, seen from below. The same numbers find articulation points, the nodes whose removal disconnects the graph: a non-root `u` is one when some tree child `v` has `low[v] ≥ disc[u]`, and the root is one when it has two or more tree children.




```
FUNCTION CriticalConnections(n, connections)
    graph ← array of n empty lists
    FOR EACH (u, v) IN connections
        graph[u].append(v)
        graph[v].append(u)                // undirected: both directions
    disc ← array of n copies of −1        // discovery time, −1 = not visited
    low ← array of n zeros                // lowest disc the subtree reaches
    timer ← [0]                           // next discovery time, one box
    bridges ← empty list
    FOR start FROM 0 TO n − 1             // one DFS per component
        IF disc[start] = −1
            Dfs(graph, start, −1, disc, low, timer, bridges)
```

```
    bridges.sort()                          // a deterministic order
    RETURN bridges

FUNCTION Dfs(graph, u, parent, disc, low, timer, bridges)
    disc[u] ← timer[0]
    low[u] ← timer[0]
    timer[0] ← timer[0] + 1
    FOR EACH v IN graph[u]
        IF v = parent
            CONTINUE                        // the tree edge we came down by
        IF disc[v] = -1                     // tree edge: explore v first
            Dfs(graph, v, u, disc, low, timer, bridges)
            low[u] ← min(low[u], low[v])
            IF low[v] > disc[u]             // nothing below v reaches u or above
                bridges.append([min(u, v), max(u, v)])
        ELSE
            low[u] ← min(low[u], disc[v])   // back edge to an ancestor
```

Each node is entered once and each edge is examined from both ends, so the search runs in `O(V + E)` time and space. The recursion is as deep as the longest DFS path, up to `n` frames, so a DFS path longer than about a thousand nodes passes Python's default recursion limit; inputs that large need the same search with an explicit stack of (node, next neighbor) pairs. The outer loop over start nodes handles a disconnected graph at no extra cost. Reach for low-links when a problem asks which single link, road or cable is critical, or which single node is a single point of failure, as in Minimum Number of Days to Disconnect Island, where an island of two or more cells splits after losing one cell exactly when it has an articulation point. The classic mistake is mixing up the two tests: `≥` belongs to articulation points, and using it for bridges here also reports `0–1` and `3–4`, which both lie on triangles. Forgetting to skip the parent is the other: the edge just walked then counts as a back edge, every child reaches its parent, and no bridge is ever reported.

Practice: LC 1192 Critical Connections in a Network · LC 1568 Minimum Number of Days to Disconnect Island

117. Bellman-Ford

Relax Every Edge n - 1 Times · Shortest Path · Tier 3

Given `n` nodes, directed edges with weights that may be negative, and a source, return the shortest distance from the source to every node, ∞ for nodes it cannot reach, or report that a negative cycle is reachable, since circling it forever makes "shortest" meaningless. With edges `2→3 (1)`, `1→2 (−3)`, `0→1 (4)`, `0→2 (2)` and source 0 the answer is `[0, 4, 1, 2]`: the cheapest way to 2 detours through 1 and the negative edge.

Dijkstra (#42) settles nodes greedily, nearest first, which is only safe when no edge is negative, since a negative edge found later can undercut a node already settled. Bellman-Ford gives up on order and relies on counting instead. Relaxing an edge `u → v` asks whether going through `u` beats the best known way into `v`, and takes it if so. One round relaxes every edge once, in any order. After round `k`, every node whose shortest path has at most `k` edges holds its final distance, because each round relaxes the next edge of that path at the latest. Without a negative cycle, a shortest path never needs to repeat a node, so it has at most `n − 1` edges, and `n − 1` rounds settle everything.

Picture correct distances spreading out from the source like a wave, at least one edge further every round, whatever order the edges are scanned in. A round that changes nothing means the wave has stopped, so the loop ends early. If an extra round after `n − 1` still lowers a distance, some path keeps getting cheaper as it grows longer, and only a negative cycle can do that.

```
scan order:  2→3 (1)   1→2 (−3)   0→1 (4)   0→2 (2)          source 0

          dist:  0    1    2    3     relaxed in this round
start            0    ∞    ∞    ∞
round 1          0    4    2    ∞     0→1 to 4, 0→2 to 2
round 2          0    4    1    3     2→3 to 2 + 1 = 3, then 1→2 to 4 − 3 = 1
round 3          0    4    1    2     2→3 to 1 + 1 = 2
check            no edge still improves: no negative cycle → [0, 4, 1, 2]
```

```
FUNCTION BellmanFord(n, edges, source)
    dist ← array of n copies of ∞
    dist[source] ← 0
    FOR round FROM 1 TO n − 1              // a shortest path has ≤ n − 1 edges
        changed ← false
        FOR EACH [u, v, w] IN edges
            IF dist[u] + w < dist[v]       // ∞ + w stays ∞: no false relax
                dist[v] ← dist[u] + w      // a cheaper way into v, through u
                changed ← true
        IF NOT changed
            BREAK                          // quiet round: all distances final
    FOR EACH [u, v, w] IN edges
        IF dist[u] + w < dist[v]           // still improving after n − 1 rounds
            RETURN null                    // a negative cycle is reachable
    RETURN dist
```

The work is at most `n − 1` rounds over all `E` edges, `O(V · E)` time and `O(V)` space: slower than Dijkstra, but indifferent to negative weights. Reach for it when weights can be negative, when the question is whether a negative cycle exists (an arbitrage loop in exchange rates), or when a path may use only a limited number of edges. That last case is the flights-within-k-stops problem: run exactly `k + 1` rounds,

one per flight, and relax each round from a copy of the previous round's distances, so that a round extends paths by at most one edge. Relaxing in place there lets one round chain several edges and break the stop limit; in plain Bellman-Ford the same chaining is harmless and only speeds things up. The classic mistake in plain Bellman-Ford is an integer such as `10^9` standing in for ∞: then `10^9 − 3 < 10^9`, so unreachable nodes relax their neighbors and a negative cycle the source cannot reach gets reported. Use a true infinity, or skip edges whose start is still unreached.

Practice: LC 787 Cheapest Flights Within K Stops · LC 743 Network Delay Time · LC 1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance

118. Kruskal's Minimum Spanning Tree

Sort Edges, Let Union-Find Reject Cycles · MST · Tier 3

Given `n` nodes and weighted undirected edges, choose `n − 1` edges that connect every node with the smallest possible total weight: a minimum spanning tree. Return its weight and its edges, or nothing if the graph is disconnected. With edges `0–1 (4)`, `0–2 (1)`, `1–2 (2)`, `1–3 (5)`, `2–3 (8)`, `3–4 (3)` the tree is `0–2, 1–2, 3–4, 1–3` with weight `11`.

The cut property makes greed safe. Split the nodes into any two groups: the cheapest edge crossing between them belongs to some minimum spanning tree. If a tree left it out, adding it would close a cycle, that cycle would cross the split a second time through another edge that is no cheaper, and swapping the two gives a tree that is no heavier. Kruskal applies this one edge at a time. Sort the edges by weight and scan them cheapest first. An edge whose ends already lie in the same component would close a cycle, so it is rejected. An edge that joins two different components is the cheapest edge leaving either of them, because every cheaper edge has already been scanned and either merged its ends or found them together; by the cut property it is safe, so take it.

Picture the nodes as islands and the sorted edges as bridges offered cheapest first. Union-find (#41) remembers which islands are already linked, so each offer costs two near-constant finds: the same root means reject, different roots means build the bridge and merge. Once `n − 1` bridges stand, every island is linked and the scan stops; if the offers run out first, the graph is disconnected.

```
sorted   edge   w   components before       action                 total
  1      0–2    1   {0} {1} {2} {3} {4}     take                     1
  2      1–2    2   {0,2} {1} {3} {4}       take                     3
  3      3–4    3   {0,1,2} {3} {4}         take                     6
  4      0–1    4   {0,1,2} {3,4}           skip: same component     6
  5      1–3    5   {0,1,2} {3,4}           take                    11
  6      2–3    8   never examined: n − 1 = 4 edges already taken

tree    0 –(1)– 2 –(2)– 1 –(5)– 3 –(3)– 4          total 11
```

```
FUNCTION Find(parent, x)
    IF parent[x] ≠ x
        parent[x] ← Find(parent, parent[x])   // path compression
    RETURN parent[x]

FUNCTION Union(parent, size, a, b)
    root_a ← Find(parent, a)
    root_b ← Find(parent, b)
    IF root_a = root_b
        RETURN false                          // already one component
    IF size[root_a] < size[root_b]
        swap(root_a, root_b)                  // hang the smaller tree below
    parent[root_b] ← root_a
    size[root_a] ← size[root_a] + size[root_b]
    RETURN true

FUNCTION Kruskal(n, edges)
    order ← empty list
    FOR EACH [u, v, w] IN edges
        order.append((w, u, v))               // weight first: sort by weight
```

```
    order.sort()                                // cheapest edge first
    parent ← [0, 1, …, n − 1]                   // every node is its own root
    size ← array of n copies of 1
    total ← 0
    tree ← empty list
    FOR EACH (w, u, v) IN order
        IF Union(parent, size, u, v)            // two components: edge is safe
            total ← total + w
            tree.append([u, v, w])
            IF len(tree) = n − 1
                BREAK                           // n − 1 edges span all n nodes
    IF len(tree) < n − 1
        RETURN null                             // the graph is disconnected
    RETURN (total, tree)
```

Sorting dominates: `O(E log E)` time, the same as `O(E log V)`, plus near-constant union-find work per edge and `O(V)` space beyond the edges. Reach for Kruskal when the graph arrives as an edge list, when the question is the cheapest way to connect everything (cities, points, network links), or when single edges must be forced in or kept out: finding the critical edges of a minimum spanning tree means rerunning Kruskal with one edge banned or taken first. On a dense graph given as points, the array version of Prim (#119) avoids building and sorting all `O(n²)` edges. The classic mistakes are sorting `[u, v, w]` triples as they come, which orders them by `u` instead of by weight, and returning the weight of a forest without checking that `n − 1` edges were taken.

Practice: LC 1584 Min Cost to Connect All Points · LC 1135 Connecting Cities With Minimum Cost · LC 1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

119. Prim's Minimum Spanning Tree

Grow One Tree with a Min-Heap · MST · Tier 3

Given `n` nodes and weighted undirected edges, return the total weight of a minimum spanning tree, the cheapest set of edges that connects every node, or nothing if the graph is disconnected. For the edges `0–1 (4)`, `0–2 (1)`, `1–2 (2)`, `1–3 (5)`, `2–3 (8)`, `3–4 (3)` the answer is `11`.

Kruskal (#118) grows a forest whose pieces slowly merge; Prim grows a single tree from node 0 and never lets go of it. At every moment the tree and the rest of the graph form a cut, and by the cut property the cheapest edge crossing it is safe to add. So the whole algorithm is one repeated question: what is the cheapest edge from the tree to a node outside it? A min-heap answers it. Whenever a node joins the tree, push each of its edges to an outside node as `(weight, neighbor)`. Pop the smallest entry. If its node is already in the tree, the entry is stale, left over from before a cheaper edge pulled that node in, so skip it; otherwise the node joins the tree at that cost.

Picture an ink blot spreading over the graph, always seeping along the cheapest edge on its rim. The heap is the rim: every edge that leaves the blot, plus a few old ones that now point back inside, which the in-tree check throws away as they surface. This lazy deletion spares the heap any removal from its middle. The popped costs are not sorted, either: edge `3–4 (3)` comes after `1–3 (5)`, because it only touches the rim once node 3 is in.

```
popped    action                      total   heap after the step
(0, 0)    add 0, push (4,1) (1,2)         0   (1,2) (4,1)
(1, 2)    add 2, push (2,1) (8,3)         1   (2,1) (4,1) (8,3)
(2, 1)    add 1, push (5,3)               3   (4,1) (5,3) (8,3)
(4, 1)    skip: 1 is already in           3   (5,3) (8,3)
(5, 3)    add 3, push (3,4)               8   (3,4) (8,3)
(3, 4)    add 4: all 5 nodes in          11   (8,3) is never popped

entries are (weight, node); the heap is listed smallest first
```

```
FUNCTION Prim(n, edges)
    graph ← array of n empty lists
    FOR EACH [u, v, w] IN edges
        graph[u].append((w, v))
        graph[v].append((w, u))               // undirected: both directions
    in_tree ← array of n copies of false
    heap ← empty min-heap
    heap.push((0, 0))                         // (cost to attach, node)
    total ← 0
    count ← 0
    WHILE heap is not empty AND count < n
        (cost, node) ← heap.pop()
        IF in_tree[node]
            CONTINUE                          // stale: joined by a cheaper edge
        in_tree[node] ← true                  // mark when popped, not pushed
        total ← total + cost
        count ← count + 1
        FOR EACH (weight, neighbor) IN graph[node]
            IF NOT in_tree[neighbor]
                heap.push((weight, neighbor))
    IF count < n
```

```
        RETURN null                          // some node was never reached
    RETURN total
```

Each edge enters the heap at most once, from whichever end joins the tree first, and a push or pop costs `O(log E) = O(log V)`, so Prim runs in `O(E log V)` time and `O(V + E)` space. It has the shape of Dijkstra's loop (#42), but the heap key is the weight of the single edge into the node, not a distance summed from the start, so negative weights do no harm. On a dense graph, such as points on a plane where every pair is an edge, drop the heap: keep an array `best[v]` of the cheapest edge from the tree to each outside node, pick the next node by an `O(V)` scan, refresh `best` from it in `O(V)`, and finish in $O(V^2)$ without ever listing the edges, the standard answer to Min Cost to Connect All Points. The classic mistake is marking a node as in the tree when it is pushed rather than popped, which freezes the first edge found to it instead of the cheapest.

Practice: LC 1584 Min Cost to Connect All Points · LC 1135 Connecting Cities With Minimum Cost · LC 1168 Optimize Water Distribution in a Village

120. Rabin-Karp

Rolling Hash Pattern Matching · Strings · Tier 3

Given a text and a pattern, return every index where the pattern starts in the text. In `abracadabra` the pattern `abra` starts at `0` and at `7`, so the answer is `[0, 7]`.

The brute force compares the pattern with every window of `m` characters, `O(n · m)` time in the worst case. Rabin-Karp compares numbers instead. Read a string as a number written in base 256, one digit per character, so `abra` is the four digits `97 98 114 97`; kept modulo a large prime, that number is the string's hash. Equal strings always have equal hashes, so a window whose hash differs from the pattern's is dismissed by one integer comparison. Equal hashes almost always mean equal strings, but not always, so every hash hit is confirmed by comparing the characters.

The trick that makes it fast is the roll. Sliding a window over the decimal digits of `31415926` from `31415` to `14159` means removing the leading 3, worth 30000, shifting left by multiplying by 10, and adding the 9. A window of text slides the same way in base 256: subtract the leading character times `BASE^(m − 1)`, multiply by `BASE`, add the incoming character, all modulo the prime. Each slide costs `O(1)`, however long the pattern is.

```
"abra" = base-256 digits 97 98 114 97 → pattern_hash 633841754 (MOD 10^9 + 7)

start   window   window_hash   hash hit?   compare characters
  0      abra     633841754      yes        abra = abra ✓   match at 0
  1      brac     651663196      no
  2      raca     918985050      no
  3      acad     633902941      no
  4      cada     667327066      no
  5      adab     633968475      no
  6      dabr     684103787      no
  7      abra     633841754      yes        abra = abra ✓   match at 7

roll 0 → 1:  drop 'a': (633841754 − 97 × 16777216) mod MOD = 6451809
             add 'c':  (6451809 × 256 + 99) mod MOD = 651663196
```

```
BASE ← 256                    // one digit per character code
MOD ← 1000000007              // a large prime keeps collisions rare

FUNCTION PolyHash(s)
    value ← 0
    FOR EACH ch IN s
        value ← (value * BASE + code(ch)) mod MOD   // shift left, add a digit
    RETURN value

FUNCTION RabinKarp(text, pattern)
    n ← len(text)
    m ← len(pattern)
    IF m = 0 OR m > n
        RETURN empty list
    high ← BASE^(m − 1) mod MOD               // weight of the leading digit
    pattern_hash ← PolyHash(pattern)
    window_hash ← PolyHash(text[0..m − 1])
    matches ← empty list
    FOR start FROM 0 TO n − m
```

```
        IF window_hash = pattern_hash AND text[start..start + m − 1] = pattern
            matches.append(start)             // a hash hit, confirmed
        IF start + m < n                      // roll the window one step right
            window_hash ← (window_hash − code(text[start]) * high) mod MOD
            window_hash ← (window_hash * BASE + code(text[start + m])) mod MOD
    RETURN matches
```

Hashing the pattern and the first window costs `O(m)`, each of the `n − m` rolls costs `O(1)`, and each hash hit costs `O(m)` to verify, so the expected time is `O(n + m)` when matches are few; many true matches, or a modulus small enough to collide often, push it toward `O(n · m)`. Collisions are real: with these constants `neonatus` and `plumular` share the hash 7749640, which is why the compare stays. KMP (#86) guarantees `O(n + m)` without luck, so Rabin-Karp earns its place where hashing many substrings is the point. Repeated DNA Sequences hashes every length-10 window into a set. Longest Duplicate Substring binary searches the length (#15), since a repeat of length `L` contains a repeat of length `L − 1`, and tests each length with a set of rolling hashes. The classic mistakes are trusting a hash hit without the compare and, outside Python, letting the subtraction go negative: `%` in Java or C++ keeps the sign, so add `MOD` back before reducing.

Practice: LC 28 Find the Index of the First Occurrence in a String · LC 187 Repeated DNA Sequences · LC 1044 Longest Duplicate Substring · LC 1316 Distinct Echo Substrings

121. Longest Common Prefix

Vertical Scan · Strings · Tier 3

Given a list of strings, return the longest string that is a prefix of every one of them, or the empty string if they share none. For `["flower", "flow", "flight"]` the answer is `"fl"`, and for `["dog", "racecar", "car"]` it is `""`.

Stack the strings on top of each other, left-aligned, and read them as columns. The common prefix is the run of columns, from the left, in which every row shows the same character. The vertical scan walks those columns one at a time, takes the character of the first string in the current column and checks every string against it. It stops at the first column where some string holds a different character or has already ended, and returns the first string cut just before that column; if nothing stops it, the first string is a prefix of all the others and is the answer. Because it stops at the first bad column, the scan never reads past the prefix plus one column. Folding the list pairwise instead (the prefix of the first two, then of that and the third) is linear too, but it can spend its time matching long early strings that a short later one cuts down at once.

Two other views are worth knowing. Sorting puts the lexicographically smallest and largest strings at the two ends, and the common prefix of the whole list equals the common prefix of just those two, because any string that sits between them in sorted order must agree with both on that prefix; `min` and `max` find the two ends in linear time, so the sort itself is unnecessary. A trie (#43) built from all the strings turns the answer into the path from the root that never branches and never passes the end of a word, which pays off when many queries ask about one fixed set of strings.

```
column →      0   1   2   3   4   5
"flower"      f   l   o   w   e   r
"flow"        f   l   o   w
"flight"      f   l   i   g   h   t
              ✓   ✓   ✗
                      ↑ col 2: ch = 'o' from "flower", but "flight" has 'i'
answer: first[0..1] = "fl"     9 comparisons, columns 3 to 5 never touched
```

```
FUNCTION LongestCommonPrefix(strs)
    IF len(strs) = 0
        RETURN ""
    first ← strs[0]
    FOR col FROM 0 TO len(first) − 1
        ch ← first[col]
        FOR EACH word IN strs
            IF col = len(word) OR word[col] ≠ ch    // word ended, or mismatch
                RETURN first[0..col − 1]
    RETURN first                                    // first is a prefix of all
```

The scan reads at most `n` characters in each of `L + 1` columns, where `L` is the length of the answer, so it costs `O(n × (L + 1))`, at most linear in the size of the input, with `O(1)` extra space besides the answer. The signals are a prefix shared by every string, autocomplete (#97), and grouping strings by how they start; when one fixed set is queried many times, build the trie instead. The classic mistake is reading `word[col]` before checking that `word` is long enough: the end of the shortest string is a stopping condition just like a mismatch, which is why the test `col = len(word)` comes first, and it is also what makes a list containing `""` return `""`.

Practice: LC 14 Longest Common Prefix · LC 3043 Find the Length of the Longest Common Prefix · LC 2416 Sum of Prefix Scores of Strings

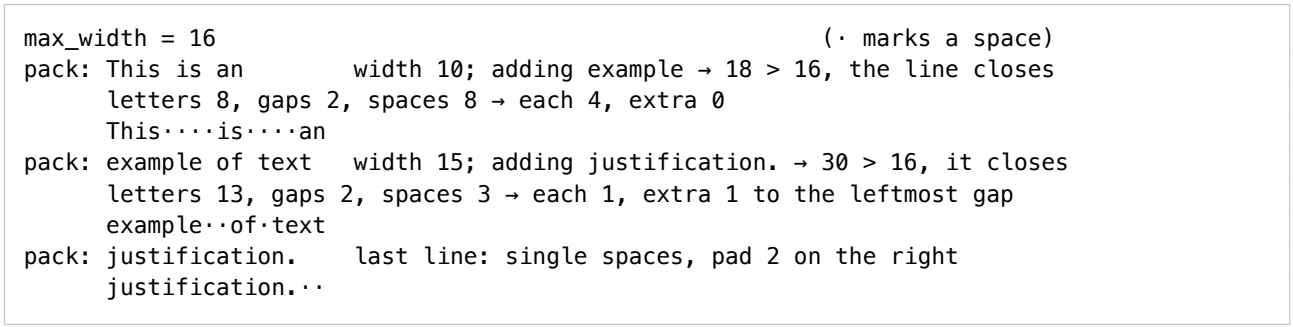
122. Text Justification

Greedy Line Packing, Then Spread the Spaces · Strings · Tier 3

Given a list of words and a width, lay the words out as lines of exactly `max_width` characters. Each line takes as many words as fit with at least one space between neighbors, and its leftover spaces are spread across the gaps as evenly as possible, the leftmost gaps taking one extra when they do not divide evenly. The last line, and any line that holds a single word, is left-justified instead: single spaces between words, then padding on the right. With width 16, the words `["This", "is", "an", "example", "of", "text", "justification."]` become `"This is an"`, `"example of text"` and `"justification. "`.

The problem splits into two independent decisions. Which words go on a line is greedy, and the statement demands it: starting from the first word not yet placed, keep taking the next word while the letters so far, plus one space per gap, plus the new word still fit. The running `width` counts exactly that, so the test is `width + 1 + len(words[j]) ≤ max_width`, where the 1 is the space the new word brings with it. How to spread the spaces is arithmetic, not search. A line with `letters` letters and `gaps` gaps must place `spaces = max_width − letters` blanks, so every gap gets `floor(spaces / gaps)` and the first `spaces mod gaps` gaps get one more. A brute force that adds one space at a time, cycling through the gaps and remeasuring the line, reaches the same layout slowly; one division does it at once.

Picture a ruler of length `max_width`. Lay the words of a line end to end and the rest of the ruler is `spaces`; cut it into `gaps` equal pieces and drop the remainder, one unit each, into the leftmost gaps. The last line and a lonely word skip the cutting: the words sit flush left and the remainder becomes one block of trailing spaces.



```
max_width = 16                                          (· marks a space)
pack: This is an        width 10; adding example → 18 > 16, the line closes
      letters 8, gaps 2, spaces 8 → each 4, extra 0
      This····is····an
pack: example of text   width 15; adding justification. → 30 > 16, it closes
      letters 13, gaps 2, spaces 3 → each 1, extra 1 to the leftmost gap
      example··of·text
pack: justification.    last line: single spaces, pad 2 on the right
      justification.··
```

```
FUNCTION FullJustify(words, max_width)
    lines ← empty list
    i ← 0
    WHILE i < len(words)
        j ← i + 1                              // the line is words[i..j−1]
        width ← len(words[i])                  // letters plus one space per gap
        WHILE j < len(words) AND width + 1 + len(words[j]) ≤ max_width
            width ← width + 1 + len(words[j])
            j ← j + 1
        is_last ← (j = len(words))
        lines.append(JustifyLine(words, i, j, max_width, is_last))
        i ← j
    RETURN lines

FUNCTION JustifyLine(words, i, j, max_width, is_last)
    IF is_last OR j − i = 1                    // left-justify, pad on the right
        line ← join(words[i..j−1], " ")
```

```
        RETURN line + " " × (max_width − len(line))
    letters ← 0
    FOR k FROM i TO j − 1
        letters ← letters + len(words[k])
    gaps ← j − i − 1
    spaces ← max_width − letters              // blanks to place in the gaps
    each ← floor(spaces / gaps)
    extra ← spaces mod gaps                   // leftmost gaps get one more
    pieces ← [words[i]]
    FOR k FROM i + 1 TO j − 1
        gap ← each
        IF k − i ≤ extra                      // gap number k − i, from 1
            gap ← gap + 1
        pieces.append(" " × gap)
        pieces.append(words[k])
    RETURN join(pieces, "")
```

Packing looks at every word a constant number of times and each line is built in `O(max_width)`, so the time is `O(n + L × max_width)` for `n` words and `L` lines, linear in the size of the output. The signals are fixed-width layout of any kind: wrapping text, padding columns, or dealing a total out as evenly as possible. That last move, `each = floor(total / parts)` with the first `total mod parts` parts taking one more, appears far beyond text. The classic mistakes are forgetting to count the separating space when testing whether the next word fits, dividing by zero gaps on a one-word line instead of treating it like the last line, and fully justifying the last line instead of padding it on the right.

Practice: LC 68 Text Justification · LC 1592 Rearrange Spaces Between Words · LC 418 Sentence Screen Fitting

123. String Compression

Run-Length Encoding with Read and Write Pointers · Strings · Tier 3

Compress an array of characters in place with run-length encoding: every maximal run of one repeated character becomes the character, followed by the length of the run when that length is more than `1`, written one digit per cell. Return the new length; whatever lies beyond it does not matter. `["a", "a", "b", "b", "c", "c", "c"]` becomes `["a", "2", "b", "2", "c", "3"]` with length `6`, and a run of twelve `b`s becomes `"b", "1", "2"`.

Building the answer in a second list is easy and costs `O(n)` extra space; the in-place version runs two pointers left to right over the same array. The read pointer finds runs: from `read` it walks forward while the character repeats, so the run is `chars[start..read − 1]` and its length is `read − start`. The write pointer lays down the encoding of each finished run, the character and then the digits of the count. Writing over the input is safe because the encoding of a run is never longer than the run itself: one cell for a single character, and `1 + d` cells for a run of length `k ≥ 2` whose count has `d` digits, where `1 + d ≤ k`. So `write` never passes `read`, and every cell it overwrites has already been read.

Picture the read pointer as a scout running ahead to the end of each run and the write pointer as a scribe following behind, compacting the ground the scout has covered. The gap between them grows only when a run of three or more shrinks into fewer cells, and the scribe never steps onto a cell the scout has not yet passed.

```
slot            0   1   2   3   4   5   6
start           a   a   b   b   c   c   c
run a × 2       a   2   b   b   c   c   c      write → 2, read → 2
run b × 2       a   2   b   2   c   c   c      write → 4, read → 4
run c × 3       a   2   b   2   c   3   c      write → 6, read → 7
                                       ↑ past the new length: ignored
return 6: the compressed text is chars[0..5] = a 2 b 2 c 3
```

```
FUNCTION Compress(chars)
    write ← 0
    read ← 0
    WHILE read < len(chars)
        ch ← chars[read]
        start ← read
        WHILE read < len(chars) AND chars[read] = ch
            read ← read + 1                  // walk to the end of the run
        chars[write] ← ch
        write ← write + 1
        count ← read − start
        IF count > 1
            FOR EACH digit IN ToString(count)     // 12 is written as "1", "2"
                chars[write] ← digit
                write ← write + 1
    RETURN write
```

Each cell is passed once by the scout and written at most once by the scribe, so the time is `O(n)`, and the only extra memory is the handful of digits of the current count. The signals are runs of equal neighbors, “in place” paired with “return the new length”, and any pair of encode and decode formats. Decoding is the mirror image: read a character, read digits while they are digits to rebuild the count (no digits

means 1), and emit the character that many times. That works only when the characters themselves are never digits; text that may contain digits needs an unambiguous frame such as the length prefix of #88. Count and Say (LC 38) applies run-length encoding, with the count first and 1s written out, to its own output again and again. The classic mistakes are writing the count into one cell (`"12"` instead of `"1"`, `"2"`), writing a `"1"` after single characters, and, in the version that flushes a run only when the character changes, forgetting to flush the final run.

Practice: LC 443 String Compression · LC 38 Count and Say · LC 271 Encode and Decode Strings

124. Reverse Words in a String

Reverse All, Then Reverse Each Word · Strings · Tier 3

Given a string, return its words in reverse order. Words are separated by one or more spaces and the string may start or end with spaces; the result separates the words by single spaces and has none at either end. `"  the sky   is blue "` becomes `"blue is sky the"`.

In Python the library answers in one line, `" ".join(reversed(s.split()))`: `split()` with no argument drops every run of spaces, so trimming and squeezing come for free. That is the answer to give first. The follow-up asks for `O(1)` extra space on a mutable character array (LC 186), and there the trick is two reversals. Reversing the whole array puts the words in the right order but spells each one backwards; reversing each word where it lies then repairs the spelling. Before that, one compaction pass with a read and a write pointer copies the characters forward, keeps only the first space after each word and drops the trailing one, so the words sit at the front separated by single spaces.

Picture the sentence as a strip of tape. Flipping the whole strip end over end carries the last word to the front, but every word now reads mirrored; flipping each word back in place fixes its letters without moving it. The outer flip reverses both the order and the letters, and the inner flips undo the letters alone.

```
start            "  the sky   is blue "     20 characters
squeeze spaces   "the sky is blue"          length 15
reverse 0..14    "eulb si yks eht"          order flipped, letters backwards
reverse 0..3     "blue si yks eht"
reverse 5..6     "blue is yks eht"
reverse 8..10    "blue is sky eht"
reverse 12..14   "blue is sky the"          every word reads forwards again
```

```
FUNCTION ReverseWords(s)
    chars ← the characters of s as an array
    length ← SqueezeSpaces(chars)         // the words fill chars[0..length−1]
    Reverse(chars, 0, length − 1)         // order flipped, letters backwards
    start ← 0
    FOR i FROM 0 TO length
        IF i = length OR chars[i] = " "   // a word ends just before i
            Reverse(chars, start, i − 1)  // spell it forwards again
            start ← i + 1
    RETURN chars[0..length − 1] as a string

FUNCTION SqueezeSpaces(chars)
    write ← 0
    FOR read FROM 0 TO len(chars) − 1
        IF chars[read] ≠ " "
            chars[write] ← chars[read]
            write ← write + 1
        ELSE IF write > 0 AND chars[write − 1] ≠ " "
            chars[write] ← " "            // keep one space after a word
            write ← write + 1
    IF write > 0 AND chars[write − 1] = " "
        write ← write − 1                 // drop the trailing space
    RETURN write

FUNCTION Reverse(chars, i, j)
```

```
    WHILE i < j
        swap(chars[i], chars[j])
        i ← i + 1
        j ← j − 1
```

Squeezing moves each character once and each of the two reversal passes swaps it at most once, so the time is `O(n)`; on a mutable array the extra space is `O(1)`, while in Python `list(s)` and the final join cost `O(n)` because strings are immutable. The signals are "reverse the order of the words" and in-place rotation: rotating an array right by `k` is the same trick, reversing the whole array and then its first `k` and remaining elements separately. Reversing the letters of each word without moving it (LC 557) is the second pass alone. The classic mistakes are forgetting the last word, which no space follows (the loop runs to `i = length` for exactly that reason), and leaving a trailing space after squeezing, which the full reversal then moves to the front.

Practice: LC 151 Reverse Words in a String · LC 186 Reverse Words in a String II · LC 557 Reverse Words in a String III · LC 189 Rotate Array

125. Multiply Strings

Grade-School Multiplication into a Digit Array · Strings · Tier 3

Multiply two non-negative integers given as decimal strings with no leading zeros and return the product as a string, without converting the inputs to numbers; they can be far longer than any machine integer. `"123"` times `"456"` is `"56088"`.

The method is the one from school, minus the partial rows written out on paper. Index both strings from the left. Digit `i` of `num1` is worth `10^(m−1−i)` and digit `j` of `num2` is worth `10^(n−1−j)`, so their product is worth `10^(m+n−2−i−j)`. A product of an `m`-digit and an `n`-digit number has at most `m + n` digits, so it fits in an array `digits` of `m + n` slots, where slot `p` is worth `10^(m+n−1−p)`; matching the two powers lands the product of digits `i` and `j` in slot `i + j + 1`. Walk both strings from the right, add each digit product into its slot, keep the ones digit there and push the tens into slot `i + j`, one to the left. The textbook alternative builds one shifted partial product per digit of `num2` and adds them up with the Add Strings loop; it makes the same digit multiplications but also allocates `n` intermediate strings and adds them one by one.

Picture the `m × n` grid of digit pairs. Every cell on one anti-diagonal, where `i + j` is constant, has the same place value, so the whole anti-diagonal pours into the same slot, and carries ripple right to left like the column sums on paper. No final carry pass is needed: every slot except the leftmost is last touched by its own `mod 10`, and the leftmost cannot overflow because the product fits in `m + n` digits.

```
 i  j   num1[i] * num2[j]   slot i+j+1   total   digits after the step
 2  2     3 * 6 = 18            5          18     [0, 0, 0, 0, 1, 8]
 2  1     3 * 5 = 15            4          16     [0, 0, 0, 1, 6, 8]
 2  0     3 * 4 = 12            3          13     [0, 0, 1, 3, 6, 8]
 1  2     2 * 6 = 12            4          18     [0, 0, 1, 4, 8, 8]
 1  1     2 * 5 = 10            3          14     [0, 0, 2, 4, 8, 8]
 1  0     2 * 4 =  8            2          10     [0, 1, 0, 4, 8, 8]
 0  2     1 * 6 =  6            3          10     [0, 1, 1, 0, 8, 8]
 0  1     1 * 5 =  5            2           6     [0, 1, 6, 0, 8, 8]
 0  0     1 * 4 =  4            1           5     [0, 5, 6, 0, 8, 8]
digits[0] is 0 (the product needs only 5 digits), so the answer is "56088"
```

```
FUNCTION Multiply(num1, num2)
    IF num1 = "0" OR num2 = "0"
        RETURN "0"
    m ← len(num1)
    n ← len(num2)
    digits ← array of m + n zeros          // the product has at most m + n
    FOR i FROM m − 1 DOWNTO 0
        FOR j FROM n − 1 DOWNTO 0
            product ← ToInt(num1[i]) * ToInt(num2[j])
            total ← digits[i + j + 1] + product
            digits[i + j + 1] ← total mod 10                    // ones digit
            digits[i + j] ← digits[i + j] + floor(total / 10)   // carry left
    start ← 0
    IF digits[0] = 0                       // only m + n − 1 digits needed
        start ← 1
    result ← empty list
    FOR k FROM start TO m + n − 1
        result.append(ToString(digits[k]))
```

```
    RETURN join(result, "")
```

The double loop does `m × n` digit multiplications, so the time is `O(m × n)` and the space is the `O(m + n)` digit array. The signals are numbers given as strings or digit arrays, "without converting to integers", or values too big for 64 bits. The same right-to-left carry loop with `total = a + b + carry` is all of Add Strings (LC 415), and Add Binary (LC 67) is that loop in base 2; Karatsuba multiplies faster, in `O(n^1.585)`, but no interview expects it. The classic mistakes are forgetting the `"0"` case, which otherwise returns a string of zeros or nothing at all, adding the carry into slot `i + j + 1` instead of `i + j`, and keeping the leading zero when the product needs only `m + n − 1` digits.

Practice: LC 43 Multiply Strings · LC 415 Add Strings · LC 67 Add Binary

126. Spiral Matrix

Walk the Boundary, Then Shrink It · Matrix · Tier 3

Given an `m × n` matrix, return all of its elements in clockwise spiral order: along the top row, down the right column, back along the bottom row, up the left column, and then the same again one ring further in. The 3 by 4 matrix with rows `1 2 3 4`, `5 6 7 8` and `9 10 11 12` reads `1 2 3 4 8 12 11 10 9 5 6 7`.

The direct simulation walks cell by cell with a direction vector and a grid of visited marks, turning right whenever the next cell is off the matrix or already read, and spends `O(m × n)` extra memory on the marks. The marks are wasted, because after every side the unread cells form a rectangle, and four numbers describe that rectangle completely: rows `top` to `bottom`, columns `left` to `right`. Each side of the walk reads one edge of the rectangle and shrinks it by one: after the top row `top` moves down, after the right column `right` moves left, after the bottom row `bottom` moves up, and after the left column `left` moves right. The loop runs while the rectangle is non-empty, peeling the matrix ring by ring from the outside in.

The first two sides of a ring need no check: the loop condition promises an unread top row, and when that row was the last one, the right column's range is empty and reads nothing. By the third side the rectangle may have collapsed. If the top row was the last unread row, `top` has just passed `bottom`, and a bottom row would read that same row again backwards; if the right column was the last unread column, the left column would repeat it upwards. The guards `top ≤ bottom` and `left ≤ right` before the last two sides matter only when the last ring is a strip: a single row, a single column, or the strip left in the middle of a wide or tall matrix. In the example the second ring is the strip `6 7`, and the guard stops the `6` from being read twice.

```
  1 →  2 →  3 →  4
               ↓
  5 →  6 →  7    8
  ↑              ↓
  9 ← 10 ← 11 ← 12

side           cells read   top  bottom  left  right   (after the side)
top row →      1 2 3 4       1     2      0     3
right col ↓    8 12          1     2      0     2
bottom row ←   11 10 9       1     1      0     2
left col ↑     5             1     1      1     2
top row →      6 7           2     1      1     2
right col ↓    none          2     1      1     1
bottom row     skipped: top 2 > bottom 1, row 1 would repeat the 6
left col ↑     none          2     1      2     1
top > bottom: the loop stops with all 12 cells read exactly once
```

```
FUNCTION SpiralOrder(matrix)
    result ← empty list
    IF len(matrix) = 0
        RETURN result
    top ← 0
    bottom ← len(matrix) − 1
    left ← 0
    right ← len(matrix[0]) − 1
```

```
    WHILE top ≤ bottom AND left ≤ right
        FOR col FROM left TO right               // top row, left to right
            result.append(matrix[top][col])
        top ← top + 1
        FOR row FROM top TO bottom               // right column, downward
            result.append(matrix[row][right])
        right ← right − 1
        IF top ≤ bottom                          // a bottom row is unread
            FOR col FROM right DOWNTO left       // bottom row, leftward
                result.append(matrix[bottom][col])
            bottom ← bottom − 1
        IF left ≤ right                          // a left column is unread
            FOR row FROM bottom DOWNTO top       // left column, upward
                result.append(matrix[row][left])
            left ← left + 1
    RETURN result
```

Every cell is read once, so the walk costs `O(m × n)` time and `O(1)` extra space beyond the output. Reach for the four shrinking bounds whenever a matrix is processed ring by ring: reading or filling in spiral order, or rotating layer by layer as in the four-way swap of #127. Spiral Matrix II runs the same walk and writes `1, 2, 3, …` instead of reading, and Spiral Matrix III walks an outward spiral from a starting cell with side lengths `1, 1, 2, 2, 3, 3, …`, keeping only the cells inside the grid. The classic mistake is dropping the two guards. Every square matrix still comes out right, which is why the bug survives casual testing, but the single row `[1, 2, 3]` comes back as `[1, 2, 3, 2, 1]`, and the 3 by 4 example gains a second `6` at the end.

Practice: LC 54 Spiral Matrix · LC 59 Spiral Matrix II · LC 885 Spiral Matrix III

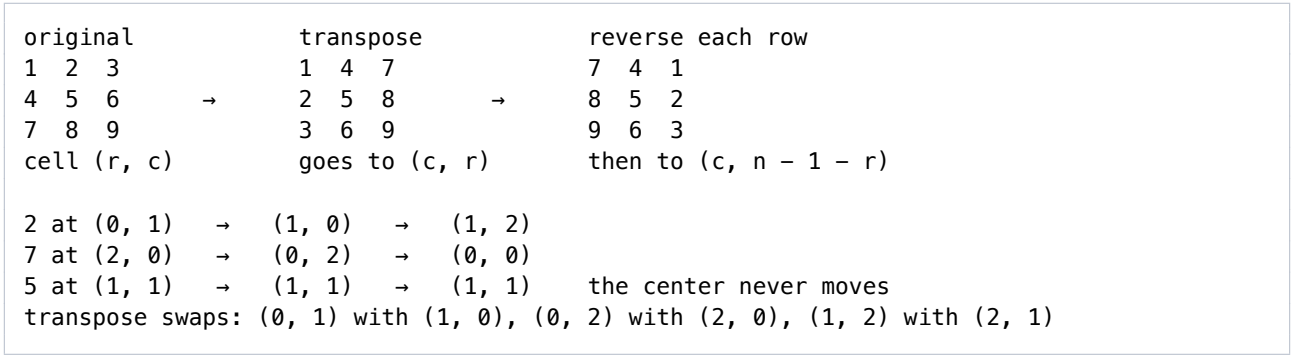
127. Rotate Image

Transpose, Then Reverse Each Row · Matrix · Tier 3

Rotate an `n × n` matrix by 90 degrees clockwise in place, without allocating a second matrix. The matrix with rows `1 2 3`, `4 5 6` and `7 8 9` becomes `7 4 1`, `8 5 2`, `9 6 3`: the first row turns into the last column, read from top to bottom.

A rotation is a rule about coordinates: the cell at row `r`, column `c` lands at row `c`, column `n − 1 − r`. Writing `rotated[c][n − 1 − r] ← matrix[r][c]` into a fresh matrix is easy and costs `O(n²)` extra space, so the in-place requirement is the whole problem. The way through is to split the rotation into two moves made only of swaps. The transpose mirrors the matrix across its main diagonal, sending `(r, c)` to `(c, r)`: every cell above the diagonal trades places with its partner below, and the diagonal stays put. Reversing each row mirrors left to right, sending `(r, c)` to `(r, n − 1 − c)`. Together they send `(r, c)` to `(c, r)` and then to `(c, n − 1 − r)`, exactly the rotation.

Picture the matrix as a square card. Flip it over its diagonal, then over its vertical midline, and it has turned a quarter turn clockwise, because two mirror images always compose into a rotation by twice the angle between the mirror lines, here twice 45 degrees. Every swap of the transpose pairs a cell above the diagonal with its mirror below, so the loop visits only the cells with `col > row`.




```
FUNCTION Rotate(matrix)                     // 90 degrees clockwise, in place
    n ← len(matrix)
    FOR row FROM 0 TO n − 1                 // transpose: mirror on the diagonal
        FOR col FROM row + 1 TO n − 1       // above the diagonal only
            swap(matrix[row][col], matrix[col][row])
    FOR row FROM 0 TO n − 1
        ReverseRow(matrix[row])             // mirror left to right

FUNCTION ReverseRow(cells)
    left ← 0
    right ← len(cells) − 1
    WHILE left < right
        swap(cells[left], cells[right])
        left ← left + 1
        right ← right − 1
```

Each pass touches every cell a constant number of times, so the rotation costs `O(n²)` time and `O(1)` extra space. Counter-clockwise sends `(r, c)` to `(n − 1 − c, r)`: reverse each row and then transpose, or transpose and then reverse the order of the rows. The alternative is the four-way cycle swap: for every cell `(r, c)` on the top side of each ring except its right-hand corner, move the four cells `(r, c) → (c,`

`n − 1 − r) → (n − 1 − r, n − 1 − c) → (n − 1 − c, r)` one step around with a single temporary. It moves each element once instead of about twice, but its index arithmetic is easy to get wrong under pressure. The signals are a grid to rotate, flip or transpose without extra memory; LC 1886 just tries the four quarter turns, and Transpose Matrix, whose matrix need not be square, must build a new array because the shape changes. The classic mistake is starting the inner loop at `0`: every pair is swapped twice, the transpose undoes itself, and the result is the mirror image `3 2 1`, `6 5 4`, `9 8 7`. Reversing the order of the rows instead of each row is the other: it turns the matrix counter-clockwise.

Practice: LC 48 Rotate Image · LC 1886 Determine Whether Matrix Can Be Obtained By Rotation · LC 867 Transpose Matrix

128. GCD and LCM

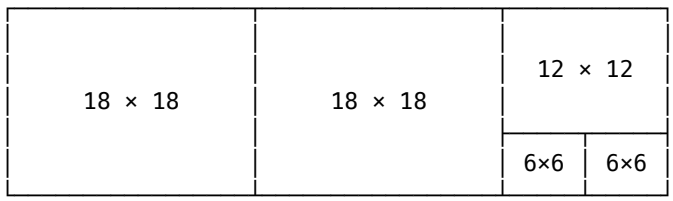
Euclid's Algorithm · Math · Tier 3

The greatest common divisor of two integers is the largest number that divides both, and the least common multiple is the smallest positive number that both divide. For `48` and `18` the gcd is `6` and the lcm is `144`.

The brute force tries every candidate from `min(a, b)` down to `1`, which costs `O(min(a, b))` divisions. Euclid's insight is that the remainder keeps every common divisor. Since `a mod b = a − q · b`, any number that divides `a` and `b` also divides `a mod b`, and any number that divides `b` and `a mod b` divides `a` back. So `gcd(a, b) = gcd(b, a mod b)`: the pair shrinks at every step while its common divisors stay exactly the same, and when the second number reaches `0` the first one is the answer, because `gcd(a, 0) = a`.

Picture an `a × b` rectangle. Taking `a mod b` cuts as many `b × b` squares as fit off its long side and leaves a thinner rectangle, and Euclid repeats the cut on the leftover until the squares tile it exactly. The side of that last square is the gcd: the largest square that tiles the original rectangle with no gap. The lcm then follows from `gcd · lcm = a · b`. Dividing before multiplying, `a / gcd · b`, keeps every intermediate value no larger than the answer, which matters in languages where `a · b` overflows. The gcd of a whole list folds the pair version across it, starting from `0` because `gcd(0, x) = x`, and stops early once it reaches `1`.

```
  a    b    a = q × b + r      cut q squares of side b off the a × b block
 48   18   48 = 2 × 18 + 12    two 18 × 18, leaving 12 × 18
 18   12   18 = 1 × 12 + 6     one 12 × 12, leaving 12 × 6
 12    6   12 = 2 × 6  + 0     two 6 × 6, nothing left
  6    0   b = 0: gcd = 6      lcm = 48 / 6 × 18 = 144
```




```
FUNCTION Gcd(a, b)
    a ← abs(a)
    b ← abs(b)
    WHILE b ≠ 0
        remainder ← a mod b            // every common divisor survives the mod
        a ← b
        b ← remainder
    RETURN a                           // gcd(a, 0) = a

FUNCTION Lcm(a, b)
    IF a = 0 OR b = 0
        RETURN 0
    RETURN (abs(a) / Gcd(a, b)) * abs(b)  // exact division first

FUNCTION GcdList(nums)
    result ← 0
```

```
    FOR EACH x IN nums
        result ← Gcd(result, x)
        IF result = 1
            BREAK                       // it can never go below 1
    RETURN result
```

Every two steps at least halve the larger number: if `b ≤ a / 2` the remainder is below `b`, and otherwise it is `a − b < a / 2`. Within two steps the larger number is at most `min(a, b)`, so Euclid runs in `O(log min(a, b))` steps, and the slowest inputs are consecutive Fibonacci numbers (#90), where almost every step cuts off a single square. Reach for the gcd when a problem asks for a common period or block size, for equal groups that must divide every count, or for a canonical key for a fraction: dividing `(dy, dx)` by their gcd is how Max Points on a Line (#134) compares slopes exactly. Strings follow the same logic: two strings are built from a common repeating block exactly when `s + t = t + s`, and then the longest such block has length `gcd(len(s), len(t))`. The classic mistake is overwriting `a` before taking the remainder, `a ← b` then `b ← a mod b`, which computes `b mod b = 0` and returns `b` after a single step.

Practice: LC 1979 Find Greatest Common Divisor of Array · LC 1071 Greatest Common Divisor of Strings · LC 914 X of a Kind in a Deck of Cards · LC 2413 Smallest Even Multiple

129. Sieve of Eratosthenes

Cross Out Multiples Starting at p squared · Math · Tier 3

List every prime below `n`, and decide whether a single number is prime. For `n = 30` the primes are `2, 3, 5, 7, 11, 13, 17, 19, 23, 29`, and `91` is not prime because `91 = 7 × 13`.

Testing each number on its own with trial division costs `O(√k)` for the number `k`, so `O(n√n)` for the whole range. The sieve turns the question around: instead of every number searching for a divisor, every prime announces its multiples. Lay out the numbers `2` to `n − 1` and walk upward. The first number that is not yet crossed out has no smaller prime factor, so it is prime, and it crosses out its multiples. Whatever survives the walk is prime.

Two bounds make the walk short. A prime `p` starts crossing at `p²`, not at `2p`: every multiple `k · p` with `2 ≤ k < p` has the smaller factor `k`, so a smaller prime has already crossed it out, which is why `5` finds `10`, `15` and `20` gone and starts at `25`. And the outer loop stops once `p² ≥ n`, because a composite below `n` always has a prime factor of at most `√n`; by then every composite is crossed out and the survivors need no more work. Trial division for a single number rests on the same square root: divisors come in pairs `d` and `n / d`, and one of the two is at most `√n`, so testing `d` while `d · d ≤ n` is enough.

```
n = 30: p runs 2, 3, 4, 5 and the loop stops at p = 6, since 6 × 6 ≥ 30
p = 2  crosses 4 6 8 10 12 14 16 18 20 22 24 26 28
p = 3  crosses 9 (12) 15 (18) 21 (24) 27       (12) = crossed again
p = 4  skipped: already crossed, its multiples are multiples of 2
p = 5  crosses 25                  10, 15, 20 fell to 2 and 3 earlier

       +0  +1  +2  +3  +4  +5  +6  +7  +8  +9     ×p = first crossed by p
   0    ·   ·   2   3  ×2   5  ×2   7  ×2  ×3      · = 0 and 1, never prime
  10   ×2  11  ×2  13  ×2  ×3  ×2  17  ×2  19
  20   ×2  ×3  ×2  23  ×2  ×5  ×2  ×3  ×2  29
survivors: 2 3 5 7 11 13 17 19 23 29, the ten primes below 30
```

```
FUNCTION Sieve(n)
    crossed ← array of n copies of false  // true once a smaller prime divides i
    p ← 2
    WHILE p * p < n
        IF NOT crossed[p]                 // p is prime: nothing crossed it out
            multiple ← p * p              // below p * p: already crossed
            WHILE multiple < n
                crossed[multiple] ← true
                multiple ← multiple + p
        p ← p + 1
    primes ← empty list
    FOR i FROM 2 TO n − 1
        IF NOT crossed[i]
            primes.append(i)
    RETURN primes

FUNCTION IsPrime(n)
    IF n < 2
        RETURN false
    d ← 2
    WHILE d * d ≤ n                       // d and n / d: one is at most √n
        IF n mod d = 0
```

```
            RETURN false
        d ← d + 1
    RETURN true
```

The sieve runs in `O(n log log n)` time, because the prime `p` crosses out at most `n / p` numbers and the sum of `1 / p` over the primes grows only like `log log n`; the array costs `O(n)` space. Trial division is `O(√n)` for one number. Reach for the sieve when a problem asks about many numbers in a range, such as counting the primes below `n` or finding the closest pair of primes in `[left, right]`, and store the smallest prime factor instead of a boolean when every value in an array must be factored. Reach for trial division when the question is about one number. The classic mistake is `d · d < n` in trial division, which declares `49` prime because its only small factor is exactly `√49 = 7`; the sieve's version of the slip is counting up to `n` inclusive when Count Primes asks for the primes strictly below `n`.

Practice: LC 204 Count Primes · LC 2523 Closest Prime Numbers in Range · LC 952 Largest Component Size by Common Factor

130. Fast Exponentiation

Square and Multiply · Math · Tier 3

Compute `x` raised to the power `n`, where `n` may be negative, and compute `x^n mod m` for a huge `n` without ever building the full number. For example `3^13 = 1594323`, `2^-2 = 0.25`, and `3^13 mod 1000 = 323`.

Multiplying `x` into the result `n` times costs `O(n)` multiplications, hopeless when `n` is in the billions. Squaring is the shortcut: `x · x = x^2`, `x^2 · x^2 = x^4`, and `k` squarings climb to `x^(2^k)`. Every exponent is a sum of powers of two, which are its binary digits, so `x^n` is the product of the rungs of that ladder where `n` has a 1 bit: `13 = 8 + 4 + 1` gives `3^13 = 3^8 · 3^4 · 3^1`. The loop reads the bits of `n` from the lowest. While `x` climbs `x, x^2, x^4, x^8, …`, the test `n mod 2 = 1` asks whether the current rung belongs in the product, and halving `n` throws away the bit just read.

A negative exponent flips the base once at the start, since `x^-n = (1 / x)^n`, and from then on the loop sees only a non-negative `n`. The modular version is the same loop with `mod m` after every product. That is legal because remainders multiply like the numbers they came from, `(a · b) mod m = ((a mod m) · (b mod m)) mod m`, and it keeps every intermediate value below `m²`: `3^1000000 mod 1000000007` takes 20 squarings of small numbers instead of building a number with 477,122 digits.

```
13 = 8 + 4 + 1 = 1101 in binary, so 3^13 = 3^1 · 3^4 · 3^8
  n   n mod 2   result after the step     x after squaring
 13      1      1 · 3 = 3                 3^2  = 9
  6      0      3          (bit 0: skip)  3^4  = 81
  3      1      3 · 81 = 243              3^8  = 6561
  1      1      243 · 6561 = 1594323      3^16 = 43046721
  0             stop: 4 squarings and 3 multiplies instead of 12 multiplies

mod 1000, the same walk on remainders:
  result 3 → 3 → 243 → 243 · 561 mod 1000 = 323     x 9 → 81 → 561 → 721
```

```
FUNCTION Power(x, n)
    IF n < 0
        x ← 1 / x                       // x^-n = (1 / x)^n
        n ← -n
    result ← 1
    WHILE n > 0
        IF n mod 2 = 1                  // lowest bit is 1: this square counts
            result ← result * x
        x ← x * x                       // x, x^2, x^4, x^8, …
        n ← floor(n / 2)                // drop the lowest bit
    RETURN result

FUNCTION ModPower(x, n, m)
    result ← 1 mod m                    // 0 when m is 1
    x ← x mod m
    WHILE n > 0
        IF n mod 2 = 1
            result ← (result * x) mod m
        x ← (x * x) mod m               // numbers stay below m * m
        n ← floor(n / 2)
    RETURN result
```

The loop runs once per bit of `n`, one squaring each time plus one multiply per 1 bit, so it costs `O(log n)` multiplications and `O(1)` space. Reach for it when an exponent is huge, when an answer is requested modulo `10^9 + 7`, or when independent choices multiply: Count Good Numbers is `5^ceil(n / 2) · 4^floor(n / 2)`, and Super Pow feeds in an exponent given as a digit array through `a^(10k + d) = (a^k)^10 · a^d`. Replacing the product of numbers by a product of matrices gives matrix powers, the source of fast Fibonacci (#90). The classic mistake is negating `n` in a language with 32-bit integers, where `-(-2^31)` overflows back to `-2^31`; widen to 64 bits before negating. Its modular twin is reducing only the result and not the squared base, which lets `x` grow without bound.

Practice: LC 50 Pow(x, n) · LC 372 Super Pow · LC 1922 Count Good Numbers

131. Reservoir Sampling

Keep the i-th Item with Probability k / i · Math · Tier 3

A stream delivers items one at a time, and its length `n` is unknown until it ends. Keep a random sample of `k` items in one pass and `O(k)` memory, so that every item ends up in the sample with the same chance `k / n`. For the stream `10, 20, 30, 40, 50` with `k = 2`, each of the five items must be kept two times in five.

Storing the whole stream and choosing at the end costs `O(n)` memory, which a stream too large to store cannot pay. The reservoir keeps the sample current instead: after `i` items it is a uniform sample of those `i`. The first `k` items fill it. From then on item `i` draws `j` uniformly from `1..i` and enters only if `j ≤ k`, which gives it probability `k / i`, exactly its fair share of the first `i`. The same draw names the slot it overwrites, and that slot is uniform over the `k` slots. A newcomer that loses the draw is simply forgotten.

The proof for `k = 1` is a telescoping product. Item `i` wins its own draw with probability `1 / i`. Each later item `m` takes the single slot with probability `1 / m`, so item `i` survives step `m` with probability `(m − 1) / m`. Its chance of being the final answer is `1/i · i/(i + 1) · (i + 1)/(i + 2) · … · (n − 1)/n`, where every numerator cancels the denominator before it, leaving `1 / n` for every `i`. For a general `k`, the newcomer at step `m` evicts one particular slot with probability `k/m · 1/k = 1/m`, so the same product starts at `k / i`, or at `1` for the first `k` items, and ends at `k / n`.

```
k = 2, stream 10 20 30 40 50, one run with random.seed(0)
  i   item   chance k/i   j = randint(1, i)   j ≤ k?          reservoir
  1    10        –               –            fill            [10]
  2    20        –               –            fill            [10, 20]
  3    30       2/3              2            ✓ into slot 2   [10, 30]
  4    40       2/4              4            ✗ forgotten     [10, 30]
  5    50       2/5              1            ✓ into slot 1   [50, 30]

item 30 lasts to the end: enters 2/3 · survives 3/4 · survives 4/5 = 2/5
20000 runs with random.seed(1), times each item was kept (expected 8000):
  10 → 8145    20 → 7898    30 → 7904    40 → 7953    50 → 8100
```

```
FUNCTION ReservoirSample(stream, k)
    reservoir ← empty list
    i ← 0
    FOR EACH item IN stream
        i ← i + 1                       // item is the i-th seen (1-based)
        IF i ≤ k
            reservoir.append(item)      // fill phase
        ELSE
            j ← RandomInt(1, i)         // uniform over 1..i
            IF j ≤ k                    // chance k / i
                reservoir[j − 1] ← item  // evict a uniform slot
    RETURN reservoir
```

Each item costs `O(1)` work, so the pass takes `O(n)` time and `O(k)` memory. Reach for it whenever a uniform choice must come from something walked once or of unknown length: Linked List Random Node is `k = 1` over the nodes, and Random Pick Index is `k = 1` over the positions holding the target, so the `c`-th match replaces the pick with probability `1 / c`. The weighted form, where each rectangle enters

with probability equal to its point count over the running total, solves Random Point in Non-overlapping Rectangles in one pass, and the same product telescopes. The classic mistake is an off-by-one in the draw: `randint(1, i − 1)` gives item `i` the chance `k / (i − 1)`, and a 0-based `randint(0, i)` gives `k / (i + 1)`, so late items come out over- or under-represented. The uniformity check in the Python book catches both slips.

Practice: LC 382 Linked List Random Node · LC 398 Random Pick Index · LC 497 Random Point in Non-overlapping Rectangles

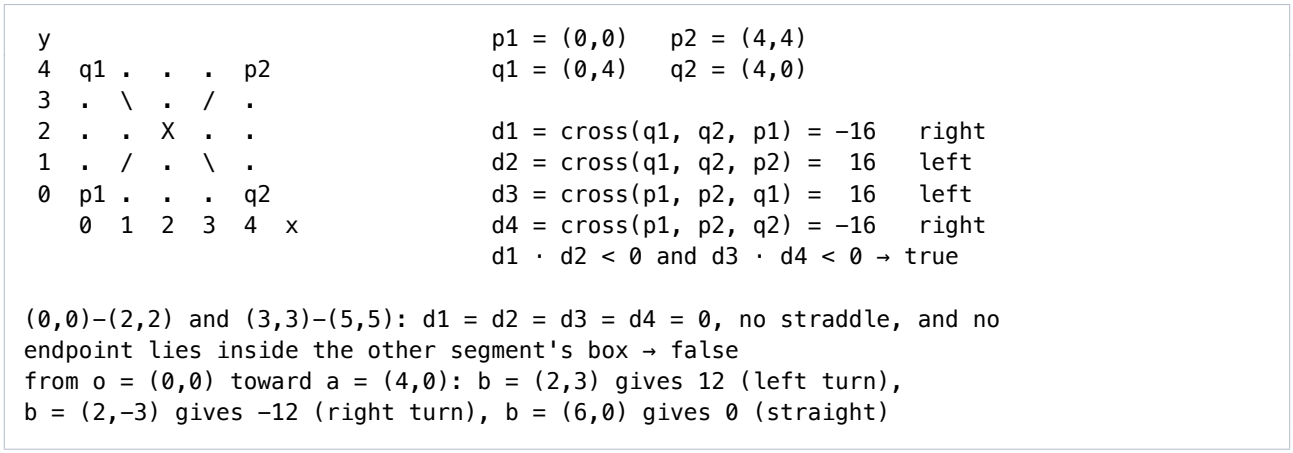
132. Orientation and Segment Intersection

The Cross Product Decides Left or Right · Geometry · Tier 3

Given three points, decide whether the path `o → a → b` turns left, turns right or runs straight; given two segments, decide whether they share a point. Coordinates are integers. The path `(0, 0) → (4, 0) → (2, 3)` turns left, the segments `(0, 0)–(4, 4)` and `(0, 4)–(4, 0)` cross at `(2, 2)`, while `(0, 0)–(2, 2)` and `(3, 3)–(5, 5)` lie on one line without touching.

Slopes bring division, a special case for vertical lines and rounding error; the cross product needs none of them. For the vectors `o → a` and `o → b`, `cross(o, a, b) = (a.x − o.x)(b.y − o.y) − (a.y − o.y)(b.x − o.x)` is the signed area of the parallelogram they span. It is positive when `b` lies to the left of the ray from `o` through `a`, a counter-clockwise turn; negative when `b` lies to the right, a clockwise turn; and zero when the three points are collinear. On integers it is exact.

Two segments cross cleanly when each straddles the other's line: `p1` and `p2` lie on opposite sides of the line `q1 q2`, and `q1` and `q2` on opposite sides of the line `p1 p2`. Opposite sides means opposite signs, so the test is `d1 · d2 < 0` and `d3 · d4 < 0`. One straddle is not enough: `(5, −1)–(5, 1)` straddles the line `y = 0` yet misses `(0, 0)–(2, 0)`. The sign test misses the touching cases, where an endpoint sits exactly on the other segment: its cross product is `0`, and it lies on the segment rather than on the line beyond it exactly when it falls inside the box spanned by the segment's ends. One such check per endpoint covers T-junctions, shared endpoints and collinear overlaps.




```
(0,0)–(2,2) and (3,3)–(5,5): d1 = d2 = d3 = d4 = 0, no straddle, and no
endpoint lies inside the other segment's box → false
from o = (0,0) toward a = (4,0): b = (2,3) gives 12 (left turn),
b = (2,−3) gives −12 (right turn), b = (6,0) gives 0 (straight)
```

```
FUNCTION Cross(o, a, b)                 // > 0 left turn, < 0 right, 0 straight
    RETURN (a.x − o.x) * (b.y − o.y) − (a.y − o.y) * (b.x − o.x)

FUNCTION OnSegment(p, q, r)             // r is on the line pq: inside the box?
    inside_x ← min(p.x, q.x) ≤ r.x ≤ max(p.x, q.x)
    inside_y ← min(p.y, q.y) ≤ r.y ≤ max(p.y, q.y)
    RETURN inside_x AND inside_y

FUNCTION SegmentsIntersect(p1, p2, q1, q2)
    d1 ← Cross(q1, q2, p1)              // side of p1 seen from q1 → q2
    d2 ← Cross(q1, q2, p2)
    d3 ← Cross(p1, p2, q1)              // side of q1 seen from p1 → p2
    d4 ← Cross(p1, p2, q2)
    IF d1 * d2 < 0 AND d3 * d4 < 0
        RETURN true                     // both pairs straddle: a clean crossing
    IF d1 = 0 AND OnSegment(q1, q2, p1)
```

```
        RETURN true                    // p1 lies on segment q1 q2
    IF d2 = 0 AND OnSegment(q1, q2, p2)
        RETURN true
    IF d3 = 0 AND OnSegment(p1, p2, q1)
        RETURN true
    IF d4 = 0 AND OnSegment(p1, p2, q2)
        RETURN true
    RETURN false
```

Each test is a handful of multiplications, so both questions cost `O(1)`. With coordinates between `-10^9` and `10^9` the cross product can reach `4 · 10^18`, which fits a signed 64-bit integer but overflows 32 bits; the product `d1 · d2` overflows even 64 bits, so outside Python compare the signs instead of multiplying. Reach for the cross product whenever a problem asks about turns, sides, collinearity or convexity: three points are collinear exactly when it is `0`, which decides Check If It Is a Straight Line and Valid Boomerang, and the convex hull (#133) is built by popping every point that does not turn left. The classic mistake is comparing slopes as floating-point numbers, which divides by zero on vertical lines and rounds the distinct slopes `10^9 / (10^9 - 1)` and `(10^9 + 1) / 10^9` to the same double; Minimum Lines to Represent a Line Chart is known for inputs that break exactly that. The second is skipping the touching checks, which calls segments that share only an endpoint disjoint.

Practice: LC 1232 Check If It Is a Straight Line · LC 1037 Valid Boomerang · LC 2280 Minimum Lines to Represent a Line Chart · LC 587 Erect the Fence

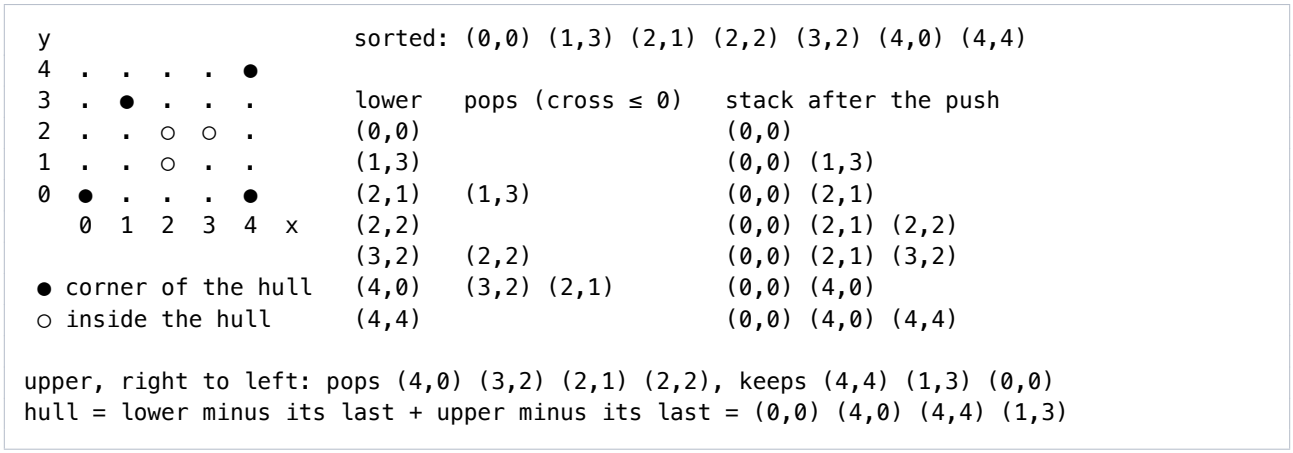
133. Convex Hull

Andrew's Monotone Chain · Geometry · Tier 3

Given points in the plane, return the corners of the smallest convex polygon that contains them all, in counter-clockwise order. Picture a rubber band snapped around nails: it bends only at the hull's corners. For the points `(0, 0), (1, 3), (2, 1), (2, 2), (3, 2), (4, 0), (4, 4)` the hull is `(0, 0), (4, 0), (4, 4), (1, 3)`, and the other three lie inside it.

The brute force tests every pair of points as a candidate edge and keeps it when all other points lie on one side, which costs `O(n^3)`. Andrew's monotone chain sorts once and then walks. Sorted by `x`, the points run from the leftmost to the rightmost, and those two extremes split the hull into a lower chain and an upper chain. Walking the lower chain from left to right, a convex boundary turns left at every corner, which is exactly what the cross product of #132 measures. So the chain is a stack: before pushing a new point, pop the top while the two top points and the new point fail to turn left, `cross ≤ 0`. A popped point sits on or above the segment that now skips it, so it cannot be a corner of the lower boundary, and it never comes back.

The upper chain is the same walk from right to left, where a convex top boundary also turns left at every corner, so the same function builds it. The two chains share their endpoints, the leftmost and the rightmost points, so dropping the last point of each before joining them lists every corner exactly once, counter-clockwise from the leftmost point, the lowest one on a tie.




```
FUNCTION Cross(o, a, b)                 // > 0 when o → a → b turns left
    RETURN (a.x − o.x) ∗ (b.y − o.y) − (a.y − o.y) ∗ (b.x − o.x)

FUNCTION HalfHull(pts)
    chain ← empty list                  // used as a stack: chain[−1] is the top
    FOR EACH p IN pts
        WHILE len(chain) ≥ 2 AND Cross(chain[−2], chain[−1], p) ≤ 0
            chain.pop()                 // last turn is not counter−clockwise
        chain.append(p)
    RETURN chain

FUNCTION ConvexHull(points)
    pts ← sorted(set(points))           // by x, then y; duplicates dropped
    IF len(pts) ≤ 2
        RETURN pts
    lower ← HalfHull(pts)
```

```
    upper ← HalfHull(reversed(pts))
    RETURN lower[0..len(lower) − 2] + upper[0..len(upper) − 2]
```

Sorting costs `O(n log n)`, and each chain is `O(n)` because every point is pushed once and popped at most once; the stacks take `O(n)` space. Reach for the hull when a problem asks for a fence or an enclosing boundary, for the extreme points of a set, or for the farthest pair, which always sits on the hull. The popping rule decides what happens to collinear points: `cross ≤ 0` drops a point in the middle of an edge, while Erect the Fence wants such points kept, so it pops only when `cross < 0` and then removes duplicates, because when all the points are collinear both chains contain every one of them. The classic mistake is joining the chains without dropping their last points, which lists the leftmost and rightmost corners twice; the second is sorting by `x` alone, which lets points that share an `x` arrive in any order: if the input `(3, 3), (3, 0), (3, 1)` arrives in that order, the chains keep the middle point `(3, 1)` and lose the corner `(3, 0)`.

Practice: LC 587 Erect the Fence · LC 812 Largest Triangle Area · LC 1232 Check If It Is a Straight Line

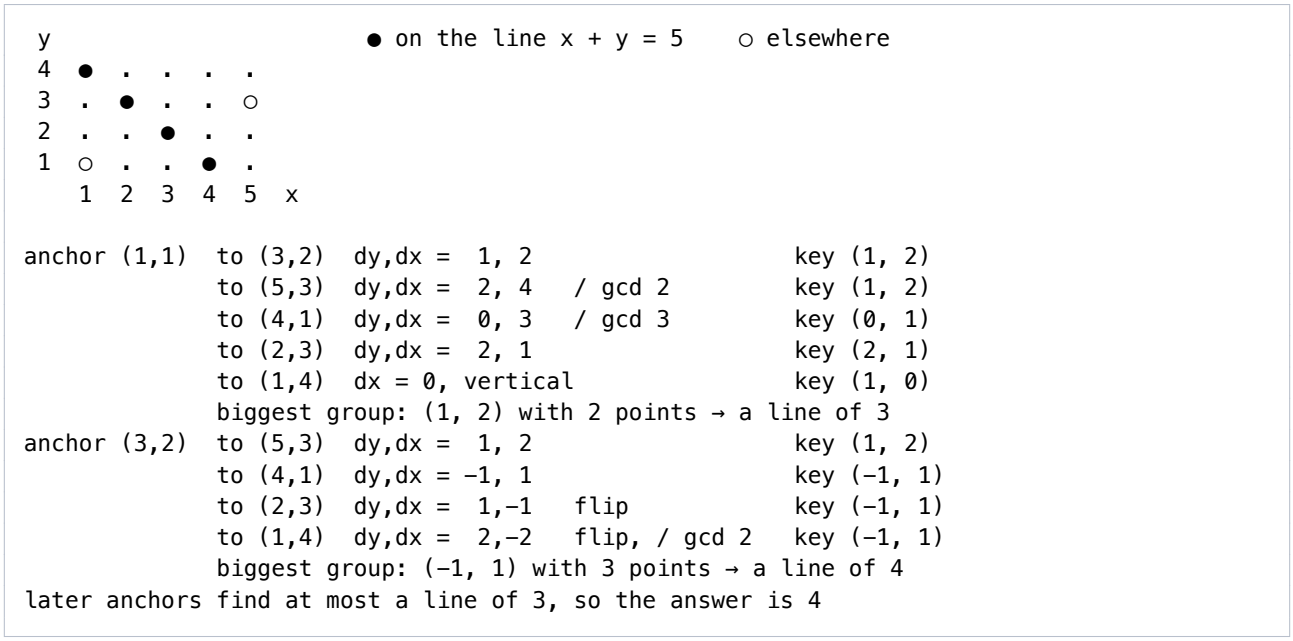
134. Max Points on a Line

Slopes as Reduced Fractions · Geometry · Tier 3

Given distinct points with integer coordinates, return the largest number of them that lie on one straight line. For `(1, 1), (3, 2), (5, 3), (4, 1), (2, 3), (1, 4)` the answer is 4, because `(1, 4), (2, 3), (3, 2), (4, 1)` all lie on the line `x + y = 5`.

The brute force takes every pair of points as a line and counts the points on it with the cross product of #132, which costs `O(n^3)`. The faster view fixes one anchor point at a time. Every line through the anchor is pinned down by its direction alone, so the other points fall into groups by their slope from the anchor, and the biggest group plus the anchor itself is the best line through that anchor. A hash map from slope to count does the grouping in one pass, and the best over all anchors is the answer. Pairing each anchor only with the points after it is enough, because every line is counted in full from its first point.

Everything rests on the slope key being exact. A floating slope `dy / dx` fails twice: vertical lines divide by zero, and distinct slopes can round to the same double, as `94911150 / 94911151` and `94911151 / 94911152` do. A reduced fraction is exact and canonical. Dividing `dy` and `dx` by their gcd (#128) turns `2/4` and `1/2` into the same key `(1, 2)`. Flipping both signs when `dx < 0` makes the two directions along a line agree, so points on either side of the anchor share a key. Every vertical pair gets the single key `(1, 0)`, and horizontal pairs come out as `(0, 1)` with no special case, because `gcd(0, dx) = dx`.




```
FUNCTION SlopeKey(p, q)
    dx ← q.x − p.x
    dy ← q.y − p.y
    IF dx = 0
        RETURN (1, 0)                  // vertical line
    IF dx < 0
        dx ← −dx                       // p → q and q → p: the same line
        dy ← −dy
    g ← Gcd(dy, dx)                    // Euclid (#128), positive since dx > 0
    RETURN (dy / g, dx / g)            // exact divisions
```

```
FUNCTION MaxPoints(points)
    n ← len(points)
    IF n ≤ 2
        RETURN n
    best ← 0
    FOR i FROM 0 TO n − 1
        counts ← empty map              // slope key → points on that line
        FOR j FROM i + 1 TO n − 1
            key ← SlopeKey(points[i], points[j])
            counts[key] ← counts.get(key, 0) + 1
            IF counts[key] + 1 > best
                best ← counts[key] + 1  // + 1 for the anchor itself
    RETURN best
```

The two loops visit `n(n − 1) / 2` pairs, each with a gcd that costs `O(log C)` for coordinates up to `C`, so the time is `O(n^2 log C)`, and one anchor's map holds fewer than `n` keys, so the space is `O(n)`. Reach for slope keys whenever points, segments or vectors must be grouped by direction: deciding whether points are collinear, merging the pieces of a line chart that share a slope, or counting the lines through a point. The classic mistake, besides the floating key, is skipping the sign normalization, which files `(1, −1)` and `(−1, 1)` under different keys although they are the same slope. Max Points on a Line promises distinct points; with duplicates, a copy of the anchor lies on every line through it and needs its own counter, added to the biggest group.

Practice: LC 149 Max Points on a Line · LC 1232 Check If It Is a Straight Line · LC 2280 Minimum Lines to Represent a Line Chart

135. Single Number

XOR Cancels Pairs · Bit Manipulation · Tier 3

Every element of a non-empty integer array appears exactly twice except one, which appears once; find that one in linear time with constant extra space. In `[4, 1, 2, 1, 2]` the answer is `4`.

Counting with a hash map works but costs `O(n)` space, and sorting to find the element without a twin costs `O(n log n)` time. XOR does it in one pass with one integer, thanks to three facts: `x XOR x = 0`, `x XOR 0 = x`, and XOR is commutative and associative, so the order of the terms does not matter. XOR the whole array together. The terms can be regrouped so that each pair sits side by side, every pair collapses to 0, and the lone element is all that remains. In Python the operator is `^`.

Geometrically, stack the numbers in binary, one per row. XOR works on each bit column separately: a column's result is 1 exactly when it holds an odd number of ones. A pair adds two ones or none to every column, so it never changes a column's parity, and the parities that remain spell out the single number.

```
               bit 2 1 0   result after the XOR
value 4            1 0 0   100
value 1            0 0 1   101
value 2            0 1 0   111
value 1            0 0 1   110
value 2            0 1 0   100
ones per column    1 2 2   odd, even, even: the single is 100 = 4

missing number in [3, 0, 1], starting from n = 3:
XOR in   i=0  v=3  i=1  v=0  i=2  v=1
result    3    0    1    1    3    2      0, 1 and 3 cancel; 2 is left
```

```
FUNCTION SingleNumber(nums)
    result ← 0
    FOR EACH value IN nums
        result ← result XOR value          // a pair cancels to 0
    RETURN result

FUNCTION MissingNumber(nums)
    result ← len(nums)                     // index n has no slot of its own
    FOR i FROM 0 TO len(nums) − 1
        result ← result XOR i              // value v cancels against index v
        result ← result XOR nums[i]
    RETURN result
```

One pass, `O(n)` time and `O(1)` space. The same cancellation finds the missing number in an array that holds `0..n` with one value absent (LC 268): XOR every index and every value, starting from n, the one index without a slot; each present value cancels against the index equal to it, wherever it sits, and the missing one has no partner. Find the Difference (LC 389) is the same trick on character codes. When two numbers are single (LC 260), the XOR of everything is `a XOR b`, which is not 0; its lowest set bit (`x & −x`, from #136) is a bit where a and b differ, so splitting the array by that bit puts them in different halves, and XOR-ing each half yields each one. XOR cancels pairs only: when the others appear three times (LC 137), count each bit modulo 3 instead. The classic mistake in Missing Number is starting from 0 instead of n: the result is then the missing value XOR n, so `[3, 0, 1]` gives `1` instead of `2`.

Practice: LC 136 Single Number · LC 268 Missing Number · LC 389 Find the Difference · LC 260 Single Number III

136. Count Set Bits

Brian Kernighan's n & (n - 1) · Bit Manipulation · Tier 3

Count the 1 bits in the binary form of a non-negative integer, its popcount or Hamming weight. `44` is `101100` in binary, so the answer is `3`.

The obvious loop tests the last bit with `n & 1` (bitwise AND) and shifts right with `n >> 1`, once per bit position: 32 rounds for a 32-bit number, however few ones it holds. Brian Kernighan's trick loops once per set bit instead. Subtracting 1 from n turns its lowest 1 into 0 and every 0 below it into 1, and leaves the higher bits alone; AND-ing that with n keeps the higher bits and wipes the flipped block. So `n & (n - 1)` is n with its lowest set bit cleared, and the number of times it can be applied before n reaches 0 is the number of ones.

Two more facts fall out of the same picture. A power of two has exactly one set bit, so n is a power of two exactly when `n > 0` and `n & (n - 1) = 0`. And `n & -n` keeps only the lowest set bit, because in two's complement `-n` is `NOT n` plus 1, which matches n at that bit and below it and disagrees with n everywhere above. For the counts of every number from 0 to n (LC 338) a one-line recurrence beats running the loop n times: `i >> 1` is i with its last bit dropped, and its count is already known, so `bits[i] = bits[i >> 1] + (i & 1)`.

```
     n       n - 1    n & (n - 1)   count
  101100    101011      101000        1      n = 44
  101000    100111      100000        2
  100000    011111      000000        3      n is 0: three set bits
n - 1 flips the lowest 1 and every 0 below it; the AND wipes that block

44 = ...00101100, -44 = ...11010100, 44 & -44 = 00000100 = 4

i               1   2   3   4   5
bits[i >> 1]    0   1   1   1   1
i & 1           1   0   1   0   1
bits[i]         1   1   2   1   2      bits for 0..5: [0, 1, 1, 2, 1, 2]
```

```
FUNCTION CountSetBits(n)
    count ← 0
    WHILE n > 0
        n ← n & (n - 1)                    // the lowest 1 becomes 0
        count ← count + 1
    RETURN count

FUNCTION IsPowerOfTwo(n)
    RETURN n > 0 AND (n & (n - 1)) = 0      // exactly one set bit

FUNCTION CountBitsUpTo(n)
    bits ← array of n + 1 zeros
    FOR i FROM 1 TO n
        bits[i] ← bits[i >> 1] + (i & 1)    // i >> 1 drops i's last bit
    RETURN bits
```

Kernighan's loop runs once per set bit, `O(k)` for k ones and never more than the number of bits, in `O(1)` space; the table for `0..n` takes `O(n)`, one step per number. Signals: popcount, Hamming weight, Hamming distance (LC 461 is the popcount of `x XOR y`), power-of-two tests, and the bitmask loops

of #137. Python integers have no fixed width, so a negative number behaves as if it had infinitely many leading ones; when a problem means the 32-bit pattern of a negative number, mask it with `n & 0xFFFFFFFF` first. The classic mistake is dropping `n > 0` from the power-of-two test: `0 & (0 − 1)` is `0 & −1 = 0`, so the bare test calls 0 a power of two.

Practice: LC 191 Number of 1 Bits · LC 231 Power of Two · LC 338 Counting Bits · LC 461 Hamming Distance

137. Bitmask Subset Enumeration

Every Integer from 0 to 2^n - 1 Is a Subset · Bit Manipulation · Tier 3

List every subset of an array of n distinct elements, this time without recursion. For `[1, 2, 3]` the eight subsets come out as `[]`, `[1]`, `[2]`, `[1, 2]`, `[3]`, `[1, 3]`, `[2, 3]` and `[1, 2, 3]`.

A subset is n yes-or-no answers, and n yes-or-no answers are an n-bit number. Let bit j of an integer `mask` say whether `nums[j]` is in. Then the integers from 0 to `2^n − 1`, written `(1 << n) − 1`, are exactly the `2^n` subsets, each once, and counting upward in binary walks the whole power set. Decoding a mask is a loop over j that tests `(mask >> j) & 1`, bit j shifted down to position 0. Compared with the backtracking tree of #30 this is flat: no recursion and no path to restore, just two loops, and each subset is an integer that can index an array or key a map.

The same view gives a second, subtler loop, over the submasks of a mask: every subset of a subset. Start from `sub = mask` and step with `sub = (sub − 1) & mask`. Subtracting 1 clears the lowest set bit of sub and sets every bit below it; the AND then throws away the bits that are not in mask, which leaves exactly the next smaller submask. The walk visits all `2^k` submasks of a mask with k set bits, each once and in decreasing order, and the empty submask, 0, is added after the loop stops.

```
nums = [1, 2, 3]    bit j of the mask says whether nums[j] is in
mask   bit 2 1 0    subset
  0        0 0 0    []
  1        0 0 1    [1]
  2        0 1 0    [2]
  3        0 1 1    [1, 2]
  4        1 0 0    [3]
  5        1 0 1    [1, 3]
  6        1 1 0    [2, 3]
  7        1 1 1    [1, 2, 3]

submasks of 13 = 1101, each step sub ← (sub − 1) & 1101:
1101 → 1100 → 1001 → 1000 → 0101 → 0100 → 0001 → 0000
e.g. 1100 − 1 = 1011, and & 1101 drops bit 1, which is not in the mask: 1001
```

```
FUNCTION SubsetsByMask(nums)
    n ← len(nums)
    result ← empty list
    FOR mask FROM 0 TO (1 << n) − 1         // 2^n masks, one per subset
        subset ← empty list
        FOR j FROM 0 TO n − 1
            IF (mask >> j) & 1 = 1          // bit j is set: nums[j] is in
                subset.append(nums[j])
        result.append(subset)
    RETURN result

FUNCTION Submasks(mask)
    result ← empty list
    sub ← mask
    WHILE sub > 0
        result.append(sub)
        sub ← (sub − 1) & mask              // borrow, keep only mask's bits
    result.append(0)                        // the empty submask ends the walk
    RETURN result
```

Listing all subsets costs `O(n · 2^n)`, `2^n` masks with n bit tests each, the same as backtracking. Walking the submasks of one mask costs `O(2^k)`, and walking them for every mask of n bits costs `O(3^n)` in total, because each element is outside the mask, in the mask but not the submask, or in both. Signals: n up to about 20, all subsets, a set used as a key, or states that are sets of chosen items. Count Number of Maximum Bitwise-OR Subsets (LC 2044) runs the plain loop; Number of Valid Words for Each Puzzle (LC 1178) turns every word into a 26-bit letter mask and enumerates the submasks of each puzzle's mask. The classic mistake is testing membership with `mask & j`, which looks at the bits of the number j, instead of `(mask >> j) & 1` or `mask & (1 << j)`: `nums[0]` is then never picked, because `mask & 0` is always 0.

Practice: LC 78 Subsets · LC 2044 Count Number of Maximum Bitwise-OR Subsets · LC 1178 Number of Valid Words for Each Puzzle

138. Rate Limiter

Token Bucket and Sliding Log · Design · Tier 3

A rate limiter decides, for each request arriving at time `t`, whether to let it through, so that a client cannot exceed an agreed rate; the two classic designs are the token bucket and the sliding log. With a budget of `3` requests per `6` seconds, four requests at time `0` and one at time `6` come out as allowed, allowed, allowed, rejected, allowed.

The token bucket holds up to `capacity` tokens and gains `rate` tokens per second; a request that finds a whole token spends it and passes, and one that does not is rejected. No timer is needed: each call first credits the tokens earned since the previous call, `(t − last) × rate`, caps the total at `capacity`, and only then decides, so the refill is lazy and the state is two numbers. A full bucket lets a burst of `capacity` requests through at once, and after that requests pass at the refill rate.

The sliding log keeps the times of the requests it allowed during the last `window` seconds in a queue, oldest at the front. A new request first drops every front time `≤ t − window`, which has left the window for good, and passes only if fewer than `limit` remain. Times arrive in order, so the expired ones always sit at the front; each is enqueued once and dequeued at most once, which makes the cost `O(1)` amortized. The log never holds more than `limit` times, so a ring buffer of that size (#110) can replace the queue.

Picture the bucket as a tank refilling at a constant trickle and the log as a frame sliding along the time axis. At the same long-run rate of half a request per second, the bucket spreads requests out after its burst, while the log refuses everything until its first three times slide out of the frame, then admits a fresh burst.

```
          token bucket: capacity 3,        sliding log: limit 3,
          rate 0.5 per second              window 6 seconds
   t      tokens after refill              log after expiry
   0      3     ✓                          []          ✓
   0      2     ✓                          [0]         ✓
   0      1     ✓                          [0, 0]      ✓
   0      0     ✗                          [0, 0, 0]   ✗
   1      0.5   ✗                          [0, 0, 0]   ✗
   2      1     ✓                          [0, 0, 0]   ✗
   4      1     ✓                          [0, 0, 0]   ✗
   6      1     ✓                          []          ✓   0 ≤ 6 − 6: expired
   6      0     ✗                          [6]         ✓
   7      0.5   ✗                          [6, 6]      ✓
allowed   0 0 0 2 4 6                      0 0 0 6 6 7
          a burst, then a steady trickle   a burst, a wall, a new burst
```

```
CLASS TokenBucket
    FUNCTION Init(capacity, rate)
        self.capacity ← capacity
        self.rate ← rate                      // tokens added per second
        self.tokens ← capacity                // start full: a burst may pass
        self.last ← 0                         // time of the last refill

    FUNCTION Allow(t)
        refill ← (t − self.last) * self.rate  // earned since the last call
        self.tokens ← min(self.capacity, self.tokens + refill)
        self.last ← t
```

```
        IF self.tokens ≥ 1
            self.tokens ← self.tokens − 1
            RETURN true
        RETURN false

CLASS SlidingLog
    FUNCTION Init(limit, window)
        self.limit ← limit
        self.window ← window
        self.log ← empty queue                // allowed times, oldest first

    FUNCTION Allow(t)
        WHILE len(self.log) > 0 AND self.log.front() ≤ t − self.window
            self.log.dequeue()                // older than the window: expired
        IF len(self.log) < self.limit
            self.log.enqueue(t)
            RETURN true
        RETURN false
```

Both limiters cost `O(1)` per request, amortized for the log; the bucket stores two numbers per client and the log up to `limit` times. The trade-off is burst against precision. The bucket is cheap and smooth in the long run but admits up to `capacity + rate × T` requests in a span of `T` seconds, so a short span can beat the nominal rate: five requests pass before time `6` above, where the log lets only three through. The log enforces at most `limit` per window exactly and pays with memory that grows with the limit; a fixed-window counter is cheaper still but lets twice the limit through across a window boundary. The signals are “at most k per window”, logger and hit-counter designs, and API throttling, where every client key gets a limiter of its own. The classic mistakes are refilling without the cap, so an idle client banks unlimited tokens, and an off-by-one at the window edge: a time `s` has expired exactly when `s ≤ t − window`.

Practice: LC 359 Logger Rate Limiter · LC 362 Design Hit Counter · LC 933 Number of Recent Calls

139. Consistent Hashing

A Hash Ring with Virtual Nodes · Design · Tier 3

A cache or storage cluster spreads keys over servers, and servers come and go. Every client must be able to compute a key's server from the key and the server list alone, and a change to the list should move as few keys as possible, because every moved key is a cache miss or a copy across the network. With 1000 keys on servers A, B and C, a fourth server D should take about a quarter of the keys, the share it will own, and nothing else should move.

The obvious placement, server number `hash(key) mod n`, fails that test badly. When `n` grows to `n + 1` a key stays put only if its hash leaves the same remainder for both, which happens for one hash in `n + 1`, so going from three servers to four moves three keys in four. Consistent hashing puts servers and keys on one circle of `2^32` positions. Each server owns points on the circle, and a key belongs to the first server point clockwise from its own position, wrapping from the top back to 0, so every point owns the arc that ends at it. A new point lands inside some arc and takes over the stretch from the previous point up to itself: the only keys that move are the ones in those stretches, and all of them move to the new server. A removed server's arcs merge into the next points clockwise, and no other key notices.

One point per server leaves the arcs badly uneven: with a single point each, the same hash splits the circle 25, 15 and 60 percent between A, B and C, and D would join with 5. Virtual nodes fix it. Each server hashes `name#0`, `name#1`, … to `v` points, so its share is a sum of `v` small arcs that evens out, and a newcomer takes a little from every server instead of a lot from one neighbor; with 100 points each the shares are 36, 32 and 32 percent. The ring is a sorted list of `(position, server)` pairs, and a lookup is the lower bound of #14 on the positions, where running off the end wraps to index 0. The hash must agree across processes, which rules out Python's `hash()` of a string, salted per run, and FNV-1a alone sends names that differ only in their last character, like `A#1` and `A#2`, to nearby points, so a finalizer scrambles its bits.

```
2 virtual nodes per server; positions as a percent of the ring, 100 = 0
sorted points:  B 3.9   C 14.5   B 29.7   A 35.2   A 54.8   C 89.9
0 ──B──────C──────────B──A─────────────A──────────────────────C──────── 100
each point owns the arc that ends at it: C 89.9 owns (54.8, 89.9]
key2 at 6.6:  the first point at or after it is C 14.5        → server C
key1 at 97.7: past the last point, wrap to index 0, B 3.9     → server B

D joins with points at 59.9 and 97.6
0 ──B──────C──────────B──A─────────────A──D──────────────────────C────D── 100
D takes (54.8, 59.9] from C and (89.9, 97.6] from B: 121 of 1000 keys move,
every one of them to D, and no other key changes server
100 virtual nodes each: 235 keys move, all to D; hash mod n moves 738
```

```
FUNCTION RingHash(text)                 // FNV-1a, then MurmurHash3's finalizer
    h ← 2166136261                      // FNV-1a offset basis
    FOR EACH ch IN text
        h ← h XOR code(ch)
        h ← (h * 16777619) mod RING     // FNV prime; RING = 2^32
    h ← h XOR (h >> 16)                 // avalanche: names one letter apart
    h ← (h * 2246822507) mod RING       // end up far apart on the ring
    h ← h XOR (h >> 13)
    h ← (h * 3266489909) mod RING
    h ← h XOR (h >> 16)
```

```
    RETURN h

CLASS HashRing
    FUNCTION Init(replicas)
        self.replicas ← replicas        // virtual nodes per server
        self.points ← empty list        // (position, server), sorted

    FUNCTION LowerBound(position)       // first point at or after position
        lo ← 0
        hi ← len(self.points)
        WHILE lo < hi
            mid ← floor((lo + hi) / 2)
            IF self.points[mid][0] < position
                lo ← mid + 1
            ELSE
                hi ← mid
        RETURN lo

    FUNCTION AddServer(server)
        FOR i FROM 0 TO self.replicas − 1
            position ← RingHash(server + "#" + str(i))
            index ← self.LowerBound(position)
            self.points.insert(index, (position, server))   // stays sorted

    FUNCTION RemoveServer(server)       // its arcs pass to the next points
        kept ← empty list
        FOR EACH point IN self.points
            IF point[1] ≠ server
                kept.append(point)
        self.points ← kept

    FUNCTION GetServer(key)
        IF len(self.points) = 0
            RETURN null
        index ← self.LowerBound(RingHash(key))
        IF index = len(self.points)
            index ← 0                   // past the last point: wrap around
        RETURN self.points[index][1]
```

A lookup hashes the key and binary searches the `n × v` points, `O(log(n × v))`. Adding a server costs `O(v × n × v)` here because each insert shifts the list, `O(v log(n × v))` with a balanced search tree, and the ring holds `O(n × v)` points. The signals are distributed caches, sharded storage, sticky load balancing and any design in which servers join and leave: Amazon's Dynamo, Cassandra and the ketama scheme of memcached clients all place each server on a ring at many points. A server with twice the capacity gets twice the virtual nodes, and replication walks clockwise to the next distinct servers to place the copies. Rendezvous hashing is the ring-free alternative: score every server against the key and take the highest, `O(n)` per lookup. No LeetCode problem asks for a ring; the practice problems drill the hashing underneath. The classic mistakes are forgetting the wrap-around, which misroutes or crashes every key past the last point, and a hash that differs between processes, like Python's `hash()`, or one that clusters, like a base-31 polynomial hash, which puts `A#0` to `A#9` at ten consecutive positions and piles a server's virtual nodes into one spot.

Practice: LC 706 Design HashMap · LC 705 Design HashSet · LC 535 Encode and Decode TinyURL

140. Stock Price Tracker

Hash Map + Heaps with Lazy Deletion · Design · Tier 3

A feed delivers stock prices as `(timestamp, price)` records, not necessarily in time order, and a later record may correct the price at an earlier timestamp. Design `update(timestamp, price)`, `current()` for the price at the latest timestamp, and `maximum()` and `minimum()` over the current prices of all timestamps. After `update(1, 10)`, `update(2, 5)` and the correction `update(1, 3)`, the current price is `5`, the maximum is `5` and the minimum is `3`: the `10` no longer exists.

A map from timestamp to price, plus the largest timestamp seen, answers `update` and `current` in `O(1)`. The extremes are the hard part. Rescanning the map costs `O(n)` per query, and a plain binary heap has no fast way to do what a correction asks for, which is to delete an arbitrary old price from its middle. The way out is not to delete it at all. Every update pushes a record `(price, timestamp)` into a max-heap and a min-heap and leaves the older records of that timestamp where they are. A record is stale when the map now holds a different price for its timestamp, and since the map is the single source of truth, telling stale from live costs one lookup. A query looks at its heap's top and pops it while it is stale; the first record that agrees with the map is the answer, because every live price has a record in each heap, so anything that outranks the true maximum must be stale.

Picture each heap as a deck in which outdated cards stay put until they surface and are thrown away. Each record is pushed once and popped at most once, so however long one query's cleanup runs, the total work over all calls is bounded by the pushes, which makes every call amortized `O(log n)`. Stale records buried below the top cost memory but never a wrong answer.

```
price@time is a record; ✗ marks one the map has since corrected
call            prices              max-heap, top first   min-heap, top first
update(1, 10)   {1: 10}             10@1                  10@1
update(2, 5)    {1: 10, 2: 5}       10@1 5@2              5@2 10@1
current() → 5   latest is 2, and prices[2] = 5
maximum() → 10  top 10@1 agrees with prices[1] = 10
update(1, 3)    {1: 3, 2: 5}        10@1✗ 5@2 3@1         3@1 5@2 10@1✗
maximum() → 5   pop 10@1✗, since prices[1] = 3; then top 5@2 agrees
                                    5@2 3@1
minimum() → 3   top 3@1 agrees; 10@1✗ stays buried in the min-heap
update(4, 2)    {1: 3, 2: 5, 4: 2}  5@2 3@1 2@4           2@4 3@1 5@2 10@1✗
current() → 2   latest is 4;  minimum() → 2 at once: top 2@4 agrees
heap sizes 3 and 4: the min-heap still holds the stale 10@1
```

```
CLASS StockPrice
    FUNCTION Init()
        self.prices ← empty map             // timestamp → its current price
        self.latest ← 0                     // the largest timestamp so far
        self.max_heap ← empty max-heap      // (price, timestamp) records
        self.min_heap ← empty min-heap      // (price, timestamp) records

    FUNCTION Update(timestamp, price)
        self.prices[timestamp] ← price      // older records of it go stale
        IF timestamp > self.latest
            self.latest ← timestamp
        self.max_heap.push((price, timestamp))
        self.min_heap.push((price, timestamp))
```

```
    FUNCTION Current()
        RETURN self.prices[self.latest]

    FUNCTION Maximum()
        WHILE true
            (price, timestamp) ← self.max_heap.top()
            IF self.prices[timestamp] = price
                RETURN price                // the record is still true
            self.max_heap.pop()             // stale: corrected since

    FUNCTION Minimum()
        WHILE true
            (price, timestamp) ← self.min_heap.top()
            IF self.prices[timestamp] = price
                RETURN price
            self.min_heap.pop()
```

`update` costs `O(log n)` for its two pushes, `current` costs `O(1)`, and `maximum` and `minimum` cost amortized `O(log n)`, where `n` counts the updates; the heaps hold one record per update, so a storm of corrections to one timestamp bloats them, and rebuilding both from the map with the `O(n)` heapify of #59 whenever they pass twice the number of live prices keeps the memory in check. The sorted-container alternative keeps the current prices in sorted order, as counts per price in Java's `TreeMap`, a balanced search tree, or as a multiset in the third-party Python `SortedList`: an update removes the old price and adds the new one, and the extremes are the two ends, `O(log n)` per call and never a stale entry. Lazy deletion is the template for tracker designs. One map holds the truth, ordered views are allowed to fall behind it, and every read checks its answer against the map: leaderboards and top-k trackers keep score maps with lazy heaps, Design a Number Container System keeps a min-heap of indices per number and discards an index whose number has changed, and the expiring cache of #94 checks every deadline it pops against its map the same way. The classic mistakes are trusting the heap top without the map check, which reports the corrected-away `10` as the maximum, and moving `latest` to every incoming timestamp, which makes `current()` return `3` after the correction `update(1, 3)` instead of `5`.

Practice: LC 2034 Stock Price Fluctuation · LC 1244 Design A Leaderboard · LC 2349 Design a Number Container System

141. All O(1) Data Structure

A Linked List of Count Buckets · Design · Tier 3

Design a counter of string keys in which every operation runs in `O(1)`. `inc(key)` adds one to the key's count, starting from 0 for a new key; `dec(key)` subtracts one, and a key whose count reaches 0 disappears; `get_max_key()` and `get_min_key()` return any key with the largest or the smallest count, or the empty string when no key is left. After `inc("hello")` twice, both queries return `hello`; after `inc("leet")`, the maximum is still `hello` and the minimum is `leet`.

A map from key to count makes `inc` and `dec` constant time but leaves each query a scan of all the keys, `O(n)`, while a heap or a balanced tree keeps the order at `O(log n)` per update. The way out is that a count only ever moves by one. Group the keys into buckets by count and keep the buckets in a doubly linked list in increasing order of count, between a head and a tail sentinel. The keys with the smallest count sit in the first bucket after the head and those with the largest in the last bucket before the tail, and a key whose count changes by one only hops to a neighboring bucket. A map from each key to its bucket finds the key's place in `O(1)`.

`inc` looks at the bucket right after the key's current one, which is the head for a new key. If that bucket holds c + 1, where c is the key's count, the key moves in; otherwise a new bucket for c + 1 is spliced in between, and the list stays sorted, because no count lies strictly between c and c + 1. `dec` mirrors this on the left with c - 1, except that a key at count 1 simply leaves the map. The bucket the key left is unlinked as soon as it is empty, so every bucket in the list holds at least one key, and the two ends of the list are always the true minimum and maximum.

```
operation    buckets, head to tail, counts rising   what changed
inc(hello)   head — [1: hello] — tail               new bucket 1 after head
inc(hello)   head — [2: hello] — tail               new 2, then 1 unlinked
inc(leet)    head — [1: leet] — [2: hello] — tail   new 1 after head
             bucket_of: leet → [1], hello → [2]
             min = head.next → leet, max = tail.prev → hello
inc(leet)    head — [2: hello, leet] — tail         2 reused, 1 unlinked
dec(hello)   head — [1: hello] — [2: leet] — tail   new 1 before 2
dec(hello)   head — [2: leet] — tail                count 0: hello is gone
dec(leet)    head — [1: leet] — tail                new 1, then 2 unlinked
dec(leet)    head — tail                            empty: both queries give ""
```

```
CLASS Bucket
    FUNCTION Init(count)
        self.count ← count
        self.keys ← empty set                  // every key with exactly count
        self.prev ← null
        self.next ← null

FUNCTION AnyKey(keys)
    FOR EACH key IN keys
        RETURN key                             // the first one met will do

CLASS AllOne
    FUNCTION Init()
        self.head ← new Bucket(0)              // sentinel before the smallest
        self.tail ← new Bucket(0)              // sentinel after the largest
        self.head.next ← self.tail
```

```
        self.tail.prev ← self.head
        self.bucket_of ← empty map              // key → the bucket holding it

    FUNCTION InsertAfter(bucket, count)
        fresh ← new Bucket(count)
        fresh.prev ← bucket
        fresh.next ← bucket.next
        bucket.next.prev ← fresh
        bucket.next ← fresh
        RETURN fresh

    FUNCTION RemoveKey(bucket, key)
        bucket.keys.remove(key)
        IF len(bucket.keys) = 0                 // empty: unlink the bucket
            bucket.prev.next ← bucket.next
            bucket.next.prev ← bucket.prev

    FUNCTION Inc(key)
        IF key IN self.bucket_of
            current ← self.bucket_of[key]
        ELSE
            current ← self.head                 // a new key comes from count 0
        target ← current.next
        IF target = self.tail OR target.count ≠ current.count + 1
            target ← self.InsertAfter(current, current.count + 1)
        target.keys.add(key)
        self.bucket_of[key] ← target
        IF current ≠ self.head
            self.RemoveKey(current, key)        // only after target is linked

    FUNCTION Dec(key)
        IF key NOT IN self.bucket_of
            RETURN
        current ← self.bucket_of[key]
        IF current.count = 1
            self.bucket_of.remove(key)          // count 0: the key disappears
        ELSE
            target ← current.prev
            IF target = self.head OR target.count ≠ current.count - 1
                target ← self.InsertAfter(current.prev, current.count - 1)
            target.keys.add(key)
            self.bucket_of[key] ← target
        self.RemoveKey(current, key)

    FUNCTION GetMaxKey()
        IF self.tail.prev = self.head
            RETURN ""
        RETURN AnyKey(self.tail.prev.keys)

    FUNCTION GetMinKey()
        IF self.head.next = self.tail
            RETURN ""
        RETURN AnyKey(self.head.next.keys)
```

Each operation is a few map and set operations plus a fixed number of pointer changes, so all four run in `O(1)`, on average because of hashing, and n keys need at most n buckets. It is the LRU cache's trick (#44) applied to counts instead of recency: a hash map gives direct access into a doubly linked list whose order is kept by local moves alone. LFU Cache (#93) keeps the same kind of frequency buckets, each holding its keys in LRU order. The signals are min and max queries over counters that change by one, such as the most and least active users. The classic mistake is the order inside `inc`: removing the key from its old bucket first can unlink that bucket before the new one is spliced in after it. From a single key at count 1, the forward walk from the head then finds no bucket at all, so `get_min_key` returns the empty string while `get_max_key` still answers. Forgetting to unlink an empty bucket is the other: the queries then read a bucket with no keys.

Practice: LC 432 All O`one Data Structure · LC 460 LFU Cache · LC 895 Maximum Frequency Stack

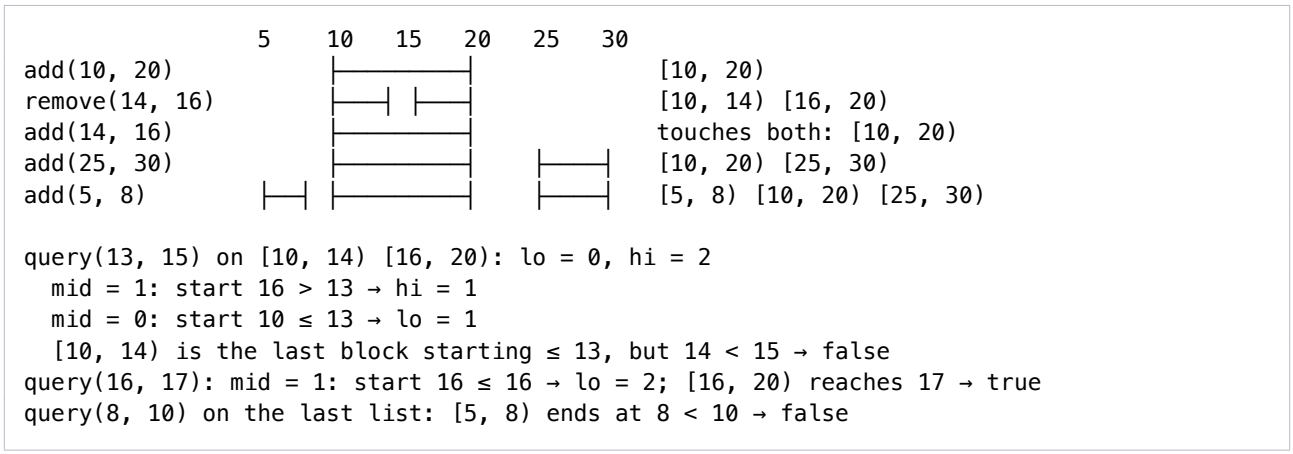
142. Range Module

A Sorted List of Disjoint Intervals · Design · Tier 3

Design a module that tracks ranges of numbers as half-open intervals `[left, right)` with `left < right`, which contain `left` but not `right`. `add_range(left, right)` starts tracking every number in the range, `remove_range(left, right)` stops tracking them, and `query_range(left, right)` asks whether every number in the range is tracked right now; the calls interleave in any order. After `add_range(10, 20)` and `remove_range(14, 16)` the module tracks `[10, 14)` and `[16, 20)`, so `query_range(10, 14)` is true, `query_range(13, 15)` is false because 14 is no longer tracked, and `query_range(16, 17)` is true.

The brute force keeps a set of every tracked number and visits each number of the range on every call, `O(right - left)` per call, hopeless when a range spans a billion values. The tracked numbers form blocks on the number line, and a block is fully described by its two ends, so store the blocks: a list of `[start, end)` pairs sorted by start, pairwise disjoint and never touching, because two blocks that touch are really one block. That last rule carries the design. Between any two stored blocks lies a gap of untracked numbers, so a range that is fully tracked sits inside a single block. A query needs only that block, the last one starting at or before `left`, and succeeds exactly when it reaches `right`. The upper-bound search of #14 finds the first block that starts after `left`, and the candidate is the block just before it; if there is none, no tracked number lies at or before `left`.

An add is Insert Interval (#65) with half-open ends. Copy the blocks that end before `left`, absorb every block that starts at or before `right` by stretching the new range to the smaller start and the larger end, then copy the rest. The tests are `end < left` and `start ≤ right`, so a block that merely touches the new range is absorbed too, as `[10, 14)` and `[16, 20)` are when `[14, 16)` is added back. A remove walks the blocks once: a block the range misses survives whole, a block it covers disappears, and a block it cuts keeps its piece left of `left`, its piece right of `right`, or both. A removal never creates touching blocks, because it leaves the gap `[left, right)` between its pieces.




```
CLASS RangeModule
    FUNCTION Init()
        self.intervals ← empty list     // sorted [start, end), gaps between

    FUNCTION AddRange(left, right)
        merged ← empty list
        i ← 0
        n ← len(self.intervals)
```

```
        WHILE i < n AND self.intervals[i][1] < left     // ends before: keep
            merged.append(self.intervals[i])
            i ← i + 1
        WHILE i < n AND self.intervals[i][0] ≤ right    // overlaps or touches
            left ← min(left, self.intervals[i][0])
            right ← max(right, self.intervals[i][1])
            i ← i + 1
        merged.append([left, right])
        WHILE i < n                                     // starts after: keep
            merged.append(self.intervals[i])
            i ← i + 1
        self.intervals ← merged

    FUNCTION RemoveRange(left, right)
        kept ← empty list
        FOR EACH [start, end] IN self.intervals
            IF end ≤ left OR start ≥ right
                kept.append([start, end])       // no overlap: untouched
            ELSE
                IF start < left
                    kept.append([start, left])  // the piece on the left
                IF end > right
                    kept.append([right, end])   // the piece on the right
        self.intervals ← kept

    FUNCTION QueryRange(left, right)
        lo ← 0
        hi ← len(self.intervals)
        WHILE lo < hi                   // first interval starting after left
            mid ← floor((lo + hi) / 2)
            IF self.intervals[mid][0] ≤ left
                lo ← mid + 1
            ELSE
                hi ← mid
        IF lo = 0
            RETURN false                // every interval starts after left
        RETURN self.intervals[lo − 1][1] ≥ right
```

Each update rebuilds the list in `O(n)` for `n` stored blocks and each query costs `O(log n)`; an add creates at most one block and a remove splits at most one, so `n` never exceeds the number of calls. Splicing a Python list shifts its tail anyway, so a binary search would not make the updates sublinear. Under heavy update traffic, a balanced search tree keyed by start (Java's `TreeMap`) finds the affected blocks in `O(log n)`, and since each block is deleted at most once after it is created, updates become amortized `O(log n)`; a segment tree (#107) extended with lazy range assignment is the other heavy-duty answer when the coordinates are known in advance. The signals are ranges added and removed over time, coverage questions on a huge number line, and free/busy or booking designs like #98. The classic mistake is letting touching blocks stay apart, for instance by testing `end ≤ left` in the first loop: adding `[14, 16)` back then copies `[10, 14)` as if it lay wholly before, the list becomes `[10, 14) [14, 20)`, and `query_range(12, 15)` fails although every number in it is tracked.

Practice: LC 715 Range Module · LC 57 Insert Interval · LC 352 Data Stream as Disjoint Intervals

143. In-Memory File System

A Trie of Path Components · Design · Tier 3

Design a file system that lives in memory and answers four calls on absolute paths such as `/a/b/c`. `mkdir(path)` creates the directory and every missing directory above it; `add_content_to_file(path, content)` creates the file if needed and appends the content; `read_content_from_file(path)` returns what the file holds; and `ls(path)` returns the sorted names inside a directory, or a one-item list with the file's own name when the path is a file. After `mkdir("/a/b/c")` and `add_content_to_file("/a/b/c/d", "hello")`, `ls("/")` is `["a"]`, `ls("/a/b/c/d")` is `["d"]`, and reading `/a/b/c/d` gives `"hello"`.

The brute force is a flat map from full path to content. Creating and reading are single lookups, but `ls` must scan every stored path for the ones exactly one level below, `O(P · L)` for `P` paths of length `L`. A directory tree is already a trie, the trie of #43 with whole names on the edges instead of letters. Each node holds a map from child name to child node, an `is_file` flag and the file's content, and the children of a directory node are exactly its listing, so `ls` reads them off and sorts them. Every call splits the path on `/` and walks down from the root one name at a time. Splitting `/a/b/c/d` gives `['', 'a', 'b', 'c', 'd']`; the empty string before the leading slash is skipped, as is any left by a trailing or doubled slash, which makes `/` itself the empty walk that stays at the root.

One walk serves all four calls, with a single switch. `mkdir` and `add_content_to_file` walk with `create` on, so every missing name becomes a fresh node on the way down and `mkdir("/a/b/c")` builds the whole chain in one pass. `ls` and `read_content_from_file` walk with `create` off and stop at the first missing name, so asking about a path never brings it into existence. A file is a node like any other with its flag set. When `ls` lands on one, it answers with the last name of the path, because a node does not store its own name; its parent's map does.

```
mkdir("/a/b/c"), add "hello" then " world" to /a/b/c/d,
mkdir("/a/b/apps"), add "todo" to /a/notes:

root
└── a
    ├── b
    │   ├── c
    │   │   └── d        file: "hello world"
    │   └── apps
    └── notes            file: "todo"

ls("/a/b")       [a, b]: root → a → b, a directory → sorted children
                 stored in insertion order c, apps → ['apps', 'c']
ls("/a/b/c/d")   [a, b, c, d]: ends on a file → its own name ['d']
ls("/a")         ['b', 'notes']: files and directories list alike
```

```
CLASS FileNode
    FUNCTION Init()
        self.children ← empty map          // name → FileNode
        self.is_file ← false
        self.content ← ""

FUNCTION SplitPath(path)
    names ← empty list
```

```
    FOR EACH part IN path.split("/")
        IF part ≠ ""                          // '/' at the start or end leaves ''
            names.append(part)
    RETURN names

CLASS FileSystem
    FUNCTION Init()
        self.root ← new FileNode()

    FUNCTION Walk(path, create)
        node ← self.root
        FOR EACH name IN SplitPath(path)
            IF name NOT IN node.children
                IF NOT create
                    RETURN null               // the path does not exist
                node.children[name] ← new FileNode()
            node ← node.children[name]
        RETURN node

    FUNCTION Ls(path)
        node ← self.Walk(path, false)
        IF node = null
            RETURN empty list
        IF node.is_file
            names ← SplitPath(path)
            RETURN [names[-1]]                // the last name in the path
        RETURN sorted(node.children)          // the child names, in order

    FUNCTION Mkdir(path)
        self.Walk(path, true)

    FUNCTION AddContentToFile(path, content)
        node ← self.Walk(path, true)
        node.is_file ← true
        node.content ← node.content + content

    FUNCTION ReadContentFromFile(path)
        node ← self.Walk(path, false)
        IF node = null
            RETURN ""
        RETURN node.content
```

Every call visits one node per name, so it costs `O(L)` for a path of `L` names plus the characters read while splitting it; `ls` on a directory adds `O(k log k)` to sort its `k` children, and the tree holds one node per directory or file. Appending copies the old content, since Python strings are immutable, so a file grown by many small appends would keep a list of chunks and join them on read. Deleting a directory is one removal from its parent's map, which drops the whole subtree at once. Reach for a trie of tokens whenever keys are paths: file systems, URL routers, nested configuration keys. Design File System (LC 1166) is the strict cousin, where creating a path succeeds only if its parent exists and the path does not, which is the walk with `create` off for all but the last name; Simplify Path (LC 71) splits the same way and keeps the names on a stack, popping on `..` and skipping `.`. The classic mistake is treating every path in `ls` as a directory: `ls("/a/b/c/d")` must return `["d"]`, the file's own name, not the children of the file's node, which are none.

Practice: LC 588 Design In-Memory File System · LC 1166 Design File System · LC 71 Simplify Path

144. Design a Text Editor

Two Stacks Around the Cursor · Design · Tier 3

Design a text editor with a cursor. `add_text(text)` types the text at the cursor, which ends up just after it; `delete_text(k)` backspaces up to `k` characters and returns how many it actually deleted; `cursor_left(k)` and `cursor_right(k)` move the cursor up to `k` places, stopping at either end of the text, and return the last `min(10, len)` characters to the left of the cursor, where `len` is how many there are. Typing `leetcode`, deleting 4 and typing `practice` leaves `leetpractice|`, with `|` marking the cursor; then `cursor_left(8)` gives `leet|practice` and returns `"leet"`.

The brute force keeps one string and a cursor index, and every insertion or deletion in the middle rebuilds the string, `O(n)` per call even when `k` is tiny. Yet every operation happens at the cursor, and the cursor splits the text into two halves that change only at their facing ends. A stack changes only at its top. So keep the text before the cursor on one stack, `left`, with the character just before the cursor on top, and the text after the cursor on a second stack, `right`, stored reversed so that its top is the character just after the cursor. Typing pushes onto `left` and backspace pops from it; the text far from the cursor is never touched.

Moving the cursor pours characters from one stack onto the other. One step left pops the top of `left`, the character just before the cursor, and pushes it onto `right`, where it becomes the character just after the cursor; one step right is the mirror image. Both loops stop early when the stack they pop from runs empty, which is exactly the cursor reaching an end of the text, and `delete_text` counts its pops so that it can report a short delete. The answer string is always the top ten of `left`, read in text order from the bottom up.

```
call                       before the cursor │ after the cursor  returns
add_text("leetcode")         l e e t c o d e │
delete_text(4)                       l e e t │                   4
add_text("practice") l e e t p r a c t i c e │
cursor_right(3)      l e e t p r a c t i c e │                   "etpractice"
cursor_left(8)                       l e e t │ p r a c t i c e   "leet"
delete_text(10)                              │ p r a c t i c e   4
cursor_left(2)                               │ p r a c t i c e   ""
cursor_right(6)                  p r a c t i │ c e               "practi"
both stacks keep their top at the cursor │; right is stored reversed,
so at the end it holds ['e', 'c'] with 'c' on top
```

```
CLASS TextEditor
    FUNCTION Init()
        self.left ← empty stack         // text before the cursor, top = nearest
        self.right ← empty stack        // text after the cursor, reversed

    FUNCTION AddText(text)
        FOR EACH ch IN text
            self.left.push(ch)

    FUNCTION DeleteText(k)
        deleted ← 0
        WHILE deleted < k AND len(self.left) > 0
            self.left.pop()
            deleted ← deleted + 1
        RETURN deleted
```

```
FUNCTION CursorLeft(k)
    WHILE k > 0 AND len(self.left) > 0
        ch ← self.left.pop()
        self.right.push(ch)         // it is now just after the cursor
        k ← k − 1
    RETURN self.LastTen()

FUNCTION CursorRight(k)
    WHILE k > 0 AND len(self.right) > 0
        ch ← self.right.pop()
        self.left.push(ch)          // it is now just before the cursor
        k ← k − 1
    RETURN self.LastTen()

FUNCTION LastTen()
    start ← max(0, len(self.left) − 10)
    RETURN join(self.left[start..len(self.left) − 1])
```

Every call does at most one pop and one push per character it types, deletes or moves past, plus at most ten steps to build the returned string, so a request for `k` characters costs `O(k)` however long the document is, and the two stacks hold the text once, `O(n)` space. This is the gap buffer that editors such as Emacs use: one array holding the text before the cursor at its front, the text after it at its back, and free space in between, with the two stacks as the two sides of the gap. A doubly linked list of characters with a cursor node supports the same calls in `O(k)` but pays a node per character, and a rope, a balanced tree of string chunks, takes over when documents are huge and edits jump around. Reach for two facing stacks whenever a position moves back and forth through a sequence and edits happen only where it stands; Browser History (#95) has the same shape, with back pages on one side of the cursor and forward pages on the other, but there a visit erases the forward pages, so one array and a cursor can jump any distance in `O(1)`. The classic mistake is joining all of `left` and then keeping its last ten characters, which costs `O(n)` on every cursor move and turns the design back into the brute force; slice the last ten first, as `LastTen` does.

Practice: LC 2296 Design a Text Editor · LC 1472 Design Browser History · LC 844 Backspace String Compare

145. Unique ID Generator

Snowflake: Time, Machine and Sequence Bits · Design · Tier 3

Design a generator of unique 64-bit ids for many machines that create records independently, with ids that sort roughly by creation time. Twitter's Snowflake packs three fields into one integer: 41 bits of milliseconds since a custom epoch, 10 bits of machine id and 12 bits of sequence number within the millisecond, with the sign bit left at 0. `next_id(now_ms)` takes the time as an argument, which keeps it deterministic and testable. On machine 5, two ids requested in millisecond 1000 after the epoch and one in millisecond 1001 are `4194324480`, `4194324481` and `4198518784`.

The obvious designs either coordinate or sprawl. A single database counter makes every id a round trip to one point of failure; a random 128-bit UUID needs no coordination but is twice as wide and scattered in time, so new rows land all over a database index instead of at its end. Snowflake removes the coordination by splitting the id space. The machine field gives each machine a private slice, so two machines never produce the same id, and within one machine the pair (millisecond, sequence) only has to never repeat, which takes nothing but local state. With the time in the highest bits, every id from a later millisecond is larger than every id from an earlier one, so sorting ids sorts them by time across the fleet, roughly: machine clocks drift apart by a few milliseconds, and inside one millisecond the machine id, not the true order, decides.

The generator remembers the millisecond of its latest id and that id's sequence number. A call in a new millisecond restarts the sequence at 0, and a call in the same millisecond takes the next number. After 4096 ids in one millisecond the 12 bits would overflow; the real generator spins until the clock ticks, and with the time passed in, it moves its stamp to the next millisecond itself, borrowing it from the future. Since the stamp can then run ahead of the clock, the generator keeps the latest clock reading separately: a call whose time is not past the stamp continues the sequence there, and only a time earlier than the previous call means the clock ran backwards, which the generator refuses rather than stamp ids with a time it cannot trust. Packing is two shifts and two ORs; decoding reverses them with shifts and masks, and AND-ing with `MAX_SEQUENCE = 4095`, twelve 1 bits, keeps exactly the low field.

```
machine 5, epoch E = 1700000000000 (2023-11-14 22:13:20 UTC)
now - E   last_ms - E   sequence   id
1000      1000          0          4194324480    a new millisecond
1000      1000          1          4194324481    the same millisecond
1001      1001          0          4198518784    a new millisecond
999       clock moved backwards: 999 < 1001, no id

the second id, 4194324481, as 64 bits from bit 63 down to bit 0:
0 00000000000000000000000000000001111101000 0000000101 000000000001
| └─── 41 bits: 1000 ms since the epoch ──┘ └─ 5 ──┘ └──── 1 ──┘
sign                                        machine    sequence
decode: id >> 22 = 1000, (id >> 12) & 1023 = 5, id & 4095 = 1

machine 7, 4097 calls at now - E = 50: calls 1 to 4096 get
[50, 7, 0] to [50, 7, 4095]; call 4097 moves to the next ms: [51, 7, 0]
```

```
SEQUENCE_BITS ← 12                          // 4096 ids per millisecond
MACHINE_BITS ← 10                           // 1024 machines
TIME_SHIFT ← SEQUENCE_BITS + MACHINE_BITS   // time starts at bit 22
MAX_SEQUENCE ← (1 << SEQUENCE_BITS) - 1     // 4095: twelve 1 bits
MAX_MACHINE ← (1 << MACHINE_BITS) - 1       // 1023: ten 1 bits
```

```
CLASS SnowflakeGenerator
    FUNCTION Init(machine_id, epoch_ms)
        self.machine_id ← machine_id    // 0..1023, different on every machine
        self.epoch_ms ← epoch_ms        // custom epoch: time 0 for these ids
        self.last_clock ← -1            // the latest time a caller passed
        self.last_ms ← -1               // the millisecond in the latest id
        self.sequence ← 0               // the latest id's number in last_ms

    FUNCTION NextId(now_ms)
        IF now_ms < self.last_clock
            RAISE "clock moved backwards"
        self.last_clock ← now_ms
        IF now_ms > self.last_ms
            self.last_ms ← now_ms       // a new millisecond: restart at 0
            self.sequence ← 0
        ELSE
            self.sequence ← self.sequence + 1   // the same millisecond
            IF self.sequence > MAX_SEQUENCE
                self.last_ms ← self.last_ms + 1 // 4096 ids used: next ms
                self.sequence ← 0
        elapsed ← self.last_ms - self.epoch_ms
        time_part ← elapsed << TIME_SHIFT                   // bits 22..62
        machine_part ← self.machine_id << SEQUENCE_BITS     // bits 12..21
        RETURN time_part | machine_part | self.sequence     // bits 0..11

FUNCTION DecodeId(snowflake_id)
    sequence ← snowflake_id & MAX_SEQUENCE                  // low 12 bits
    machine_id ← (snowflake_id >> SEQUENCE_BITS) & MAX_MACHINE
    elapsed ← snowflake_id >> TIME_SHIFT                    // the rest
    RETURN [elapsed, machine_id, sequence]
```

Each id costs `O(1)` time and a handful of numbers of state. The field widths set the capacity: 41 bits of milliseconds last about 69.7 years, so counting from the Unix epoch would run out in 2039, while the epoch above lasts until 2093; 10 bits allow 1024 machines, and 12 bits allow 4096 ids per millisecond per machine, about four million a second. Machine ids are the only coordination, assigned once at deployment from configuration or a service such as ZooKeeper; Twitter split the field into 5 bits of data center and 5 of worker. Reach for this layout when ids must be unique across servers without a central counter, ordered by time and small enough for a 64-bit column; the URL shortener of #100 can encode such ids in base 62. The classic mistake is wrapping the sequence with `(sequence + 1) & 4095` and carrying on, so the 4097th id of a millisecond repeats the first. Another is a wrong shift: moving the time by 12 instead of 22 lands its low bits on the machine field, and two machines can mint the same id.

Practice: LC 535 Encode and Decode TinyURL · LC 190 Reverse Bits · LC 191 Number of 1 Bits

146. Bloom Filter

k Hash Functions into One Bit Array · Design · Tier 3

A Bloom filter remembers a set without storing its items. `add(item)` records an item, and `might_contain(item)` answers false, which is always right, or true, which is wrong only for a small, tunable share of items that were never added. With 19 bits and 3 hash functions, after adding `cat`, `dog` and `fish`, `might_contain("cat")` is true, `might_contain("owl")` is false, and `might_contain("colt")` is true although `colt` was never added.

A hash set answers exactly but stores every item in full, too much for a billion URLs. When a wrong yes costs only a wasted lookup, a Bloom filter makes do with about 10 bits per item, however large the items. It is an array of m bits, all 0, and k hash functions, each sending an item to one bit. `add` sets the item's k bits to 1 and `might_contain` reads them. A single 0 proves the item was never added, since adding it would have set that bit and bits are never cleared: there are no false negatives. All k bits at 1 only says probably, because other items may have set them.

Two hashes are enough for all k probes, as Kirsch and Mitzenmacher showed: probe i lands on bit `(h1 + i × step) mod m`, with h1 and h2 polynomial string hashes as in Rabin-Karp (#120), bases 31 and 37. The step `1 + h2 mod (m − 1)` is never 0, so with m prime the k probes hit k different bits. Python's `hash()` would be a mistake: it is salted afresh in every process, so a filter saved by one run would be probed at the wrong bits by the next. Picture the array as a strip of lights. Each add switches on k of them, and a query asks whether all k of its own are on. After n adds a bit is still off with probability about `e^(−kn/m)`, so the false-positive rate is about `(1 − e^(−kn/m))^k`. The best `k = (m / n) ln 2` leaves half the lights on, and 10 bits per item with 7 hashes give 0.82 percent.

```
m = 19 bits, k = 3; probe i sets bit (h1 + i·step) mod 19, step = 1 + h2 mod 18
add "cat"     h1 mod 19 = 13, step  7   →   bits 13,  1,  8
add "dog"     h1 mod 19 =  8, step  9   →   bits  8, 17,  7    shares bit 8
add "fish"    h1 mod 19 = 10, step 13   →   bits 10,  4, 17    shares bit 17

  bit   0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
        0  1  0  0  1  0  0  1  1  0  1  0  0  1  0  0  0  1  0   7 of 19 set

might_contain("cat")    bits 13, 1, 8    read 1 1 1   → true, it was added
might_contain("owl")    bits 2, 17, 13   read 0 1 1   → false: bit 2 is 0
might_contain("colt")   bits 7, 10, 13   read 1 1 1   → true, a false positive:
                        dog set bit 7, fish bit 10 and cat bit 13
```

```
MOD ← 1000000007                          // a large prime, as in Rabin-Karp

FUNCTION StringHash(word, base)
    h ← 0
    FOR EACH ch IN word
        h ← (h * base + code(ch)) mod MOD
    RETURN h

CLASS BloomFilter
    FUNCTION Init(m, k)
        self.m ← m                        // number of bits, best a prime
        self.k ← k                        // number of hash functions
        self.bits ← array of m zeros
```

```
    FUNCTION Positions(item)
        h1 ← StringHash(item, 31)
        h2 ← StringHash(item, 37)          // a second, different hash
        step ← 1 + h2 mod (self.m − 1)     // 1..m−1: the probes always move
        result ← empty list
        FOR i FROM 0 TO self.k − 1
            result.append((h1 + i * step) mod self.m)
        RETURN result

    FUNCTION Add(item)
        FOR EACH p IN self.Positions(item)
            self.bits[p] ← 1

    FUNCTION MightContain(item)
        FOR EACH p IN self.Positions(item)
            IF self.bits[p] = 0
                RETURN false               // adding it would have set this bit
        RETURN true                        // added, or a false positive
```

Each operation costs `O(L)` to hash an item of length L plus `O(k)` bit operations, and the filter stays m bits however many items arrive; a target rate p needs `m = −n ln p / (ln 2)²` bits, about 9.6 per item for 1 percent. The signals are a huge set, a membership check in front of an expensive lookup, and tolerance for a rare false yes: LSM-tree stores such as Bigtable, Cassandra and RocksDB keep a filter per file on disk and skip every file whose filter says no. Guava builds h1 and h2 from the two halves of one 128-bit MurmurHash3, a far stronger hash than these. Bits are shared, so a filter cannot delete; a counting Bloom filter keeps a small counter per position instead. The classic mistake is deleting anyway: clearing the bits of `cat` turns off bit 8, and `dog`, which was added, starts coming back false.

Practice: LC 705 Design HashSet · LC 187 Repeated DNA Sequences

147. Count-Min Sketch

Approximate Counts for Heavy Hitters · Design · Tier 3

A Count-Min sketch counts the items of a stream approximately, in fixed memory. `add(item, count)` records occurrences and `estimate(item)` returns a count that is never too small and rarely much too large; a size-k heap beside it tracks the k most frequent items, the heavy hitters. With 3 rows of 5 counters, the stream `cat dog cat fish cat dog bird cat dog` gives the estimates 4, 3, 1 and 2, although `bird` came once, and the top two are `cat` and `dog`.

Exact counting needs a hash map entry per distinct item, and a stream of search queries or IP addresses has billions. The sketch is a grid of d rows by w counters, each row with its own hash function: `add` bumps the item's counter in every row, and `estimate` returns the smallest of its d counters. A counter holds the item's count plus the counts of everything that collided with it in that row, never less, so the minimum keeps the least polluted one. A single clean row gives the exact answer; only an item that collides in every row, like `bird`, comes out too large.

Row r reads the item as a number x with a base-31 polynomial hash and sends it to column `((a·x + b) mod p) mod w`, with p the large prime `MOD` and a and b drawn for that row from a seeded generator, a universal hash family. Two items then share a counter with probability about `1 / w`, so a row overcounts by at most `N / w` on average over a stream of N items, and by more than `e·N / w` with probability at most `1 / e`; the rows are independent, so all d overshoot together with probability at most `e^(-d)`.

The sketch cannot list what it has seen, so the heavy hitters live in the size-k min-heap of Top-K (#19) beside it: a newcomer is pushed with its estimate, and at k + 1 entries the weakest leaves, a tie evicting the name that sorts first, since the heap compares `(estimate, item)` pairs. A leader's count grows while its heap entry stays behind, so entries go stale; a stale entry popped from the top goes back with its current count, and the first fresh entry popped is the true weakest. A dropped item keeps its count in the sketch, so `dog` returns with all of it.

```
width 5, depth 3, seed 6; stream cat dog cat fish cat dog bird cat dog
columns in rows 0, 1, 2:  cat 1 0 3   dog 1 1 4   fish 4 2 2   bird 4 2 3
         col 0   col 1   col 2   col 3   col 4
row 0      0       7       0       0       2      cat 4 + dog 3 share col 1
row 1      4       3       2       0       0      fish 1 + bird 1 share col 2
row 2      0       0       1       5       3      cat 4 + bird 1 share col 3
estimate(cat)  = min(7, 4, 5) = 4     exact: row 1 is clean
estimate(fish) = min(2, 2, 1) = 1     exact: row 2 is clean
estimate(bird) = min(2, 2, 5) = 2     true 1: a collision in every row

item   est   heap of k = 2, smallest first   what happened
cat     1    (1 cat)
dog     1    (1 cat) (1 dog)
cat     2    (1 cat) (1 dog)                the entry (1 cat) is now stale
fish    1    (1 fish) (2 cat)               (1 cat) back as (2 cat); drop dog
cat     3    (1 fish) (2 cat)
dog     2    (2 cat) (2 dog)                dog returns with 2; drop fish
bird    2    (2 cat) (2 dog)                tie at 2, bird sorts first: drop it
cat     4    (2 cat) (2 dog)
dog     3    (2 cat) (2 dog)                top() = [(4, 'cat'), (3, 'dog')]
```

```
MOD ← 1000000007                        // a large prime

FUNCTION StringHash(word)
    h ← 0
    FOR EACH ch IN word
        h ← (h * 31 + code(ch)) mod MOD
    RETURN h

CLASS CountMinSketch
    FUNCTION Init(width, depth, seed)
        self.width ← width
        self.depth ← depth
        self.table ← empty list         // depth rows of width counters
        self.a ← empty list             // row r sends x to column
        self.b ← empty list             // ((a[r]·x + b[r]) mod MOD) mod width
        rng ← RandomGenerator(seed)     // the same seed, the same hashes
        FOR row FROM 0 TO depth - 1
            self.table.append(array of width zeros)
            self.a.append(rng.RandomInt(1, MOD - 1))
            self.b.append(rng.RandomInt(0, MOD - 1))

    FUNCTION Columns(item)
        x ← StringHash(item)
        result ← empty list
        FOR row FROM 0 TO self.depth - 1
            h ← (self.a[row] * x + self.b[row]) mod MOD
            result.append(h mod self.width)
        RETURN result

    FUNCTION Add(item, count)
        cols ← self.Columns(item)
        FOR row FROM 0 TO self.depth - 1
            self.table[row][cols[row]] ← self.table[row][cols[row]] + count

    FUNCTION Estimate(item)
        cols ← self.Columns(item)
        best ← self.table[0][cols[0]]
        FOR row FROM 1 TO self.depth - 1
            best ← min(best, self.table[row][cols[row]])
        RETURN best                     // collisions only ever add

CLASS HeavyHitters
    FUNCTION Init(k, width, depth, seed)
        self.k ← k
        self.sketch ← CountMinSketch(width, depth, seed)
        self.counts ← empty map         // leader → estimate when last seen
        self.heap ← empty min-heap      // (estimate, leader), one per leader

    FUNCTION Add(item)
        self.sketch.Add(item, 1)
        estimate ← self.sketch.Estimate(item)
        IF item IN self.counts
            self.counts[item] ← estimate  // its heap entry is now stale
            RETURN
        self.counts[item] ← estimate
```

```
        self.heap.push((estimate, item))
        IF len(self.heap) > self.k      // one too many: drop the weakest
            (estimate, weakest) ← self.heap.pop()
            WHILE estimate ≠ self.counts[weakest]  // stale: it grew since
                self.heap.push((self.counts[weakest], weakest))
                (estimate, weakest) ← self.heap.pop()
            self.counts.remove(weakest)

    FUNCTION Top()
        pairs ← empty list
        FOR EACH item IN self.counts
            pairs.append((self.counts[item], item))
        pairs.sort()
        RETURN reverse(pairs)           // largest estimate first
```

Each `add` costs `O(L + d)` for an item of length L, plus `O(log k)` amortized for the heap, since each re-push repairs one earlier update. The memory is `d × w` counters and k leaders, whatever the number of distinct items: in the demo 800 counters find the three heavy words among 3003, and $w = 2719$ with $d = 5$ keeps each estimate within 0.1 percent of N with 99 percent probability. Sketches built from the same seed merge by adding their grids cell by cell, so each server counts its own traffic and a collector sums the grids. The signals are counts over a stream too big to store and trending top-k lists; counts that fit in memory deserve the exact heap of #19 or buckets of #61. The classic mistake is one hash for every row: the rows become copies, a collision repeats in all of them, and the minimum is no better than one row.

Practice: LC 347 Top K Frequent Elements · LC 692 Top K Frequent Words · LC 1636 Sort Array by Increasing Frequency

148. Quadtree

Points in Squares, Split When Full · Design · Tier 3

A quadtree stores points in the plane so that a box query reads only the part of the map the box covers, the query behind “find the drivers near this rider”. `insert(x, y)` adds a point and `range_query(left, bottom, right, top)` returns the points inside the box, edges included. With nine drivers in a 16 by 16 square of city blocks, the box from `(2, 2)` to `(6, 6)` around a rider at `(4, 4)` returns `(3, 5)`, `(5, 6)` and `(6, 3)`.

Scanning every driver costs `O(n)` per query, and a busy city can send thousands of queries a second. A quadtree cuts the square into four equal quarters and keeps cutting wherever points crowd. Each node covers a square, given by its lower-left corner and its side, and a leaf holds up to 4 points. Inserting into a full leaf splits it into four children of half the side, south-west, south-east, north-west and north-east, and its points move down into the children whose squares hold them. Busy downtown blocks end up deep inside small squares while an empty suburb stays one big leaf, so the tree follows the density of the data.

A query walks down from the root and asks each node one question: does its square overlap the box? If not, the whole square is skipped and every point beneath it goes unread. If so, a leaf tests its few points and an inner node asks its four children. A square owns its left and bottom edges but not its right and top ones, so each point lives in exactly one leaf. A leaf whose side has shrunk to `MIN_SIZE` keeps every point it receives, so a pile of identical points cannot split forever.

```
insert (1, 2) (3, 5) (6, 1) (2, 7): they fill the root, a leaf of 4
insert (5, 6): the root splits, all 4 points fall into SW, and SW splits too
insert (12, 3) (10, 12) (14, 9) (6, 3)

root  [0,16)×[0,16)
├── SW  [0,8)×[0,8)
│   ├── SW  [0,4)×[0,4)      (1, 2)
│   ├── SE  [4,8)×[0,4)      (6, 1) (6, 3)
│   ├── NW  [0,4)×[4,8)      (3, 5) (2, 7)
│   └── NE  [4,8)×[4,8)      (5, 6)
├── SE  [8,16)×[0,8)         (12, 3)
├── NW  [0,8)×[8,16)         empty
└── NE  [8,16)×[8,16)        (10, 12) (14, 9)

range_query(2, 2, 6, 6), the box around a rider at (4, 4):
  it overlaps root, SW and all four children of SW, which hold 6 points
  (1, 2) ✗   (6, 1) ✗   (6, 3) ✓   (3, 5) ✓   (2, 7) ✗   (5, 6) ✓
  SE, NW and NE miss the box: skipped, their 3 points never read
  → sorted: [(3, 5), (5, 6), (6, 3)]
```

```
CAPACITY ← 4                            // points a leaf holds before it splits
MIN_SIZE ← 1                            // a leaf this small never splits

CLASS QuadTree
    FUNCTION Init(x, y, size)
        self.x ← x                      // lower-left corner
        self.y ← y
        self.size ← size
        self.points ← empty list        // only leaves hold points
```

```
        self.children ← null            // SW, SE, NW, NE once split

    FUNCTION Contains(px, py)           // left and bottom edges included
        inside_x ← self.x ≤ px AND px < self.x + self.size
        inside_y ← self.y ≤ py AND py < self.y + self.size
        RETURN inside_x AND inside_y

    FUNCTION Insert(px, py)
        IF NOT self.Contains(px, py)
            RETURN false                // not in this square
        IF self.children = null
            IF len(self.points) < CAPACITY OR self.size ≤ MIN_SIZE
                self.points.append((px, py))
                RETURN true
            self.Split()                // a full leaf becomes four leaves
        FOR EACH child IN self.children
            IF child.Insert(px, py)     // exactly one child holds it
                RETURN true
        RETURN false

    FUNCTION Split()
        half ← self.size / 2
        self.children ← empty list      // SW, SE, NW, NE
        FOR EACH (dx, dy) IN [(0, 0), (half, 0), (0, half), (half, half)]
            self.children.append(QuadTree(self.x + dx, self.y + dy, half))
        old_points ← self.points
        self.points ← empty list
        FOR EACH (px, py) IN old_points
            self.Insert(px, py)         // now lands in a child

    FUNCTION RangeQuery(left, bottom, right, top)
        found ← empty list
        IF left ≥ self.x + self.size OR right < self.x
            RETURN found                // no overlap in x: prune
        IF bottom ≥ self.y + self.size OR top < self.y
            RETURN found                // no overlap in y: prune
        FOR EACH (px, py) IN self.points
            IF left ≤ px AND px ≤ right AND bottom ≤ py AND py ≤ top
                found.append((px, py))
        IF self.children ≠ null
            FOR EACH child IN self.children
                found.extend(child.RangeQuery(left, bottom, right, top))
        RETURN found
```

An insert walks one path down from the root, `O(depth)`, and for evenly spread points the depth is `O(log n)`. A query costs the nodes it visits plus the points it reads: a box about the size of a leaf touches a few leaves and their ancestors, roughly `O(log n + k)` for k answers, and a radius query is a box query plus a distance check. The signals are points in the plane and questions about what is near: the drivers around a rider, the restaurants within a mile, the pins in a map's visible window. A geohash writes the same picture as a string: interleaved bits of longitude and latitude pick a half, then a half of that, so nearby points share a prefix, though a query must also read the 8 neighboring cells, since a point just across an edge can get a different prefix. Google's S2 library numbers quadtree cells on the faces of a cube along a Hilbert curve. A k-d tree splits at the median instead, alternating x and y, and stays balanced for points known in advance. The classic mistake is pruning by corners, asking whether a

corner of either shape lies inside the other: a thin box that crosses a square from edge to edge has no such corner, so the square is skipped and the points in its middle are lost.

Practice: LC 427 Construct Quad Tree · LC 558 Logical OR of Two Binary Grids Represented as Quad-Trees · LC 973 K Closest Points to Origin

149. Job Scheduler

A Min-Heap of Next Run Times · Design · Tier 3

Design a scheduler that runs jobs at given times. `schedule(name, run_at, interval)` registers a job that first runs at `run_at` and then every `interval` time units, or only once when `interval` is 0; `cancel(name)` drops a job; and `run_due(now)` runs every job that is due by `now`, in time order with ties going to the entry inserted first, and returns the runs as `[time, name]` pairs. The caller passes the time, the way an event loop passes its clock. With `ping` every 5 units from time 5, `save` once at 10 and `mail` once at 12, `run_due(10)` returns `[[5, 'ping'], [10, 'save'], [10, 'ping']]`.

The brute force scans every job on every tick to find the due ones, `O(n)` per check even when nothing is due. A scheduler only ever asks which run comes first, and a min-heap keyed by run time keeps that answer at its top, at `O(log n)` per push and pop. So keep one heap entry per pending run, `(run_at, seq, name)`, and let `run_due` pop while the top is due. The `seq` comes from a counter stamped on every push: it breaks ties between equal times in insertion order, and since no two entries share it, the heap never compares names. A recurring job goes back in at `run_at + interval` right after it runs; if that time is still at or before `now`, the same loop pops it again, so a scheduler that fell behind catches up on every missed run, in order.

Cancelling cannot reach into the middle of a heap cheaply, so it does not try. Each name maps to the `seq` of its one live entry; `cancel` deletes that mapping in `O(1)`, and the orphaned entry stays in the heap until it surfaces at the top, where `run_due` sees that it is not the live entry and discards it. Comparing the `seq`, not just the name, matters once a cancelled name is scheduled again: the old entry and the new one carry the same name, and only the number tells them apart. Rescheduling a live name works the same way.

```
entries are (run_at, seq, name), listed earliest first; ✗ = stale
schedule(ping, 5, 5)    heap (5,1,ping)
schedule(save, 10, 0)   heap (5,1,ping) (10,2,save)
schedule(mail, 12, 0)   heap (5,1,ping) (10,2,save) (12,3,mail)
run_due(10)             pop (5,1,ping): run, push (10,4,ping)
                        pop (10,2,save): run, a one-off: done
                        pop (10,4,ping): run, push (15,5,ping)
                        top (12,3,mail) is after 10: stop
                        returns [[5, 'ping'], [10, 'save'], [10, 'ping']]
cancel(mail)            heap (12,3,mail)✗ (15,5,ping)
run_due(12)             pop (12,3,mail): stale, skip; returns []
run_due(26)             pops 15, 20, 25 and pushes 20, 25, 30: catch-up
                        heap (30,8,ping)
cancel(ping)            heap (30,8,ping)✗
schedule(ping, 35, 0)   heap (30,8,ping)✗ (35,9,ping)
run_due(40)             pop (30,8,ping): live seq is 9, so stale, skip
                        pop (35,9,ping): run; returns [[35, 'ping']]
```

```
CLASS JobScheduler
    FUNCTION Init()
        self.heap ← empty min-heap      // (run_at, seq, name), earliest on top
        self.interval ← empty map       // name → repeat interval, 0 for one-off
        self.live ← empty map           // name → seq of its one live entry
        self.seq ← 0                    // insertion counter: breaks ties
```

```
    FUNCTION Push(name, run_at)
        self.seq ← self.seq + 1
        self.live[name] ← self.seq
        self.heap.push((run_at, self.seq, name))

    FUNCTION Schedule(name, run_at, interval)
        self.interval[name] ← interval
        self.Push(name, run_at)         // an older entry for name goes stale

    FUNCTION Cancel(name)
        IF name NOT IN self.live
            RETURN false
        self.live.remove(name)
        self.interval.remove(name)
        RETURN true

    FUNCTION RunDue(now)
        ran ← empty list
        WHILE len(self.heap) > 0 AND self.heap.top()[0] ≤ now
            (run_at, seq, name) ← self.heap.pop()
            IF name NOT IN self.live OR self.live[name] ≠ seq
                CONTINUE                // cancelled or replaced: stale
            ran.append([run_at, name])
            interval ← self.interval[name]
            IF interval > 0
                self.Push(name, run_at + interval)  // if still due: catch up
            ELSE
                self.live.remove(name)  // a one-off job is done
                self.interval.remove(name)
        RETURN ran
```

Scheduling costs `O(log n)` for `n` heap entries, each run costs one pop and at most one push, and a cancel is `O(1)`; a stale entry is popped once, so lazy deletion adds at most one pop per cancel or reschedule. Dead entries do occupy the heap until they surface, so a scheduler whose timers are mostly cancelled, as timeouts usually are, rebuilds the heap from its live entries once the dead ones outnumber them; the expiring cache of #94 and the price tracker of #140 lean on the same lazy deletion. For millions of timers, kernels and network libraries use a timing wheel: a circular array of buckets, one per tick, where a timer due `d` ticks ahead goes into the bucket `d` slots past the current one, so adding and cancelling cost `O(1)` and each tick empties one bucket, with coarser wheels stacked above for long delays. Replaying missed runs is a policy; a job that should not replay them re-arms at the first `run_at + k × interval` after `now`. The signals are “run at time t”, “every k seconds”, timeouts, delayed retries, and simulations that serve tasks or meetings in time order, such as Single-Threaded CPU and Meeting Rooms III, which builds on the end-time heap of #27. The classic mistake is checking only the name for staleness: above, the dead entry `(30,8,ping)` would pass that test and run `ping` five units early, and the real run at 35 would then be thrown away.

Practice: LC 1834 Single-Threaded CPU · LC 621 Task Scheduler · LC 2402 Meeting Rooms III

150. Inverted Index

Postings Lists and Sorted Intersection · Design · Tier 3

An inverted index answers keyword search. `add_document(doc_id, text)` indexes a document, `search_and(terms)` returns the ids of the documents that contain every term, and `search_or(terms)` those that contain any of them. It is the index at the back of a book turned into a data structure, from each word to the places it appears. Over six short documents, `search_and(["quick", "dog"])` returns `[3, 4, 6]` and `search_or(["fox", "lazy"])` returns `[1, 2, 4]`.

Scanning every document for every query rereads the whole collection each time. The index does the reading once, when a document arrives: it maps each term to its postings list, the ids of the documents that contain it, and a query then touches only the lists of its own terms. `add_document` lowercases the text, splits it on spaces and appends `doc_id` to each word's list. Ids arrive in increasing order and a list only ever grows at its end, so every postings list is sorted without a single sort. A word repeated in one document finds the document's id already at the end of its list, so checking the last id keeps each document once.

Sorted lists turn AND into a two-pointer walk, the merge of merge sort (#17) with a different rule for what to keep. Put a finger on the front of each list. Equal ids mean the document holds both terms, so keep it and move both fingers; otherwise the smaller id cannot be in the other list, whose remaining ids are all larger, so its finger moves past it. OR is the same walk keeping every id once. With several terms, AND starts from the shortest list: the running result can only shrink, so it is never longer than that first list.

```
doc  text                          postings, ids in the order added
1    the quick brown fox           the    [1, 2, 4, 6]     doc 4 only once
2    the lazy dog                  quick  [1, 3, 4, 6]
3    a quick brown dog             dog    [2, 3, 4, 5, 6]
4    the dog chased the quick fox  fox    [1, 4]
5    my dog sleeps                 lazy   [2]
6    quick thinking saved the dog  brown  [1, 3]

search_and(quick, dog): quick is the shorter list, so it leads
  i  j   quick[i]  dog[j]   step
  0  0      1        2      1 < 2: 1 is not in dog, i moves
  1  0      3        2      3 > 2: 2 is not in quick, j moves
  1  1      3        3      equal: keep 3, both move
  2  2      4        4      equal: keep 4, both move
  3  3      6        5      6 > 5: j moves
  3  4      6        6      equal: keep 6
  → [3, 4, 6]
search_or(fox, lazy): the same walk keeps every id once → [1, 2, 4]
```

```
FUNCTION Intersect(a, b)
    result ← empty list
    i ← 0
    j ← 0
    WHILE i < len(a) AND j < len(b)
        IF a[i] = b[j]
            result.append(a[i])            // in both: keep it
            i ← i + 1
            j ← j + 1
```

```
        ELSE IF a[i] < b[j]
            i ← i + 1                       // a[i] is not in b
        ELSE
            j ← j + 1                       // b[j] is not in a
    RETURN result

FUNCTION Union(a, b)
    result ← empty list
    i ← 0
    j ← 0
    WHILE i < len(a) AND j < len(b)
        IF a[i] = b[j]
            result.append(a[i])             // in both: keep one copy
            i ← i + 1
            j ← j + 1
        ELSE IF a[i] < b[j]
            result.append(a[i])
            i ← i + 1
        ELSE
            result.append(b[j])
            j ← j + 1
    result.extend(a[i..len(a) - 1])         // one list ran out: copy the rest
    result.extend(b[j..len(b) - 1])
    RETURN result

CLASS InvertedIndex
    FUNCTION Init()
        self.postings ← empty map           // term → sorted doc ids

    FUNCTION AddDocument(doc_id, text)
        lowered ← lowercase(text)
        FOR EACH word IN split(lowered)
            IF word NOT IN self.postings
                self.postings[word] ← empty list
            ids ← self.postings[word]
            IF len(ids) = 0 OR ids[len(ids) - 1] ≠ doc_id  // once per document
                ids.append(doc_id)

    FUNCTION SearchAnd(terms)
        lists ← empty list
        FOR EACH term IN terms
            word ← lowercase(term)
            IF word NOT IN self.postings
                RETURN empty list           // a missing term matches nothing
            lists.append(self.postings[word])
        IF len(lists) = 0
            RETURN empty list
        lists.sort(by length)               // shortest list first
        result ← copy of lists[0]           // the index stays intact
        FOR k FROM 1 TO len(lists) - 1
            result ← Intersect(result, lists[k])
        RETURN result

    FUNCTION SearchOr(terms)
        result ← empty list
```

```
        FOR EACH term IN terms
            word ← lowercase(term)
            IF word IN self.postings
                result ← Union(result, self.postings[word])
        RETURN result
```

Indexing costs `O(W)` for a document of W words. An AND query takes time linear in the total length of its lists, one step per id passed; OR folds t lists of N ids in all pairwise in `O(t · N)`, and the heap of list fronts from #20 merges them in `O(N log t)`. Real engines build on the same sorted lists. Each posting also records where the word sits in the document, so a phrase query such as `quick fox` checks that the positions are adjacent. Skip pointers, one every √L entries of a list of L ids, let a finger leap ahead when the other is far in front, so the short list drives the cost. The ids are stored as gaps between neighbors, small numbers that compress well, and matches are ranked by TF-IDF: how often the term appears in the document times the log of how rare it is in the collection. A large index is split by document across machines, each shard answering for its slice. The signals are keyword search and "which records have all of these tags". The classic mistake is appending the id once per occurrence: doc 4 says `the` twice, the list of `the` becomes `[1, 2, 4, 4, 6]`, and `search_or(["the"])` reports document 4 twice.

Practice: LC 349 Intersection of Two Arrays · LC 350 Intersection of Two Arrays II · LC 1268 Search Suggestions System